

Pengembangan Arsitektur Platform DataOps dan MLOps Terintegrasi pada Kubernetes dengan Pemanfaatan *Open Source Tools*

Draf Laporan Tugas Akhir

Oleh

Bryan P. Hutagalung
18222130



**PROGRAM STUDI SISTEM DAN TEKNOLOGI INFORMASI
SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA
INSTITUT TEKNOLOGI BANDUNG**

Juli 2026

LEMBAR PENGESAHAN

Pengembangan Arsitektur Platform DataOps dan MLOps Terintegrasi pada Kubernetes dengan Pemanfaatan *Open Source Tools*

Draf Laporan Tugas Akhir

Oleh

**Bryan P. Hutagalung
18222130**

Program Studi Sistem dan Teknologi Informasi
Sekolah Teknik Elektro dan Informatika
Institut Teknologi Bandung

Laporan Tugas Akhir ini telah disetujui dan disahkan
di Bandung, pada tanggal 9 Juli 2026

Pembimbing

Achmad Imam Kistijantoro, S.T, M.Sc., Ph.D.

NIP. 197308092006041001

PERNYATAAN ORISINALITAS

Saya yang bertanda tangan di bawah ini menyatakan dengan sesungguhnya bahwa:

1. Tugas Akhir ini adalah hasil karya saya sendiri yang dibuat dengan sebenarnya sesuai dengan kaidah ilmiah dan etika akademik.
2. Semua sumber rujukan dan data yang saya gunakan dalam penyusunan laporan tugas akhir ini, baik yang berupa kutipan langsung maupun tidak langsung, telah saya cantumkan sumbernya dengan baik dan benar sesuai dengan ketentuan penulisan laporan tugas akhir.
3. Isi laporan ini belum pernah diajukan pada program pendidikan di institusi mana pun.

Apabila di kemudian hari terbukti bahwa laporan ini melanggar hal-hal di atas, saya bersedia menerima sanksi sesuai dengan peraturan yang berlaku di Institut Teknologi Bandung.

Bandung, 9 Juli 2026

Bryan P. Hutagalung

NIM: 18222130

PERNYATAAN PENGGUNAAN AI

Saya yang bertanda tangan di bawah ini menyatakan bahwa dalam penulisan dokumen Tugas Akhir ini, saya menggunakan alat bantu kecerdasan artifisial (AI) untuk beberapa aspek tertentu di dalam proses penulisan, seperti yang tercantum dalam tabel berikut.

No.	Jenis	Alat Bantu	Tingkat	Tempat
1	Memeriksa ejaan dan tata bahasa	Claude	Tinggi	Semua bab
2	Pembuatan teks	Tidak ada	Tidak ada	Tidak ada
3	Pembuatan grafik, gambar, atau ilustrasi	Tidak ada	Tidak ada	Tidak ada
4	Pencarian informasi atau referensi	Claude	Sangat tinggi	Bab 1 - Bab 5
5	Penyusunan bahan diskusi	Claude	Sangat tinggi	Semua bab
6	Penyusunan kode program	Claude	Sedang	Bab 5 dan Lampiran A
7	Penyusunan formula atau perhitungan matematis	Tidak ada	Tidak ada	Tidak ada
8	Penyusunan konsep desain atau algoritma	Claude	Rendah	Bab 4
9	Penyusunan analisis data	Tidak ada	Tidak ada	Tidak ada
10	Penyusunan kesimpulan atau rekomendasi	Tidak ada	Tidak ada	Tidak ada

Bandung, 9 Juli 2026

Bryan P. Hutagalung

NIM: 18222130

ABSTRAK

Pengembangan Arsitektur Platform DataOps dan MLOps Terintegrasi pada Kubernetes dengan Pemanfaatan *Open Source Tools*

oleh

Bryan P. Hutagalung

18222130

Pengembangan model *machine learning* ke tahap produksi masih menghadapi fragmentasi *tool* dan pemisahan kerja antara tim data dan tim model. Praktik DataOps memperkuat kualitas dan kecepatan alur data, sementara MLOps memperkuat keandalan siklus hidup model, namun keduanya sering tumbuh terpisah sehingga *lineage*, deteksi *drift*, dan penyajian fitur antara mode *batch* dan *streaming* tidak konsisten lintas siklus. Penelitian ini mengembangkan arsitektur platform yang mengintegrasikan DataOps dan MLOps pada Kubernetes dengan komponen *open source*, sehingga satu bidang kendali menyatukan tujuh lapis arsitektur, yaitu infrastruktur, ingestasi data, pemrosesan, penyimpanan fitur, siklus hidup model, penyajian model, serta tata kelola dan observabilitas. Metode yang digunakan adalah *Design Science Research Methodology* (DSRM) yang mencakup identifikasi masalah, penyusunan tujuan artefak, perancangan tujuh lapis arsitektur, implementasi berbasis GitOps, serta uji coba dengan *use case* pasar aset kripto sebagai kasus verifikasi karena beroperasi tanpa henti. Arsitektur menempatkan Kubernetes sebagai bidang orkestrasi, Apache Kafka dan Apache Flink untuk *streaming*, Apache Spark untuk pemrosesan *batch*, Feast sebagai *feature store* ganda dengan ClickHouse *offline*, Valkey sebagai *online store* fitur, dan Qdrant sebagai indeks vektor, MLflow dan KubeFlow untuk siklus hidup model, KServe untuk penyajian, serta DataHub untuk katalog dan *lineage*. Deteksi *drift* berbasis *Population Stability Index* dan Kolmogorov-Smirnov memicu pelatihan ulang otomatis melalui Argo CronWorkflow, sementara *point-in-time correctness* dijaga Feast pada fase pelatihan dan penyajian *online*. Argo CD menyatukan GitOps untuk reproduksibilitas *deployment* dari satu sumber Git, sedangkan Prometheus, Grafana, OpenTelemetry, Loki, serta Grafana Tempo melengkapi observabilitas pada tiga lapis: metrik, log, dan *trace*. Hasil evaluasi menunjukkan keempat tujuan platform terpenuhi secara struktural pada lingkungan *node* tunggal, mencakup rekonstruksi seluruh komponen dari satu sumber Git, tata kelola otomatis sepanjang *pipeline*, pemicuan pelatihan ulang oleh deteksi *drift*, dan penyajian fitur *dual-store* dengan kontrak konsisten antara pelatihan dan *inference*. Pengukuran beban kuantitatif penuh seperti latensi p99 dan *throughput* pada kluster multi-*node* menjadi arah pengembangan lanjutan sesuai batasan penelitian.

Kata kunci: DataOps, MLOps, Kubernetes, Feature Store, Drift Detection, GitOps, Open Source.

KATA PENGANTAR

Puji syukur penulis panjatkan ke hadirat Tuhan Yang Maha Esa atas rahmat dan karunia-Nya, penulis dapat menyelesaikan penulisan Laporan Tugas Akhir yang berjudul “Pengembangan Arsitektur Platform DataOps dan MLOps Terintegrasi pada Kubernetes dengan Pemanfaatan *Open Source Tools*”. Laporan ini disusun sebagai salah satu syarat untuk menyelesaikan program Sarjana di Program Studi Sistem dan Teknologi Informasi, Sekolah Teknik Elektro dan Informatika, Institut Teknologi Bandung.

Penulis menyadari bahwa Tugas Akhir ini tidak akan selesai tanpa bantuan dan dukungan dari banyak pihak. Oleh karena itu, penulis menyampaikan terima kasih kepada:

1. Tuhan Yang Maha Esa atas rahmat dan penyertaan-Nya sepanjang proses pengerjaan Tugas Akhir.
2. Bapak Ir. I Gusti Bagus Baskara Nugraha, S.T., M.T., Ph.D. selaku Ketua Program Studi Sistem dan Teknologi Informasi sekaligus dosen wali penulis.
3. Tim Tugas Akhir Program Studi Sistem dan Teknologi Informasi atas koordinasi dan kelancaran selama pelaksanaan Tugas Akhir.
4. Bapak Achmad Imam Kistijantoro, S.T, M.Sc., Ph.D. selaku dosen pembimbing atas arahan, masukan, dan diskusi selama pengerjaan Tugas Akhir.
5. Orang tua penulis, papa dan mama, atas doa dan dukungan yang tidak pernah putus.
6. Pasangan penulis, Tamara, atas kesabaran, dukungan, dan motivasi yang terus menerus selama proses penulisan Tugas Akhir.
7. Teman-teman penulis, yaitu Nangorians (Yusril, Timothy, Ardra, Jojo, Andhika, Natha, Panji, Jebe), S2 DS (Tamara, Yovanka), Lakara (Anin, Lina, Angie, Jendra, Zaki, Tamara, Shazya, Scifo, Dama, Maulvi, Marcell, dan seluruh anggota Ring 2), Vishvanava, Panitia Perayaan Wisokto HMIF ITB 2024 (Hira, Tamara, Attara, Erwan, Andhika, Jeremy, Angie, Juan), Sarenda Adikara, Bermakna (Nahda, Tarisha, SPP, Tamara, Aziz, Fadhil, Arfa, Akram, Sam, dan seluruh anggota Ring 12), GARAKA, dan BYTE atas kebersamaan dan dukungannya.

Penulis menyadari bahwa laporan ini masih memiliki kekurangan. Oleh karena itu, kritik dan saran yang membangun sangat diharapkan untuk perbaikan di masa mendatang. Semoga laporan ini bermanfaat bagi pembaca dan pengembangan platform DataOps dan MLOps di lingkungan akademik maupun industri.

Bandung, 9 Juli 2026
Penulis

Bryan P. Hutagalung

DAFTAR ISI

LEMBAR PENGESAHAN	i
PERNYATAAN ORISINALITAS	ii
PERNYATAAN PENGGUNAAN AI	iii
ABSTRAK	iv
KATA PENGANTAR	v
DAFTAR ISI	vii
DAFTAR GAMBAR	xi
DAFTAR TABEL	xiii
DAFTAR <i>LISTING</i>	xv
DAFTAR SIMBOL	xvii
DAFTAR SINGKATAN	xix
I PENDAHULUAN	1
I.1 Latar Belakang	1
I.2 Rumusan Masalah	2
I.3 Tujuan	3
I.4 Batasan Masalah	5
I.5 Metodologi	6
I.6 Sistematika Penulisan	8
II STUDI LITERATUR	11
II.1 DataOps dan MLOps	11
II.1.1 DataOps	11
II.1.2 MLOps	12
II.1.3 Tingkat Kematangan MLOps dan DataOps	13
II.1.4 Integrasi DataOps dan MLOps	17
II.2 Kubernetes dan Orkestrasi <i>Cloud-Native</i>	17
II.2.1 Peran, Objek Dasar, dan Distribusi	18
II.2.2 Penskalaan Otomatis dan Ekosistem <i>Operator</i>	18
II.3 Manajemen Data: Ingestasi, Pemrosesan, dan Penyimpanan	19
II.3.1 Pola Lambda dan Kappa	19
II.3.2 Apache Kafka	20
II.3.3 Apache Flink	21
II.3.4 Apache Spark dan dbt	22
II.3.5 Format Tabel <i>Lakehouse</i>	22
II.3.6 Penyimpanan Objek dan Versi Data	23
II.3.7 Penyimpanan Relasional dan Pencarian	24
II.4 Layanan Fitur (<i>Feature Store</i>)	24
II.4.1 Konsep dan Peran	24
II.4.2 Penyimpanan <i>Offline</i> dan <i>Online</i>	25
II.4.3 Pencarian Vektor dan HNSW	26
II.4.4 <i>Point-in-Time Correctness</i>	26
II.5 Siklus Hidup Model	27

II.5.1	Eksperimen dan <i>Tracking</i>	27
II.5.2	Orkestrasi Pelatihan	27
II.5.3	Eksperimen Interaktif dan <i>Tuning</i>	28
II.5.4	Penyajian Model	29
II.6	Tata Kelola Data: Katalog, <i>Lineage</i> , dan Kualitas	29
II.7	Deteksi <i>Drift</i>	30
II.8	Observabilitas <i>Cloud-Native</i>	30
II.9	GitOps dan <i>Continuous Delivery</i>	31
II.10	Keamanan Platform: Identitas, Rahasia, <i>Mesh</i> , dan Kebijakan	32
II.10.1	Identitas dan Pengelolaan Rahasia	32
II.10.2	<i>Service Mesh</i> , <i>API Gateway</i> , dan Penegakan Kebijakan	33
II.11	Penelitian Terkait	33
III	ANALISIS MASALAH	35
III.1	Analisis Kondisi Saat Ini	35
III.1.1	Beban Konfigurasi Platform dari Nol	35
III.1.2	Biaya Berlangganan Layanan Terkelola	36
III.1.3	Efek Domino Fragmentasi Lintas Peran	37
III.2	Analisis Kebutuhan	39
III.2.1	Identifikasi Masalah Pengguna	39
III.2.2	Kebutuhan Fungsional	40
III.2.3	Kebutuhan Nonfungsional	43
III.3	Analisis Pemilihan Solusi	47
III.3.1	Alternatif Solusi	47
III.3.2	Analisis Penentuan Solusi	49
III.3.3	Posisi terhadap Penelitian Terkait	51
III.4	Analisis Pemilihan Open Source Tools	57
III.4.1	Lapis Orkestrasi dan Infrastruktur	58
III.4.2	Lapis Ingestasi dan Pemrosesan Data	60
III.4.3	Lapis Penyimpanan Data	62
III.4.4	Lapis Layanan Fitur	64
III.4.5	Lapis Siklus Hidup Model	66
III.4.6	Lapis Tata Kelola Data	68
III.4.7	Lapis Observabilitas	69
III.4.8	Lapis GitOps dan <i>Continuous Delivery</i>	71
III.4.9	Lapis Keamanan Platform	72
IV	PERANCANGAN ARSITEKTUR PLATFORM DATAOPS DAN MLOPS	77
IV.1	Gambaran Umum Platform	77
IV.2	Perancangan Arsitektur Terintegrasi	80
IV.2.1	Lapisan Infrastruktur dan Bidang Kendali	80
IV.2.2	Lapisan Ingestasi, Pemrosesan, dan Penyimpanan Fitur	81
IV.2.3	Lapisan Siklus Hidup Model dan Penyajian	84
IV.2.4	Lapisan Tata Kelola dan Observabilitas	86
IV.3	Perancangan Sub-sistem Tata Kelola Data	86
IV.3.1	Katalog Metadata dan <i>Lineage</i>	88

IV.3.2	Kontrak Data dan Kualitas	88
IV.3.3	Identitas dan Kontrol Akses	89
IV.4	Perancangan Sub-sistem Deteksi <i>Drift</i> dan <i>Continuous Training</i>	89
IV.4.1	Pengukuran <i>Drift</i>	90
IV.4.2	Alur Pelatihan Ulang Otomatis	90
IV.4.3	Mekanisme Penanganan Kasus Khusus	91
IV.5	Perancangan Sub-sistem Layanan Fitur <i>Dual-Store</i>	92
IV.5.1	Penyimpanan <i>Offline</i> dan <i>Online</i>	92
IV.5.2	Dukungan <i>Vector Embeddings</i> dan HNSW	93
IV.5.3	Jaminan <i>Point-in-Time Correctness</i>	93
IV.5.4	Aliran Materialisasi Fitur	94
IV.6	Alur Kerja <i>End-to-End</i>	94
V	IMPLEMENTASI ARSITEKTUR PLATFORM DATAOPS DAN MLOPS	97
V.1	Lingkungan Implementasi	97
V.1.1	Konfigurasi Dasar <i>Cluster</i>	98
V.1.2	Manajemen Rahasia dan Identitas	98
V.1.3	Pola GitOps	99
V.2	Implementasi Arsitektur Terintegrasi	100
V.2.1	Lapisan Ingestasi	100
V.2.2	Lapisan Pemrosesan	100
V.2.3	Lapisan Penyimpanan	101
V.2.4	Lapisan Siklus Hidup Model dan Penyajian	101
V.2.5	Lapisan Observabilitas	102
V.3	Implementasi Sub-sistem Tata Kelola Data	103
V.3.1	Katalog Metadata dan <i>Lineage</i>	103
V.3.2	Kontrak Data dan Kualitas	104
V.3.3	Kontrol Akses	104
V.4	Implementasi Sub-sistem Deteksi <i>Drift</i> dan <i>Continuous Training</i>	105
V.4.1	Detektor <i>Drift</i>	105
V.4.2	<i>Control Loop</i> Pelatihan Ulang	106
V.4.3	Penanganan Kasus Khusus	106
V.5	Implementasi Sub-sistem Layanan Fitur <i>Dual-Store</i>	107
V.5.1	Konfigurasi Feast	107
V.5.2	Dukungan <i>Vector Embeddings</i>	108
V.5.3	Materialisasi dan <i>Point-in-Time Correctness</i>	108
V.6	Verifikasi Implementasi	109
V.6.1	Lingkup Verifikasi	109
V.6.2	Pengamatan Hasil Awal	110
VI	EVALUASI	113
VI.1	Metode Evaluasi	113
VI.1.1	Kasus Verifikasi dan Lingkungan Uji	114
VI.1.2	Evaluasi Tujuan T-1: Arsitektur Terintegrasi dan GitOps	116
VI.1.3	Evaluasi Tujuan T-2: Tata Kelola Data	117
VI.1.4	Evaluasi Tujuan T-3: Deteksi <i>Drift</i> dan Pelatihan Ulang	117

VI.1.5 Evaluasi Tujuan T-4: Layanan Fitur <i>Dual-Store</i>	118
VI.1.6 Matriks Cakupan dan Uji Penerimaan	119
VI.2 Hasil Evaluasi	122
VI.3 Perbandingan Biaya Infrastruktur dan Tenaga Kerja	127
VI.4 Pembahasan Hasil Evaluasi	131
VII PENUTUP	133
VII.1 Kesimpulan	133
VII.2 Saran	134
Lampiran A KATALOG PERINTAH VERIFIKASI USE CASE	153
A.1 Tujuan T-1: Arsitektur Terintegrasi dan GitOps	153
A.2 Tujuan T-2: Tata Kelola Data	155
A.3 Tujuan T-3: Deteksi <i>Drift</i> dan Pelatihan Ulang	155
A.4 Tujuan T-4: Layanan Fitur <i>Dual-Store</i>	156
Lampiran B PARAMETER PENILAIAN KRITERIA EVALUASI AL-TERNATIF	159
B.1 Kriteria Lintas Tabel	159
B.2 Kriteria Spesifik per Lapis	160

DAFTAR GAMBAR

II.1	MLOps <i>level</i> 0: proses manual	13
II.2	MLOps <i>level</i> 1: otomasi <i>pipeline</i> pelatihan	14
II.3	MLOps <i>level</i> 2: orkestrasi CI/CD/CT	14
III.1	Gambaran Umum Fragmentasi DataOps dan MLOps	38
IV.1	Gambaran Umum Platform DataOps dan MLOps Terintegrasi	78
IV.2	Rincian Lapisan Infrastruktur dan Bidang Kendali	82
IV.3	Rincian Lapisan Ingestasi, Pemrosesan, dan Penyimpanan Fitur . . .	83
IV.4	Rincian Lapisan Siklus Hidup Model dan Penyajian	85
IV.5	Rincian Lapisan Tata Kelola dan Observabilitas	87
VI.1	Proyeksi Biaya Kumulatif Tiga Tahun Tiga Skenario	130

DAFTAR TABEL

II.1 Perbandingan <i>Framework</i> Tingkat Kematangan MLOps	16
III.1 Kebutuhan Fungsional Sistem	41
III.2 Kebutuhan Nonfungsional Sistem	44
III.3 Evaluasi Pemenuhan Kebutuhan Fungsional	50
III.4 Evaluasi Pemenuhan Kebutuhan Nonfungsional	50
III.5 Total Skor Pemenuhan Kebutuhan Fungsional dan Nonfungsional	50
III.6 Perbandingan Detail Arsitektur Saat Ini dengan yang Diusulkan	51
III.7 Evaluasi Alternatif Distribusi Kubernetes <i>Open-Source</i>	59
III.8 Evaluasi Alternatif <i>Message Broker</i> untuk Tulang Punggung <i>Streaming</i>	60
III.9 Evaluasi Alternatif <i>Schema Registry</i> untuk Apache Kafka	61
III.10 Evaluasi Alternatif <i>Framework</i> Pemrosesan Data	61
III.11 Evaluasi Alternatif Format Tabel <i>Lakehouse Open-Source</i>	62
III.12 Evaluasi Alternatif Penyimpanan Objek S3-Kompatibel <i>Open-Source</i>	62
III.13 Evaluasi Alternatif Pengelola Versi Data <i>Open-Source</i> pada Lapis Penyimpanan Objek	63
III.14 Evaluasi Alternatif <i>Feature Store Open-Source</i>	64
III.15 Evaluasi Alternatif Penyimpanan <i>Offline</i> (OLAP)	65
III.16 Evaluasi Alternatif Penyimpanan <i>Online</i> (Latensi Rendah)	65
III.17 Evaluasi Alternatif <i>Vector Index Open-Source</i> untuk Pencarian Pendekatan Tetangga Terdekat	65
III.18 Evaluasi Alternatif <i>Experiment Tracking</i> dan <i>Model Registry</i>	66
III.19 Evaluasi Alternatif Orkestrasi ML <i>Open-Source</i>	67
III.20 Evaluasi Alternatif <i>Model Serving</i>	67
III.21 Evaluasi Alternatif Manajemen Metadata dan Tata Kelola	68
III.22 Evaluasi Alternatif <i>Monitoring</i> dan <i>Observability</i>	69
III.23 Evaluasi Alternatif <i>Business Intelligence</i> dan <i>Dashboarding Open-Source</i>	70
III.24 Evaluasi Alternatif <i>GitOps Controller Open-Source</i>	71
III.25 Evaluasi Alternatif <i>Identity Provider Open-Source</i>	72
III.26 Evaluasi Alternatif Pengelola Rahasia	72
III.27 Evaluasi Alternatif <i>Service Mesh</i>	73
III.28 Evaluasi Alternatif <i>API Gateway Open-Source</i>	73

III.29	Evaluasi Alternatif <i>Policy Engine Open-Source</i>	74
III.30	Evaluasi Alternatif <i>Runtime Security Open-Source</i> pada Kubernetes	74
IV.1	Pemetaan Peran Pengguna ke Antarmuka dan <i>Tool Platform</i>	79
V.1	Struktur Direktori Utama Proyek dan Fungsinya	98
VI.1	Skenario Uji Penerimaan per Tujuan Platform	119
VI.2	Target Metrik, Instrumen Ukur, dan Status Evaluasi per Kebutuhan	124
VI.3	Perbandingan Biaya Bulanan Infrastruktur per 16 Juni 2026	128
VI.4	Ilustrasi Biaya Infrastruktur dan Tenaga Kerja Tiga Tahun per 16 Juni 2026	129
B.1	Parameter Penilaian Kriteria Lintas Tabel	159
B.2	Parameter Penilaian Kriteria Spesifik per Lapis	160

DAFTAR LISTING

VI.1	Kutipan <i>overlay</i> Kustomize <i>use case</i> : prefiks <i>crypto-</i> , pemetaan registri <i>image</i> , <i>replica idle</i> satu, dan batas HPA <i>node</i> tunggal	115
A.1	Langkah persiapan pemuatan konfigurasi domain dan <i>namespace</i> pembangkit beban (dari ~/documents/ta/)	153
A.2	Perintah verifikasi tujuan T-1 (dari ~/documents/ta/)	153
A.3	Perintah verifikasi tujuan T-2 (dari ~/documents/ta/)	155
A.4	Perintah verifikasi tujuan T-3 (dari ~/documents/ta/)	155
A.5	Perintah verifikasi tujuan T-4 (dari ~/documents/ta/)	156

DAFTAR SIMBOL

Notasi	Deskripsi	Pemakaian pertama kali
p99	Persentil ke-99 dari distribusi latensi permintaan, dipakai sebagai indikator <i>tail latency</i> pada SLO layanan <i>inference</i> .	Bab IV

DAFTAR SINGKATAN

Singkatan	Deskripsi	Pemakaian pertama kali
ACID	<i>Atomicity, Consistency, Isolation, Durability</i> , sifat transaksi basis data.	Bab II
AGPL	<i>GNU Affero General Public License</i> , lisensi <i>copyleft</i> kuat yang juga mencakup penggunaan melalui jaringan.	Bab III
ANN	<i>Approximate Nearest Neighbor</i> , kelas algoritma pencarian tetangga terdekat secara aproksimasi pada ruang vektor.	Bab II
API	<i>Application Programming Interface</i> .	Bab I
APISIX	Apache APISIX, <i>API gateway</i> berbasis Nginx dan etcd.	Bab II
ASF	<i>Apache Software Foundation</i> , yayasan nirlaba yang menaungi proyek <i>open source</i> Apache.	Bab III
BSD	<i>Berkeley Software Distribution</i> , keluarga lisensi permisif perangkat lunak.	Bab III
BSL	<i>Business Source License</i> , lisensi <i>source-available</i> dengan pembatasan pemakaian komersial.	Bab III
BUSL	Penulisan SPDX untuk <i>Business Source License</i> versi 1.1.	Bab III
CD	<i>Continuous Delivery</i> atau <i>Continuous Deployment</i> .	Bab I
CDC	<i>Change Data Capture</i> , teknik penangkapan perubahan data pada basis data sumber.	Bab II
CI	<i>Continuous Integration</i> .	Bab II
CNCF	<i>Cloud Native Computing Foundation</i> , yayasan yang menaungi proyek <i>cloud native</i> di bawah Linux Foundation.	Bab II
CNPG	<i>CloudNativePG</i> , operator Kubernetes untuk PostgreSQL.	Bab IV
CPU	<i>Central Processing Unit</i> .	Bab II
CR	<i>Custom Resource</i> , objek konkret yang dibuat dari sebuah CRD pada Kubernetes.	Bab V

CRD	<i>Custom Resource Definition</i> , definisi tipe sumber daya kustom pada Kubernetes.	Bab II
CT	<i>Continuous Training</i> , pelatihan model secara berkelanjutan dengan pemicu <i>drift</i> atau jadwal.	Bab II
DAG	<i>Directed Acyclic Graph</i> , graf berarah tanpa siklus untuk menyusun urutan tugas pada <i>orchestrator</i> .	Bab IV
DSRM	<i>Design Science Research Methodology</i> , kerangka metodologi penelitian rekayasa artefak.	Bab I
eBPF	<i>extended Berkeley Packet Filter</i> , mekanisme menjalankan program tervalidasi di dalam kernel Linux.	Bab II
ECB	<i>European Central Bank</i> , sumber kurs referensi EUR ke USD.	Bab VI
ESO	<i>External Secrets Operator</i> , operator Kubernetes penyinkron rahasia dari pengelola rahasia eksternal.	Bab V
FTE	<i>Full-Time Equivalent</i> , satuan alokasi penuh satu tenaga kerja per bulan.	Bab VI
GMS	<i>Generalized Metadata Service</i> , komponen layanan metadata inti pada DataHub.	Bab V
GPU	<i>Graphics Processing Unit</i> .	Bab II
gRPC	<i>gRPC Remote Procedure Call</i> , kerangka pemanggilan prosedur jarak jauh berbasis HTTP/2.	Bab III
HNSW	<i>Hierarchical Navigable Small World</i> , struktur graf untuk ANN.	Bab II
HPA	<i>Horizontal Pod Autoscaler</i> , kontrol penskalaan jumlah <i>pod</i> pada Kubernetes.	Bab I
HTML	<i>HyperText Markup Language</i> .	Bab IV
JISDOR	<i>Jakarta Interbank Spot Dollar Rate</i> , kurs referensi Bank Indonesia untuk USD ke IDR.	Bab VI
JSON	<i>JavaScript Object Notation</i> .	Bab II
KEDA	<i>Kubernetes Event-driven Autoscaling</i> , <i>autoscaler</i> berbasis sinyal eksternal.	Bab I

KES	<i>Key Encryption Service</i> , layanan kunci enkripsi sisi <i>server</i> untuk MinIO.	Bab II
KMS	<i>Key Management Service</i> , layanan pengelolaan kunci enkripsi.	Bab II
KS	<i>Kolmogorov-Smirnov test</i> , uji statistik kesetaraan dua distribusi.	Bab II
LF	<i>Linux Foundation</i> , yayasan nirlaba yang menaungi ekosistem proyek <i>open source</i> .	Bab III
MLOps	<i>Machine Learning Operations</i> , disiplin praktik operasional pengembangan dan penyajian model <i>machine learning</i> .	Bab I
MPL	<i>Mozilla Public License</i> , lisensi <i>copyleft</i> lemah berbasis berkas.	Bab III
mTLS	<i>mutual Transport Layer Security</i> , autentikasi TLS dua arah antar layanan.	Bab II
OIDC	<i>OpenID Connect</i> , protokol identitas berbasis OAuth 2.0.	Bab II
OLAP	<i>Online Analytical Processing</i> , pemrosesan kueri analitis pada data bervolume besar.	Bab III
ONNX	<i>Open Neural Network Exchange</i> , format pertukaran model lintas <i>framework</i> .	Bab IV
PSI	<i>Population Stability Index</i> , metrik pergeseran distribusi fitur antar dua periode.	Bab II
REST	<i>Representational State Transfer</i> .	Bab II
RPO	<i>Recovery Point Objective</i> , batas maksimum kehilangan data yang dapat ditoleransi saat pemulihan.	Bab VI
RTO	<i>Recovery Time Objective</i> , batas maksimum waktu pemulihan layanan setelah gangguan.	Bab VI
SDK	<i>Software Development Kit</i> .	Bab III
SLO	<i>Service Level Objective</i> .	Bab II
SQL	<i>Structured Query Language</i> .	Bab II
SSO	<i>Single Sign-On</i> , autentikasi tunggal untuk banyak aplikasi.	Bab VI
TLS	<i>Transport Layer Security</i> .	Bab I
VPA	<i>Vertical Pod Autoscaler</i> , kontrol penyetelan <i>request/limit</i> sumber daya <i>pod</i> .	Bab I

YAML

YAML Ain't Markup Language, format serialisasi data untuk berkas konfigurasi.

Bab II

BAB I

PENDAHULUAN

Bab ini meletakkan dasar bagi keseluruhan laporan Tugas Akhir. Pembahasan dibuka dengan latar belakang yang memetakan tekanan praktik DataOps dan MLOps pada lingkungan produksi, kemudian diturunkan menjadi empat rumusan masalah dan empat tujuan yang saling berpasangan. Batasan masalah menetapkan lingkup pengerjaan, metodologi menjelaskan kerangka *Design Science Research Methodology* yang diikuti, dan sistematika penulisan menutup bab dengan peta isi laporan. Penomoran RM-1 hingga RM-4 dan T-1 hingga T-4, serta batasan penelitian BP-1 hingga BP-3 dan batasan implementasi BI-1 hingga BI-2 yang diperkenalkan pada bab ini menjadi rujukan tetap bagi seluruh bab berikutnya.

I.1 Latar Belakang

Keberhasilan *machine learning* pada lingkungan produksi ditentukan oleh kemampuan memindahkan model dari eksperimen ke layanan yang stabil, terpantau, dan dapat diperbarui, bukan oleh ketepatan model semata. Sculley dkk. (2015) menunjukkan kode model hanya sebagian kecil dari sistem nyata, sedangkan sisanya berupa pengelolaan data, konfigurasi, pemantauan, penyajian, dan *lineage*. Paleyes, Urma, dan Lawrence (2022) memetakan tantangan *deployment* dari kualitas data hingga pemeliharaan model, sementara adaptasi DevOps oleh komunitas *machine learning* melahirkan MLOps (Kreuzberger, Köhl, dan Hirschl 2023) dan DataOps tumbuh serupa untuk *pipeline* data. Rella (2022) mencatat keduanya berbagi prinsip otomasi yang sama tetapi kerap berjalan sebagai dua *tech stack* terpisah, sedangkan Jain dan Das (2025) menempatkan integrasinya sebagai syarat *pipeline* yang skalabel dan tangguh. Upaya integrasi tersebut dihadapkan pada dua tekanan utama yang, ketika tidak teratasi, memicu tekanan ketiga berupa efek domino.

1. Beban membangun platform dari nol di atas kumpulan komponen yang terfragmentasi

Kajian sistematis Amou Najafabadi dkk. (2024) atas 43 studi primer mengategorikan 35 komponen arsitektur MLOps beserta ragam *tool* pendukungnya, menandai luasnya ruang pilihan sebelum satu platform utuh terbentuk. Burgueño-Romero dkk. (2025) menambahkan bahwa komponen *open source* MLOps cenderung berdiri sendiri alih-alih membentuk kerangka kohesif,

sehingga beban integrasi, kurva belajar, dan penyelarasan versi ditanggung tim pemakai sejak awal.

2. **Biaya berlangganan layanan terkelola dan *trade-off* antara waktu pengembangan dengan anggaran berjalan**

Layanan terkelola mempercepat adopsi (Li dkk. 2023), tetapi studi empiris Hamza, Akbar, dan Capilla (2024) menemukan bahwa penagihan bayar per pakai dan pelaporan biaya *cloud* yang tidak transparan menyulitkan estimasi anggaran dan dapat menjadi tidak hemat pada beban besar, sejalan dengan Adusumilli (2025). Ditambah lagi dengan risiko keterikatan penyedia (Mo dkk. 2023), organisasi terjebak antara membangun tim platform dengan waktu pengembangan panjang atau berlangganan dengan biaya sulit diestimasi dan ruang kustomisasi yang sempit.

3. **Efek domino fragmentasi yang merambat ke seluruh rantai DataOps dan MLOps**

Ketika kedua tekanan tidak terjawab oleh satu bidang kendali, sambungan yang terputus antar lapis ingestasi, fitur, pelatihan, dan penyajian memaksa integrasi dirakit ulang tiap proyek, sementara metadata, identitas, dan *lineage* dikelola sendiri-sendiri sehingga asal data sulit ditelusuri dan nilai fitur rawan berbeda antara pelatihan dan penyajian. Shankar dkk. (2022) mendokumentasikan lewat wawancara delapan belas insinyur bahwa pemeliharaan *pipeline* dan integrasi antar komponen menjadi keluhan berulang, dengan rincian per lapisan pada Bab III.

Ketiga tekanan tersebut menggerakkan pengembangan platform yang menyatukan DataOps dan MLOps di atas Kubernetes dengan komponen *open source*. Kubernetes dipilih sebagai standar *de facto* orkestrasi *cloud-native* yang teradopsi luas di lingkungan *enterprise* (Lakka 2025), sedangkan komponen *open source* membuat arsitektur dapat ditiru tanpa keterikatan penyedia sekaligus menghapus biaya lisensi. Rancangan dijaga *domain-agnostic* agar dapat diadaptasi lintas domain tanpa mengubah lapis kendali, dengan keluaran empat *sub-sistem* pemutus rantai fragmentasi. Seluruh komponen ditata pada Gitea dengan Argo CD sebagai mesin rekonsiliasi deklaratif, sehingga tujuh lapis arsitektur, dari ingestasi hingga tata kelola, terhubung pada satu jalur GitOps yang dirinci pada Bab IV.

I.2 **Rumusan Masalah**

Latar belakang di atas mengarah pada empat rumusan masalah yang ingin diselesaikan oleh platform yang dikembangkan. Keempat butir dirumuskan sebagai perta-

nyaan agar setiap permasalahan memiliki batas teknis yang jelas dan rute jawaban berupa *sub-sistem* atau kontrak antar komponen pada bab-bab berikutnya. Setiap pertanyaan menysasar properti sistem yang belum terpenuhi, bukan ketiadaan satu *tool* tertentu, agar jawabannya berlaku lintas pilihan komponen dan menjaga sifat *domain-agnostic* platform. Keempat butir di bawah ini selanjutnya dirujuk dengan singkatan RM-1 hingga RM-4 pada bab-bab berikutnya.

1. Bagaimana menyatukan *tech stack* DataOps dan MLOps yang terfragmentasi pada lapis ingestasi, pemrosesan, penyimpanan, pelatihan, dan penyajian model ke dalam satu bidang kendali pada Kubernetes?
2. Bagaimana mewujudkan tata kelola data yang konsisten lintas siklus data, fitur, dan model sehingga katalog metadata, *lineage*, dan kontrol kualitas tidak berjalan sebagai pekerjaan susulan?
3. Bagaimana mengotomasi deteksi *drift* dan menjamin *point-in-time correctness* agar tidak terjadi kebocoran temporal antara mode *batch* dan *streaming*?
4. Bagaimana menyajikan fitur multimodal, yang mencakup fitur tabular, agregat, dan *embedding* vektor berdimensi tinggi, dari satu sumber kebenaran dengan nilai yang konsisten antara fase pelatihan dan penyajian?

Keempat rumusan masalah di atas dijawab secara berurutan oleh empat tujuan pada Subbab I.3, dengan rincian kebutuhan fungsional dan nonfungsional diturunkan pada Bab III. Keempat rumusan tidak saling tumpang tindih karena masing-masing menysasar satu titik kegagalan yang berbeda, yaitu perakitan lintas lapis, tata kelola data, deteksi *drift* beserta pencegahan kebocoran temporal, dan penyajian fitur multimodal, sekaligus bersama-sama mencakup rentang dari siklus data hingga penyajian model. Karena dirumuskan dengan batas teknis yang jelas dan berpijak pada kegagalan yang terdokumentasi pada latar belakang, keempat rumusan menjadi acuan tetap yang jawabannya ditunjukkan melalui artefak yang berjalan. Pemenuhan tiap rumusan ditelusuri kembali pada Bab VI melalui skenario uji yang menautkan rumusan dengan bukti verifikasinya.

I.3 Tujuan

Penelitian ini merumuskan empat tujuan yang masing-masing menjawab satu rumusan masalah, dengan keluaran berupa artefak teknis yang dapat ditunjukkan dan dipakai ulang. Setiap tujuan menetapkan keluaran berupa rancangan arsitektur, manifest *deployment* pada repositori Gitea, dan kontrak antar komponen yang dapat diuji pada *use case* verifikasi. Keempat tujuan di bawah ini selanjutnya dirujuk dengan singkatan T-1 hingga T-4 pada bab-bab berikutnya, dengan pemetaan satu lawan satu

terhadap RM-1 hingga RM-4. Tiap tujuan dirumuskan sebagai tindakan merancang dan mewujudkan artefak yang dapat ditunjukkan, bukan sebagai aktivitas mengukur atau menguji.

1. Merancang dan mewujudkan arsitektur platform DataOps dan MLOps terintegrasi di atas Kubernetes yang menyatukan ingestasi, pemrosesan, penyimpanan fitur, pelatihan, penyajian model, observabilitas, dan tata kelola pada satu bidang kendali deklaratif dengan praktik GitOps, menjawab RM-1.
2. Membangun *sub-sistem* tata kelola data yang menyediakan katalog metadata, perekaman *lineage*, dan kontrol kualitas data sebagai layanan bersama yang dijalankan secara otomatis di seluruh *pipeline* data, fitur, dan model, menjawab RM-2.
3. Mengimplementasikan *sub-sistem* deteksi *drift* terotomasi dengan kebijakan pelatihan ulang berbasis ambang batas serta menjamin *point-in-time correctness* pada penyajian fitur lintas mode *batch* dan *streaming*, menjawab RM-3.
4. Mengembangkan layanan fitur *dual-store* yang melayani fitur tabular dan agregat melalui satu kontrak API dengan jaminan konsistensi nilai antara fase pelatihan dan penyajian, serta melayani *embedding* vektor berdimensi tinggi pada indeks vektor terpisah dengan ruang *embedding* yang konsisten pada kedua fase tersebut, menjawab RM-4.

Kriteria keberhasilan keseluruhan platform diukur dari empat sudut yang masing-masing menautkan satu tujuan. Setiap kriteria dirumuskan agar dapat diperiksa pada artefak yang berjalan, bukan pada rancangan di atas kertas. Pemeriksaannya dilakukan melalui skenario uji pada Bab VI dengan penerimaan fungsional dan struktural sesuai lingkup batasan penelitian. Keempat kriteria berikut menjadi tolok ukur minimum sebelum platform dinyatakan menjawab rumusan masalah.

1. Seluruh artefak dapat dipasang dari satu repositori Git melalui Argo CD tanpa intervensi manual setelah *bootstrap* awal klaster (T-1).
2. Seluruh *pipeline* data, fitur, dan model terdaftar pada katalog metadata dengan *lineage* yang dapat ditelusuri (T-2).
3. Mekanisme deteksi *drift* memicu pelatihan ulang secara otomatis ketika ambang batas dilewati pada *use case* verifikasi (T-3).
4. Fitur tabular, agregat, dan *embedding* tersaji melalui satu kontrak API dengan nilai yang konsisten antara penyimpanan *offline* dan *online* (T-4).

I.4 Batasan Masalah

Pengembangan platform pada Tugas Akhir ini dibatasi pada dua tingkatan yang perlu dibedakan agar penilaian terarah. Batasan penelitian menetapkan ruang lingkup yang sengaja tidak dicakup oleh kontribusi, sedangkan batasan implementasi menyangkut pilihan perwujudan yang dapat diganti tanpa mengubah kontribusi selama fungsi yang sama tetap terpenuhi. Perbedaan ini penting karena pilihan seperti jumlah *node*, patokan versi, atau sintaks komponen tertentu bukan bagian dari kontribusi rancangan, melainkan detail perwujudan pada lingkungan tertentu. Batasan penelitian dirujuk dengan singkatan BP-1 hingga BP-3 dan batasan implementasi dengan BI-1 hingga BI-2 pada bab-bab berikutnya.

Batasan penelitian berikut menetapkan lingkup yang tidak dicakup oleh kontribusi rancangan. Ketiganya membatasi klaim pada lapis kendali platform, cakupan keamanan antar layanan, dan lingkup evaluasi. Batas ini menjaga penilaian tetap tertuju pada kontribusi arsitektur, bukan pada pengembangan model atau karakterisasi beban penuh. Konsekuensi tiap batasan pada evaluasi dibahas kembali pada Bab VI.

BP-1 Fokus pada lapis kendali platform

Fokus berada pada lapis kendali platform, bukan pengembangan model spesifik. *Use case* domain hanya dipakai sebagai kasus verifikasi rancangan sehingga pembahasan platform dijaga *domain-agnostic*.

BP-2 Cakupan keamanan pada akses antar layanan internal

Aspek keamanan berfokus pada akses antar layanan internal melalui *mutual TLS* Istio, manajemen rahasia OpenBao, dan kebijakan jaringan Kubernetes. Autentikasi pengguna akhir pada lapisan aplikasi berada di luar cakupan penelitian.

BP-3 Lingkup evaluasi pada penerimaan fungsional dan struktural

Evaluasi difokuskan pada penerimaan fungsional dan struktural, sedangkan karakterisasi beban kuantitatif penuh seperti latensi p99, *throughput* puncak, dan skalabilitas horizontal ditempatkan sebagai arah pengembangan lanjutan. Evaluasi tidak mencakup *deployment* produksi pada lingkungan *multi-tenant* dengan trafik publik.

Batasan implementasi berikut menyangkut pilihan perwujudan yang dapat diganti tanpa mengubah sifat kontribusi. Keduanya menetapkan lingkungan *k3s* satu *node* dan patokan versi April 2026 sebagai kondisi pengujian yang dapat direproduksi. Penggantian jumlah *node*, versi komponen, atau sintaks alat dengan padanan berka-

pabilitas setara tidak mengubah rancangan. Kedua batasan ini dipetakan kembali pada pembahasan hasil di Bab VI.

BI-1 Lingkungan Kubernetes *k3s* satu *node*

Platform berjalan di atas *k3s* pada satu *node* sebagai lingkungan pengembangan dan pengujian, dengan skala diatur oleh HPA, VPA, dan KEDA ketika beban nyata muncul. *Overlay* Kustomize untuk *environment* multi-*node* tetap dapat ditambahkan tanpa modifikasi basis kode, sehingga jumlah *node* bersifat perwujudan dan bukan bagian dari kontribusi.

BI-2 Komponen *open source* dengan patokan versi April 2026

Versi pustaka dan basis *image container* dipatok pada rilis stabil tertinggi per April 2026 agar pengujian dapat direproduksi pada lingkungan yang sama. Patokan versi maupun pilihan serta sintaks komponen tertentu dapat diganti dengan komponen lain berkapabilitas setara tanpa mengubah rancangan.

Batasan di atas menjadi konteks pembacaan seluruh laporan, terutama pada Bab III dan Bab VI. Batasan penelitian menjaga fokus kontribusi pada lapis kendali yang *domain-agnostic*, sedangkan batasan implementasi menandai keputusan perwujudan yang terbuka untuk diperluas ke lingkungan multi-*node* dan multi-domain. Asumsi *open source* dan patokan versi memberi peluang verifikasi arsitektur tanpa kontak dengan penyedia layanan berbayar. Perbedaan kedua tingkatan ini menegaskan bahwa keterbatasan *node* tunggal dan patokan versi tidak membatasi keberlakuan rancangan.

I.5 Metodologi

Penelitian ini mengikuti kerangka *Design Science Research Methodology* (DSRM) yang dirumuskan oleh Peffers dkk. (2007) untuk penelitian sistem informasi yang menghasilkan artefak. Keluaran utama Tugas Akhir ini berupa platform yang berjalan, yaitu artefak jenis instansiasi (*instantiation*) dalam taksonomi *design science*, sehingga kerangka penelitian berorientasi artefak menjadi pilihan yang tepat (Hevner dkk. 2004). DSRM dipilih dibandingkan metodologi rekayasa perangkat lunak murni karena DSRM menempatkan iterasi antara perancangan, demonstrasi, dan evaluasi sebagai bagian inti proses penelitian, bukan sekadar tahapan pengembangan. Kerangka proses *data science* seperti CRISP-DM dan TDSP tidak menjadi pilihan karena keduanya menata pembangunan satu model pada satu domain data, sedangkan keluaran Tugas Akhir ini adalah platform lintas domain dan model hanya menjadi salah satu beban kerja yang dilayani. Tahapan yang dijalankan meliputi enam fase berikut dengan keluaran teknis yang dapat diukur.

1. **Identifikasi masalah dan motivasi**

Studi literatur DataOps, MLOps, dan tata kelola data dilakukan untuk menyusun peta tantangan. Tiga pemicu dan empat rumusan masalah (RM-1 sampai RM-4) menjadi keluaran fase ini.

2. **Penetapan tujuan artefak**

Tujuan dirumuskan sebagai daftar artefak teknis yang dapat dipasang, dioperasikan, dan diaudit, dipetakan satu lawan satu terhadap rumusan masalah. Kebutuhan fungsional KF-01 sampai KF-14 dan kebutuhan nonfungsional KNF-01 sampai KNF-12 disusun pada Bab III.

3. **Perancangan dan pengembangan artefak**

Arsitektur tujuh lapis dirancang sebagai bidang kendali deklaratif berbasis Kubernetes, bertolak dari alternatif arsitektur terpilih hasil matriks evaluasi multi-kriteria pada Bab III. Pengembangan manifes dilakukan modular pada repositori Gitea dengan *folder* per komponen dan *overlay* Kustomize.

4. **Demonstrasi**

Platform dipasang pada lingkungan *k3s* dan diberi beban kerja dari satu *use case* domain nyata sebagai kasus verifikasi, yang dirinci pada Bab VI. Demonstrasi diatur sebagai empat tahap iteratif, dengan setiap tahap memvalidasi satu *sub-sistem* sesuai T-1 hingga T-4.

5. **Evaluasi**

Evaluasi dilaksanakan pada akhir siklus implementasi dengan tiga dimensi: fungsional, nonfungsional (skalabilitas, ketersediaan, observabilitas, keamanan, biaya), serta tata kelola (*lineage* tertelusur, kontrak data ditegakkan, kebijakan akses dapat diaudit). Angka kuantitatif evaluasi dilaporkan pada Bab VI.

6. **Komunikasi**

Hasil rancangan dan implementasi didokumentasikan pada laporan ini, beserta repositori Gitea yang menjadi acuan reproduksi. Keluaran fase ini berupa laporan, repositori sumber, dan presentasi ujian akhir.

Keenam fase di atas dijalankan secara iteratif, dengan kemungkinan kembali ke fase sebelumnya ketika temuan baru muncul pada fase berikutnya. Iterasi tersebut sejalan dengan karakter DSRM sebagai metodologi berorientasi artefak yang menempatkan umpan balik dari demonstrasi dan evaluasi sebagai pemicu perbaikan rancangan. Pola pengembangan serupa, yaitu membangun platform atau *pipeline* MLOps berbasis Kubernetes lalu menilainya melalui demonstrasi dan eksperimen, dipakai oleh Hsu dkk. (2024) dan Rodrigues dkk. (2025). Pelaporan tiap fase di-

susun pada bab yang sesuai sehingga keterhubungan antara fase metodologi dan struktur laporan tetap eksplisit.

I.6 Sistematika Penulisan

Laporan Tugas Akhir disusun atas tujuh bab dengan urutan yang mengikuti DSRM dan praktik pelaporan rekayasa perangkat lunak. Pemetaan antara rumusan masalah, tujuan, *sub-sistem*, dan butir kesimpulan dijaga konsisten lewat penomoran RM-N, T-N, dan butir pada Bab IV hingga Bab VII. Bab IV dan Bab V membentuk pasangan perancangan dan implementasi yang dibaca berurutan, sedangkan Bab VI dan Bab VII menutup siklus dengan evaluasi serta kesimpulan dan saran. Urutan bab beserta isi utamanya dirinci pada poin-poin berikut.

1. Bab I Pendahuluan

Memuat latar belakang, rumusan masalah, tujuan, batasan masalah, metodologi penelitian, serta sistematika penulisan.

2. Bab II Studi Literatur

Meninjau konsep DataOps dan MLOps, Kubernetes, manajemen data, layanan fitur, siklus hidup model, tata kelola data, deteksi *drift*, observabilitas, praktik GitOps, dan keamanan platform sebagai landasan teoretis, lalu mengidentifikasi kesenjangan pada penelitian terkait.

3. Bab III Analisis Masalah

Memetakan kondisi sistem dan menyusun kebutuhan fungsional KF-01 hingga KF-14 serta kebutuhan nonfungsional KNF-01 hingga KNF-12 sebagai turunan empat tujuan. Empat alternatif arsitektur dibandingkan melalui matriks evaluasi multi-kriteria, lalu posisi platform terhadap penelitian terkait dipetakan berdasarkan kebutuhan yang sama.

4. Bab IV Perancangan Arsitektur Platform DataOps dan MLOps

Menjabarkan arsitektur tujuh lapis dan perancangan empat *sub-sistem* yang menjawab T-1 hingga T-4. Diagram arsitektur disajikan dalam gambaran umum beserta rincian per kelompok lapisan.

5. Bab V Implementasi Arsitektur Platform DataOps dan MLOps

Memaparkan implementasi tiap *sub-sistem* yang sejalan dengan Bab IV, dengan praktik GitOps melalui Argo CD pada repositori Gitea sebagai sumber kebenaran tunggal.

6. Bab VI Evaluasi

Berisi metode evaluasi, hasil pengukuran, dan pembahasan terhadap kebutuh-

an fungsional dan nonfungsional pada Bab III.

7. **Bab VII Penutup**

Menyimpulkan jawaban atas T-1 hingga T-4 dan memberikan saran pengembangan lanjutan untuk lingkungan multi-*node* dan multi-domain.

Setelah Bab VII, dokumen ditutup oleh Daftar Pustaka yang memuat seluruh referensi terkutip dan Lampiran yang memuat berkas pendukung. Daftar Pustaka memakai gaya *Chicago author-date* (`biblatex-chicago`) dengan label Indonesia agar konsisten dengan tubuh laporan. Lampiran memuat daftar artefak platform dan hasil verifikasi *use case*. Penomoran lampiran mengikuti abjad (Lampiran A dan B) sebagaimana konvensi laporan Tugas Akhir di STI ITB.

BAB II

STUDI LITERATUR

Bab ini merangkum landasan teori dan ekosistem perangkat lunak yang relevan untuk perancangan platform pada Bab IV. Tinjauan disusun mulai dari konsep DataOps dan MLOps, dilanjutkan dengan komponen teknis yang menjadi bahan baku perancangan, kemudian ditutup dengan posisi penelitian terdahulu dan kebaruan yang ditawarkan. Susunan ini mengikuti urutan lapis arsitektur platform, dari orkestrasi infrastruktur, jalur data, layanan fitur, dan siklus hidup model, hingga tata kelola, deteksi *drift*, observabilitas, praktik GitOps, dan keamanan, sebagaimana dirinci pada Bab IV. Sumber rujukan menggabungkan publikasi akademik dengan dokumentasi resmi komponen *open source* agar setiap pilihan teknis dapat ditelusuri ke spesifikasi sumber.

Setiap subbab teknis memperkenalkan konsep lapis tersebut, menautkannya ke rujukan akademik, lalu menyebut beberapa alat *open source* sebagai contoh perwujudan. Perbandingan berskor antar alat dan pemilihan komponen konkret tidak dibahas pada bab ini, melainkan pada Subbab III.4 di Bab III yang menilai tiap alat terhadap kriteria kelayakan platform. Pemisahan ini menjaga bab ini tetap membahas konsep secara umum, sementara keputusan pemilihan alat ditegaskan pada bab analisis dengan bukti skor. Dengan begitu, keputusan teknis pada Bab IV dan Bab V dapat ditelusuri dari konsep pada bab ini ke pemilihan pada Bab III.

II.1 DataOps dan MLOps

Dua disiplin operasional yang menjadi pokok penelitian ini diuraikan terlebih dahulu sebelum komponen teknis pendukungnya. DataOps dibahas sebagai praktik otomasi siklus hidup data, kemudian MLOps sebagai perluasan praktik DevOps ke siklus hidup model. Kerangka tingkat kematangan MLOps dan model evolusi DataOps dipakai untuk menempatkan kondisi awal serta target kapabilitas platform pada jenjang yang dapat diukur. Subbab penutup membahas integrasi kedua disiplin yang menjadi celah penelitian sekaligus posisi platform ini.

II.1.1 DataOps

DataOps merupakan adaptasi praktik DevOps untuk siklus hidup data. Munappy dkk. (2020) mendefinisikan DataOps sebagai pendekatan yang mengadopsi prak-

tik *agile* dan DevOps serta mengandalkan otomatisasi tahapan siklus hidup data untuk mempercepat penyediaan hasil analitik yang berkualitas. Kleppmann (2017) memberi fondasi konseptual lewat penjelasan tentang sistem berbasis data yang andal, dapat diskalakan, dan dapat dipelihara, sementara Stopford (2018) memperkenalkan pola arsitektur *event-driven* dengan Apache Kafka sebagai tulang punggung. Pada praktiknya, DataOps menuntut otomatisasi *pipeline* ingestasi dan transformasi, pengujian kontrak data, perekaman *lineage*, dan pemantauan kualitas data sebagai bagian permanen dari operasi.

Empat pilar operasi DataOps yang dipakai pada platform ini adalah otomatisasi *pipeline*, kontrak skema, pengujian kualitas data, dan pemantauan SLO. Otomatisasi *pipeline* disusun dengan Apache Airflow untuk orkestrasi *batch* terjadwal, kontrak skema dijaga oleh Karapace sebagai *schema registry*, pengujian kualitas dijalankan oleh Great Expectations sebagai gerbang pada *pipeline*, dan SLO didefinisikan melalui Sloth dalam bentuk aturan Prometheus. Polyzotis dkk. (2018) merinci praktik validasi data berkelanjutan yang menyatu dengan *pipeline machine learning*, mencakup pemeriksaan skema, rentang nilai, dan anomali distribusi pada setiap proses ingestasi. Kombinasi keempat pilar tersebut menjadi prasyarat bagi integrasi DataOps dan MLOps yang dibahas pada Subbab II.1.4.

II.1.2 MLOps

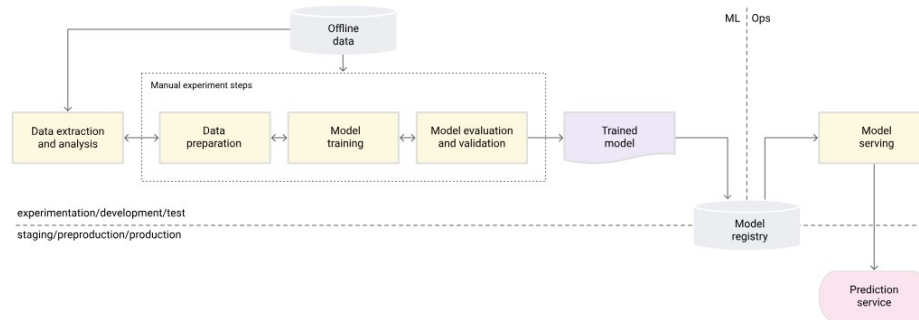
MLOps memperluas praktik DevOps ke seluruh tahap siklus hidup model *machine learning*. Kreuzberger, Kühl, dan Hirschl (2023) memberikan definisi terpadu: MLOps sebagai gabungan budaya kerja dan perangkat lunak untuk mengotomasi eksperimen, pelatihan, evaluasi, *deployment*, dan pemantauan model. Sculley dkk. (2015) menjelaskan akar persoalannya: *hidden technical debt* pada sistem *machine learning* muncul terutama dari komponen non-model seperti perekayasa fitur, konfigurasi, perekaman, pengujian data, dan layanan pemantauan. Paleyes, Urma, dan Lawrence (2022) melengkapi pemetaan dengan survei tantangan operasionalisasi, dan Shankar dkk. (2022) membawa perspektif praktisi melalui wawancara dengan tim MLOps lintas industri.

Tiga karakteristik penting yang membedakan MLOps dari DevOps tradisional adalah *continuous training*, *continuous monitoring*, dan *model registry* sebagai sumber kebenaran versi model. Amershi dkk. (2019) menggambarkan sembilan tahap rekayasa perangkat lunak *machine learning* mulai dari pengumpulan data hingga pemantauan, dengan setiap tahap memerlukan dukungan otomatisasi tersendiri. Breck dkk. (2017) memperkenalkan *rubric* kesiapan produksi yang menjadi acuan banyak tim

untuk menilai kematangan *pipeline* mereka. Platform ini mengoperasionalkan ketiga karakteristik tersebut dengan Kubeflow Pipelines untuk *continuous training*, Evidently dan Prometheus untuk *continuous monitoring*, serta MLflow Model Registry sebagai sumber kebenaran versi model yang ditelaah ulang pada Subbab II.5.1.

II.1.3 Tingkat Kematangan MLOps dan DataOps

Beberapa kerangka kematangan diusulkan untuk menggambarkan jenjang kapabilitas tim. Google Cloud (2024) membagi MLOps menjadi tiga level: *level 0* dengan proses manual, *level 1* dengan otomasi *pipeline* pelatihan, dan *level 2* dengan otomasi *deployment* berbasis CI/CD/CT. Tiga level ini diilustrasikan pada Gambar II.1, Gambar II.2, dan Gambar II.3. Microsoft Azure (2023) memberi varian dengan lima level dari 0 hingga 4, Kartakis dkk. (2022) menguraikan empat fase kematangan dari *initial* hingga *scalable* pada peta jalan MLOps AWS, dan McKnight (2020) menambahkan perspektif organisasi. Perbandingan ringkas dapat dilihat pada Tabel II.1.



Gambar II.1 MLOps *level 0*: proses manual dengan penyerahan model satu arah (Google Cloud 2024).

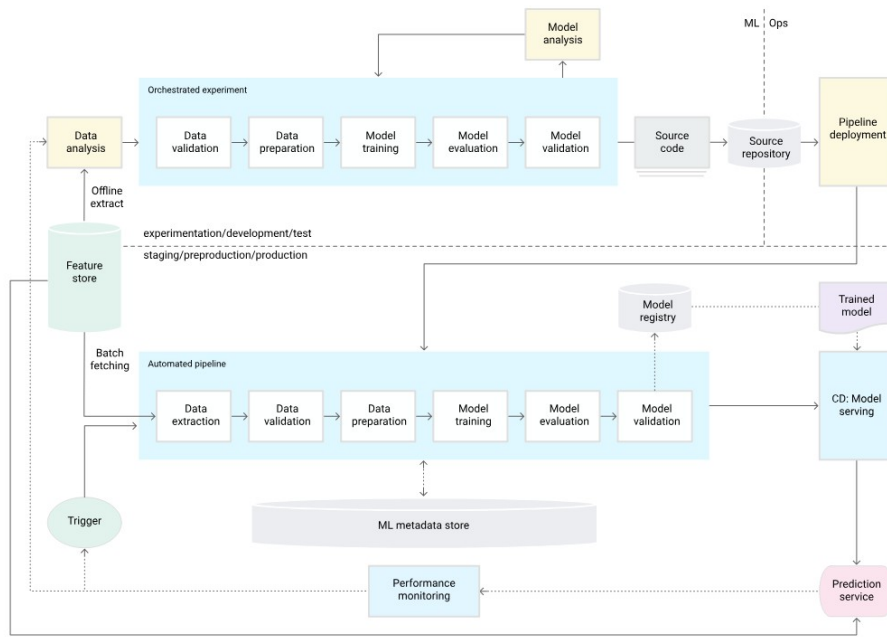
Platform yang dikembangkan menargetkan kematangan setara *level 2* pada kerangka Google Cloud. Target ini menuntut tiga jalur otomasi yang bekerja sebagai satu siklus, bukan sekadar tersedianya alat pada tiap tahap. Ketiganya menutup siklus otomasi dari pelatihan, *deployment*, hingga pemantauan yang memicu pelatihan ulang. Rincian tiap jalur pada platform ini adalah sebagai berikut.

1. **Pipeline pelatihan otomatis**

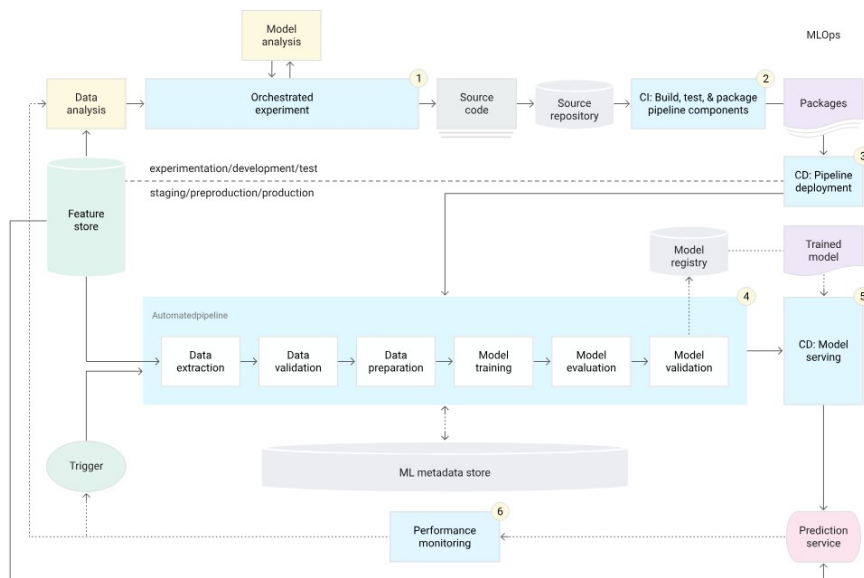
Pipeline pelatihan didefinisikan sebagai *pipeline* Kubeflow Pipelines yang dipicu secara terjadwal maupun ketika ambang *drift* terlampaui.

2. **Deployment model otomatis**

Deployment model dijalankan otomatis melalui KServe InferenceService yang versinya dipromosikan langsung oleh *pipeline* pelatihan.



Gambar II.2 MLOps *level 1*: otomasi *pipeline* pelatihan dengan komponen yang dapat dipakai ulang (Google Cloud 2024).



Gambar II.3 MLOps *level 2*: orkestrasi CI/CD/CT dengan deteksi *drift* sebagai pemicu pelatihan ulang (Google Cloud 2024).

3. Pemantauan model berkelanjutan

Pemantauan model dilakukan secara berkelanjutan oleh *job* deteksi *drift* terjadwal yang melaporkan metrik *drift* ke Prometheus serta menyimpan skor historis pada tabel analitik untuk audit.

Ketiga jalur otomasi tersebut menjadi acuan implementasi pada Bab V dan dasar

evaluasi pada Bab VI. Jalur pelatihan diwujudkan sebagai *pipeline* Kubeflow Pipelines, jalur *deployment* sebagai promosi versi pada KServe, dan jalur pemantauan sebagai *job* deteksi *drift* yang melapor ke Prometheus. Ketiganya dihubungkan oleh CronWorkflow Argo yang membaca skor *drift* dan memanggil *pipeline* pelatihan ketika ambang terlampaui. Dengan rantai itu, kematangan *level 2* dapat diperiksa sebagai perilaku sistem, bukan sekadar daftar komponen.

Tabel II.1 Perbandingan *Framework* Tingkat Kematangan MLOps

Frame- work	Level	DevOps Practices	ML Pipeline Auto- mation	CI/CD Inte- gration	CT (Conti- nuous Training)	Moni- toring & Feedback Loop	Gover- nance & Lineage
GCP	L0	0	0	0	0	0	0
	L1	1	2	1	2	2	2
	L2	2	2	2	2	2	2
Azure	L0	0	0	0	0	0	0
	L1	1	0	1	0	0	0
	L2	1	2	0	2	1	2
	L3	2	2	2	2	1	2
	L4	2	2	2	2	2	2
AWS	Initial	0	0	0	0	0	0
	Repeat- able	1	2	0	2	1	2
	Reliable	2	2	2	2	2	2
	Scalable	2	2	2	2	2	2
Giga Om	L0	0	0	0	0	0	0
	L1	1	1	0	0	1	1
	L2	2	2	2	1	1	2
	L3	2	2	2	2	2	2
	L4	2	2	2	2	2	2

Keterangan skor: 0 = praktik belum hadir pada level tersebut, 1 = hadir sebagian atau semi-otomatis, 2 = otomatis penuh; nilai dipetakan dari deskripsi level tiap kerangka.

Padanan kerangka pada sisi data diberikan oleh model evolusi DataOps yang disusun Munappy dkk. (2020) dari studi kasus industri telekomunikasi, berbentuk tangga lima tahap: analisis data ad hoc, analisis data semi-otomatis dengan *pipeline* data, praktik *agile data science*, pengujian dan pemantauan berkelanjutan, lalu DataOps. Tahap awal model tersebut ditandai silo data, keputusan bisnis yang hanya bertumpu pada sebagian kecil data, dan pemantauan *pipeline* yang masih manual, sedangkan tahap akhirnya menuntut CI/CD untuk analitik data, orkestrasi *pipeline*, dan pemantauan seluruh siklus hidup data. Laporan ini memakai kedua kerangka secara berpasangan untuk membingkai kondisi awal dan target: kondisi terfragmentasi pada Gambar III.1 di Bab III setara dengan *level 0* sekaligus tahap awal evolusi DataOps, sedangkan arsitektur pada Bab IV menargetkan *level 2* sekaligus tahap

DataOps. Dengan pasangan tersebut, perpindahan kapabilitas dapat dinilai pada sisi data dan sisi model sekaligus, bukan hanya pada salah satunya.

II.1.4 Integrasi DataOps dan MLOps

Jain dan Das (2025) menyatakan bahwa integrasi DataOps dan MLOps menjadi prasyarat skalabilitas dan ketahanan *pipeline machine learning* modern. Sementara itu, Burgueño-Romero dkk. (2025) mengusulkan arsitektur MLOps *open source* berbasis komponen, namun cakupannya berfokus pada jalur model dan belum membahas penyajian fitur *dual-store* maupun kebijakan deteksi *drift* sebagai satu kesatuan dengan jalur data. Rodrigues dkk. (2025) membahas arsitektur untuk *stream learning* mendekati waktu nyata, dengan penekanan pada deteksi *drift* dan pembaruan model. Penelitian ini berada pada irisan ketiganya: arsitektur *open source* yang menyatukan DataOps dan MLOps, dengan layanan fitur *dual-store* dan deteksi *drift* terotomasi sebagai komponen inti.

Integrasi tersebut diwujudkan melalui tiga jembatan teknis yang dijelaskan pada bab berikutnya.

1. **Feature contract pada Feast**

Feast menyatukan *offline store* dan *online store* dengan satu definisi FeatureView, sehingga jalur DataOps yang menulis fitur dan jalur MLOps yang membaca fitur memakai kontrak yang sama.

2. **Lineage terbuka melalui OpenLineage**

OpenLineage menyalurkan jejak transformasi *pipeline* data dari Airflow, termasuk langkah dbt di dalamnya, ke satu jalur penerimaan DataHub sehingga asal-usul *dataset* pelatihan dapat dirunut dari katalog yang sama.

3. **Bidang kendali deklaratif Argo CD**

Argo CD merekonsiliasi manifes komponen DataOps dan MLOps dari satu repositori Gitea, sehingga kebijakan dan versi komponen tetap konsisten lintas dua disiplin.

II.2 Kubernetes dan Orkestrasi *Cloud-Native*

Seluruh komponen platform berjalan di atas Kubernetes, sehingga karakter lapis orkestrasi ini perlu dipahami sebelum membahas komponen di atasnya. Pembahasan dibagi dua, yaitu peran Kubernetes beserta objek dasar dan pilihan distribusinya, kemudian mekanisme penskalaan otomatis beserta ekosistem *operator*. Urutan ini

menempatkan konsep dasar sebelum mekanisme yang dibangun di atasnya. Rincian penerapannya pada tiap komponen dibahas pada Bab IV.

II.2.1 Peran, Objek Dasar, dan Distribusi

Kubernetes berperan sebagai bidang orkestrasi yang menjalankan beban kerja ter-kontainerisasi pada *cluster* mesin. Lakka (2025) mendokumentasikan adopsi Kubernetes pada skala perusahaan dan menyoroti perannya sebagai standar *de facto cloud-native*. Adusumilli (2025) memperluas perspektif tersebut dengan pola *serverless* di atas Kubernetes, yang relevan untuk beban kerja *inference* dengan pola lalu lintas berfluktuasi. Cloud Native Computing Foundation (2024a) memberikan referensi resmi mengenai objek-objek inti seperti Pod, Deployment, StatefulSet, Service, dan Ingress yang menjadi landasan setiap komponen platform.

Pemilihan Kubernetes sebagai bidang dasar memberi tiga keuntungan teknis: orkestrasi beban kerja melalui objek deklaratif, mekanisme penskalaan otomatis, dan ekosistem operator yang memperluas API *cluster* dengan *Custom Resource Definition* (CRD). Distribusi *lightweight* seperti *k3s* mengakomodasi pengembangan dan pengujian pada satu *node* tanpa mengorbankan kompatibilitas API. Pilihan distribusi tersebut sejalan dengan asumsi BI-1 pada Bab I bahwa lingkungan pengujian memakai satu *node* dengan *footprint* sumber daya yang ringkas. Mekanisme *cluster autoscaler* tidak diaktifkan pada lingkungan satu *node*, sedangkan HPA, VPA, dan KEDA menjadi mekanisme penskalaan beban kerja yang dipakai.

Selain *k3s*, distribusi Kubernetes ringan lain yang umum dipertimbangkan untuk lingkungan satu *node* adalah *k0s*, MicroK8s, dan *kind*. Keempat distribusi lolos uji *conformance* CNCF sehingga API Kubernetes yang dipakai komponen platform tetap identik, sedangkan perbedaannya terletak pada *footprint*, kesiapan produksi, dan dependensi *runtime*. Perbedaan itu antara lain *containerd* yang tertanam pada *k3s* dan *k0s*, ketergantungan MicroK8s pada *snap*, serta *kind* yang menjalankan *node* di dalam kontainer Docker. Perbandingan berskor keempat distribusi terhadap kriteria kelayakan platform beserta alasan pemilihan *k3s* dibahas pada Subbab III.4.

II.2.2 Penskalaan Otomatis dan Ekosistem Operator

Penskalaan otomatis dijalankan oleh tiga komponen yang saling melengkapi. *Horizontal Pod Autoscaler* (HPA) menambah atau mengurangi jumlah *replica* pada Deployment dan StatefulSet berdasarkan metrik CPU dan memori yang dipasok oleh Metrics Server. *Vertical Pod Autoscaler* (VPA) (Kubernetes Autoscaling

SIG 2025) merekomendasikan dan menyetel *request* sumber daya pada level *pod* berdasarkan pengukuran historis. KEDA (KEDA Authors 2025) memperluas penskalaan ke sumber peristiwa eksternal melalui *ScaledObject*, termasuk *lag* antrean Kafka dan kueri metrik Prometheus, sehingga jumlah *replica* pemroses peristiwa mengikuti panjang antrean. Pada platform ini, batas bawah *minReplicaCount* dipertahankan satu agar konsumen tetap siaga saat *idle*.

Komponen operator dan lapisan *runtime* menyediakan abstraksi tingkat tinggi bagi perangkat data, model, dan layanan. Strimzi (Strimzi Authors 2025) mengoperasikan kluster Apache Kafka beserta *Kafka Connect* melalui CRD seperti *Kafka*, *KafkaUser*, dan *KafkaTopic*. CloudNativePG (CloudNativePG Contributors 2025) mengelola siklus hidup kluster PostgreSQL, termasuk pencadangan dan *failover*. Operator Apache Flink (Apache Flink Authors 2025) dan operator Apache Spark (Apache Spark Authors 2025) menyiapkan *FlinkDeployment* dan *SparkApplication* sebagai unit kerja deklaratif, sedangkan operator Altinity untuk ClickHouse (Altinity 2025) menyediakan CRD *ClickHouseInstallation* untuk replikasi dan *sharding*. Pada sisi *runtime*, Knative Serving (Knative Authors 2025) menambah kemampuan penskalaan ke nol pada beban kerja *request-response* dan menjadi fondasi bagi KServe. Kueue (Kueue Authors 2024) menyediakan antrean kerja untuk beban kerja *batch* sehingga prioritas dan kuota dapat ditegakkan lintas *namespace*. cert-manager (cert-manager Contributors 2025) mengotomasi penerbitan dan rotasi sertifikat TLS bagi *Ingress* dan *webhook* komponen platform.

II.3 Manajemen Data: Ingestasi, Pemrosesan, dan Penyimpanan

Lapis manajemen data mencakup jalur masuk data sampai tempat penyimpanannya. Pembahasan dimulai dari pola arsitektur aliran data, dilanjutkan komponen ingestasi dan pemrosesan, yaitu Kafka, Flink, serta Spark dan dbt. Tiga subbab berikutnya membahas lapis penyimpanan, dari format tabel *lakehouse*, penyimpanan objek dengan versi data, hingga penyimpanan relasional dan indeks pencarian. Urutan tersebut mengikuti arah aliran data pada platform, dari peristiwa masuk sampai data siap dikonsumsi oleh lapis layanan fitur.

II.3.1 Pola Lambda dan Kappa

Marz dan Warren (2015) memperkenalkan arsitektur Lambda yang memisahkan *batch layer* dan *speed layer*. Kreps (2014) mengkritisi pola tersebut karena menduplikasi logika pemrosesan dan mengusulkan arsitektur Kappa yang menyatukan keduanya menjadi satu jalur *stream-first*. Amazon Web Services (2024) memberik-

an rangkuman pola *event-driven* yang banyak diadopsi di lingkungan *cloud-native*. Perbedaan pokok kedua pola terletak pada jumlah jalur pemrosesan yang harus dipelihara, sebab Lambda menuntut dua implementasi logika yang sama sedangkan Kappa memakai satu *event log* sebagai sumber pemutaran ulang.

Platform ini mengadopsi pola Kappa dengan modifikasi pada lapis pemrosesan: aliran utama berjalan pada Flink untuk *streaming* sedangkan *backfill* historis dijalankan oleh Spark dan dbt pada referensi waktu yang sama. Pola tersebut menjaga konsistensi temporal antara fitur *streaming* dan fitur *batch* sebagaimana dijabarkan pada Subbab II.4.4. Pendekatan ini juga sejalan dengan rekomendasi Stopford (2018) mengenai *event log* sebagai sumber kebenaran tunggal. Hasilnya, kebutuhan pelatihan ulang model yang melibatkan data historis dapat dijawab tanpa menyalin data ke jalur terpisah, sebab *offline store* ClickHouse dipopulasi dari *topic* Kafka yang sama.

II.3.2 Apache Kafka

Apache Kafka berperan sebagai *distributed log* yang menyimpan kejadian dalam urutan tertentu dan memungkinkan konsumen memutar ulang aliran data dari posisi tertentu (Apache Software Foundation 2024c). Kafka mendukung jaminan *at-least-once* secara baku dan *exactly-once* dengan transaksi, sehingga cocok untuk dipakai sebagai tulang punggung penghubung antara produsen data dan konsumen pemroses. Beberapa *message broker* lain yang umum dipertimbangkan adalah Redpanda (Redpanda Data, Inc. 2025), Apache Pulsar (Apache Software Foundation 2025), dan NATS JetStream (NATS Authors 2025). Keempatnya menyediakan model *log* yang serupa namun berbeda pada jaminan pengiriman, kematangan *operator* Kubernetes, dan model lisensi. Perbandingan berskor *message broker* beserta alasan pemilihan Kafka dibahas pada Subbab III.4.

Registrasi skema dipisahkan ke Karapace (Aiven Karapace Authors 2025) sebagai *schema registry* kompatibel-Confluent yang berlisensi Apache 2.0, dengan dukungan Avro, JSON Schema, dan Protobuf serta verifikasi kompatibilitas saat publikasi. Verifikasi tersebut menahan perubahan skema yang memutus konsumen lama, sehingga evolusi kontrak data berlangsung mundur-kompatibel. Karapace menyimpan skema pada *topic* internal Kafka sehingga tidak menambah basis data baru pada platform. Perbandingan beberapa pilihan *schema registry* pada ekosistem Kafka dibahas pada Subbab III.4.

Pada platform ini, Kafka dioperasikan melalui *operator* Strimzi yang menyediak-

an CRD Kafka, KafkaUser, KafkaTopic, dan KafkaConnect, sehingga seluruh konfigurasi klaster, autentikasi pengguna, daftar *topic*, dan *connector* berada dalam bidang kendali yang sama dengan komponen lain. Mode autentikasi yang dipakai adalah SCRAM-SHA-512 melalui Strimzi KafkaUser dengan kredensial yang disebarkan oleh External Secrets Operator dari OpenBao. Topologi *topic* bawaan menggunakan enam *partition* dengan faktor replikasi satu pada lingkungan satu *node*, dan faktor replikasi dapat dinaikkan menjadi tiga pada lingkungan multi-*node*. Pemantauan *lag* antrean dilakukan oleh KEDA ScaledObject yang membaca metrik Kafka Exporter melalui Prometheus.

Jalur ingestasi berbasis konektor dilengkapi oleh Kafka Connect, kerangka integrasi pada ekosistem Kafka yang menjalankan konektor sumber dan tujuan sebagai beban kerja terkelola (Apache Software Foundation 2024c). Debezium (Debezium Community 2025) dipakai sebagai konektor *change data capture* (CDC) yang membaca *write-ahead log* PostgreSQL dan menerbitkan perubahan baris sebagai peristiwa Kafka, sehingga tabel relasional dapat diikuti ke jalur *streaming* tanpa kueri berkala. Antarmuka penelusuran klaster disediakan oleh Kafbat UI (Kafbat Authors 2025) untuk memeriksa *topic*, *consumer group*, dan skema pada operasi harian. Ketiganya memakai mekanisme autentikasi SCRAM yang sama dengan klien Kafka lain sehingga kebijakan akses *topic* tetap satu pintu.

II.3.3 Apache Flink

Apache Flink memproses aliran data dengan dukungan *event-time* dan *stateful operations*. Carbone dkk. (2015) menjelaskan dasar *pipeline streaming* pada Flink, dan dokumentasi resmi (Apache Software Foundation 2024b) menjabarkan fitur *exactly-once processing* dengan *checkpointing*. Flink dipakai untuk transformasi fitur waktu nyata dan agregasi *rolling window* yang dibutuhkan oleh model *streaming*. Penggunaan *event-time* alih-alih *processing-time* membuat hasil transformasi tetap konsisten ketika data terlambat tiba.

Pada platform ini, Flink dijalankan melalui FlinkDeployment CRD yang disediakan oleh *operator* Apache Flink. *Checkpoint* dan *savepoint* ditulis ke MinIO sebagai penyimpanan objek S3-kompatibel, sehingga pemulihan *job* dapat dilakukan tanpa kehilangan keadaan. Aliran utama *job* membaca *topic* dari Kafka melalui konektor Flink Kafka, melakukan transformasi pada *event-time* dengan *watermark* per partisi, lalu menerbitkan hasil ke *topic sink*. Lapis analitik mengonsumsi *topic* tersebut melalui tabel *Kafka engine* ClickHouse. Latensi pemrosesan dipantau melalui metrik Flink yang diekspos pada Prometheus dan diturunkan menjadi SLO melalui Sloth.

II.3.4 Apache Spark dan dbt

Apache Spark menyediakan kemampuan pemrosesan *batch* dan *micro-batch* pada skala besar (Apache Software Foundation 2024d). Zaharia dkk. (2016) memberikan ringkasan arsitektur Spark sebagai mesin terpadu. Spark dipakai untuk transformasi fitur dengan referensi waktu historis dan untuk pelatihan model yang membutuhkan pemrosesan data besar. Transformasi SQL deklaratif pada lapisan *warehouse* dikerjakan oleh dbt (dbt Labs 2025), yang memetakan model ke berkas SQL dengan dukungan pengujian skema dan dokumentasi otomatis. Perbandingan pilihan mesin pemrosesan data menurut kriteria kelayakan platform dibahas pada Subbab III.4.

Pada platform ini, Spark disediakan sebagai kapabilitas *batch* sesuai permintaan: *job* diserahkan langsung ke penjadwal Kubernetes tanpa kluster *master-worker* yang menganggur, dengan Airflow sebagai pemicu *job* terjadwal. Pembagian peran menempatkan dbt sebagai pengolah utama lapisan *warehouse*, sementara Spark menangani pemrosesan yang melampaui kapasitas satu mesin SQL. dbt menjalankan transformasi SQL pada *warehouse* ClickHouse dengan model *ref()* yang menjaga *lineage* antar tabel *medallion* dari *staging* hingga tabel fitur. Eksekusi dbt berlangsung di dalam *task* Airflow sehingga jejak *lineage* ikut dikirim ke DataHub melalui jalur OpenLineage pada Subbab II.6.

II.3.5 Format Tabel *Lakehouse*

Penyimpanan jangka panjang pada *data lake* memerlukan format tabel yang menyediakan transaksi ACID, evolusi skema yang aman, dan ketertelusuran lintas *engine* pemrosesan. Apache Iceberg, Delta Lake, dan Apache Hudi merupakan tiga format tabel *open-source* yang umum dipakai untuk arsitektur *lakehouse*, sementara penyimpanan langsung dalam berkas Parquet tanpa lapisan format tabel tidak menjamin transaksi maupun evolusi skema yang aman. Pada platform ini, Apache Iceberg dipakai sebagai format tabel *lakehouse*, dengan katalog REST melalui Lakekeeper (Lakekeeper Authors 2025) dan kueri federasi melalui Trino (Trino Software Foundation 2024). Perbandingan format tabel *lakehouse* terhadap kriteria yang relevan beserta alasan pemilihan Iceberg dibahas pada Subbab III.4.

Lakekeeper berperan sebagai *catalog server* REST yang menyimpan metadata Iceberg pada PostgreSQL, sehingga *engine* pemrosesan (Trino, Spark, Flink) dapat membaca dan menulis tabel Iceberg yang sama. Trino berperan sebagai *query engine* federatif yang memungkinkan kueri lintas ClickHouse, PostgreSQL, dan tabel Iceberg melalui satu jalur SQL. Kombinasi tersebut menghadirkan dua manfaat:

schema evolution yang aman tanpa migrasi data fisik dan *time-travel* untuk *snapshot* historis. Pemilihan ini juga sejalan dengan asumsi *open source* pada batasan BI-2 yang menghindari ketergantungan pada katalog terkelola berbayar.

II.3.6 Penyimpanan Objek dan Versi Data

Lapis penyimpanan objek menjadi landasan bagi *data lake*, *model registry*, dan pencadangan. Empat penyedia objek S3-kompatibel *open source* yang umum dipertimbangkan adalah MinIO (MinIO, Inc. 2024), Ceph RGW (Ceph Authors 2025), SeaweedFS (SeaweedFS Authors 2025), dan Garage (Deuxfleurs Association 2025). Pada platform ini MinIO dipakai sebagai penyedia objek, dan perbandingan berskor keempatnya beserta alasan pemilihannya dibahas pada Subbab III.4. *Encrypted storage* pada MinIO memakai *Server-Side Encryption with KMS* (SSE-KMS) dengan KES sebagai *gateway* ke OpenBao sebagaimana dirinci pada Subbab II.10.

Versi data pada lapis objek dikelola oleh lakeFS (Treeverse Ltd. 2025) yang menambahkan model *branching* dan *commit* bergaya Git pada *bucket* objek, sehingga jejak versi data dapat diruntut tanpa menggandakan berkas. Alternatif pengelola versi data yang umum dipertimbangkan adalah DVC (Iterative, Inc. 2025) yang menautkan versi data ke repositori Git, Project Nessie (Project Nessie Authors 2025) yang memberi versi pada katalog Iceberg, dan Pachyderm (Pachyderm, Inc. 2025) yang menggabungkan versi data dengan eksekusi *pipeline*. Perbandingan berskor pengelola versi data beserta alasan pemilihan lakeFS dibahas pada Subbab III.4. Pola pemakaiannya pada platform mengikuti alur *branch-merge*: setiap eksekusi *pipeline batch* menulis pada *branch* terpisah dan baru digabungkan ke *branch* utama setelah gerbang kualitas lolos, sehingga data yang dipublikasikan selalu melewati pemeriksaan.

Pada platform ini, MinIO disusun dengan *bucket* terpisah per kepentingan, antara lain *mlflow* untuk artefak eksperimen, *feast* untuk *registry feature store* berbasis berkas, *warehouse* untuk data *lakehouse*, *flink-checkpoints* untuk *checkpoint streaming*, serta *velero-backups* untuk pencadangan. Pemisahan per *bucket* membuat kebijakan versi, retensi, dan akses dapat diatur per kepentingan tanpa saling mengganggu. Kombinasi MinIO dan lakeFS memberi fleksibilitas untuk dua pola pemakaian: penyimpanan datar untuk artefak yang tidak memerlukan versi serta penyimpanan bercabang untuk data *pipeline* yang memerlukan *point-in-time*. Pemantauan kapasitas dilakukan melalui metrik MinIO yang diekspos ke Prometheus.

II.3.7 Penyimpanan Relasional dan Pencarian

Sebagian besar layanan platform memerlukan basis data relasional untuk menyimpan metadata. PostgreSQL (PostgreSQL Global Development Group 2025) dipakai sebagai basis data utama dan dioperasikan melalui *operator* CloudNativePG sebagaimana dibahas pada Subbab II.2. MySQL (Oracle Corporation 2025) dipakai khusus untuk *metadata store* pada layanan yang menuntut kompatibilitas *wire-protocol* MySQL, sedangkan OpenSearch (OpenSearch Foundation 2025) dipakai sebagai indeks pencarian dan penyimpanan graf metadata yang menjadi tulang punggung DataHub. Pencadangan pada ketiga lapis tersebut dijalankan oleh Velero dengan jadwal harian dan retensi 30 hari.

Pemisahan tiga lapis penyimpanan tersebut mengikuti karakter beban kerja. PostgreSQL melayani transaksi metadata MLflow, Airflow, DataHub, lakeFS, Lakekeeper, SpiceDB, dan Superset. MySQL melayani *metadata store* Kubeflow Pipelines (ML Metadata) dan Katib yang terikat pada protokol MySQL, sedangkan OpenSearch melayani indeks pencarian DataHub serta *lineage graph* dengan dukungan kueri graf. PostgreSQL dioperasikan dengan Cluster CRD CloudNativePG dengan *barman-cloud* sebagai *plugin* pencadangan ke MinIO. MySQL berjalan sebagai Deployment dengan *persistent volume* pada *storage class* lokal. OpenSearch berjalan sebagai StatefulSet dengan satu peran gabungan pada lingkungan satu *node*, dan dapat dipecah menjadi peran terpisah (master, data, ingest) pada lingkungan multi-*node*.

II.4 Layanan Fitur (*Feature Store*)

Layanan fitur menghubungkan jalur data dengan jalur model sehingga keduanya membaca definisi fitur yang sama. Konsep dan peran *feature store* dibahas terlebih dahulu, disusul pemisahan penyimpanan *offline* dan *online* yang menjadi inti arsitektur *dual-store*. Dua subbab berikutnya membahas pencarian vektor untuk fitur *embedding* serta jaminan *point-in-time correctness* yang mencegah kebocoran temporal. Keempat topik tersebut menjadi dasar perancangan lapis layanan fitur pada Bab IV.

II.4.1 Konsep dan Peran

Feature store berperan sebagai lapis penyatu antara produksi fitur dan konsumsi fitur. Lamer dkk. (2024) menjelaskan posisi *feature store* di antara *data lake*, *data warehouse*, dan *datamart*, sebagai komponen yang menyiapkan fitur untuk konsumsi model. Li dkk. (2023) menjabarkan arsitektur *feature store geo-distributed* dan

menekankan pentingnya kontrak fitur yang seragam antara *batch* dan *online*. Kartakis dkk. (2022) menempatkan *feature store* terkelola sebagai salah satu komponen pondasi dalam peta jalan MLOps tingkat perusahaan.

Tiga peran utama *feature store* adalah: (1) menyimpan fitur dengan kontrak skema yang konsisten lintas konsumen, (2) penyedia *point-in-time join* antara entitas dan fitur historis untuk pelatihan, dan (3) penyedia *lookup* fitur berlatensi rendah untuk *inference*. Tanpa *feature store*, ketiga peran tersebut harus dirakit ulang oleh setiap tim yang menyajikan model, sehingga muncul duplikasi kontrak dan risiko *train-serving skew* yang dibahas oleh Breck dkk. (2017). Platform ini menempatkan Feast sebagai bidang kendali *feature store* dengan *offline store* ClickHouse, *online store* Valkey untuk fitur tabular dan agregat, serta Qdrant untuk fitur *embedding* vektor. Tiga lapis penyimpanan tersebut terhubung pada satu FeatureView sehingga konsumen pelatihan dan konsumen penyajian membaca kontrak yang sama.

II.4.2 Penyimpanan *Offline* dan *Online*

Penyimpanan *offline* dipakai untuk pelatihan dan *batch scoring*, sedangkan penyimpanan *online* dipakai untuk *inference* dengan latensi rendah. Pemisahan ini mengikuti perbedaan pola akses, yaitu pemindaian historis bervolume besar pada satu sisi dan *lookup* per entitas dalam hitungan milidetik pada sisi lain. Pada platform ini, Feast (Feast Project 2024) dipakai sebagai kerangka *feature store*, ClickHouse (ClickHouse Inc 2024) sebagai *offline store*, dan Valkey (Valkey Contributors 2025) sebagai *online store*. Perbandingan berskor kerangka *feature store* serta pilihan penyimpanan *offline* dan *online* beserta alasan pemilihan ketiganya dibahas pada Subbab III.4.

Karakter beban kerja antara *offline* dan *online* secara mendasar berbeda. *Offline store* ClickHouse dioptimasi untuk kueri agregasi pada jutaan baris dengan kompresi kolumnar LZ4 atau ZSTD, sehingga cocok untuk pembuatan *training dataset* dengan *point-in-time join*. *Online store* Valkey memakai struktur data kunci-nilai pada memori dengan latensi *lookup* sub-milidetik, sehingga cocok untuk *inference* yang membutuhkan ribuan *lookup* per detik. Materialisasi dari *offline* ke *online* dijalankan oleh *job* terjadwal Kubernetes berdasarkan FeatureView pada Feast, dengan dukungan inkremental untuk fitur yang sering berubah dan dukungan penuh untuk fitur historis.

II.4.3 Pencarian Vektor dan HNSW

Fitur *embedding* berdimensi tinggi memerlukan struktur indeks pendekatan tetangga terdekat (*approximate nearest neighbor*, ANN). Malkov dan Yashunin (2020) memperkenalkan *Hierarchical Navigable Small World* (HNSW) sebagai struktur indeks dengan keseimbangan baik antara kecepatan kueri dan kualitas hasil. Alternatif lain seperti *Product Quantization* (Jegou, Douze, dan Schmid 2011) dan pencarian skala miliar dengan GPU (Johnson, Douze, dan Jégou 2021) relevan untuk skala yang berbeda. Untuk platform ini, Qdrant (Qdrant Team 2025) dipakai sebagai *vector index* terpisah dari *online store* tabular. Perbandingan beberapa pilihan *vector index open-source* beserta alasan pemilihan Qdrant dibahas pada Subbab III.4.

Pemilihan Qdrant juga mempertimbangkan dukungan *hybrid search* yang menggabungkan kueri vektor dengan filter atribut, sebagaimana dibahas oleh Bruch, Gai, dan Ingber (2023) sebagai pola pencarian yang banyak diadopsi pada sistem *retrieval* modern. Devlin dkk. (2019) dan OpenAI (2022) memberi konteks mengenai sumber *embedding* dari model bahasa yang banyak dipakai sebagai masukan ke *vector index*. Pada platform ini, Qdrant dijalankan sebagai Deployment dengan *persistent volume*, PodDisruptionBudget, serta kunci API dari External Secrets Operator. Metrik jarak *collection* dikonfigurasi pada layanan *embedding* platform dengan nilai bawaan kosinus untuk *embedding* yang dinormalkan. Indeks HNSW dikonfigurasi dengan parameter *m* dan *ef_construct* yang dapat ditingkatkan ketika rasio *recall* perlu diperbesar.

II.4.4 Point-in-Time Correctness

Jaminan *point-in-time correctness* berarti nilai fitur yang dipakai saat pelatihan model harus identik dengan nilai yang tersedia pada saat itu, bukan nilai versi terkini. Kaufman dkk. (2012) merumuskan kebocoran data sebagai kondisi ketika proses pelatihan memakai informasi yang belum tersedia pada saat prediksi dilakukan, dan menunjukkan bahwa kondisi tersebut membuat hasil evaluasi model tampak lebih baik daripada kinerja sesungguhnya. Feast menyediakan operasi *get_historical_features* yang melakukan *point-in-time join* antara entitas dan riwayat fitur, sehingga konsistensi nilai antara fase pelatihan dan fase *inference* dapat ditegakkan. Vela dkk. (2022) menambah perspektif evaluasi temporal pada pemodelan urutan waktu yang sejalan dengan kebutuhan *point-in-time*.

Pada platform ini, jaminan *point-in-time* dijamin oleh dua mekanisme. Pertama, FeatureView Feast mendaftarkan kolom *event_timestamp* sebagai acuan *join* his-

toris dan kolom `created_timestamp` sebagai penanda waktu penulisan fitur. Kedua, *online store* memakai versi terbaru dari nilai fitur sehingga *lookup* saat *inference* membaca nilai yang konsisten dengan `event_timestamp` terkini. Kombinasi keduanya menekan peluang kebocoran temporal saat pelatihan dan menjaga konsistensi nilai antara kedua fase.

II.5 Siklus Hidup Model

Siklus hidup model mencakup tahap eksperimen sampai model tersaji sebagai layanan. Pembahasan dimulai dari *tracking* eksperimen dan *registry* model, dilanjutkan orkestrasi *pipeline* pelatihan. Dua subbab berikutnya membahas lingkungan eksperimen interaktif beserta *hyperparameter tuning*, kemudian penyajian model pada lapis *inference*. Urutan tersebut mengikuti perjalanan model dari kode eksperimen sampai *endpoint* yang dipantau.

II.5.1 Eksperimen dan *Tracking*

MLflow (MLflow Contributors 2024) menyediakan *tracking* eksperimen, *registry* model, dan API *deployment*. Pimentel dkk. (2019) menyoroti masalah reproduktibilitas *notebook*, yang menjadi alasan mengapa *tracking* terpusat menjadi prasyarat. Perbandingan beberapa pilihan platform *tracking* dibahas pada Subbab III.4. Anaconda, Inc. (2020) memberikan konteks lapangan mengenai praktik kerja *data scientist* yang dapat dijadikan acuan saat memilih alat *tracking*.

Pada platform ini, MLflow dijalankan sebagai Deployment dengan PostgreSQL sebagai *backend store* dan MinIO sebagai *artifact store*. Akses konsol MLflow melewati jalur autentikasi terpusat platform yang dijelaskan pada Subbab II.10, sehingga tidak ada mekanisme kredensial terpisah per konsol. *Model registry* dipakai sebagai sumber kebenaran versi model yang akan dipromosikan oleh *pipeline continuous training* ke KServe InferenceService. Artefak model tersimpan pada *bucket* MinIO sehingga kebijakan enkripsi dan pencadangan lapis objek berlaku seragam untuk artefak eksperimen.

II.5.2 Orkestrasi Pelatihan

Dua pilihan utama orkestrasi pelatihan pada Kubernetes adalah Kubeflow Pipelines (Kubeflow Contributors 2024) dan Argo Workflows (Argo Project 2025). Apache Airflow (Apache Software Foundation 2024a) masih relevan untuk *pipeline* data berorientasi *batch*. Kubeflow Pipelines dipakai untuk *pipeline* pelatihan dan *de-*

ployment model, sedangkan Argo Workflows dipakai pada *control loop* berbentuk CronWorkflow yang membaca skor *drift* dan memicu pelatihan ulang. Perbandingan ketiganya berdasarkan kebutuhan domain dibahas pada Subbab III.4.

Pada platform ini, *controller* Argo Workflows disertakan oleh instalasi Kubeflow Pipelines sehingga kedua *orchestrator* berjalan pada *namespace* yang sama. Keduanya mengakses *registry* MLflow dan Feast melalui Service Kubernetes tanpa jalur khusus. Pembagian perannya tetap dijaga, yaitu Kubeflow Pipelines menjalankan *pipeline* pelatihan sedangkan CronWorkflow Argo membaca skor *drift* sebagai *control loop*. Susunan ini menghindari dua instalasi *workflow engine* yang tumpang tindih pada kluster yang sama.

II.5.3 Eksperimen Interaktif dan *Tuning*

Kubeflow Notebooks (Kubeflow Authors 2025b) menyediakan lingkungan *notebook* multi-pengguna di atas Kubernetes dengan tata kelola sumber daya per pengguna. Untuk *hyperparameter tuning*, Katib (Kubeflow Authors 2025a) menjalankan pencarian *grid*, *random*, *Bayesian*, dan *neural architecture search* melalui Experiment CRD yang algoritme pencariannya dilayani oleh *suggestion service* terpisah. Kubeflow Trainer (Kubeflow Authors 2025c) memperluas dukungan ke pelatihan terdistribusi melalui CRD TrainJob dengan *runtime* bawaan untuk kerangka seperti PyTorch dan DeepSpeed. Vangala, Mehta, dan Singh (2022) memberi gambaran praktik MLOps di lapangan yang menempatkan ketiga komponen tersebut sebagai bagian dari satu lapis pengembangan model.

Pada platform ini, Kubeflow Notebooks dipasang dalam bentuk *standalone* tanpa Profile CRD, sebab kebutuhan isolasi *multi-tenant* tidak termasuk dalam lingkup pengujian satu *node*. Komponen notebook-controller merekonsiliasi CRD Notebook menjadi StatefulSet per pengguna, dan jupyter-web-app berperan sebagai antarmuka pembuatan *notebook*. Kuota sumber daya ditegakkan melalui *request* dan *limit* pada spesifikasi *notebook* bersama kebijakan validasi Kyverno yang mewajibkan batas sumber daya pada setiap *pod*. Katib menjalankan *trial* sebagai Job terpisah dengan parameter yang diberi label sehingga hasilnya dapat dirunut pada konsol Katib. Kubeflow Trainer menyediakan CRD TrainJob untuk pelatihan terdistribusi yang relevan ketika kluster diperluas menjadi multi-*node*.

II.5.4 Penyajian Model

KServe (KServe Contributors 2024) menyediakan kerangka penyajian model di atas Kubernetes dengan dukungan banyak *runtime*, otomatisasi penskalaan, dan integrasi dengan praktik GitOps. Hsu dkk. (2024) memperlihatkan ujicoba *deployment* terotomasi dengan KServe yang sejalan dengan pendekatan platform ini. Liang dkk. (2024) menyoroti integrasi sistem dengan *machine learning* sebagai pendorong otomatisasi pelatihan dan *deployment*. Ahmad dkk. (2024) memberikan tinjauan strategi keamanan MLOps yang relevan untuk lapis penyajian. Perbandingan KServe dengan kerangka penyajian model lain menurut kriteria kelayakan platform dibahas pada Subbab III.4.

Pada platform ini, KServe dijalankan melalui InferenceService CRD dengan dukungan *predictor*, *transformer*, dan *explainer* yang dapat dirangkai pada satu jalur. *Runtime* penyajian yang didefinisikan adalah ClusterServingRuntime berbasis MLServer untuk model dari *registry* MLflow, sementara *runtime* bawaan KServe lain seperti Triton Inference Server dapat ditambahkan ketika model jaringan saraf dalam dibutuhkan. Kemampuan penskalaan ke nol disediakan oleh Knative Serving sebagai fondasi KServe, dengan batas bawah satu *replica* dipertahankan pada platform agar permintaan pertama tidak menanggung *cold start*. *Deployment* progresif pada layanan di depan model dijalankan oleh Argo Rollouts dengan strategi *canary*. Pemantauan kualitas prediksi tidak dibebankan pada jalur *inference*: deteksi *drift* membaca tabel fitur pada lapis analitik melalui *job* terjadwal sebagaimana dibahas pada Subbab II.7.

II.6 Tata Kelola Data: Katalog, *Lineage*, dan Kualitas

Bernardo dkk. (2024) memetakan tata kelola data sebagai disiplin lintas fungsi yang mencakup metadata, kualitas, akses, dan kepemilikan. Stoyanovich, Howe, dan Jagadish (2020) memberi konteks etis dengan menempatkan tanggung jawab atas data pada level platform, bukan hanya level aplikasi. Chien (2022) memberi panduan praktis untuk peningkatan kualitas data berbasis pengukuran. Pada platform ini, aspek metadata dan kepemilikan diwadahi oleh DataHub sebagai katalog, aspek kualitas dijaga oleh Great Expectations sebagai kontrak kualitas, aspek akses ditegakkan oleh lapis keamanan pada Subbab II.10, dan jejak transformasi direkam melalui OpenLineage sebagai protokol *lineage* terbuka.

DataHub (DataHub Project 2024) dipakai sebagai katalog metadata pada platform ini. Great Expectations (Great Expectations 2024) dipakai untuk pengujian kontrak

data yang dijalankan pada *pipeline* produksi. OpenLineage (OpenLineage Authors 2025) dipakai sebagai standar terbuka pengumpulan *lineage* antar komponen *pipeline*, sehingga jejak dari ingestasi hingga model dapat disusun dengan kosa kata yang seragam. Perbandingan beberapa pilihan katalog beserta alasan pemilihan DataHub dibahas pada Subbab III.4. Kontrak kualitas data dikemas sebagai *expectation suite* yang tersimpan pada repositori Git, dimuat dari ConfigMap, dan dijalankan oleh Great Expectations sebagai gerbang kualitas terjadwal pada *pipeline* Airflow, sementara *pipeline lakehouse* menambah pemeriksaan kualitas melalui kueri Trino sebelum *branch* lakeFS digabungkan ke *branch* utama.

II.7 Deteksi *Drift*

Lu dkk. (2019) memberikan tinjauan luas tentang pembelajaran di bawah *concept drift*. Bayram, Ahmed, dan Kassler (2022) mengelompokkan keluarga detektor berbasis performa, dan Hinder dkk. (2023) memperluas pembahasan ke arah penjelasan model. Beberapa metrik yang relevan untuk *covariate drift* dan *prior probability shift* antara lain *Population Stability Index* (PSI), uji Kolmogorov-Smirnov (Reis dkk. 2016), dan jarak Wasserstein. Céspedes Sisniega dkk. (2024) memperlihatkan implementasi deteksi *drift* pada lingkungan *serverless*, sementara Jourdan dkk. (2023) membahas penanganan *drift* pada *deep learning* untuk pemantauan proses.

Pada platform ini, PSI dan uji Kolmogorov-Smirnov dipakai sebagai detektor utama, dengan ambang batas yang dikonfigurasi per fitur. Detektor berjalan sebagai *job* terjadwal yang membandingkan jendela data terkini terhadap jendela referensi bergulir pada beberapa skala waktu, dari hitungan jam hingga puluhan hari, langsung dari tabel fitur ClickHouse, lalu menuliskan skor *drift* ke tabel analitik dan metrik Prometheus. CronWorkflow Argo membaca skor tersebut sebagai *control loop* dan memicu pelatihan ulang melalui Kubeflow Pipelines apabila ambang dilewati. Evidently (Evidently AI 2025) melengkapi jalur tersebut sebagai layanan pelaporan yang menghasilkan laporan *data drift* dan ringkasan data ke *workspace* Evidently untuk ditelaah secara visual.

II.8 Observabilitas *Cloud-Native*

Cloud Native Computing Foundation (2024b) dan IBM (2024) mendefinisikan observabilitas sebagai gabungan tiga pilar: metrik, log, dan *trace*. Untuk platform ini, Prometheus (Prometheus Authors 2024) dipakai untuk metrik, Loki (Grafana Labs 2024b) untuk log, dan Grafana Tempo (Grafana Labs 2024c) untuk *trace*,

dengan OpenTelemetry (OpenTelemetry Authors 2025) sebagai *collector* bersama dan Grafana (Grafana Labs 2024a) sebagai antarmuka visualisasi. Sloth (Sloth Authors 2025) membantu menurunkan definisi *service-level objective* ke dalam aturan Prometheus, sedangkan Evidently (Evidently AI 2025) memberi laporan kualitas data dan model yang melengkapi pilar observabilitas. Pyroscope (Grafana Labs 2025) menyediakan *continuous profiling* untuk profil CPU dan memori, Pushgateway (Prometheus Authors 2025) menjadi titik kumpul metrik untuk *job* berdurasi pendek di luar siklus *scrape*, dan OpenCost (OpenCost Authors 2025) memantau biaya per beban kerja pada klaster.

Apache Superset (Apache Superset Authors 2025) berperan sebagai antarmuka BI yang menyatukan kueri SQL lintas ClickHouse, Trino, dan MySQL pada satu lapis dasbor. Perbandingan beberapa pilihan *BI open-source* dibahas pada Subbab III.4. Pemantauan tambahan untuk kualitas data dan model dirangkai sebagai dasbor pada Grafana sehingga pengamatan terhadap kondisi sistem dapat dilakukan dari satu antarmuka. Perbandingan beberapa pilihan *observability stack* dibahas pada Subbab III.4. Korelasi antara metrik, log, dan *trace* dimungkinkan melalui label *trace_id* yang dilampirkan pada baris log Loki dan rentang Tempo, sehingga pelacakan kegagalan dapat berpindah antar pilar tanpa kehilangan konteks.

II.9 GitOps dan *Continuous Delivery*

GitOps menempatkan repositori Git sebagai sumber kebenaran tunggal untuk konfigurasi sistem. Limoncelli (2018) menjelaskan pola GitOps sebagai pendekatan IT *self-service* yang memetakan operasi ke kontrol versi. Argo CD (Argo Project 2024) dipakai sebagai *operator* GitOps yang merekonsiliasi keadaan *cluster* dengan deklarasi pada Git. Kim dkk. (2016) memberi dasar teoretis tentang alur kerja CI/CD yang menjadi induk GitOps.

Repositori sumber kebenaran dijalankan secara mandiri pada Gitea (Gitea Contributors 2025) sebagai layanan Git *self-hosted* yang ringan dan kompatibel dengan API yang umum, sehingga seluruh manifes, *chart* Helm, dan *overlay* Kustomize berada pada satu bidang kendali yang dapat ditarik oleh Argo CD tanpa ketergantungan terhadap penyedia eksternal. Tekton (Tekton Contributors 2024) dipakai sebagai *pipeline* CI dengan rangkaian *task* kloning sumber, penggabungan *overlay* konfigurasi, *lint*, pengujian, serta *build* dan *push image*, diakhiri *task finally* yang melaporkan metrik *pipeline* ke Pushgateway. Argo Rollouts (Argo Rollouts Authors 2025) melengkapi rantai dengan strategi *deployment* progresif berbasis *canary* dan analisis

metrik. Perbandingan beberapa *GitOps controller open-source* berdasarkan kriteria yang relevan dibahas pada Subbab III.4.

II.10 Keamanan Platform: Identitas, Rahasia, *Mesh*, dan Kebijakan

Lapisan keamanan menjadi prasyarat bagi platform yang melayani DataOps dan MLOps secara berdampingan. Empat aspek dipertimbangkan: identitas pengguna terpadu, pengelolaan rahasia, *service mesh*, dan *policy engine*. Empat aspek tersebut bekerja sebagai lapisan pertahanan yang saling melengkapi, sesuai prinsip *defense in depth* yang dibakukan pada panduan rekayasa sistem aman NIST SP 800-160 (Ross, Winstead, dan McEvilly 2022). Pelanggaran pada satu lapis tidak otomatis membuka akses ke lapis berikutnya sebab tiap lapis menegakkan kontrol yang berbeda jenis: identitas mengontrol *siapa*, rahasia mengontrol *apa yang dipegang*, *mesh* mengontrol *siapa berbicara ke siapa*, dan kebijakan mengontrol *apa yang boleh dibuat di cluster*. Dua aspek pertama dibahas pada Subbab II.10.1, sedangkan dua aspek berikutnya bersama pemantauan *runtime* dibahas pada Subbab II.10.2.

II.10.1 Identitas dan Pengelolaan Rahasia

Untuk identitas, Dex (Dex Authors 2025) dipakai sebagai *identity provider* berbasis OpenID Connect (OIDC) pada platform ini. Untuk antarmuka konsol yang tidak menyediakan OIDC secara *native*, oauth2-proxy (OAuth2 Proxy Authors 2025) dipakai sebagai *reverse proxy* yang menanyakan sesi Dex sebelum meneruskan permintaan ke layanan, sehingga seluruh konsol pengembang berbagi satu jalur autentikasi. Untuk otorisasi granular pada layanan yang membutuhkan kontrol relasi, SpiceDB (AuthZed 2025) menyediakan basis data otorisasi gaya Google Zanzibar dengan skema relasi deklaratif dan API permintaan izin yang konsisten. Perbandingan beberapa *identity provider open-source* beserta alasan pemilihan Dex dibahas pada Subbab III.4.

Untuk pengelolaan rahasia, OpenBao (OpenBao Authors 2025) dipakai sebagai *fork open-source* dari HashiCorp Vault di bawah Linux Foundation yang menjamin penyimpanan rahasia terenkripsi serta penerbitan kredensial dinamis. Data OpenBao disimpan pada *integrated storage Raft* sehingga tidak bergantung pada basis data eksternal. Integrasi pada beban kerja dilakukan melalui External Secrets Operator (External Secrets Authors 2025) yang menyalin rahasia ke Secret Kubernetes dengan rotasi otomatis melalui CRD ExternalSecret dan ClusterSecretStore. Enkripsi data *at rest* pada MinIO dijembatani oleh MinIO KES (MinIO, Inc. 2025) sebagai *stateless KMS gateway* yang meneruskan permintaan kunci ke OpenBao tan-

pa menyimpan kunci pada *cluster*. Perbandingan beberapa pengelola rahasia *open-source* dibahas pada Subbab III.4.

II.10.2 *Service Mesh, API Gateway, dan Penegakan Kebijakan*

Untuk komunikasi antar-layanan, Istio (Istio Authors 2024) dipakai sebagai *service mesh* yang menyediakan *mutual TLS* (mTLS) seragam, kontrol lalu lintas lapis aplikasi melalui *VirtualService* dan *AuthorizationPolicy*, serta integrasi telemetri dengan *OpenTelemetry*. Perbandingan beberapa *service mesh open-source* berdasarkan kriteria fitur, ekosistem, dan kematangan dibahas pada Subbab III.4. Pada perbatasan *mesh*, Apache APISIX (Apache APISIX Authors 2025) berperan sebagai *API gateway* yang menyaring lalu lintas masuk, menerapkan kebijakan kuota, dan menjembatani mTLS terhadap konsumen di luar *mesh*. Perbandingan beberapa *API gateway open-source* dibahas pada Subbab III.4.

Untuk penegakan kebijakan, Kyverno (Kyverno Authors 2024) dipakai sebagai *policy engine native* Kubernetes dengan sintaks YAML yang ringkas serta dukungan *mutate*, *validate*, dan *generate*. Open Policy Agent (OPA) Gatekeeper (Open Policy Agent Authors 2024) juga umum dipakai sebagai alternatif berbasis Rego. Pada platform ini, kebijakan Kyverno dipakai antara lain untuk mewajibkan batas sumber daya pada setiap *pod* dan menegakkan profil keamanan baku lintas *namespace*. Perbandingan beberapa *policy engine open-source* dibahas pada Subbab III.4.

Untuk pemantauan ancaman pada lapis *runtime*, Falco (Falco Authors 2025) dipakai sebagai detektor berbasis eBPF yang memantau *syscall* dan menerbitkan peristiwa keamanan, sedangkan Trivy Operator (Aqua Security 2025) memindai *image* dan beban kerja secara terjadwal. Pencadangan *cluster* dan *persistent volume* dijaga oleh Velero (Velero Authors 2025) dengan arsip yang ditulis ke *bucket* MinIO khusus pencadangan. Uji ketahanan disediakan oleh Chaos Mesh (Chaos Mesh Authors 2025) melalui CRD eksperimen seperti PodChaos untuk menyuntikkan kegagalan terkontrol. Perbandingan beberapa *runtime security open-source* dibahas pada Subbab III.4.

II.11 Penelitian Terkait

Beberapa penelitian terdahulu memiliki lingkup yang berdekatan dengan platform ini. Burgueño-Romero dkk. (2025) mengusulkan arsitektur MLOps *open source*, namun belum memasukkan layanan fitur dengan dukungan vektor sebagai komponen utama. Amou Najafabadi dkk. (2024) memetakan beragam arsitektur MLOps

tetapi tidak menggabungkan DataOps secara menyeluruh. Rodrigues dkk. (2025) menyentuh *near real-time stream learning*, namun tidak meletakkan tata kelola data sebagai bidang kendali yang ditegakkan. Ahmad dkk. (2024) membahas strategi keamanan MLOps, tetapi belum menautkannya pada bidang kendali tata kelola yang menegakkan kebijakan akses sepanjang siklus data dan model.

Kesenjangan yang berulang pada penelitian-penelitian tersebut adalah belum adanya satu rancangan yang sekaligus menyatukan DataOps dan MLOps pada satu bidang kendali, menegakkan tata kelola data sebagai kontrak yang dijalankan otomatis, dan menyediakan layanan fitur multimodal dari satu sumber kebenaran. Kesenjangan inilah yang menjadi titik berangkat analisis kebutuhan dan pemilihan solusi pada Bab III.

BAB III

ANALISIS MASALAH

Bab ini membahas analisis masalah pada pengembangan platform DataOps dan MLOps melalui empat langkah, yaitu pemetaan kondisi saat ini, perumusan kebutuhan fungsional dan nonfungsional, pemilihan alternatif solusi, dan pemilihan komponen *open source* untuk tiap lapis arsitektur. Subbab III.1 memetakan tiga sumber tekanan yang menjadi akar pengembangan platform, kemudian Subbab III.2 menurunkan empat belas kebutuhan fungsional dan dua belas kebutuhan nonfungsional sebagai tolok ukur kelayakan. Subbab III.3 menilai empat alternatif solusi memakai skor pemenuhan kebutuhan lalu memilih alternatif berskor tertinggi, kemudian Subbab III.4 menutup bab dengan menurunkan pemilihan komponen *open source* konkret untuk tiap lapis arsitektur. Penomoran kebutuhan yang disusun pada bab ini menjadi rujukan tetap ketika Bab IV memetakannya ke komponen dan Bab VI mengujinya.

III.1 Analisis Kondisi Saat Ini

Kondisi pengembangan platform DataOps dan MLOps saat ini dipengaruhi oleh dua tekanan utama yang, ketika tidak teratasi, memicu efek domino sebagai tekanan ketiga. Pertama, beban konfigurasi puluhan komponen *open source* dari nol membatasi kecepatan pembangunan platform internal. Kedua, biaya berlangganan layanan terkelola membatasi kelayakan operasional pada skala produksi sekaligus menghadirkan pertukaran antara waktu pengembangan dan anggaran berjalan. Ketiga, fragmentasi yang merambat lintas peran (*business user*, *data engineer*, *data scientist*, dan *machine learning engineer*) memicu efek domino pada tata kelola data, deteksi *drift*, dan penyajian fitur multimodal. Ketiga subbab berikut menelaah setiap sumber tekanan tersebut secara berurutan.

III.1.1 Beban Konfigurasi Platform dari Nol

Organisasi yang membangun platform dari nol harus merangkai puluhan komponen lintas lapis ingestasi, pemrosesan, penyimpanan, pelatihan, dan penyajian model, dengan ruang pilihan *tool* yang sangat luas untuk tiap lapis. Setiap komponen memiliki kontrak konfigurasi, protokol akses, dan model identitas masing-masing, sehingga *glue code* antar lapis terus tumbuh seiring penambahan komponen. Perubahan versi pustaka pada satu komponen kerap memicu *rework* pada komponen

sekitarnya, terutama ketika skema data dan kontrak API ikut berubah. Tim platform juga menanggung kurva belajar yang panjang karena harus menguasai paradigma operasional yang berbeda untuk basis data, sistem antrian, mesin pemrosesan, registri model, dan layanan penyajian. Ketiga mekanisme tersebut berjalan serentak sehingga beban konfigurasi tidak berhenti pada pemasangan awal, melainkan berulang pada setiap penambahan komponen dan pergantian versi.

Beban tersebut bersifat akumulatif karena waktu yang habis untuk integrasi tidak dapat dipulihkan menjadi pengembangan kapabilitas. Shankar dkk. (2022) mendokumentasikan praktik MLOps yang sering tersendat justru karena lapisan integrasi yang tidak pernah selesai disempurnakan oleh tim platform. Kondisi tersebut menjelaskan mengapa banyak inisiatif platform internal terhenti pada tahap arsitektur referensi dan tidak pernah mencapai jalur produksi yang lengkap. Tantangan akan semakin sulit ketika tim berukuran kecil harus menjaga kontrak layanan untuk ingestasi, pemrosesan, pelatihan, penyajian, dan tata kelola sekaligus, karena setiap pergantian versi menimbulkan rantai uji regresi yang panjang. Beban ini diturunkan pada Subbab III.2 menjadi kebutuhan konfigurasi deklaratif, rekonsiliasi otomatis, dan modularitas komponen agar pemasangan ulang platform tidak mengulang biaya integrasi dari nol.

III.1.2 Biaya Berlangganan Layanan Terkelola

Sebagai alternatif terhadap konfigurasi dari nol, organisasi dapat berlangganan layanan terkelola dari penyedia *cloud*. Li dkk. (2023) memperlihatkan bahwa layanan *feature store* terkelola mempercepat adopsi dengan membebaskan tim dari pembangunan infrastruktur penyimpanan fitur. Akan tetapi, Hamza, Akbar, dan Capilla (2024) menemukan bahwa model penagihan bayar per pakai pada layanan terkelola menyulitkan estimasi anggaran dan cenderung meningkat seiring volume *ingest*, jumlah *training run*, dan trafik *inference*. Biaya tersebut bersifat berulang setiap bulan dan jarang dapat ditekan dengan optimasi internal karena pengguna tidak memegang kendali penuh atas alokasi sumber daya pada lapis bawah. Ketergantungan pada satu penyedia menimbulkan keterikatan vendor yang membuat migrasi menjadi mahal, terutama ketika format *feature view*, *registry model*, dan *lineage* memakai skema kepemilikan tertutup. Beban ini bertambah ketika organisasi memakai layanan terkelola dari lapis berbeda yang dipasok beberapa penyedia, sebab perbedaan model identitas, protokol akses, dan format *audit log* memaksa tim platform tetap merakit otomatisasi tata kelola lintas penyedia secara manual.

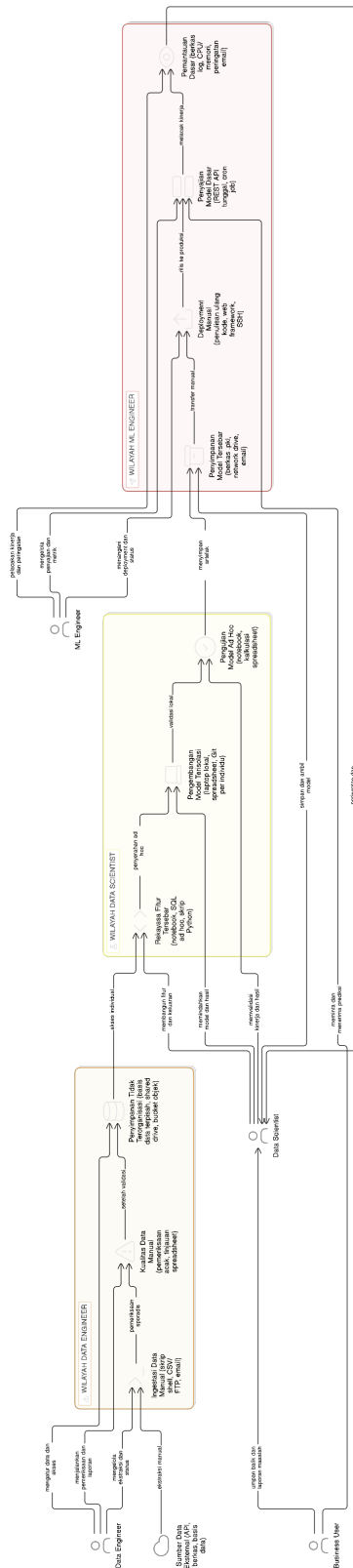
Kedua jalur tersebut menghadirkan pertukaran mendasar antara waktu pengembang-

an dan anggaran berjalan. Sifat pertukarannya tidak simetris: biaya membangun dari nol terkonsentrasi di awal dan menurun setelah jalur *end-to-end* stabil, sedangkan biaya berlangganan tumbuh mengikuti volume *ingest*, jumlah *training run*, dan trafik *inference* sehingga justru membesar ketika adopsi berhasil. Organisasi yang berpindah dari satu jalur ke jalur lain bahkan menanggung kedua beban sekaligus selama masa transisi, karena integrasi internal harus dirakit sambil kontrak layanan terkelola masih berjalan. Karakter pertukaran ini dipakai pada dua titik analisis berikutnya, yaitu kebutuhan nonfungsional efisiensi sumber daya dan portabilitas pada Subbab III.2.3 serta penilaian empat alternatif solusi pada Subbab III.3.

III.1.3 Efek Domino Fragmentasi Lintas Peran

Pengembangan model *machine learning* untuk tahap produksi melibatkan empat peran utama, yaitu *business user*, *data engineer*, *data scientist*, dan *machine learning engineer*. Kreuzberger, Köhl, dan Hirschl (2023) memperlihatkan bahwa peran tersebut sering menggunakan *tool* yang berbeda sehingga muncul fragmentasi yang menghambat kolaborasi lintas tim. Paley, Urma, dan Lawrence (2022) menambahkan bahwa fragmentasi tersebut melahirkan integrasi yang lemah pada sambungan data dan model serta menurunkan kecepatan rilis dan kualitas keluaran. Gambar III.1 merangkum gambaran umum fragmentasi pada keempat peran tersebut. Gambaran umum dipilih karena beban inti yang menjadi titik tolak penelitian terletak bukan pada satu peran tertentu, melainkan pada sambungan antar peran yang terputus di lapisan ingestasi, fitur, pelatihan, dan penyajian. Efek domino muncul ketika satu kontrak data antar lapis pecah lalu menyebar ke lapis di atasnya, yaitu merusak skema pada lapis ingestasi merambat ke pemrosesan, lalu ke fitur yang dipakai untuk pelatihan, sehingga model pada lapis penyajian menerima distribusi yang berbeda dengan saat pelatihan. Pada kerangka kematangan Subbab II.1.3, kondisi ini setara dengan MLOps *level 0* yang prosesnya manual dan berbasis skrip, eksperimennya terpisah dari operasi, serta berjalan tanpa CI/CD dan tanpa pemantauan kinerja aktif (Google Cloud 2024). Kondisi yang sama berada pada tahap awal model evolusi DataOps karena data terkumpul dalam silo dan pemantauan *pipeline* masih dilakukan manual (Munappy dkk. 2020).

Amou Najafabadi dkk. (2024) dan Burgueño-Romero dkk. (2025) memperlihatkan bahwa fragmentasi tidak hanya muncul pada antarmuka tim, tetapi juga pada bidang kendali infrastruktur, kebijakan akses, dan observabilitas. Kondisi inilah yang menjadi titik tolak penelitian, karena keempat rumusan masalah pada Bab I pada hakikatnya merupakan konsekuensi dari rangkaian sambungan yang terputus terse-



Gambar III.1 Gambaran Umum Fragmentasi DataOps dan MLOps, setara MLOps *level 0* (Google Cloud 2024) dan tahap awal evolusi DataOps (Munappy dkk. 2020).

but. Konsekuensi tersebut tidak dapat ditangani satu per satu, sebab perbaikan pada satu lapis akan kembali terkikis selama kontrak antar lapis tetap dirakit secara ad hoc. Analisis kebutuhan pada Subbab III.2 karena itu menempatkan kontrak yang seragam lintas lapis sebagai tuntutan yang berlaku pada keseluruhan arsitektur, dan perbandingan pada Subbab III.3 menilai empat alternatif solusi berdasarkan kemampuannya memutus efek domino ini.

III.2 Analisis Kebutuhan

Setelah pemetaan kondisi saat ini, kebutuhan platform dirumuskan dalam dua kategori: kebutuhan fungsional yang menggambarkan fitur yang harus dimiliki platform, dan kebutuhan nonfungsional yang menggambarkan atribut kualitas yang harus dipenuhi. Identifikasi kebutuhan menggunakan empat rumusan masalah dari Bab I sebagai titik awal, dengan mempertimbangkan beban pada Subbab III.1.1 dan Subbab III.1.2 agar rancangan tetap layak operasional dan finansial. Setiap kebutuhan fungsional dipetakan langsung pada satu sub-bidang arsitektur agar tidak terjadi tumpang tindih cakupan antar kebutuhan, sementara setiap kebutuhan nonfungsional disertai target metrik yang dapat diuji pada Bab VI. Identifikasi awal masalah pengguna pada Subbab III.2.1 memberikan rangkuman kebutuhan kontekstual sebelum diturunkan menjadi daftar formal pada Subbab III.2.2 dan Subbab III.2.3.

III.2.1 Identifikasi Masalah Pengguna

Identifikasi masalah pengguna diturunkan dari empat rumusan masalah pada Bab I dengan menelusuri titik sentuh tiap peran pada rantai produksi model. Penelusuran memetakan setiap rumusan masalah ke kontrak antar lapis yang terputus pada lapisan ingestasi, fitur, pelatihan, dan penyajian, lalu mencatat peran yang paling merasakan akibat putusnya kontrak tersebut. Penyusunan sengaja menempatkan kebutuhan pada sambungan antar lapis sebagai dasar, bukan pada preferensi *tool* satu peran tertentu, agar daftar tetap ringkas dan setara untuk keempat peran. Keempat kebutuhan berikut menjadi titik awal penyusunan daftar formal kebutuhan fungsional dan nonfungsional, dan masing-masing dapat dirunut kembali ke rumusan masalah yang melahirkannya serta ke peran yang paling merasakannya.

1. Bidang kendali terpadu lintas peran

Bidang kendali terpadu lintas peran dibutuhkan untuk mengatur ingestasi, pelatihan, penyajian, dan pemantauan, sehingga sambungan antar lapis tidak lagi dirakit secara manual dan integrasi *tool* dapat dipulihkan dari sumber kebenaran tunggal.

2. Tata kelola data yang seragam lintas siklus

Tata kelola yang seragam di antara lapisan data, fitur, dan model dibutuhkan agar *lineage*, katalog, dan validasi kualitas tetap menyatu ketika data berpindah antar lapis, terutama bagi *data engineer* yang menjaga kontrak data sepanjang siklus.

3. Deteksi *drift* otomatis dan jaminan validitas temporal

Deteksi *drift* yang terotomasi sekaligus jaminan *point-in-time correctness* dibutuhkan pada sambungan antara lapis data dan lapis model agar evaluasi model tidak mengandung kebocoran temporal sebagaimana ditegaskan oleh Vela dkk. (2022) dan Kaufman dkk. (2012). Pihak yang paling bergantung pada kontrak ini adalah *data scientist* dan *machine learning engineer* yang menilai kelayakan model sebelum produksi.

4. Dukungan fitur multimodal berlatensi rendah

Dukungan fitur multimodal yang terdiri dari fitur tabular, urutan waktu, dan *embedding* berdimensi tinggi dengan latensi rendah pada saat *inference* dibutuhkan pada lapis penyajian fitur. Kebutuhan ini berakar pada kerja *data scientist* yang merekayasa fitur lintas modalitas.

Empat kebutuhan pengguna tersebut menyiratkan satu pola yang sama, yaitu kebutuhan untuk memutus sambungan ad hoc antar lapis dengan kontrak yang seragam dan dapat dipanggil dari satu antarmuka. Pengguna pada peran *business user* menjadi penerima manfaat terakhir karena hanya melihat keluaran model pada lapis penyajian, sehingga kualitas kontrak pada lapis bawah menentukan apakah keputusan bisnis dapat diambil tepat waktu. Identifikasi tersebut menjadi titik awal untuk menyusun daftar formal kebutuhan fungsional pada Subbab III.2.2 dan kebutuhan nonfungsional pada Subbab III.2.3. Daftar formal tersebut selanjutnya dipetakan ke komponen arsitektur pada Bab IV dan diuji pemenuhannya pada Bab VI, sehingga setiap kebutuhan tetap terhubung dari sumber masalah hingga bukti pemenuhannya.

III.2.2 Kebutuhan Fungsional

Tabel III.1 menyajikan empat belas kebutuhan fungsional yang disusun mengikuti lapisan arsitektur platform, bergerak dari lapisan fitur, lalu siklus hidup model dan tata kelola, hingga lapisan operasional, sehingga penomorannya berurutan menurut lapisan dan bukan menurut tujuan. Empat kebutuhan pertama, yaitu manajemen fitur terpusat, penyajian fitur ganda *online-offline*, jaminan validitas temporal, dan dukungan *vector embedding*, merangkum kontrak pada lapisan fitur. Enam kebutuhan berikutnya menyatukan siklus hidup model dan tata kelola dalam satu rentang,

mencakup otomatisasi *pipeline* pelatihan, otomatisasi *deployment* model, pemantauan *drift* otomatis, tata kelola dan penemuan, pelacakan eksperimen, serta registri model. Empat kebutuhan terakhir, yaitu pemrosesan *batch* dan *stream*, konfigurasi deklaratif, API dan SDK terpadu, serta manajemen sumber daya, merangkum lapisan operasional yang menopang seluruh lapisan di atasnya. Kolom Tujuan pada Tabel III.1 menautkan setiap kebutuhan ke satu dari empat tujuan pada Bab I, pada dimensi yang terpisah dari urutan lapisan berdasarkan *sub-sistem* yang paling bertanggung jawab mewujudkan kebutuhan tersebut, sehingga setiap kebutuhan fungsional memiliki satu tujuan pemilik yang jelas dan tetap dapat dirunut ketika dipetakan ke komponen pada Bab IV dan ke skenario uji pada Bab VI.

Tabel III.1 Kebutuhan Fungsional Sistem

ID	Kebutuhan Fungsional	Deskripsi	Tujuan
KF-01	Manajemen Fitur Terpusat	Sistem menyediakan <i>feature registry</i> terpusat berbasis Feast dengan <i>file-based registry</i> pada MinIO untuk mendefinisikan, mengelola, dan melakukan <i>versioning</i> fitur. <i>Registry</i> menyimpan metadata seperti deskripsi, tipe data, <i>owner</i> , dan dependensi, serta dikelola di dalam repositori Gitea agar mendukung <i>workflow</i> GitOps dan penelusuran perubahan.	T-1
KF-02	Penyajian Fitur Ganda	Sistem mampu melakukan <i>serving</i> fitur dalam dua mode, yaitu <i>online serving</i> berbasis Valkey melalui protokol RESP untuk <i>real-time inference</i> dengan target latensi p99 sub-detik, dan <i>offline serving</i> berbasis ClickHouse untuk <i>historical retrieval</i> berkapasitas besar yang mendukung <i>point-in-time correct join</i> pada data historis.	T-4
KF-03	Jaminan Validitas Temporal	Sistem secara bawaan menjamin <i>point-in-time correctness</i> ketika menyusun <i>training dataset</i> untuk data <i>time-series</i> , sehingga mencegah kebocoran data temporal yang dapat membuat evaluasi dan model menjadi tidak valid.	T-3
KF-04	Dukungan <i>Vector Embeddings</i>	Sistem mendukung penyimpanan, pengindeksan, dan <i>serving vector embeddings</i> pada indeks vektor terpisah berbasis Qdrant dengan <i>similarity search</i> HNSW melalui antarmuka gRPC dan filter <i>payload</i> , dengan target latensi p99 sub-detik untuk volume permintaan menengah.	T-4
KF-05	Otomatisasi <i>Pipeline</i> Pelatihan	Sistem menyediakan <i>pipeline</i> otomatis untuk mengorkestrasi proses pelatihan model dari ujung ke ujung melalui Kubeflow Pipelines, terintegrasi dengan Feast sebagai <i>feature store</i> , MLflow untuk <i>experiment tracking</i> dan <i>model registry</i> , serta Katib untuk <i>hyperparameter tuning</i> , sehingga mendukung otomasi setara MLOps <i>level 2</i> .	T-3

ID	Kebutuhan Fungsional	Deskripsi	Tujuan
KF-06	Otomatisasi <i>Deployment</i> Model	Sistem mengotomatisasi <i>deployment</i> model yang telah lulus validasi ke <i>inference endpoint</i> produksi melalui KServe <i>InferenceService</i> di atas Knative Serving, serta mendukung strategi <i>rolling update</i> , <i>canary deployment</i> , dan <i>progressive delivery</i> berbasis Argo Rollouts dengan mekanisme <i>rollback</i> otomatis ketika terjadi degradasi.	T-1
KF-07	Pemantauan <i>Drift</i> Otomatis	Sistem secara otomatis dan berkala memantau <i>data drift</i> dan <i>concept drift</i> dengan membandingkan distribusi fitur di produksi terhadap distribusi pada saat pelatihan menggunakan uji statistik (misalnya PSI dan Kolmogorov-Smirnov) dengan ambang adaptif, serta memicu pelatihan ulang otomatis melalui Argo CronWorkflow ketika ambang terlampaui.	T-3
KF-08	Tata Kelola & Penemuan Otomatis	Sistem menangkap dan menyajikan metadata serta <i>lineage</i> untuk fitur, <i>dataset</i> , model, dan <i>pipeline</i> pada katalog DataHub melalui ingestasi otomatis dan peristiwa <i>lineage</i> berbasis OpenLineage, sehingga memudahkan penemuan, analisis dampak, dan pemenuhan kebutuhan <i>compliance</i> .	T-2
KF-09	Pelacakan Eksperimen Terintegrasi	Sistem menyediakan <i>experiment tracking</i> yang terintegrasi untuk mencatat parameter, metrik, artefak, dan versi kode, sehingga proses eksperimen dapat direproduksi dan dibandingkan secara sistematis.	T-2
KF-10	<i>Versioning</i> dan Registri Model	Sistem menyediakan <i>model registry</i> terpusat untuk mengelola versi model dan status siklus hidupnya (<i>staging</i> , <i>production</i> , <i>archived</i>) disertai <i>audit trail</i> lengkap untuk mendukung <i>governance</i> .	T-2
KF-11	Pemrosesan <i>Batch</i> dan <i>Stream</i>	Sistem mendukung <i>batch processing</i> untuk analisis dan rekayasa fitur historis serta <i>stream processing</i> untuk komputasi fitur <i>real-time</i> , dan keduanya terhubung dengan <i>feature store</i> sehingga definisi fitur tetap konsisten.	T-4
KF-12	Konfigurasi Deklaratif	Sistem memungkinkan konfigurasi komponen seperti fitur, <i>pipeline</i> , dan <i>deployment</i> dalam format deklaratif (manifes Kubernetes, <i>overlay</i> Kustomize, dan definisi Feast berbasis Python) yang dikontrol versinya di Gitea dan disinkronkan oleh Argo CD sebagai <i>single source of truth</i> .	T-1
KF-13	API dan SDK Terpadu	Sistem menyediakan SDK Python yang seragam (Feast, MLflow, dan klien KServe v2 <i>inference</i>) dalam satu <i>environment</i> uv sebagai antarmuka klien standar untuk membaca fitur <i>online</i> , menelusuri <i>run</i> dan registri model, serta memanggil <i>endpoint inference</i> , sehingga kompleksitas integrasi menurun dan pola pemakaian menjadi konsisten.	T-1

ID	Kebutuhan Fungsional	Deskripsi	Tujuan
KF-14	Manajemen Sumber Daya	Sistem menyediakan kemampuan pengelolaan sumber daya komputasi yang efisien melalui Kueue untuk <i>job queuing</i> , prioritas, dan kuota berbasis ClusterQueue, dipadukan dengan HPA, VPA, dan KEDA untuk <i>dynamic provisioning</i> pada lingkungan <i>multi-tenant</i> agar tidak ada satu <i>workload</i> yang menghabiskan sumber daya klaster.	T-1

Penomoran KF-01 sampai KF-14 dipakai konsisten pada Bab IV ketika setiap kebutuhan dipetakan ke komponen, pada Bab V ketika setiap kebutuhan dipetakan ke berkas manifes, dan pada Bab VI ketika setiap kebutuhan dipetakan ke skenario uji. Pemetaan satu lawan satu antara kebutuhan fungsional dan sub-sistem dipilih untuk menghindari tumpang tindih cakupan, sehingga jika satu kebutuhan tidak terpenuhi maka komponen yang bertanggung jawab dapat segera ditemukan tanpa perlu menelusuri rantai panjang. Mayoritas kebutuhan berprioritas tinggi karena menyangkut kontrak inti antar lapis, sedangkan KF-13 dan KF-14 bersifat penunjang yang memperbaiki pengalaman integrasi dan efisiensi sumber daya tanpa menghalangi fungsi utama. Kebutuhan pada lapisan fitur secara khusus merangkum kontrak *point-in-time correctness* dan dukungan multimodal yang menjadi pembeda utama dari pendekatan tradisional yang hanya menyajikan fitur tabular dari satu sumber.

III.2.3 Kebutuhan Nonfungsional

Tabel III.2 menyajikan dua belas kebutuhan nonfungsional yang berperan sebagai pembatas kualitas dan disusun mengikuti taksonomi atribut kualitas alih-alih menurut tujuan. Kinerja latensi rendah dan *throughput* tinggi menyangkut performa penyajian fitur dan *stream processing*. Skalabilitas horizontal, keandalan, dan toleransi kesalahan menyangkut kemampuan platform mengikuti pertumbuhan beban. Konsistensi dan kebenaran menegakkan kontrak kualitas data sepanjang *pipeline*, mencakup kesesuaian skema dan validitas nilai. *Observability* mengikat metrik, log, dan *trace* pada satu antarmuka. Keamanan mengikat autentikasi, otorisasi, enkripsi, dan *audit logging*. Ekstensibilitas dan modularitas memberi ruang bagi penggantian komponen tanpa mengganggu komponen lain. Kemudahan penggunaan, efisiensi sumber daya, portabilitas, dan kemudahan pemeliharaan menutup daftar dengan menyoroti aspek operasional jangka panjang. Penetapan target pada KNF skalabilitas horizontal mengacu pada taksonomi karakteristik skalabilitas dari Bondi (2000), yang membedakan beberapa dimensi skalabilitas, di antaranya skalabilitas beban dan skalabilitas struktural, sebagai faktor yang memengaruhi kinerja ketika beban

bertumbuh.

Tabel III.2 Kebutuhan Nonfungsional Sistem

ID	Kebutuhan Nonfungsional	Deskripsi	Target Metrik	Tujuan
KNF-01	Kinerja Latensi Rendah	<i>Online feature serving</i> berbasis Valkey dan <i>vector similarity search</i> berbasis Qdrant harus memberikan latensi yang stabil dan rendah untuk mendukung <i>inference</i> waktu nyata pada berbagai domain aplikasi, termasuk pada saat beban tinggi.	Latensi p99 sub-detik pada <i>feature serving</i> dan <i>vector search</i> , dengan ambang spesifik per layanan diukur dan dipantau melalui SLO Sloth	T-4
KNF-02	Throughput Tinggi	Sistem harus mampu menangani <i>workload</i> dengan <i>throughput</i> tinggi yang umum pada beban kerja <i>streaming</i> dan <i>serving</i> bervolume tinggi, dengan karakteristik skalabilitas mendekati linear ketika sumber daya ditambah.	<i>Throughput ingest</i> dan <i>serving</i> tumbuh proporsional terhadap jumlah <i>partition</i> Kafka serta <i>replica</i> Feast melalui <i>auto-scaling</i> KEDA dan HPA	T-4
KNF-03	Skalabilitas Horizontal	Seluruh komponen sistem harus mendukung skala horizontal, sehingga kapasitas dapat ditingkatkan dengan menambah <i>node</i> dan menghasilkan peningkatan kinerja yang linear atau mendekati linear.	Seluruh <i>workload</i> terstandarisasi pada HPA (CPU/memori), VPA <i>recommender</i> , dan KEDA (Kafka <i>lag</i> , <i>custom metrics</i>)	T-1
KNF-04	Keandalan & Toleransi Kesalahan	Sistem harus andal dengan komponen <i>stateful</i> (Kafka, ClickHouse, PostgreSQL, MinIO, Valkey, Qdrant) yang dapat berjalan dalam mode <i>multi-replica</i> pada lingkungan produksi <i>multi-node</i> , dan mampu melakukan pemulihan otomatis dari kegagalan sementara melalui mekanisme <i>operator</i> dan <i>snapshot</i> berkala. Pada lingkungan verifikasi <i>node</i> tunggal, <i>replica idle</i> ditetapkan menjadi satu dan ditingkatkan secara dinamis ketika beban nyata terdeteksi.	SLO <i>platform</i> dijaga melalui Sloth (Kafka 99,9%, Postgres 99,95%, MinIO p99 <500ms pada 99%), sedangkan <i>disaster recovery</i> berbasis Velero <i>snapshot</i> harian penuh dan <i>incremental</i> per <i>use case</i>	T-1

ID	Kebutuhan Nonfungsional	Deskripsi	Target Metrik	Tujuan
KNF-05	Konsistensi & Kebenaran	Sistem harus menegakkan kontrak kualitas data sepanjang <i>pipeline</i> , mencakup kesesuaian skema, validitas rentang nilai, dan kelengkapan data, sehingga hasil analisis maupun <i>inference</i> model bertumpu pada data yang sah dan konsisten.	Seluruh <i>checkpoint</i> Great Expectations lolos tanpa pelanggaran skema maupun rentang nilai pada data yang diproses ke lapis <i>gold</i>	T-2
KNF-06	Observability	Sistem harus memiliki <i>observability</i> menyeluruh, mencakup metrik (Prometheus dan Grafana), <i>structured log</i> (Loki), <i>distributed tracing</i> (Grafana Tempo melalui OpenTelemetry), <i>continuous profiling</i> (Pyroscope), dan <i>alerting</i> (Alertmanager dan Sloth) sehingga perilaku sistem dapat dianalisis dari ujung ke ujung.	Seluruh komponen terinstrumentasi metrik dan log standar, <i>trace</i> OpenTelemetry untuk lintasan kritis, serta SLO Sloth untuk indikator layanan	T-2
KNF-07	Keamanan	Sistem harus menerapkan kontrol keamanan menyeluruh, mencakup autentikasi tunggal melalui dex sebagai <i>identity provider</i> OIDC dan oauth2-proxy, otorisasi berbasis RBAC dan SpiceDB, enkripsi data <i>in transit</i> dengan Istio mTLS <i>strict</i> , enkripsi <i>at rest</i> pada MinIO melalui KES sebagai <i>KMS gateway</i> ke OpenBao, kebijakan admisi Kyverno, serta deteksi ancaman <i>runtime</i> oleh Falco dan pemindaian kerentanan <i>image</i> oleh Trivy Operator.	TLS 1.3 <i>in transit</i> , AES-256 <i>at rest</i> , dan <i>audit log</i> OpenBao serta Kubernetes <i>audit policy</i> aktif pada seluruh bidang kendali	T-2
KNF-08	Ekstensibilitas & Modularitas	Arsitektur harus modular sehingga komponen dapat diperluas atau diganti melalui antarmuka terdefinisi (<i>Custom Resource Definition</i> , helm <i>chart</i> , dan <i>overlay</i> Kustomize) tanpa mengganggu komponen lain.	Setiap komponen disusun pada <i>folder</i> terpisah dengan <i>kustomization.yaml</i> sendiri, sehingga dapat dipasang, diperbarui, atau diganti secara independen	T-1

ID	Kebutuhan Nonfungsional	Deskripsi	Target Metrik	Tujuan
KNF-09	Kemudahan Penggunaan	Antarmuka konsol pengembang dan operator platform harus dapat diakses dari satu titik autentikasi tunggal sehingga konsol Argo CD, Argo Workflows, Kubeflow, MLflow, DataHub, dan Grafana terhubung melalui satu sesi identitas, dengan dokumentasi referensi yang ringkas untuk konfigurasi setiap komponen.	Seluruh antarmuka konsol platform terhubung pada satu jalur autentikasi dex dan oauth2-proxy tanpa konfigurasi identitas tambahan per layanan, serta tiap komponen memiliki dokumentasi referensi pada repositori Gitea	T-1
KNF-10	Efisiensi Sumber Daya	Sistem harus memanfaatkan sumber daya komputasi dan penyimpanan secara efisien untuk mengendalikan biaya tanpa mengorbankan kinerja, baik pada lingkungan <i>node</i> tunggal maupun multi- <i>node</i> .	Kompresi kolumnar pada ClickHouse, <i>autoscaling</i> berbasis HPA, VPA, dan KEDA agar utilisasi sumber daya proporsional terhadap beban, serta atribusi biaya per <i>workload</i> melalui OpenCost	T-1
KNF-11	Portabilitas	Sistem harus dapat dijalankan di berbagai lingkungan <i>deployment</i> dengan perubahan terbatas pada konfigurasi dan tanpa modifikasi kode besar, sebab seluruh komponen berdiri di atas distribusi Kubernetes standar dan <i>image open source</i> .	Dapat di- <i>deploy</i> pada distribusi Kubernetes <i>managed</i> (AWS EKS, GCP GKE, Azure AKS) maupun <i>on-premise</i> (k3s, kubeadm) hanya dengan penyesuaian <i>overlay</i> Kustomize dan parameter <i>secret</i>	T-1
KNF-12	Kemudahan Pemeliharaan	Sistem harus mudah dipelihara, dengan struktur kode dan manifest yang jelas, <i>pipeline</i> CI yang otomatis pada Tekton, serta <i>deployment</i> deklaratif dari sumber kebenaran tunggal pada Gitea melalui Argo CD.	Setiap komponen dapat direkonsiliasi ulang dari <i>repository</i> Gitea tanpa intervensi manual, dan perubahan ditolak otomatis oleh <i>pull request review</i> dan <i>policy</i> Kyverno bila tidak sesuai konvensi	T-1

Setiap KNF disertai target metrik yang dapat diuji secara langsung pada Bab VI, sehingga klaim kualitas tidak berhenti pada deskripsi tetapi memiliki ambang batas yang konkret. Pemilihan ambang batas pada KNF latensi dan *throughput* mengikuti karakteristik penyajian fitur *online*, tempat *lookup* fitur harus diselesaikan dalam waktu singkat agar layanan *inference* tetap responsif terhadap data yang bergerak cepat. Pemilihan ambang batas pada KNF observabilitas mengikuti praktik yang dirangkum oleh Cloud Native Computing Foundation (2024b) bahwa metrik, log, dan *trace* harus dapat dirujuk silang pada satu antarmuka agar diagnosis dapat ditegakkan tanpa berpindah *tool*. Berbeda dari kebutuhan fungsional yang dapat dipetakan satu lawan satu ke tujuan, kebutuhan nonfungsional bersifat lintas-bidang sehingga satu atribut kualitas dapat menopang lebih dari satu tujuan sekaligus. Meskipun demikian, setiap KNF tetap memiliki tujuan utama tempat target metriknya paling langsung diuji, dan kolom Tujuan pada Tabel III.2 menautkan setiap kebutuhan nonfungsional ke tujuan utama tersebut. Konsekuensinya, tujuan ketiga yang menyangkut deteksi *drift* terotomasi dan *point-in-time correctness* tidak menjadi pemilik utama satu pun KNF, sebab kapabilitasnya ditegakkan secara fungsional melalui KF-03, KF-05, dan KF-07, sedangkan dimensi kualitasnya tertopang lintas-bidang oleh KNF-05 pada sisi kebenaran data dan KNF-06 pada sisi pemantauan sinyal *drift*.

III.3 Analisis Pemilihan Solusi

Bagian ini menyusun alternatif solusi yang dapat memenuhi kebutuhan fungsional dan nonfungsional, kemudian memilih solusi terbaik dengan menggunakan skor pemenuhan kebutuhan. Penilaian skor dilakukan terhadap empat alternatif dengan menggunakan dua belas KNF dan empat belas KF sebagai kriteria, sehingga total skor maksimum sebuah alternatif adalah lima puluh dua. Setiap skor ditetapkan dengan memetakan kapabilitas terdokumentasi tiap alternatif, sebagaimana ditinjau pada Bab II, terhadap definisi kebutuhan pada Subbab III.2, sehingga penilaian bertumpu pada bukti kapabilitas dan bukan pada preferensi penulis. Alternatif yang memperoleh skor tertinggi akan dirancang lebih lanjut pada Bab IV dan diimplementasikan pada Bab V, dengan justifikasi tertulis pada kelompok kebutuhan yang membedakan peringkat antar alternatif.

III.3.1 Alternatif Solusi

Empat alternatif solusi dipertimbangkan untuk membangun platform DataOps dan MLOps terintegrasi. Pemilihan empat alternatif didasarkan pada pemetaan pende-

katan yang umum dipakai oleh organisasi yang menerapkan MLOps pada skala produksi, mencakup pendekatan terkelola penuh, pendekatan *open source* di atas Kubernetes, pendekatan kustom dari awal, dan pendekatan hibrida antara terkelola dan *open source*. Setiap alternatif diuraikan dengan kelebihan, kekurangan, dan rujukan pustaka yang relevan agar pertimbangan dapat ditelusuri kembali. Keempat alternatif dinilai terhadap dua puluh enam kebutuhan yang sama dari Subbab III.2 sehingga perbandingannya dapat diaudit kriteria demi kriteria.

1. Alternatif A: Layanan Terkelola dari Penyedia *Cloud*

Pendekatan ini menggunakan rangkaian layanan terkelola pada satu penyedia, mencakup layanan ingestasi, *warehouse*, *feature store*, pelatihan, dan penyajian. Kelebihannya berupa kecepatan adopsi dan integrasi bawaan antar layanan, sebagaimana dibahas oleh Li dkk. (2023). Kekurangannya berupa keterikatan terhadap satu penyedia, biaya operasional pada beban tinggi, dan keterbatasan dalam menyesuaikan komponen pada level rendah, sehingga pertukaran waktu lawan anggaran pada Subbab III.1.2 tetap menjadi beban berjalan.

2. Alternatif B: Komponen *Open Source* pada Kubernetes

Pendekatan ini merangkai komponen *open source* di atas Kubernetes sebagai bidang orkestrasi. Lakka (2025) dan Burgueño-Romero dkk. (2025) memperlihatkan rancangan serupa yang menjadikan Kubernetes sebagai fondasi orkestrasi bagi komponen *open source*. Kelebihannya berupa fleksibilitas, transparansi versi, portabilitas, dan kemampuan menegakkan pola GitOps. Kekurangannya berupa kebutuhan tata kelola yang lebih ketat dan kurva belajar yang lebih panjang dibandingkan layanan terkelola, sehingga beban konfigurasi dari nol pada Subbab III.1.1 perlu dikurangi melalui rancangan referensi yang utuh.

3. Alternatif C: Pengembangan Kustom dari Awal

Pendekatan ini membangun setiap lapisan secara mandiri tanpa banyak menggunakan komponen yang sudah jadi. Kelebihannya berupa kontrol penuh atas perilaku setiap komponen. Kekurangannya berupa biaya pengembangan dan pemeliharaan yang sangat besar, ketergantungan pada tim yang sangat kompeten, dan risiko menciptakan ulang masalah yang sudah diselesaikan oleh pustaka *open source* yang teruji.

4. Alternatif D: Hibrida antara Layanan Terkelola dan *Open Source*

Pendekatan ini menggunakan beberapa layanan terkelola, terutama pada lapisan dasar seperti penyimpanan dan jaringan, dipadukan dengan komponen

open source pada lapisan DataOps dan MLOps. Kelebihannya berupa keseimbangan antara kecepatan adopsi dan fleksibilitas. Kekurangannya berupa kompleksitas integrasi antara dua jenis komponen dan kebutuhan kontrak antar lapisan yang harus dirancang dengan saksama.

Keempat alternatif tersebut sengaja dipilih agar mencakup seluruh rentang pertukaran waktu lawan anggaran yang dibahas pada Subbab III.1.2. Alternatif A dan D menukar waktu pengembangan dengan biaya berlangganan yang berjalan terus, Alternatif C menukar biaya berlangganan dengan waktu pengembangan yang sangat panjang, sedangkan Alternatif B berupaya menekan keduanya melalui rancangan referensi yang dapat dipasang ulang. Posisi relatif keempatnya diukur secara kuantitatif pada Subbab III.3.2 memakai skor pemenuhan kebutuhan fungsional dan nonfungsional. Penilaian kuantitatif itu menjadi dasar keputusan akhir sehingga pemilihan solusi tidak bertumpu pada preferensi melainkan pada bukti pemenuhan kebutuhan.

III.3.2 Analisis Penentuan Solusi

Tabel III.3 menyajikan evaluasi pemenuhan kebutuhan fungsional, Tabel III.4 menyajikan evaluasi pemenuhan kebutuhan nonfungsional, dan Tabel III.5 merekapitulasi total skor gabungan keduanya. Dua tabel pertama memuat skor untuk setiap kebutuhan pada keempat alternatif, sedangkan tabel rekapitulasi hanya menjumlahkan keduanya menjadi skor akhir yang menjadi dasar keputusan. Alternatif B memperoleh skor total tertinggi, yaitu 49 dari 52, diikuti oleh Alternatif A dan Alternatif D dengan skor 43, dan Alternatif C dengan skor 30. Selisih skor antara Alternatif B dan dua pesaing terdekatnya sebesar enam poin terutama berasal dari KF jaminan validitas temporal, KF dukungan *vector embeddings*, KNF portabilitas, dan KNF ekstensibilitas yang tidak dipenuhi penuh oleh pendekatan terkelola maupun hibrida.

Alternatif B unggul terutama pada kebutuhan yang menyangkut jaminan validitas temporal, dukungan *vector embeddings*, pemantauan *drift* otomatis, ekstensibilitas, dan portabilitas. Alternatif A unggul pada kemudahan penggunaan dan kemudahan pemeliharaan, tetapi tertinggal pada validitas temporal, dukungan *vector embeddings* pada *feature store*, dan portabilitas. Alternatif C unggul pada kontrol penuh tetapi kalah pada hampir semua kebutuhan operasional. Alternatif D mendekati Alternatif A tetapi tertinggal pada portabilitas dan ekstensibilitas. Skor nol pada Alternatif A untuk ekstensibilitas dan portabilitas mencerminkan keterikatan pada format kepemilikan tertutup yang menyulitkan penggantian komponen dan migrasi

Tabel III.3 Evaluasi Pemenuhan Kebutuhan Fungsional

Kebutuhan	Cloud	Open-Source	Kustom	Hibrida
KF-01: Manajemen Fitur Terpusat	2	2	1	2
KF-02: Penyajian Fitur Ganda	2	2	1	2
KF-03: Jaminan Validitas Temporal	1	2	2	1
KF-04: Dukungan <i>Vector Embeddings</i>	1	2	2	1
KF-05: Otomatisasi <i>Pipeline</i> Pelatihan	2	2	1	2
KF-06: Otomatisasi <i>Deployment</i> Model	2	2	1	2
KF-07: Pemantauan <i>Drift</i> Otomatis	1	2	1	1
KF-08: Tata Kelola & Penemuan	2	2	1	2
KF-09: Pelacakan Eksperimen	2	2	1	2
KF-10: <i>Versioning</i> Registri Model	2	2	1	2
KF-11: Pemrosesan <i>Batch/Stream</i>	2	2	1	2
KF-12: Konfigurasi Deklaratif	2	2	1	2
KF-13: API dan SDK Terpadu	2	1	1	1
KF-14: Manajemen Sumber Daya	2	2	1	2
Total Skor KF	25	27	16	24

Keterangan skor: 0 = tidak memenuhi sama sekali, 1 = memenuhi sebagian, 2 = memenuhi seluruhnya.

Tabel III.4 Evaluasi Pemenuhan Kebutuhan Nonfungsional

Kebutuhan	Cloud	Open-Source	Kustom	Hibrida
KNF-01: Kinerja Latensi Rendah	2	2	2	2
KNF-02: <i>Throughput</i> Tinggi	2	2	2	2
KNF-03: Skalabilitas Horizontal	2	2	1	2
KNF-04: Keandalan & Toleransi Kesalahan	2	2	1	2
KNF-05: Konsistensi & Kebenaran	1	2	2	1
KNF-06: <i>Observability</i>	2	2	1	2
KNF-07: Keamanan	2	2	1	2
KNF-08: Ekstensibilitas & Modularitas	0	2	2	1
KNF-09: Kemudahan Penggunaan	2	1	0	2
KNF-10: Efisiensi Sumber Daya	1	2	1	1
KNF-11: Portabilitas	0	2	1	1
KNF-12: Kemudahan Pemeliharaan	2	1	0	1
Total Skor KNF	18	22	14	19

Keterangan skor: 0 = tidak memenuhi sama sekali, 1 = memenuhi sebagian, 2 = memenuhi seluruhnya.

Tabel III.5 Total Skor Pemenuhan Kebutuhan Fungsional dan Nonfungsional

Alternatif	Skor KF (maks 28)	Skor KNF (maks 24)	Total (maks 52)
A: Layanan Terkelola <i>Cloud</i>	25	18	43
B: <i>Open Source</i> pada Kubernetes	27	22	49
C: Kustom dari Awal	16	14	30
D: Hibrida Terkelola dan <i>Open Source</i>	24	19	43

antar penyedia, sedangkan skor nol pada Alternatif C untuk kemudahan penggunaan dan kemudahan pemeliharaan mencerminkan beban membangun dan merawat setiap lapisan tanpa komponen siap pakai. Berdasarkan skor pemenuhan kebutuhan dan analisis komparatif yang sejalan dengan Burgueño-Romero dkk. (2025), Amou

Najafabadi dkk. (2024), dan Lakka (2025), Alternatif B dipilih sebagai solusi yang akan dirancang pada Bab IV dan diimplementasikan pada Bab V.

III.3.3 Posisi terhadap Penelitian Terkait

Alternatif B yang terpilih diwujudkan sebagai platform yang menggabungkan tiga karakter yang belum dilingkupi sekaligus pada penelitian sebelumnya: integrasi DataOps dan MLOps pada satu bidang kendali Kubernetes, layanan fitur *dual-store* dengan dukungan vektor HNSW, dan deteksi *drift* terotomasi yang memicu pelatihan ulang melalui kontrol GitOps. Hsu dkk. (2024) dan Liang dkk. (2024) memberikan basis komparatif untuk menilai posisi tersebut. Platform ini menambahkan kontrak *lineage* terbuka melalui OpenLineage dan kontrak kualitas data melalui Great Expectations yang dijalankan pada *pipeline* produksi. Tabel III.6 merinci perbandingan tiap aspek arsitektur antara kondisi saat ini dan solusi terpilih beserta rujukan penelitian yang mendasari setiap peningkatan, sehingga menegaskan bagaimana platform menutup kesenjangan yang teridentifikasi pada Bab II.

Tabel III.6 Perbandingan Detail Arsitektur Saat Ini dengan yang Diusulkan

Aspek	Arsitektur Saat Ini	Arsitektur Diusulkan	Manfaat yang Diharapkan	Referensi
Orkestrasi	<i>Compute</i> heterogen dengan <i>provisioning</i> manual, waktu tunggu <i>scaling</i> panjang, dan tingkat standardisasi rendah.	Platform Kubernetes terpadu dengan manajemen deklaratif dan alokasi sumber daya otomatis, sejalan dengan komponen arsitektur MLOps yang diidentifikasi Kreuzberger, Kühl, dan Hirschl (2023). GitOps dengan Argo CD memastikan perubahan infrastruktur dan aplikasi mengikuti <i>single source of truth</i> di Git.	Lakka (2025) menunjukkan bahwa adopsi Kubernetes pada perusahaan disertai praktik DevOps yang matang meningkatkan <i>deployment frequency</i> dan menekan <i>Mean Time To Recovery</i> (MTTR) secara signifikan, dan arsitektur yang diusulkan menargetkan karakteristik performa pada arah yang sama.	(Kreuzberger, Kühl, dan Hirschl 2023; Lakka 2025)

Aspek	Arsitektur Saat Ini	Arsitektur Diusulkan	Manfaat yang Diharapkan	Referensi
Manajemen Fitur	Logika fitur tersebar di <i>notebook</i> dan skrip dengan duplikasi tinggi dan banyak fungsi yang saling tumpang tindih.	Feat dengan <i>feature registry</i> terpusat, <i>dual serving offline/online</i> , serta dukungan <i>point-in-time correctness</i> .	Duplikasi fitur dapat ditekan, <i>training-serving skew</i> berkurang melalui pemisahan <i>offline</i> dan <i>online</i> serta <i>point-in-time retrieval</i> , dan <i>feature reuse</i> berpotensi meningkat signifikan.	(Munappy dkk. 2022; Polyzotis dkk. 2018)
<i>Vector Embeddings</i>	Fitur <i>embedding</i> tidak didukung secara <i>native</i> atau diimplementasikan secara kustom dengan latensi tinggi dan <i>throughput</i> terbatas.	Qdrant sebagai indeks vektor terpisah dari <i>online store</i> tabular Valkey, dengan HNSW <i>native</i> , <i>API gRPC</i> , dan filter <i>payload</i> .	Johnson, Douze, dan Jégou (2021) menunjukkan bahwa indeks vektor berbasis HNSW maupun FAISS dapat melayani <i>billion-scale similarity search</i> dengan latensi rendah. Desain ini memakai Qdrant agar fitur <i>embedding</i> dilayani secara <i>online</i> tanpa membebani <i>online store</i> kunci-nilai dan tanpa membangun layanan kustom.	(Johnson, Douze, dan Jégou 2021; Qdrant Team 2025)
Validitas Temporal	<i>Point-in-time join</i> dilakukan manual dan rentan kesalahan, sehingga banyak tim mengalami <i>data leakage</i> .	<i>Point-in-time join</i> otomatis dari Feast dengan pola verifikasi yang lebih sistematis.	Pendekatan ini mencegah kebocoran data temporal yang dapat menghasilkan estimasi performa yang terlalu optimistis saat evaluasi dan tidak realistis di produksi.	(Polyzotis dkk. 2018)

Aspek	Arsitektur Saat Ini	Arsitektur Diusulkan	Manfaat yang Diharapkan	Referensi
Pemrosesan Data	<i>Batch dan stream</i> diproses oleh <i>pipeline</i> terpisah tanpa arsitektur yang koheren, sehingga hasil pemrosesan tidak konsisten.	Spark sebagai mesin <i>batch processing</i> dan Flink sebagai mesin <i>stream processing</i> yang hasilnya dimaterialisasikan langsung ke <i>Feature Store</i> . Airflow mengorkestrasi <i>pipeline</i> transformasi data sebagai DAG sehingga pengembangan dan penjadwalan transformasi menjadi lebih sistematis.	Waktu pengembangan <i>pipeline</i> dapat menurun dan <i>freshness</i> data meningkat berkat pemrosesan <i>streaming</i> yang lebih efisien dan terintegrasi.	(Rodrigues dkk. 2025)
Pelatihan Model	<i>Workflow</i> pelatihan berbasis <i>notebook</i> dengan <i>tracking</i> eksperimen informal dan <i>artifact</i> tidak terkelola.	Kubeflow <i>Pipelines</i> untuk orkestrasi pelatihan dan MLflow untuk <i>tracking</i> eksperimen.	Reprodusibilitas meningkat melalui kontainerisasi dan pengelolaan eksperimen terpusat, sementara siklus pengembangan <i>pipeline</i> menjadi lebih terstruktur.	(Amershi dkk. 2019; Pimentel dkk. 2019)

Aspek	Arsitektur Saat Ini	Arsitektur Diusulkan	Manfaat yang Diharapkan	Referensi
Deployment Model	Proses <i>deployment</i> manual dengan <i>refactoring</i> besar dan sering terjadi <i>bug</i> pasca- <i>deploy</i> .	KServe dengan <i>InferenceService</i> deklaratif dan <i>deployment</i> otomatis melalui GitOps di atas Knative Serving. OpenTelemetry mengumpulkan log dan <i>trace inference</i> ke Loki dan Grafana Tempo sebagai dasar pemantauan dan analitik lanjutan.	Waktu <i>deployment</i> berpotensi turun secara signifikan, dan insiden terkait kesalahan konfigurasi <i>deployment</i> dapat dikurangi lewat otomasi.	(Paleyes, Urma, dan Lawrence 2022)
Pola Deployment Lanjut	Pola seperti <i>canary release</i> jarang digunakan, organisasi umumnya menggunakan <i>binary switchover</i> .	Dukungan <i>canary release</i> , A/B <i>testing</i> , dan <i>automatic rollback</i> prediktif.	<i>Rollout</i> dapat dilakukan secara bertahap dan aman, sehingga MTTR menurun karena kegagalan dapat dibatasi pada <i>canary traffic</i> .	(Shankar dkk. 2022)
Deteksi Drift	Deteksi <i>drift</i> bersifat reaktif dengan jeda deteksi yang panjang terhadap isu penting dan perubahan gradual.	<i>Monitoring</i> proaktif dengan uji statistik, <i>alert real-time</i> , dan penetapan ambang adaptif.	Jeda deteksi <i>drift</i> dapat dipersingkat sehingga pelatihan ulang dapat dijadwalkan lebih tepat waktu untuk menjaga kinerja model.	(Céspedes Sisniega dkk. 2024; Bayram, Ahmed, dan Kassler 2022)
Tata Kelola & Lineage	Dokumentasi dilakukan secara manual dan <i>audit trail</i> sering tidak lengkap atau tersebar.	DataHub dengan <i>metadata ingestion</i> otomatis dan <i>lineage</i> yang dapat ditelusuri dari <i>source</i> hingga model, dibantu OpenLineage sebagai protokol pengirim peristiwa lintas <i>engine</i> pemrosesan.	Waktu persiapan audit dapat berkurang dan cakupan metadata meningkat melalui proses <i>ingestion</i> dan <i>lineage</i> otomatis.	(Stoyanovich, Howe, dan Jagadish 2020; Lamer dkk. 2024)

Aspek	Arsitektur Saat Ini	Arsitektur Diusulkan	Manfaat yang Diharapkan	Referensi
<i>Observability</i>	Fokus pemantauan pada metrik infrastruktur, tanpa cakupan metrik ML spesifik di sebagian besar organisasi.	<i>Observability</i> mencakup metrik ML, dasbor kinerja model, dan <i>alert</i> berbasis indikator <i>drift</i> . Prometheus dan Grafana menjadi inti, OpenTelemetry mengalirkan log dan <i>trace</i> ke Loki dan Grafana Tempo, sementara Alertmanager mengonsolidasikan <i>alert</i> ke berbagai kanal notifikasi.	Proses <i>debugging</i> dan <i>root cause analysis</i> menjadi lebih efektif sehingga MTTR dapat diturunkan.	(Shankar dkk. 2022)
Integrasi CI/CD	CI/CD untuk <i>pipeline</i> dan model dilakukan secara manual atau semi-otomatis, dengan alur kerja yang tidak seragam.	GitOps dengan Argo CD sebagai mekanisme <i>continuous deployment</i> deklaratif, dilengkapi Tekton untuk <i>continuous integration</i> dan Argo Rollouts untuk <i>progressive delivery</i> , dengan Git sebagai sumber kebenaran tunggal yang mencakup seluruh konfigurasi DataOps dan MLOps.	Frekuensi <i>deployment</i> dapat meningkat dan variasi kesalahan <i>deployment</i> dapat ditekan melalui otomasi dan <i>peer review</i> pada <i>pull request</i> .	(Kreuzberger, Kühl, dan Hirschl 2023)

Aspek	Arsitektur Saat Ini	Arsitektur Diusulkan	Manfaat yang Diharapkan	Referensi
Pengalaman <i>Developer</i>	<i>Developer</i> menghabiskan banyak waktu untuk tugas repetitif, <i>onboarding</i> lama, dan penggunaan alat yang terpisah.	Platform terintegrasi dengan <i>self-service</i> untuk data, fitur, <i>pipeline</i> , dan akses. Dex sebagai <i>identity provider</i> OIDC dan oauth2-proxy menyatukan autentikasi tunggal di seluruh antarmuka pengembang.	<i>Onboarding</i> dapat dipercepat dan produktivitas <i>data scientist</i> meningkat karena hambatan akses infrastruktur berkurang.	(Paleyes, Urma, dan Lawrence 2022; Rella 2022)
Efisiensi Biaya	Utilisasi sumber daya tidak optimal dan tidak ada atribusi biaya yang jelas ke <i>workload</i> .	<i>Auto-scaling</i> berbasis HPA, VPA, dan KEDA, pemantauan biaya melalui OpenCost, serta alokasi sumber daya yang lebih terukur pada kluster Kubernetes. Seluruh komponen inti berbasis <i>open source</i> sehingga biaya lisensi dapat ditekan.	Penghematan biaya signifikan dapat dicapai melalui optimasi pemakaian sumber daya dan <i>auto-scaling</i> , sebagaimana ditunjukkan dalam berbagai studi kasus adopsi Kubernetes (Lakka 2025). Desain ini mengadopsi prinsip yang sama untuk mengoptimalkan pemakaian sumber daya.	(Lakka 2025)

Aspek	Arsitektur Saat Ini	Arsitektur Diusulkan	Manfaat yang Diharapkan	Referensi
Keamanan & Compliance	Kontrol akses beragam di tiap sistem, dan <i>audit</i> tersebar tanpa pandangan terpadu.	RBAC terintegrasi, <i>audit trail</i> komprehensif, dan pengecekan <i>compliance</i> yang terotomatisasi. Dex sebagai <i>identity provider</i> OIDC memusatkan identitas, oauth2-proxy menegakkan autentikasi pada antarmuka aplikasi, Kyverno menegakkan kebijakan admisi pada klaster, OpenBao mengelola <i>secrets</i> , sementara Falco dan Trivy Operator memantau ancaman <i>runtime</i> dan kerentanan <i>image</i> .	Risiko insiden keamanan dapat berkurang melalui kontrol terpusat dan proses audit menjadi lebih ringkas.	(Ahmad dkk. 2024)

Perbandingan pada Tabel III.6 menempatkan setiap aspek arsitektur pada dua kondisi, yaitu praktik yang terdokumentasi pada penelitian terdahulu dan kondisi sesudah solusi terpilih diterapkan. Setiap baris menautkan peningkatan yang diklaim dengan rujukan penelitian yang mendasarinya sehingga posisi platform dapat diperiksa aspek demi aspek. Aspek tanpa dasar pada penelitian terdahulu tidak dimasukkan agar perbandingan tetap berpijak pada sumber. Rincian perwujudan tiap aspek menjadi bahan pemilihan komponen pada Subbab III.4 dan perancangan pada Bab IV.

III.4 Analisis Pemilihan Open Source Tools

Subbab ini menurunkan pemilihan komponen konkret untuk Alternatif B, yaitu rangkaian alat *open source* di atas Kubernetes yang terpilih pada Subbab III.3.2. Pembahasan disusun mengikuti lapis arsitektur yang diperkenalkan pada Bab II, sehingga

ga tiap alat dinilai pada konteks konsep yang telah dijelaskan sebelumnya. Pengelompokan lapis merujuk pada dekomposisi komponen teknis MLOps yang disusun Kreuzberger, Kühl, dan Hirschl (2023), yang mengidentifikasi sembilan komponen teknis mulai dari CI/CD, orkestrasi alur kerja, *feature store*, infrastruktur pelatihan, *model registry*, dan *metadata store*, hingga penyajian dan pemantauan model, dipadukan dengan lapis manajemen data pada praktik DataOps yang dibahas Jain dan Das (2025) dan Rella (2022). Amou Najafabadi dkk. (2024) menegaskan bahwa pemetaan eksplisit antara alat dan komponen arsitektur masih menjadi celah pada literatur MLOps, dan subbab ini menyediakan pemetaan tersebut untuk konteks platform *open source*. Sembilan kelompok lapis pada subbab ini sengaja lebih rinci daripada tujuh lapis rancangan karena mengikuti urutan konsep pada Bab II untuk keperluan penilaian alat, kemudian dikonsolidasikan menjadi tujuh lapis arsitektur pada Subbab IV.2 yang dirunut ke enam kategori komponen pada kajian yang sama.

Setiap lapis yang memiliki lebih dari satu kandidat layak dinilai melalui tabel evaluasi dengan skala seragam 0 sampai 2 untuk tiap kriteria, dengan makna umum tiap nilai diberikan pada keterangan skor di bawah masing-masing tabel dan parameter penilaian tiap kriteria dirinci pada Lampiran B. Nilai 0 menandai kriteria yang tidak dipenuhi atau fitur yang tidak tersedia, nilai 1 menandai dukungan parsial yang masih menuntut komponen tambahan atau memikul keterbatasan berarti, dan nilai 2 menandai pemenuhan penuh oleh kapabilitas bawaan. Penilaian diturunkan dari dokumentasi resmi dan status pemeliharaan tiap proyek, sedangkan kolom Total menjumlahkan keempat kriteria sehingga nilai tertingginya delapan. Kolom Lisensi (Yayasan) mencantumkan lisensi resmi beserta yayasan yang menaunginya sebagai bukti keterbukaan tata kelola, bersifat informatif, dan tidak mengubah perhitungan kolom Total. Alat dengan nilai Total tertinggi pada tiap tabel menjadi pilihan utama platform untuk fungsi yang dievaluasi, sementara alat berskor lebih rendah dapat tetap dipakai sebagai pendamping dengan peran berbeda sebagaimana diuraikan pada tiap lapis.

III.4.1 Lapis Orkestrasi dan Infrastruktur

Lapis orkestrasi menjadi fondasi tempat seluruh komponen platform dijalankan, sebagaimana dibahas pada Subbab II.2.1. Untuk lingkungan satu *node*, distribusi Kubernetes ringan yang dipertimbangkan adalah *k3s*, *k0s*, MicroK8s, dan *kind*. Keempatnya lolos uji *conformance* CNCF sehingga API yang dipakai komponen tetap identik, dan kriteria penilaian menyoroti *footprint*, kesiapan produksi, dependensi *runtime*, serta model lisensi. Tabel III.7 merangkum perbandingan keempatnya ter-

hadap kriteria tersebut.

k3s dipilih karena biner tunggalnya di bawah seratus megabita berjalan pada memori terbatas, kesiapannya setara Kubernetes penuh untuk beban produksi, *containerd* yang tertanam menghilangkan dependensi *runtime* eksternal, dan *controller* Helm bawaannya mempermudah pemasangan komponen. Sebaliknya *kind* ditujukan untuk pengujian CI dan bukan produksi, sedangkan *k0s* dan MicroK8s dikelola masing-masing oleh satu vendor tanpa naungan yayasan netral. Nilai Total tertinggi pada *k3s* mencerminkan gabungan kesiapan produksi dan *footprint* ringan yang sesuai dengan asumsi BI-1 pada Bab I. Pilihan ini tidak mengikat rancangan pada satu *node*, sebab keempat distribusi berbagi API yang sama sehingga migrasi ke distribusi lain tidak mengubah manifes komponen.

Tabel III.7 Evaluasi Alternatif Distribusi Kubernetes *Open-Source*

Distribusi	<i>Footprint</i> Ringan	Siap Produksi	Tanpa De- pendensi	Eko- sistem Add-on	Lisensi (Yayasan)	Total
<i>k3s</i>	2	2	2	2	Apache 2.0 (CNCF)	8
<i>k0s</i>	2	2	2	1	Apache 2.0 (mandiri)	7
MicroK8s	1	2	1	2	Apache 2.0 (mandiri)	6
<i>kind</i>	1	0	1	0	Apache 2.0 (CNCF)	2

Keterangan skor: 0 = tidak memenuhi, 1 = memenuhi sebagian, 2 = memenuhi seluruhnya; parameter penilaian tiap kriteria dirinci pada Lampiran B.

III.4.2 Lapis Ingestasi dan Pemrosesan Data

Lapis ingestasi dan pemrosesan memindahkan data dari sumber ke penyimpanan analitik, dengan *message broker* sebagai tulang punggung aliran peristiwa yang dijelaskan pada Subbab II.3.2. Kandidat yang dipertimbangkan adalah Apache Kafka, Redpanda, Apache Pulsar, dan NATS JetStream, dengan perbandingan pada Tabel III.8. Kafka dipilih karena ekosistem konektornya paling luas, *operator* Strimzi menyediakan pengelolaan deklaratif yang matang pada Kubernetes, lisensinya Apache 2.0 di bawah *Apache Software Foundation*, dan protokolnya dibaca langsung oleh konektor Flink, tabel *Kafka engine* ClickHouse, serta *scaler* KEDA. Ketergantungan silang inilah yang membuat Kafka memperoleh skor penuh pada ekosistem konektor dan kompatibilitas protokol, dua kriteria tempat sebagian alternatif tertinggal.

Tabel III.8 Evaluasi Alternatif *Message Broker* untuk Tulang Punggung *Streaming*

<i>Message Broker</i>	Eko-sistem Konektor	Operator K8s	Lisensi Terbuka	Protokol Kafka	Lisensi (Yayasan)	Total
Apache Kafka	2	2	2	2	Apache 2.0 (ASF)	8
Redpanda	1	2	0	2	BSL 1.1 (mandiri)	5
Apache Pulsar	1	1	2	1	Apache 2.0 (ASF)	5
NATS JetStream	1	1	2	0	Apache 2.0 (CNCF)	4

Keterangan skor: 0 = tidak memenuhi, 1 = memenuhi sebagian, 2 = memenuhi seluruhnya; parameter penilaian tiap kriteria dirinci pada Lampiran B.

Registrasi skema dinilai terpisah karena menjadi kontrak evolusi data lintas produsen dan konsumen, sebagaimana dibahas pada Subbab II.3.2. Tabel III.9 membandingkan beberapa *schema registry* pada ekosistem Kafka. Karapace dipilih sebagai *registry* kompatibel-Confluent berlisensi Apache 2.0 dengan dukungan Avro, JSON Schema, dan Protobuf serta verifikasi kompatibilitas saat publikasi. Karapace menyimpan skema pada *topic* internal Kafka sehingga tidak menambah basis data baru pada platform.

Pemrosesan data mencakup jalur *streaming* dan *batch* yang dibahas pada Subbab II.3.3 dan Subbab II.3.4. Tabel III.10 membandingkan mesin pemrosesan menurut kriteria kelayakan platform. Apache Spark memperoleh Total tertinggi sebagai mesin pemrosesan *batch* berskala besar, sedangkan Apache Flink dipilih untuk jalur *streaming* karena dukungan *event-time* dan *exactly-once* dengan *checkpointing* yang membuatnya unggul pada kriteria *streaming* meski lebih rendah pada kriteria

Tabel III.9 Evaluasi Alternatif *Schema Registry* untuk Apache Kafka

<i>Schema Registry</i>	Lisensi <i>Open</i>	Kompati- bilitas Conflu- ent	Duku- ngan Format	Kema- tangan	Lisensi (Yayasan)	Total
Karapace (Aiven)	2	2	2	1	Apache 2.0 (mandiri)	7
Apicurio Registry	2	1	2	1	Apache 2.0 (mandiri)	6
Confluent <i>Schema Registry</i>	0	2	2	2	<i>Confluent Community</i> (mandiri)	6
Cloudera <i>Schema Registry</i>	2	0	1	1	Apache 2.0 (mandiri)	4

Keterangan skor: 0 = tidak memenuhi, 1 = memenuhi sebagian, 2 = memenuhi seluruhnya;
parameter penilaian tiap kriteria dirinci pada Lampiran B.

batch. Transformasi SQL deklaratif pada lapisan *warehouse* dikerjakan oleh dbt. Ketiganya memikul peran berbeda pada jalur data sehingga saling melengkapi tanpa tumpang tindih fungsi.

Tabel III.10 Evaluasi Alternatif *Framework* Pemrosesan Data

<i>Framework Pemrosesan</i>	Kema- tangan	Kapa- bilitas <i>Batch</i>	Kapa- bilitas <i>Stream- ing</i>	Komu- nitas	Lisensi (Yayasan)	Total
Apache Spark	2	2	1	2	Apache 2.0 (ASF)	7
Apache Flink	2	1	2	1	Apache 2.0 (ASF)	6
Apache Kafka + Streams	2	0	2	1	Apache 2.0 (ASF)	5
Apache Beam	1	1	1	1	Apache 2.0 (ASF)	4

Keterangan skor: 0 = tidak memenuhi, 1 = memenuhi sebagian, 2 = memenuhi seluruhnya;
parameter penilaian tiap kriteria dirinci pada Lampiran B.

III.4.3 Lapis Penyimpanan Data

Lapis penyimpanan menyimpan data jangka panjang beserta versinya, dengan format tabel *lakehouse* yang menjamin transaksi dan evolusi skema, sebagaimana dibahas pada Subbab II.3.5. Kandidat format adalah Apache Iceberg, Delta Lake, dan Apache Hudi, dengan perbandingan pada Tabel III.11. Apache Iceberg dipilih karena netralitas *engine*, dukungan katalog REST melalui Lakekeeper, dan kemampuan kueri federasi melalui Trino. Netralitas *engine* inilah yang membuat Iceberg unggul, sebab tabel yang sama dapat dibaca dan ditulis oleh Trino, Spark, maupun Flink tanpa migrasi data fisik.

Tabel III.11 Evaluasi Alternatif Format Tabel *Lakehouse Open-Source*

Format Tabel	Kemampuan	Transaksi ACID	Evolusi Skema	Integrasi Engine	Lisensi (Yayasan)	Total
Apache Iceberg	2	2	2	2	Apache 2.0 (ASF)	8
Delta Lake	2	2	1	1	Apache 2.0 (LF)	6
Apache Hudi	1	2	1	1	Apache 2.0 (ASF)	5
Parquet (tanpa format tabel)	2	0	1	2	Apache 2.0 (ASF)	5

Keterangan skor: 0 = tidak memenuhi, 1 = memenuhi sebagian, 2 = memenuhi seluruhnya; parameter penilaian tiap kriteria dirinci pada Lampiran B.

Penyimpanan objek menjadi landasan bagi *data lake*, *model registry*, dan pencadangan, sebagaimana dibahas pada Subbab II.3.6. Tabel III.12 membandingkan empat penyedia objek S3-kompatibel. MinIO dipilih karena *footprint* ringan pada satu *node*, kompatibilitas API S3 yang luas, dukungan enkripsi sisi *server* berbasis KMS, dan kemampuan berjalan pada modus distribusi maupun *single-node*. Kompatibilitas API S3 yang luas membuat MinIO dapat dipakai lintas komponen tanpa penyesuaian klien.

Tabel III.12 Evaluasi Alternatif Penyimpanan Objek S3-Kompatibel *Open-Source*

Penyimpanan Objek	Kompatibilitas S3	<i>Footprint</i> Satu Node	Enkripsi KMS	Dukungan K8s	Lisensi (Yayasan)	Total
MinIO	2	2	2	2	AGPL v3 (mandiri)	8
SeaweedFS	1	2	1	1	Apache 2.0 (mandiri)	5
Ceph RGW	2	0	2	2	LGPL 2.1 (LF)	6
Garage	1	2	0	1	AGPL v3 (mandiri)	4

Keterangan skor: 0 = tidak memenuhi, 1 = memenuhi sebagian, 2 = memenuhi seluruhnya; parameter penilaian tiap kriteria dirinci pada Lampiran B.

Versi data pada lapis objek dinilai terpisah karena menyangkut ketertelusuran perubahan *dataset*, sebagaimana dibahas pada Subbab II.3.6. Tabel III.13 membandingkan lakeFS, DVC, Project Nessie, dan Pachyderm. lakeFS dipilih karena percabangan *zero-copy* bekerja langsung di atas MinIO tanpa terikat satu format tabel, antarmukanya tetap S3 sehingga *engine* pemrosesan tidak perlu diubah, dan lisensinya Apache 2.0 sementara Pachyderm berlisensi *source-available*. Model percabangan bergaya Git ini menjaga jejak versi data tanpa menggandakan berkas.

Tabel III.13 Evaluasi Alternatif Pengelola Versi Data *Open-Source* pada Lapis Penyimpanan Objek

Pengelola Versi	Per-cabangan Tanpa Salin	Integrasi Objek S3	Skala Data Besar	Lisensi Terbuka	Lisensi (Yayasan)	Total
lakeFS	2	2	2	2	Apache 2.0 (mandiri)	8
Project Nessie	2	1	1	2	Apache 2.0 (mandiri)	6
DVC	1	1	1	2	Apache 2.0 (mandiri)	5
Pachyderm	1	1	1	0	<i>Source-available</i> (mandiri)	3

Keterangan skor: 0 = tidak memenuhi, 1 = memenuhi sebagian, 2 = memenuhi seluruhnya; parameter penilaian tiap kriteria dirinci pada Lampiran B.

III.4.4 Lapis Layanan Fitur

Lapis layanan fitur menghubungkan jalur data dengan jalur model melalui definisi fitur yang sama, sebagaimana dibahas pada Subbab II.4.1. Tabel III.14 membandingkan beberapa kerangka *feature store open source*. Feast dipilih karena modularitas penyimpanan dan kontrak API yang konsisten lintas *offline* dan *online*, yang tecermin pada skor penuh di seluruh kriteria termasuk ekstensibilitas dan integrasi K8s. Hopsworks dan Featureform memperoleh skor lebih rendah pada ekstensibilitas dan komunitas sehingga Total keduanya berada di bawah Feast.

Tabel III.14 Evaluasi Alternatif *Feature Store Open-Source*

<i>Feature Store</i>	Kematan- gan	Eksten- sibilitas	Komu- nitas	Integrasi K8s	Lisensi (Yayasan)	Total
Feast	2	2	2	2	Apache 2.0 (LF AI & Data)	8
Hopsworks <i>Feature Store</i>	1	1	1	1	AGPL v3 (mandiri)	4
Feathr (LinkedIn)	0	1	0	1	Apache 2.0 (LF AI & Data)	2
Featureform	1	1	0	1	MPL 2.0 (mandiri)	3

Keterangan skor: 0 = tidak memenuhi, 1 = memenuhi sebagian, 2 = memenuhi seluruhnya; parameter penilaian tiap kriteria dirinci pada Lampiran B.

Penyimpanan fitur dipisah menjadi *offline* untuk pelatihan dan *online* untuk *inference* berlatensi rendah, sebagaimana dibahas pada Subbab II.4.2. Tabel III.15 dan Tabel III.16 menilai kandidat untuk kedua peran tersebut. ClickHouse dipilih sebagai *offline store* karena kompresi kolumnar dan latensi kueri agregasi yang rendah, sedangkan Valkey dipilih sebagai *online store* karena kompatibilitas *wire-protocol* RESP dengan ekosistem klien Redis serta lisensi BSD di bawah Linux Foundation yang tetap terbuka pasca-perubahan lisensi Redis. Kedua pilihan mencerminkan perbedaan karakter beban antara kueri agregasi pada jutaan baris dan *lookup* sub-milidetik saat *inference*.

Fitur *embedding* berdimensi tinggi memerlukan indeks tetangga terdekat yang dibahas pada Subbab II.4.3. Tabel III.17 membandingkan beberapa *vector index open source*. Qdrant dipilih karena dukungan HNSW *native*, mode *gRPC* berlatensi rendah, dan kemampuan filter pada *payload* yang menyatu dengan kueri kemiripan. Pemisahan *vector index* dari *online store* tabular menjaga tiap penyimpanan tetap dioptimasi untuk karakter kuerinya sendiri.

Tabel III.15 Evaluasi Alternatif Penyimpanan *Offline* (OLAP)

Penyimpanan <i>Offline</i>	Kema- tangan	Performa <i>Query</i>	Duku- ngan <i>Ingestion</i>	Komu- nitas	Lisensi (Yayasan)	Total
ClickHouse	2	2	2	2	Apache 2.0 (mandiri)	8
Apache Pinot	1	2	2	1	Apache 2.0 (ASF)	6
Apache Druid	2	2	2	1	Apache 2.0 (ASF)	7
Apache Kudu	1	1	1	1	Apache 2.0 (ASF)	4

Keterangan skor: 0 = tidak memenuhi, 1 = memenuhi sebagian, 2 = memenuhi seluruhnya;
parameter penilaian tiap kriteria dirinci pada Lampiran B.

Tabel III.16 Evaluasi Alternatif Penyimpanan *Online* (Latensi Rendah)

Penyimpanan <i>Online</i>	Kema- tangan	Latensi	<i>Through- put</i>	Lisensi <i>Open</i>	Lisensi (Yayasan)	Total
Valkey (RESP)	2	2	2	2	BSD-3 (LF)	8
Redis <i>Community</i>	2	2	2	1	AGPL v3 (mandiri)	7
DragonflyDB	1	2	2	0	BSL 1.1 (mandiri)	5
Apache Cassandra	2	1	2	2	Apache 2.0 (ASF)	7

Keterangan skor: 0 = tidak memenuhi, 1 = memenuhi sebagian, 2 = memenuhi seluruhnya;
parameter penilaian tiap kriteria dirinci pada Lampiran B.

Tabel III.17 Evaluasi Alternatif *Vector Index Open-Source* untuk Pencarian Pendekatan Tetangga Terdekat

<i>Vector Index</i>	Kema- tangan	Latensi Rendah	Komu- nitas	Opera- bilitas K8s	Lisensi (Yayasan)	Total
Qdrant (HNSW)	2	2	1	2	Apache 2.0 (mandiri)	7
Milvus	2	2	1	1	Apache 2.0 (LF AI & Data)	6
Weaviate	1	1	1	1	BSD-3 (mandiri)	4
pgvector (PostgreSQL)	2	1	1	2	PostgreSQL <i>License</i> (komunitas)	6

Keterangan skor: 0 = tidak memenuhi, 1 = memenuhi sebagian, 2 = memenuhi seluruhnya;
parameter penilaian tiap kriteria dirinci pada Lampiran B.

III.4.5 Lapis Siklus Hidup Model

Lapis siklus hidup model mengantar model dari eksperimen sampai penyajian, dimulai dari *tracking* eksperimen dan *registry* model pada Subbab II.5.1. Tabel III.18 membandingkan beberapa platform *tracking*. MLflow dipilih karena menyatukan *tracking* eksperimen, *registry* model, dan *API deployment* pada satu layanan dengan *backend* PostgreSQL dan *artifact store* MinIO. Penyatuan ketiga fungsi ini memberi MLflow keunggulan integrasi dibanding alat *tracking* yang hanya menangani pencatatan metrik.

Tabel III.18 Evaluasi Alternatif *Experiment Tracking* dan *Model Registry*

<i>Tracking & Registry</i>	Kematan- gan	Eksten- sibilitas	Komu- nitas	Integrasi	Lisensi (Yayasan)	Total
MLflow	2	2	2	2	Apache 2.0 (LF)	8
DVC (<i>Data Version Control</i>)	1	1	1	1	Apache 2.0 (mandiri)	4
TensorBoard	2	2	1	0	Apache 2.0 (mandiri)	5
Neptune	1	1	1	1	<i>Proprietary SaaS</i> (klien Apache 2.0)	4

Keterangan skor: 0 = tidak memenuhi, 1 = memenuhi sebagian, 2 = memenuhi seluruhnya; parameter penilaian tiap kriteria dirinci pada Lampiran B.

Orkestrasi pelatihan dinilai terpisah karena menyangkut eksekusi *pipeline* pada Kubernetes, sebagaimana dibahas pada Subbab II.5.2. Tabel III.19 membandingkan Kubeflow Pipelines, Argo Workflows, dan Apache Airflow. Kubeflow Pipelines dipilih untuk jalur pelatihan karena *component* berbasis kontainer dengan kontrak input dan output yang ditegakkan Pipeline IR, sedangkan Argo Workflows dipakai untuk *control loop* CronWorkflow yang memicu pelatihan ulang. Pemisahan peran kedua *orchestrator* menjaga *pipeline* pelatihan dapat dipromosikan tanpa mengganggu *control loop* pemantauan.

Penyajian model mengakhiri pembahasan lapis ini dengan menyajikan model sebagai layanan, sebagaimana dibahas pada Subbab II.5.4. Tabel III.20 membandingkan kerangka penyajian menurut kriteria kelayakan platform. KServe dipilih karena dukungan banyak *runtime*, penskalaan otomatis termasuk ke nol melalui Knative, dan integrasi dengan praktik GitOps. Nilai Total tertinggi pada KServe mencerminkan kematangan, skalabilitas, dan dukungan multi-*framework* yang tidak dimiliki penuh oleh kerangka penyajian berbasis satu *framework*.

Tabel III.19 Evaluasi Alternatif Orkestrasi ML *Open-Source*

Orkestrasi ML	Kema- tangan	K8s- Native	Pipeline ML	Komu- nitas	Lisensi (Yayasan)	Total
Kubeflow Pipelines	1	2	2	2	Apache 2.0 (CNCF)	7
Apache Airflow	2	1	1	2	Apache 2.0 (ASF)	6
Argo Workflows	2	2	1	1	Apache 2.0 (CNCF)	6
Prefect	1	1	1	1	Apache 2.0 (mandiri)	4

Keterangan skor: 0 = tidak memenuhi, 1 = memenuhi sebagian, 2 = memenuhi seluruhnya;
parameter penilaian tiap kriteria dirinci pada Lampiran B.

Tabel III.20 Evaluasi Alternatif *Model Serving*

Model Serving	Kema- tangan	Skala- bilitas	Frame- work	Komu- nitas	Lisensi (Yayasan)	Total
KServe	2	2	2	2	Apache 2.0 (LF AI & Data)	8
Seldon Core	1	1	2	1	BSL 1.1 (mandiri)	5
BentoML	1	1	2	1	Apache 2.0 (mandiri)	5
TorchServe	0	1	0	1	Apache 2.0 (LF/PyTorch)	2

Keterangan skor: 0 = tidak memenuhi, 1 = memenuhi sebagian, 2 = memenuhi seluruhnya;
parameter penilaian tiap kriteria dirinci pada Lampiran B.

III.4.6 Lapis Tata Kelola Data

Lapis tata kelola menegakkan katalog, kualitas, dan *lineage* data sepanjang siklus, sebagaimana dibahas pada Subbab II.6. Fungsi katalog metadata menjadi pusat lapis ini karena menautkan aset data lintas komponen DataOps dan MLOps. Tabel III.21 membandingkan beberapa pilihan katalog metadata. Kriteria penilaian menyoroti keluasan integrasi, dukungan *lineage*, kematangan proyek, dan model lisensi.

DataHub dipilih sebagai katalog metadata karena dukungan integrasi luas dengan komponen DataOps dan MLOps serta penerimaan *lineage* melalui standar terbuka OpenLineage. Great Expectations melengkapi lapis ini sebagai kontrak kualitas yang dijalankan pada *pipeline* produksi, sedangkan OpenLineage menjadi protokol *lineage* yang menyeragamkan jejak dari ingestasi hingga model. Ketiganya bekerja sebagai satu bidang tata kelola, yaitu katalog untuk penemuan, kontrak untuk kualitas, dan protokol untuk asal-usul. Pemisahan peran ini menghindari tumpang tindih fungsi antara alat katalog dan alat kualitas.

Tabel III.21 Evaluasi Alternatif Manajemen Metadata dan Tata Kelola

Metadata & Tata Kelola	Kematan-tangan	Eksten-sibilitas	Komu-nitas	Integrasi	Lisensi (Yayasan)	Total
DataHub (LinkedIn)	2	2	2	2	Apache 2.0 (LF AI & Data)	8
Apache Atlas	2	1	1	1	Apache 2.0 (ASF)	5
OpenMetadata	1	1	1	2	Apache 2.0 (LF)	5
Amundsen (Lyft)	1	1	0	1	Apache 2.0 (LF AI & Data)	3

Keterangan skor: 0 = tidak memenuhi, 1 = memenuhi sebagian, 2 = memenuhi seluruhnya; parameter penilaian tiap kriteria dirinci pada Lampiran B.

III.4.7 Lapis Observabilitas

Lapis observabilitas menyatukan metrik, log, dan *trace* pada satu antarmuka, sebagaimana dibahas pada Subbab II.8. Tabel III.22 membandingkan beberapa *observability stack open source*. Kombinasi Prometheus untuk metrik, Loki untuk log, dan Grafana Tempo untuk *trace* dengan OpenTelemetry sebagai *collector* bersama dipilih karena korelasi antar pilar dapat ditegakkan melalui label `trace_id` yang seragam. Grafana menjadi antarmuka visualisasi tunggal yang menyatukan ketiga pilar tersebut.

Tabel III.22 Evaluasi Alternatif *Monitoring* dan *Observability*

<i>Monitoring & Observability</i>	Kematan-tangan	Ekstensi-sibilitas	Komunitas	Integrasi K8s	Lisensi (Yayasan)	Total
Prometheus + Grafana	2	2	2	2	Apache 2.0 (CNCF) + AGPL v3 (mandiri)	8
InfluxDB + Chronograf	1	1	1	1	MIT (mandiri)	4
Elastic Stack (ELK)	2	1	1	2	AGPL v3 (mandiri)	6
Jaeger + OpenTelemetry	2	1	2	2	Apache 2.0 (CNCF)	7

Keterangan skor: 0 = tidak memenuhi, 1 = memenuhi sebagian, 2 = memenuhi seluruhnya; parameter penilaian tiap kriteria dirinci pada Lampiran B.

Antarmuka *business intelligence* dinilai terpisah karena menyajikan hasil kueri lintas basis data pada satu lapis dasbor, sebagaimana dibahas pada Subbab II.8. Tabel III.23 membandingkan beberapa pilihan BI *open source*. Apache Superset dipilih karena menyatukan kueri SQL lintas ClickHouse, Trino, dan MySQL pada satu antarmuka dasbor. Pemisahan Superset untuk dasbor analitik dari Grafana untuk pemantauan sistem menjaga tiap antarmuka fokus pada jenis pengamatan yang berbeda.

Tabel III.23 Evaluasi Alternatif *Business Intelligence* dan *Dashboarding* *Open-Source*

<i>BI Tool</i>	Kema- tangan	Konektor SQL	Multi- Tenant K8s	Komu- nitas	Lisensi (Yayasan)	Total
Apache Superset	2	2	2	2	Apache 2.0 (ASF)	8
Metabase	2	1	1	1	AGPL v3 (mandiri)	5
Redash	1	1	1	1	BSD-2 (mandiri)	4
Lightdash	1	1	1	1	MIT (mandiri)	4

Keterangan skor: 0 = tidak memenuhi, 1 = memenuhi sebagian, 2 = memenuhi seluruhnya;
parameter penilaian tiap kriteria dirinci pada Lampiran B.

III.4.8 Lapis GitOps dan *Continuous Delivery*

Lapis GitOps menempatkan repositori Git sebagai sumber kebenaran tunggal konfigurasi sistem, sebagaimana dibahas pada Subbab II.9. Tabel III.24 membandingkan beberapa *GitOps controller open source*. Argo CD dipilih sebagai *operator* GitOps karena merekonsiliasi keadaan *cluster* dengan deklarasi pada Git secara berkelanjutan dan menyediakan antarmuka penelusuran sinkronisasi yang matang. Kriteria penilaian menyoroti model rekonsiliasi, dukungan multi-*tenant*, dan kematangan ekosistem.

Rantai *continuous delivery* dilengkapi Gitea sebagai repositori *self-hosted*, Tekton sebagai *pipeline* CI, dan Argo Rollouts untuk *deployment* progresif berbasis *canary*. Keempat komponen menjaga seluruh manifes, *chart*, dan *overlay* berada pada satu bidang kendali yang ditarik Argo CD tanpa ketergantungan penyedia eksternal. Pembagian peran menempatkan Argo CD pada rekonsiliasi keadaan dan Tekton pada pembangunan artefak sehingga keduanya tidak tumpang tindih. Rantai ini menjadi mekanisme yang memicu pelatihan ulang model melalui kontrol deklaratif yang sama.

Tabel III.24 Evaluasi Alternatif *GitOps Controller Open-Source*

<i>GitOps Controller</i>	Kemampuan	K8s-Native	Antarmuka	Komunitas	Lisensi (Yayasan)	Total
Argo CD	2	2	2	2	Apache 2.0 (CNCF)	8
Flux CD	2	2	1	1	Apache 2.0 (CNCF)	6
Rancher Fleet	1	2	1	1	Apache 2.0 (mandiri)	5
Jenkins X	1	1	1	0	Apache 2.0 (CDF)	3

Keterangan skor: 0 = tidak memenuhi, 1 = memenuhi sebagian, 2 = memenuhi seluruhnya; parameter penilaian tiap kriteria dirinci pada Lampiran B.

III.4.9 Lapis Keamanan Platform

Lapis keamanan bekerja sebagai pertahanan berlapis yang dibahas pada Subbab II.10, mencakup identitas, rahasia, *service mesh*, kebijakan, dan pemantauan *runtime*. Untuk identitas terpadu, Tabel III.25 membandingkan beberapa *identity provider open source*. Dex dipilih sebagai *identity provider* OIDC karena *footprint* ringan, dukungan federasi terhadap penyedia eksternal, dan integrasi sederhana dengan Kubernetes. *oauth2-proxy* dan *SpiceDB* melengkapi lapis identitas untuk konsol tanpa OIDC *native* dan otorisasi granular.

Tabel III.25 Evaluasi Alternatif *Identity Provider Open-Source*

<i>Identity Provider</i>	Bobot Footprint	Federasi OIDC	Opera- bilitas K8s	Kema- tangan	Lisensi (Yayasan)	Total
Dex <i>IdP</i>	2	2	2	2	Apache 2.0 (CNCF)	8
Keycloak	1	2	2	2	Apache 2.0 (CNCF)	7
Authelia	2	1	1	1	Apache 2.0 (mandiri)	5
Authentic	0	2	1	1	MIT (mandiri)	4

Keterangan skor: 0 = tidak memenuhi, 1 = memenuhi sebagian, 2 = memenuhi seluruhnya; parameter penilaian tiap kriteria dirinci pada Lampiran B.

Pengelolaan rahasia dinilai terpisah karena menyimpan kredensial dan kunci enkripsi platform, sebagaimana dibahas pada Subbab II.10.1. Tabel III.26 membandingkan beberapa pengelola rahasia *open source*. OpenBao dipilih sebagai *fork open source* dari HashiCorp Vault di bawah Linux Foundation karena menjamin penyimpanan terenkripsi dan penerbitan kredensial dinamis tanpa basis data eksternal berkat *integrated storage* Raft. External Secrets Operator menyalurkan rahasia ke Secret Kubernetes dengan rotasi otomatis.

Tabel III.26 Evaluasi Alternatif Pengelola Rahasia

Pengelola Rahasia	Lisensi Open	Rotasi Dinamis	Integrasi ESO	Kema- tangan	Lisensi (Yayasan)	Total
OpenBao	2	2	2	1	MPL 2.0 (LF)	7
HashiCorp Vault <i>Community</i>	0	2	2	2	BUSL 1.1 (mandiri)	6
Bitnami <i>Sealed Secrets</i>	2	0	0	2	Apache 2.0 (mandiri)	4
Mozilla SOPS	2	0	0	2	MPL 2.0 (CNCF)	4

Keterangan skor: 0 = tidak memenuhi, 1 = memenuhi sebagian, 2 = memenuhi seluruhnya; parameter penilaian tiap kriteria dirinci pada Lampiran B.

Komunikasi antar-layanan dan perbatasan *mesh* dibahas pada Subbab II.10.2. Tabel III.27 dan Tabel III.28 menilai *service mesh* dan *API gateway*. Istio dipilih se-

bagai *service mesh* karena *mutual TLS* seragam, kontrol lalu lintas lapis aplikasi, dan integrasi telemetry dengan OpenTelemetry, sedangkan Apache APISIX dipilih sebagai *API gateway* yang menyaring lalu lintas masuk dan menjembatani mTLS terhadap konsumen di luar *mesh*. Keduanya memisahkan kontrol lalu lintas internal dari kontrol perbatasan tanpa fungsi yang tumpang tindih.

Tabel III.27 Evaluasi Alternatif *Service Mesh*

<i>Service Mesh</i>	Kema- tangan	Fitur L7	Eko- sistem CRD	Komu- nitas	Lisensi (Yayasan)	Total
Istio <i>ambient/sidecar</i>	2	2	2	2	Apache 2.0 (CNCF)	8
Linkerd	2	1	1	1	Apache 2.0 (CNCF)	5
Cilium Service Mesh	1	1	1	2	Apache 2.0 (CNCF)	5
Consul Connect	1	1	1	1	BUSL 1.1 (mandiri)	4

Keterangan skor: 0 = tidak memenuhi, 1 = memenuhi sebagian, 2 = memenuhi seluruhnya;
parameter penilaian tiap kriteria dirinci pada Lampiran B.

Tabel III.28 Evaluasi Alternatif *API Gateway Open-Source*

<i>API Gateway</i>	Kema- tangan	Lisensi <i>Open</i>	Opera- bilitas K8s	<i>Plugin/</i> Ekstensi	Lisensi (Yayasan)	Total
Apache APISIX	2	2	2	2	Apache 2.0 (ASF)	8
Kong <i>Gateway</i> <i>OSS</i>	2	2	2	1	Apache 2.0 (mandiri)	7
Emissary Ingress (Ambassador)	1	2	2	1	Apache 2.0 (CNCF)	6
NGINX Ingress	1	2	2	1	Apache 2.0 (komunitas Kubernetes)	6

Keterangan skor: 0 = tidak memenuhi, 1 = memenuhi sebagian, 2 = memenuhi seluruhnya;
parameter penilaian tiap kriteria dirinci pada Lampiran B.

Penegakan kebijakan *admission* dinilai terpisah, sebagaimana dibahas pada Subbab II.10.2. Tabel III.29 membandingkan beberapa *policy engine open source*. Kyverno dipilih sebagai *policy engine native* Kubernetes dengan sintaks YAML ringkas serta dukungan *mutate*, *validate*, dan *generate*. Sintaks berbasis YAML membuat Kyverno lebih mudah diadopsi dibanding alternatif berbasis Rego pada konteks tim platform.

Pemantauan ancaman pada lapis *runtime* melengkapi lapis keamanan, sebagaimana dibahas pada Subbab II.10.2. Tabel III.30 membandingkan beberapa alat *runtime security open source*. Falco dipilih sebagai detektor berbasis eBPF yang memantau

Tabel III.29 Evaluasi Alternatif *Policy Engine Open-Source*

<i>Policy Engine</i>	Sintaks Native	Mode Mutate	Pelaporan Pelanggaran	Komunitas	Lisensi (Yayasan)	Total
Kyverno	2	2	2	2	Apache 2.0 (CNCF)	8
OPA Gatekeeper	1	1	1	2	Apache 2.0 (CNCF)	5
jsPolicy	1	1	1	0	Apache 2.0 (mandiri)	3
Kubewarden	1	1	2	1	Apache 2.0 (CNCF)	5

Keterangan skor: 0 = tidak memenuhi, 1 = memenuhi sebagian, 2 = memenuhi seluruhnya; parameter penilaian tiap kriteria dirinci pada Lampiran B.

syscall, dilengkapi Trivy Operator untuk pemindaian *image*, Velero untuk pencadangan, dan Chaos Mesh untuk uji ketahanan. Keempat alat mencakup dimensi berbeda dari keamanan *runtime* sehingga tidak saling menggantikan.

Tabel III.30 Evaluasi Alternatif *Runtime Security Open-Source* pada Kubernetes

<i>Runtime Security</i>	Kemampuan	Cakupan eBPF/ Kernel	Aturan Bawaan	Komunitas	Lisensi (Yayasan)	Total
Falco	2	2	2	2	Apache 2.0 (CNCF)	8
Cilium Tetragon	1	2	1	1	Apache 2.0 (CNCF)	5
Aqua Tracee	1	2	1	1	Apache 2.0 (mandiri)	5
Sysdig Secure OSS	1	1	1	1	Apache 2.0 (mandiri)	4

Keterangan skor: 0 = tidak memenuhi, 1 = memenuhi sebagian, 2 = memenuhi seluruhnya; parameter penilaian tiap kriteria dirinci pada Lampiran B.

Pemetaan pada subbab ini menautkan tiap lapis arsitektur ke satu alat utama beserta pendampingnya, sehingga rancangan pada Bab IV dapat langsung merujuk komponen terpilih tanpa mengulang pertimbangan pemilihan. Alat dengan nilai Total tertinggi pada tiap tabel menjadi pilihan utama lapisnya, sedangkan alat pendamping memikul peran yang berbeda tanpa tumpang tindih fungsi, seperti Flink untuk *streaming* di samping Spark untuk *batch* dan Argo Workflows untuk *control loop* di samping Kubeflow Pipelines untuk pelatihan. Pemisahan konsep pada Bab II dari pemilihan pada subbab ini menjaga tinjauan pustaka tetap umum sementara keputusan alat ditegakkan dengan bukti skor. Dengan pemetaan ini, keputusan teknis di seluruh laporan dapat dirunut dari konsep pada Bab II, ke pemilihan pada subbab ini, hingga perwujudan pada Bab V.

BAB IV

PERANCANGAN ARSITEKTUR PLATFORM DATAOPS DAN MLOPS

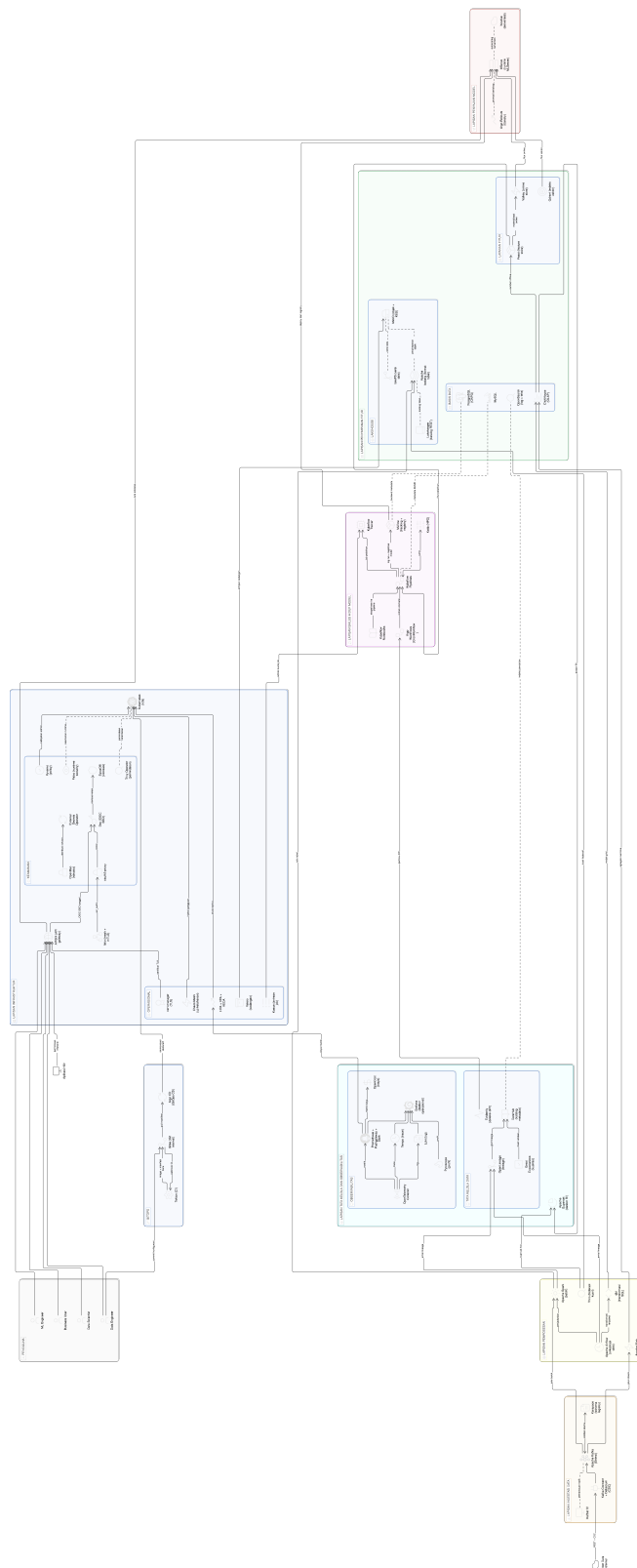
Bab ini menyajikan perancangan arsitektur platform yang mengintegrasikan DataOps dan MLOps pada Kubernetes. Pembahasan dimulai dari gambaran umum yang merangkum perbedaan kondisi sebelum dan sesudah integrasi, kemudian dilanjutkan dengan perancangan empat sub-sistem yang masing-masing memenuhi satu tujuan dari Bab I: arsitektur platform terintegrasi sebagai pemenuhan T-1, sub-sistem tata kelola data sebagai pemenuhan T-2, sub-sistem deteksi *drift* dengan pelatihan ulang otomatis sebagai pemenuhan T-3, dan sub-sistem layanan fitur *dual-store* sebagai pemenuhan T-4. Bab ini ditutup dengan alur kerja *end-to-end* yang menghubungkan keempat sub-sistem dalam satu siklus yang dapat direproduksi.

IV.1 Gambaran Umum Platform

Platform yang dirancang menempatkan Kubernetes sebagai bidang orkestrasi tunggal dan menyatukan komponen DataOps dan MLOps di atasnya. Gambar IV.1 memperlihatkan arsitektur terintegrasi yang menggantikan kondisi fragmentasi pada Gambar III.1 dari Bab III. Komponen yang sebelumnya berdiri sendiri pada perangkat lunak yang berbeda kini terhubung melalui satu bidang kendali, satu sumber kebenaran berbasis Git, dan satu antarmuka observabilitas. Penyatuan bidang kendali memungkinkan tim platform menerapkan perubahan dengan satu mekanisme *deploy* yang sama untuk seluruh komponen, sehingga jalur integrasi yang sebelumnya tersebar dapat disederhanakan. Susunan pada gambar tersebut sekaligus memenuhi tujuh komponen yang dipersyaratkan Google Cloud (2024) untuk MLOps *level 2*, yaitu kendali versi pada Gitea, layanan uji dan *build* pada Tekton, layanan *deployment* pada Argo CD, *model registry* pada MLflow, *feature store* pada Feast, penyimpanan metadata ML pada MLflow dan basis metadata Kubeflow Pipelines, serta orkestrator *pipeline* pada Kubeflow Pipelines dengan pemicu pelatihan ulang otomatis dari deteksi *drift*. Rantai yang sama menempatkan platform pada tahap DataOps dalam model evolusi Munappy dkk. (2020) karena CI/CD, orkestrasi, pengujian kualitas, dan pemantauan berlaku juga pada *pipeline* data, bukan hanya pada model.

Perbedaan utama terhadap kondisi sebelum integrasi terletak pada tiga aspek berikut.

1. **Penyatuan bidang kendali pada Kubernetes**



Gambar IV.1 Gambaran Umum Platform DataOps dan MLOps Terintegrasi, memenuhi komponen MLOps *level 2* (Google Cloud 2024) dan tahap DataOps pada model evolusi (Munappy dkk. 2020).

Bidang kendali yang sebelumnya tersebar pada bermacam *tool* digabungkan ke dalam bidang kendali Kubernetes yang sama.

2. Sumber kebenaran konfigurasi tunggal di Git

Sumber kebenaran konfigurasi yang sebelumnya berada pada berkas konfigurasi terpisah disatukan pada repositori Git yang dijaga oleh Argo CD (Argo Project 2024).

3. Penyatuan antarmuka observabilitas

Antarmuka observabilitas yang sebelumnya berbeda untuk metrik, log, dan *trace* disatukan di Grafana dengan Prometheus (Prometheus Authors 2024), Loki (Grafana Labs 2024b), dan Grafana Tempo (Grafana Labs 2024c) sebagai sumber tiga pilar observabilitas.

Ketiga perbedaan tersebut bersama-sama memutus efek domino fragmentasi yang dibahas pada Subbab III.1.3, karena tiap lapis berbagi kontrak yang sama dan dapat dirujuk silang melalui DataHub.

Platform tunggal ini dipakai oleh empat peran utama yang masing-masing masuk melalui antarmuka berbeda sesuai tanggung jawabnya. Tabel IV.1 merangkum pemetaan tiap peran ke antarmuka utama, *tool* yang diakses, dan fungsi yang dijalankan, sehingga satu titik masuk tunggal per peran menggantikan kebiasaan berpindah antar banyak *tool*. Karena seluruh komponen mendaftarkan katalog dan *lineage* ke DataHub serta berbagi satu bidang identitas yang dirinci pada Subbab IV.3.3, metadada yang dihasilkan satu peran langsung terlihat oleh peran lain tanpa berpindah sistem. Penyatuan pada lapis antarmuka inilah yang melengkapi penyatuan pada lapis kendali, sehingga keempat peran bekerja di atas satu platform tanpa kehilangan antarmuka yang sesuai dengan tugasnya.

Tabel IV.1 Pemetaan Peran Pengguna ke Antarmuka dan *Tool* Platform

Peran	Antarmuka Utama	<i>Tool</i> yang Diakses	Fungsi Utama
<i>Business User</i>	Superset, Grafana	Apache Superset, Grafana	Dasbor data dan operasional
<i>Data Engineer</i>	DataHub, Argo CD	DataHub, Argo CD, Argo Workflows	Katalog, <i>rollout</i> konfigurasi, eksekusi <i>pipeline</i>
<i>Data Scientist</i>	Kubeflow Notebooks	Kubeflow Notebooks, MLflow, Feast	Eksperimen, <i>tracking</i> , registri fitur
<i>Machine Learning Engineer</i>	Kubeflow Pipelines	Kubeflow Pipelines, KServe, Argo Rollouts	Pelatihan ulang, penyajian, promosi versi

IV.2 Perancangan Arsitektur Terintegrasi

Bagian ini menjawab T-1 dengan merancang arsitektur platform DataOps dan MLOps terintegrasi pada Kubernetes. Arsitektur disusun dalam tujuh lapis yang saling terhubung melalui kontrak antarmuka eksplisit. Tujuh lapis yang dimaksud adalah lapisan infrastruktur, lapisan ingestasi data, lapisan pemrosesan, lapisan penyimpanan fitur, lapisan siklus hidup model, lapisan penyajian model, dan lapisan tata kelola serta observabilitas. Jumlah tujuh bukan taksonomi baru, melainkan hasil penataan ulang kategori komponen arsitektur MLOps dari dua rujukan sehingga setiap lapis dapat dirunut ke kategori sumbernya.

Amou Najafabadi dkk. (2024) mensintesis 35 komponen arsitektur MLOps dari 43 studi primer menjadi enam kategori, yaitu *data curation*, *storage and versioning*, *ML training*, *CI/CD*, *inference*, serta *infrastructure and supporting services*, sedangkan Kreuzberger, Kühl, dan Hirschl (2023) merinci sembilan komponen teknis mulai dari komponen *CI/CD* dan *feature store* hingga *model registry* dan komponen pemantauan. Lapisan ingestasi data dan lapisan pemrosesan merupakan pecahan kategori *data curation*, dengan komponen *data collector* pada lapisan ingestasi dan *data preprocessor* pada lapisan pemrosesan, karena platform memisahkan jalur *batch* dan *stream*. Lapisan penyimpanan fitur menampung komponen penyimpanan dan versi data serta *feature store* dari kategori *storage and versioning*, dan lapisan siklus hidup model menampung kategori *ML training* bersama *model registry* serta menyimpan metadata ML dari kategori yang sama. Lapisan penyajian model menampung kategori *inference*, sedangkan lapisan infrastruktur menggabungkan kategori *infrastructure and supporting services* dengan *CI/CD* pada satu bidang kendali GitOps. Penyesuaian terakhir memindahkan komponen pemantauan, yang pada kategori *inference* berada satu kelompok dengan penyajian, ke lapisan tata kelola dan observabilitas, sekaligus menambahkan katalog, *lineage*, dan kualitas data pada lapisan tersebut karena integrasi DataOps menuntut tata kelola data sebagai fungsi lintas lapis (Munappy dkk. 2020).

IV.2.1 Lapisan Infrastruktur dan Bidang Kendali

Lapisan infrastruktur menggunakan Kubernetes (Cloud Native Computing Foundation 2024a) sebagai bidang orkestrasi untuk seluruh komponen platform. Istio (Istio Authors 2024) menyediakan jaringan layanan dengan mTLS yang mengamankan komunikasi antar layanan, sedangkan APISIX (Apache APISIX Authors 2025) berperan sebagai *API gateway* pada perbatasan *mesh*. OpenBao (OpenBao Authors 2025) mengelola rahasia, dan *External Secrets Operator* menyebarkan rahasia da-

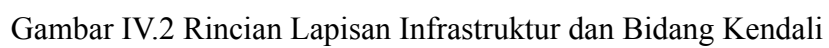
ri OpenBao ke *namespace* target tanpa menyalin rahasia ke berkas manifes. Argo CD menjadi satu-satunya jalur perubahan, menarik konfigurasi dari Git dan menerapkannya ke *cluster* melalui prinsip GitOps (Limoncelli 2018). Gambar IV.2 merinci komponen lapisan ini beserta jalur GitOps yang menyatukannya.

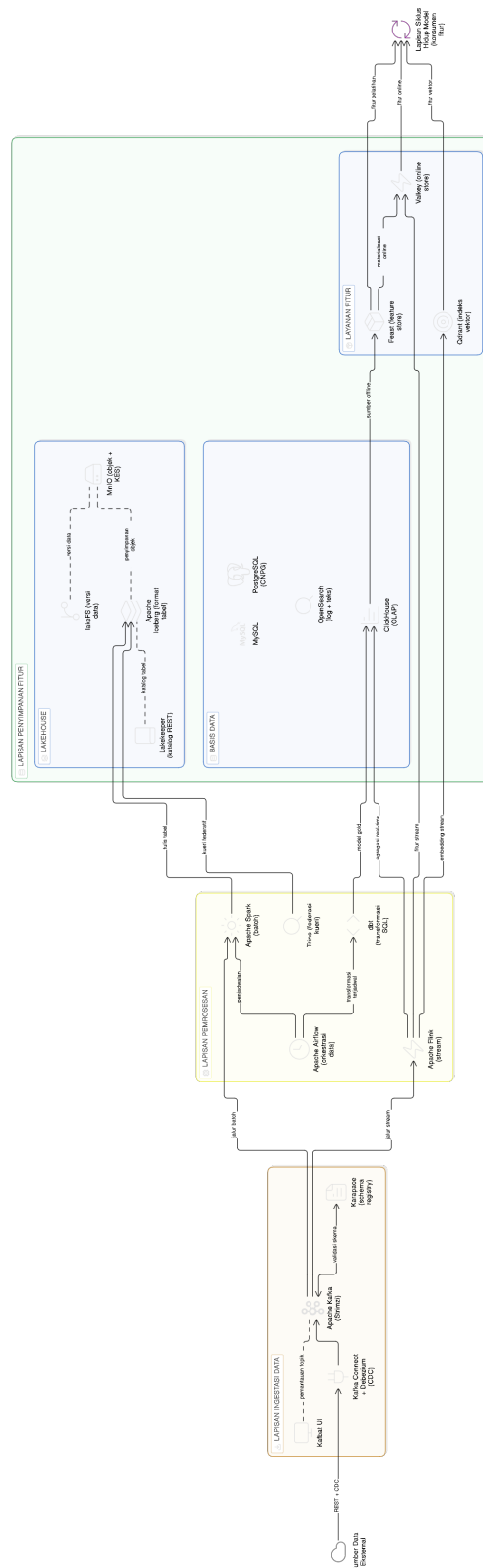
Kebijakan keamanan dijalankan dalam dua lapis: Kyverno (Kyverno Authors 2024) pada lapis admisi Kubernetes, dan Falco (Falco Authors 2025) pada lapis *runtime*. Trivy Operator (Aqua Security 2025) memindai *image* dan beban kerja secara terus menerus, sedangkan Velero (Velero Authors 2025) melakukan pencadangan sumber daya *cluster* dan *persistent volume* secara terjadwal. Penskalaan otomatis dijalankan oleh tiga komponen yang saling melengkapi yaitu HPA Kubernetes pada level Pod, VPA *recommender* untuk rekomendasi *resource request*, dan KEDA (KEDA Authors 2025) untuk penskalaan berbasis peristiwa dari Kafka atau Prometheus. Kueue (Kueue Authors 2024) mengelola antrean dan kuota *job* pelatihan melalui *ClusterQueue* sehingga sumber daya komputasi terbagi adil antar-*tenant* sesuai prioritas, dengan *job* di luar kuota berstatus *Pending* hingga kuota tersedia. Cert-manager (cert-manager Contributors 2025) mengelola sertifikat TLS, melengkapi lapisan infrastruktur dengan kontrak identitas mesin yang diperbarui otomatis.

IV.2.2 Lapisan Ingestasi, Pemrosesan, dan Penyimpanan Fitur

Lapisan ingestasi menggunakan Apache Kafka (Apache Software Foundation 2024c) sebagai bus pesan utama dengan operator Strimzi pada *cluster* Kubernetes. *Kafka Connect* digunakan untuk integrasi dengan sumber data eksternal, dan *Karapace* digunakan sebagai registri skema. Lapisan pemrosesan dibagi menjadi dua jalur, yaitu jalur *batch* dan jalur *stream*, sehingga kebutuhan latensi berbeda dapat dilayani tanpa kompromi. Jalur *batch* menggunakan Apache Spark (Apache Software Foundation 2024d) dan dbt untuk transformasi pada *data warehouse*, sedangkan jalur *stream* menggunakan Apache Flink (Apache Software Foundation 2024b) untuk pemrosesan jendela waktu, agregasi, dan deteksi pola. Gambar IV.3 merinci kedua jalur pemrosesan beserta lapisan penyimpanan dan layanan fitur yang menerimanya.

Penyimpanan jangka panjang pada *data lake* menggunakan MinIO sebagai *object store* dengan format Apache Iceberg sebagai *table format*, sedangkan Trino (Trino Software Foundation 2024) menyediakan kueri federasi pada beberapa sumber data sekaligus. Katalog Iceberg dikelola oleh Lakekeeper (Lakekeeper Authors 2025) yang memberikan *REST API* kompatibel dengan klien Iceberg standar dan dapat dipakai oleh Spark, Trino, dan Flink secara seragam. lakeFS (Treeverse Ltd. 2025) ditambahkan sebagai lapisan *version control* untuk *data lake* sehingga eksperimen





Gambar IV.3 Rincian Lapisan Ingestasi, Pemrosesan, dan Penyimpanan Fitur

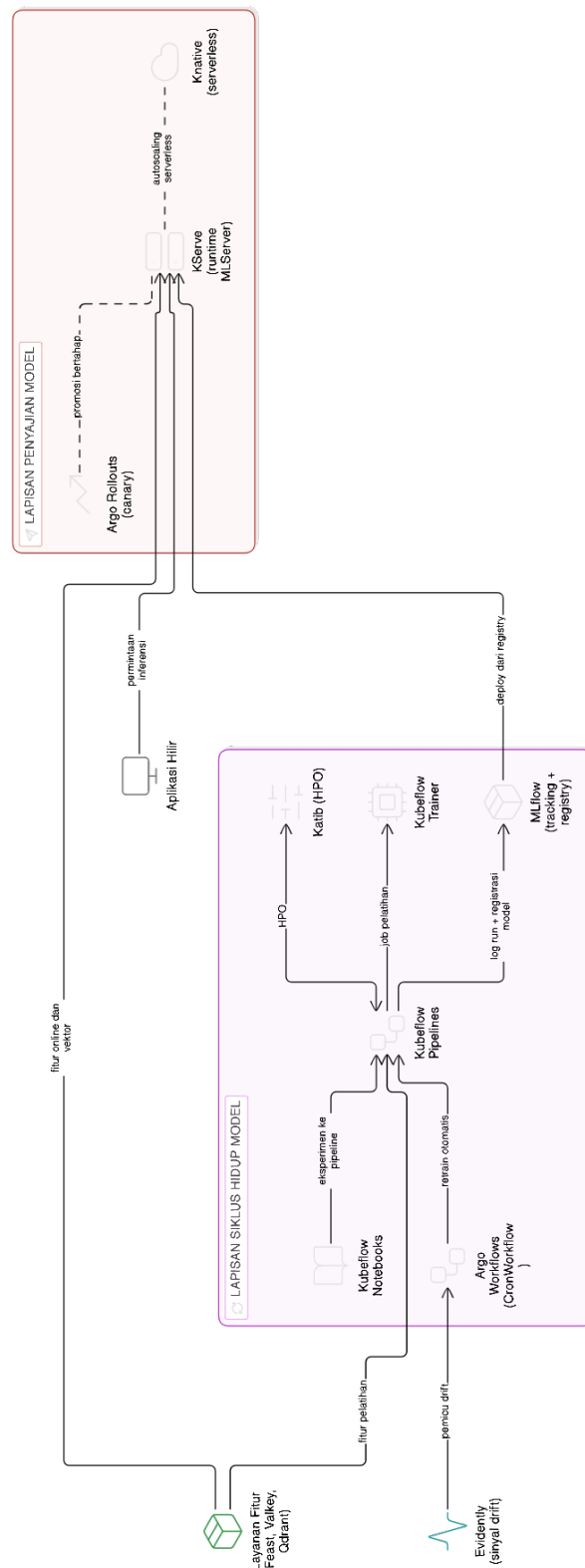
pada cabang data dapat dilakukan tanpa mengganggu cabang utama. ClickHouse (ClickHouse Inc 2024) berperan sebagai *warehouse* berorientasi kolom yang menampung tabel hasil transformasi dan menyediakan kueri analitik dengan latensi rendah untuk dasbor maupun materialisasi fitur.

Orkestrasi jalur *batch* dirancang menggunakan Apache Airflow (Apache Software Foundation 2024a) sebagai *scheduler* DAG yang merangkai langkah validasi, transformasi, dan materialisasi fitur menjadi satu *pipeline* data terjadwal. Definisi DAG ditarik dari Git melalui mekanisme *git-sync* sehingga perubahan *pipeline* mengikuti alur GitOps yang sama dengan komponen platform lain. Setiap langkah transformasi dijalankan sebagai *pod* terisolasi melalui KubernetesPodOperator agar batas sumber daya dapat ditetapkan per langkah dan kegagalan satu langkah tidak menular ke langkah lain. Pembagian peran ini menempatkan Airflow sebagai *orchestrator* bidang data dan Kubeflow Pipelines sebagai *orchestrator* pelatihan model, sehingga tanggung jawab DataOps dan MLOps tidak bercampur dalam satu *controller*.

IV.2.3 Lapisan Siklus Hidup Model dan Penyajian

Lapisan siklus hidup model menggunakan MLflow (MLflow Contributors 2024) untuk *tracking* dan registri model. Kubeflow Pipelines (Kubeflow Contributors 2024) dipakai sebagai *orchestrator* pelatihan, sedangkan Katib digunakan untuk *hyperparameter tuning*. Lapisan penyajian menggunakan KServe (KServe Contributors 2024) sebagai *inference platform* dengan dukungan banyak *runtime*, otomatisasi penskalaan, dan integrasi langsung dengan model yang tersimpan pada registri MLflow. Argo Workflows mengeksekusi *pipeline* yang tidak terkait langsung dengan pelatihan, sedangkan Argo CronWorkflow menjadwalkan *job* berkala untuk pemeliharaan dan pemicu pelatihan ulang. Basis data relasional untuk metadata MLflow disediakan oleh PostgreSQL yang dikelola operator CNPG, sedangkan metadata Kubeflow Pipelines dan Katib disimpan pada MySQL, sehingga riwayat eksperimen dan *pipeline* tersimpan pada penyimpanan yang konsisten. Gambar IV.4 merinci alur dari eksperimen dan pelatihan menuju penyajian model beserta pemicu pelatihan ulang otomatis.

Kubeflow Trainer (Kubeflow Authors 2025c) menjalankan pelatihan terdistribusi melalui TrainJob yang dipanggil dari *pipeline* Kubeflow Pipelines, sehingga pelatihan *multi-replica* tidak memerlukan *custom controller*. Penyajian model multimodal didukung KServe dengan *runtime* bawaan untuk *scikit-learn*, XGBoost, PyTorch, dan ONNX, ditambah *runtime* kustom untuk model yang memerlukan dependensi spesifik. Knative (Knative Authors 2025) berperan sebagai bi-



Gambar IV.4 Rincian Lapisan Siklus Hidup Model dan Penyajian

dang *serverless* bagi KServe, sehingga penskalaan dari nol dapat dilakukan tanpa menahan sumber daya saat trafik rendah. *Runtime* kustom didaftarkan sebagai *ClusterServingRuntime* sehingga penambahan kerangka model baru tidak mengubah definisi *InferenceService* yang sudah berjalan.

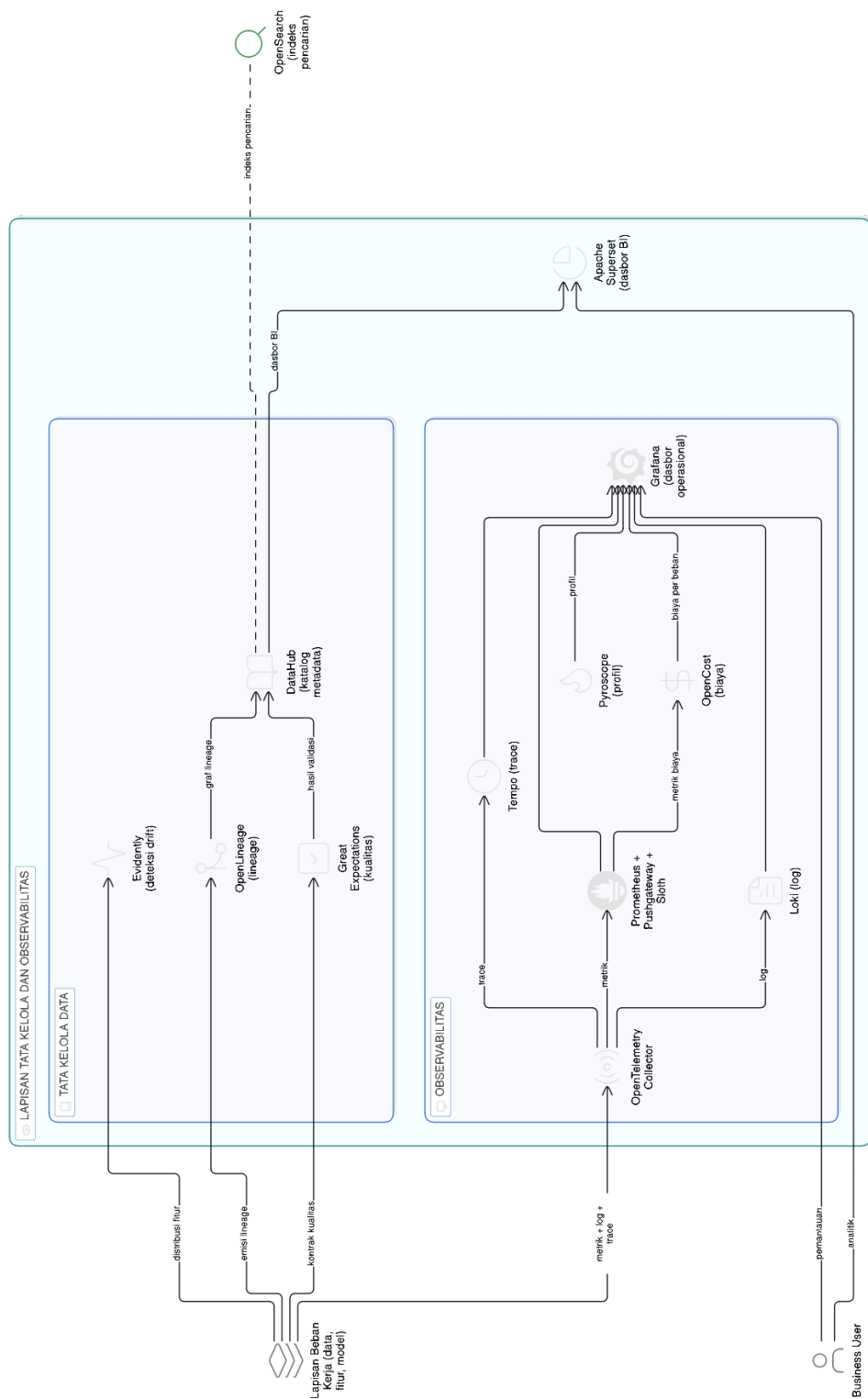
IV.2.4 Lapisan Tata Kelola dan Observabilitas

Lapisan tata kelola menggunakan DataHub (DataHub Project 2024) sebagai katalog metadata yang mengumpulkan informasi dari komponen lain melalui mekanisme *push* dan *pull*, dengan OpenLineage (OpenLineage Authors 2025) sebagai protokol pengirim peristiwa *lineage* dari komponen *pipeline*. Great Expectations (Great Expectations 2024) menjalankan pengujian kontrak data pada lapisan *batch* dan *stream*. Lapisan observabilitas menggunakan Prometheus untuk metrik, Loki untuk log, dan Grafana Tempo (Grafana Labs 2024c) untuk *trace*, dengan OpenTelemetry sebagai *collector* bersama. Grafana menyatukan ketiganya dan ditambahkan dasbor khusus untuk kualitas data, *drift* fitur, dan kinerja model. Gambar IV.5 merinci hubungan antara tata kelola data dan tiga pilar observabilitas pada satu antarmuka.

Sloth (Sloth Authors 2025) menurunkan *service-level objective* ke aturan Prometheus, sedangkan Pyroscope menyediakan profil performa berkelanjutan dan OpenCost melengkapi pemantauan biaya per beban kerja. Superset (Apache Superset Authors 2025) berperan sebagai antarmuka analitik bagi *business user* yang menampilkan dasbor di atas ClickHouse, Trino, dan MySQL. Pushgateway dipakai untuk mengumpulkan metrik *job* berdurasi pendek, antara lain CronJob validasi data dan CronWorkflow pengukur *drift*, agar metrik mereka tidak hilang setelah *job* selesai. Evidently (Evidently AI 2025) melengkapi observabilitas khusus model dengan *report* distribusi fitur dan kinerja model yang dapat diaudit dan disisipkan ke dasbor Grafana.

IV.3 Perancangan Sub-sistem Tata Kelola Data

Bagian ini menjawab T-2 dengan merancang sub-sistem tata kelola data yang konsisten lintas siklus data, fitur, dan model. Tata kelola yang dimaksud mencakup katalog metadata, *lineage*, pengujian kontrak data, dan kontrol akses berbasis identitas. Pemilihan keempat aspek tersebut mengikuti penekanan pada KF-08, KF-09, dan KF-10 serta KNF terkait keamanan dan observabilitas pada Bab III, sekaligus menjawab efek domino fragmentasi pada lapisan tata kelola yang dibahas pada Subbab III.1.3. Sub-sistem ini menjadi prasyarat bagi tiga sub-sistem lainnya karena katalog metadata berfungsi sebagai indeks bersama yang menghubungkan ingestasi,



Gambar IV.5 Rincian Lapisan Tata Kelola dan Observabilitas

pelatihan, penyajian, dan pemantauan dalam satu graf tata kelola.

IV.3.1 Katalog Metadata dan Lineage

DataHub menjadi pusat katalog metadata. Sumber metadata datang dari beberapa komponen: Kafka untuk *topic* dan skema, ClickHouse (ClickHouse Inc 2024) untuk tabel pada *warehouse*, MinIO dan Apache Iceberg dengan katalog Lakekeeper (Lakekeeper Authors 2025) untuk *dataset* pada *data lake*, Feast (Feast Project 2024) untuk definisi fitur, MLflow untuk *experiment* dan model, serta Kubeflow Pipelines untuk *pipeline*. Sambungan *lineage* antar komponen dikirim melalui peristiwa OpenLineage (OpenLineage Authors 2025) sehingga DataHub dapat merangkai graf ketertelusuran yang seragam dari ingestasi sampai prediksi. Identitas pengguna katalog diteruskan dari *identity provider* pusat yang dipakai bersama oleh seluruh antarmuka platform.

Ingestasi metadata pada DataHub dijalankan secara berkala melalui CronJob yang memanggil `datahub ingest` dengan resep YAML per sumber, sehingga penambahan sumber baru tinggal menambahkan resep tanpa memodifikasi kode pusat. Setiap resep meneruskan token OIDC dari OpenBao, sehingga autentikasi pada DataHub konsisten dengan autentikasi yang dipakai oleh komponen lain. Pengelolaan graf metadata juga memuat *glossary* domain yang ditarik dari berkas YAML pada repositori Git, memberi lapisan semantik yang dapat ditelusuri oleh *business user*. Praktik DataHub (DataHub Project 2024) menempatkan katalog metadata sebagai sumber utama jawaban ketertelusuran yang umum muncul pada audit kepatuhan, sehingga kapabilitas tersebut dapat diuji pada Bab VI.

IV.3.2 Kontrak Data dan Kualitas

Great Expectations memberi struktur untuk mendeklarasikan ekspektasi kualitas data, termasuk tipe kolom, batas nilai, dan jumlah baris minimum. Ekspektasi dijalankan pada dua titik: setelah ingestasi pada lapisan *bronze* dan setelah transformasi pada lapisan *silver* dan *gold*. Hasil pengujian dipublikasikan ke DataHub sehingga kualitas data tertampil bersama metadata lain. Chien (2022) memberi panduan agar metrik kualitas dipakai sebagai dasar pengambilan keputusan, bukan sekadar indikator pasif.

Bernardo dkk. (2024) dan Stoyanovich, Howe, dan Jagadish (2020) memberi konteks yang melebihi aspek teknis, yakni pertanggungjawaban data sebagai bagian dari tata kelola platform. Kegagalan ekspektasi pada lapisan *bronze* memicu per-

ingatan pada saluran observabilitas tanpa menghentikan jalur ingestasi, sementara kegagalan pada lapisan *silver* dan *gold* memicu penghentian *pipeline* agar data tidak berpindah ke lapisan berikutnya dengan kualitas yang tidak memenuhi kontrak. Setiap eksekusi Great Expectations menyimpan *run history* sehingga regresi kualitas dapat dirunut. dbt menjalankan model SQL pada ClickHouse dan menulis hasil ke tabel *gold* dengan kontrak yang divalidasi oleh *suite* ekspektasi yang dirancang pada *checkpoint* terjadwal.

IV.3.3 Identitas dan Kontrol Akses

Kontrol akses berbasis identitas menggunakan dex (Dex Authors 2025) sebagai *identity provider* OIDC dan oauth2-proxy (OAuth2 Proxy Authors 2025) sebagai *reverse proxy* yang menegakkan autentikasi pada antarmuka aplikasi. Identitas berlaku konsisten dari antarmuka pengguna ke *API* hingga ke akses data pada *warehouse* dan *lake*, dengan token OIDC yang sama dipakai oleh DataHub, MLflow, Grafana, Argo CD, dan Kubeflow Pipelines. Kebijakan otorisasi pada level kontainer dan beban kerja ditegakkan oleh Kyverno (Kyverno Authors 2024) pada lapis admisi Kubernetes, antara lain wajibnya *resource limits*, larangan *privileged container* di luar pengecualian eksplisit, dan kepatuhan label *Pod Security Standards*. Strategi keamanan MLOps yang dibahas Ahmad dkk. (2024) menjadi rujukan utama, karena banyak ancaman MLOps muncul ketika kontrol akses antar lapisan tidak konsisten.

SpiceDB (AuthZed 2025) dihadirkan sebagai layanan otorisasi berbasis Zanzibar yang menetapkan hubungan akses pada level objek, sehingga kebijakan akses data-set, fitur, dan model dapat dinyatakan dengan model relasi yang seragam. OpenBao menjadi sumber tunggal rahasia yang disinkronkan ke *namespace* target oleh *External Secrets Operator*, dan *Key Encryption Service* mengenkripsi rahasia tersebut pada lapis MinIO dengan kunci yang dirotasi secara berkala. Falco (Falco Authors 2025) memonitor peristiwa *runtime* dan mengirim peringatan ke Loki ketika perilaku mencurigakan terdeteksi, sehingga lapisan *runtime* tercakup oleh observasi berkelanjutan. Chaos Mesh (Chaos Mesh Authors 2025) dipakai untuk pengujian keandalan terjadwal yang memvalidasi bahwa kontrol akses tetap terjaga pada kondisi kegagalan *partial*.

IV.4 Perancangan Sub-sistem Deteksi *Drift* dan *Continuous Training*

Bagian ini menjawab T-3 dengan merancang sub-sistem deteksi *drift* pada fitur dan label yang otomatis memicu pelatihan ulang pada saat melewati ambang batas. Sub-sistem ini menjawab kombinasi *drift* dan *temporal leakage* yang menjadi salah satu

rumusan masalah. Penekanan diberikan pada otomatisasi penuh: pengukuran, pengambilan keputusan, pelatihan ulang, validasi, dan promosi versi dijalankan tanpa intervensi manual. Sub-sistem ini menjadi titik temu antara DataOps dan MLOps karena *drift* terdeteksi pada lapisan data sementara konsekuensinya dieksekusi pada lapisan model.

IV.4.1 Pengukuran *Drift*

Dua indikator utama dipilih sebagai detektor. *Population Stability Index* (PSI) dipakai untuk fitur kategorikal dan kontinu dengan *binning*, sedangkan uji Kolmogorov-Smirnov dipakai untuk distribusi nilai kontinu (Reis dkk. 2016). Pengukuran dilakukan pada jendela waktu yang bergerak dan dibandingkan dengan jendela referensi. Tingkat *drift* dikategorikan menjadi *stable*, *moderate*, dan *severe* dengan ambang yang dapat dikonfigurasi per fitur. Bayram, Ahmed, dan Kassler (2022), Lu dkk. (2019), dan Hinder dkk. (2023) menjadi rujukan untuk penyusunan kebijakan keluarga detektor dan respons platform.

Pengukuran *drift* dijalankan oleh CronJob yang menarik fitur *baseline* dan fitur jendela aktif dari ClickHouse, lalu menghitung indikator pada *namespace model-lifecycle* agar terisolasi dari beban kerja produksi lain. Hasil pengukuran dipublikasikan ke tabel *gold.drift_multi_scale* pada ClickHouse sehingga riwayat dapat dianalisis secara historis untuk audit dan pelaporan. Pushgateway menerima metrik tiap eksekusi sehingga Prometheus dapat merangkainya menjadi deret jangka panjang yang ditampilkan pada dasbor Grafana. Evidently menambah lapisan visual *report* HTML yang disimpan pada MinIO dan ditautkan dari DataHub agar dapat dirujuk oleh *data scientist* ketika menelusuri penurunan kinerja model.

IV.4.2 Alur Pelatihan Ulang Otomatis

Alur pelatihan ulang dijalankan oleh Argo CronWorkflow yang berperan sebagai *control loop*. *Workflow* ini menjalankan *job* pengukuran *drift*, kemudian memutuskan apakah pemicu pelatihan diperlukan. Jika diperlukan, *workflow* memicu *pipeline* pelatihan pada Kubeflow Pipelines. *Pipeline* pelatihan menarik fitur dari Feast dengan operasi *get_historical_features* yang menjamin *point-in-time correctness*, kemudian menulis model yang dihasilkan ke MLflow. Model kandidat didaftarkan ke registri MLflow sebagai versi baru, sedangkan kelayakannya untuk menggantikan versi produksi ditentukan pada tahap promosi *canary* yang menilai metrik penyajian versi baru secara langsung.

Promosi versi pada lapisan penyajian dikendalikan oleh Argo Rollouts (Argo Rollouts Authors 2025) dengan strategi *canary* yang menyalurkan sebagian trafik ke versi baru sebelum promosi penuh, sehingga *rollback* otomatis dapat dilakukan ketika metrik kinerja menurun. Céspedes Sisniega dkk. (2024) memberi contoh penerapan deteksi *drift* pada lingkungan *serverless*, dan pendekatan yang serupa diadaptasi ke lingkungan Kubernetes pada platform ini. Setiap langkah *canary* dijeda agar metrik terkumpul, lalu *AnalysisTemplate* mengevaluasi tingkat keberhasilan dan latensi p99 versi *canary* terhadap ambang SLO melalui kueri Prometheus dan menghentikan promosi ketika ambang dilanggar. Riwayat promosi dan *rollback* tercatat pada *namespace* Argo sehingga audit promosi dapat dijalankan langsung dari Argo CD UI tanpa berpindah *tool*.

IV.4.3 Mekanisme Penanganan Kasus Khusus

Sub-sistem ini melayani tiga kasus khusus yang umum muncul pada operasi deteksi *drift*. Ketiganya menyangkut keputusan otomasi yang tidak dapat menunggu label kebenaran tersedia. Penanganannya dirancang pada jalur kendali yang sama agar tidak menambah cabang *workflow* baru. Ketiga kasus tersebut adalah sebagai berikut.

1. ***Drift* terdeteksi sebelum label tersedia**

Ketika *drift* terdeteksi pada fitur tetapi label belum tersedia, *workflow* hanya melakukan pelatihan ulang setelah label berjalan cukup lama.

2. ***Drift* akibat perubahan distribusi sementara**

Ketika *drift* terjadi karena perubahan distribusi sementara, ambang dihitung dengan jendela bergulir agar tidak memicu pelatihan ulang yang tidak perlu.

3. **Kegagalan versi baru pada analisis *canary***

Ketika versi baru gagal pada analisis *canary*, promosi dihentikan otomatis sehingga versi stabil pada lapisan penyajian tetap dipertahankan.

Ketiga kasus tersebut dirancang untuk ditangani tanpa percabangan tambahan pada *workflow*. CronWorkflow pelatihan ulang disusun atas dua langkah berurutan, yaitu pengukuran *drift* dan langkah keputusan yang memicu pelatihan ulang, sedangkan penghentian promosi saat analisis *canary* gagal menjadi tanggung jawab Argo Rollouts pada lapisan penyajian. Ambang PSI dan KS beserta jendela *lookback* ditetapkan sebagai parameter pada CronWorkflow sehingga setiap perubahan kebijakan tetap melewati *commit* Git dan dapat diaudit. Parameter tersebut tersimpan pada manifes di repositori Git sehingga nilai ambang yang berlaku selalu sama dengan yang tercatat pada riwayat *commit*.

IV.5 Perancangan Sub-sistem Layanan Fitur *Dual-Store*

Bagian ini menjawab T-4 dengan merancang sub-sistem layanan fitur yang menyatukan fitur tabular dan fitur deret waktu dalam satu kontrak data dari satu sumber kebenaran, sedangkan *embedding* berdimensi tinggi dikelola pada indeks vektor terpisah. Feast digunakan sebagai *framework* layanan fitur yang menyatukan kontrak antara *offline* dan *online* untuk fitur tabular. Pemisahan beban kerja dilakukan secara fisik antara *store* kunci-nilai untuk fitur tabular dan indeks vektor untuk *embedding*. Fitur tabular diambil melalui API Feast, sedangkan *embedding* diambil melalui klien *native* Qdrant pada saat *inference*.

IV.5.1 Penyimpanan *Offline* dan *Online*

Offline store menggunakan ClickHouse untuk fitur tabular dan deret waktu yang dipakai pada fase pelatihan, dengan tabel kolom yang dipetakan langsung ke definisi FeatureView Feast. *Online store* menggunakan Valkey (Valkey Contributors 2025) untuk fitur yang dipakai pada fase *inference* dengan target latensi p99 di bawah sepuluh milidetik. Dasar pemilihan kedua komponen mengikuti perbandingan pada Bab II, sedangkan pada perancangan ini keduanya diikat pada satu kontrak Feast sehingga skema *offline* dan *online* berasal dari definisi yang sama. Pemisahan peran ini memungkinkan penyetelan sumber daya pelatihan dan penyajian secara mandiri tanpa menduplikasi definisi fitur.

Vector search HNSW tidak digabungkan langsung pada *online store* tabular, melainkan disediakan oleh Qdrant (Qdrant Team 2025) sebagai indeks vektor terpisah, sehingga beban kerja kunci-nilai dan beban kerja pencarian vektor terpisah secara operasional. Pemisahan *store* memudahkan penyetelan *resource* secara mandiri karena beban Valkey berorientasi kunci-nilai dengan akses *point lookup* sedangkan beban Qdrant berorientasi pencarian tetangga terdekat yang membutuhkan memori untuk indeks HNSW. Valkey dideklarasikan sebagai `online_store` pada repositori Feast, sedangkan Qdrant berdiri sebagai indeks vektor terpisah yang diakses melalui klien *native* di luar Feast, bukan sebagai *store* kedua di dalam Feast. Pembaruan skema fitur diterapkan ke ClickHouse melalui ALTER TABLE terkelola oleh dbt dan secara bersamaan ke Feast melalui `feast apply` sehingga skema *offline* dan *online* selalu sinkron.

IV.5.2 Dukungan *Vector Embeddings* dan HNSW

Embedding dihasilkan oleh model `sentence-transformers/all-mpnet-base-v2` (Reimers dan Gurevych 2019) berdimensi 768 untuk teks, dan disimpan sebagai fitur dengan dimensi yang konsisten antara fase pelatihan dan fase *inference*. Pencarian tetangga terdekat dijalankan oleh Qdrant (Qdrant Team 2025) dengan indeks HNSW (Malkov dan Yashunin 2020) berparameter `m=16` dan `ef_construct=200` yang memberikan keseimbangan yang baik antara waktu kueri dan kualitas. Qdrant menyediakan kueri *gRPC* berlatensi rendah serta penyaringan *payload* pada saat pencarian, sehingga *embedding* dapat digabung dengan fitur tabular dari Valkey untuk membentuk *request inference* yang lengkap. Bruch, Gai, dan Ingber (2023) dan Campese, Lauriola, dan Moschitti (2024) memberi rujukan untuk fungsi penggabungan hasil pencarian pada konteks *hybrid retrieval*.

Parameter HNSW dikonfigurasi melalui *collection* Qdrant yang dideklarasikan oleh *init Job* sehingga penambahan koleksi baru dapat dilakukan dengan menambah berkas YAML, tanpa intervensi langsung pada Qdrant. *Snapshot* koleksi vektor dijalankan secara berkala ke MinIO sebagai bagian dari pencadangan Velero untuk objek terkait. Pemilihan dimensi 768 berdasarkan model `all-mpnet-base-v2` memberi keseimbangan antara kualitas semantik dan biaya penyimpanan, dan dapat disesuaikan dengan model alternatif tanpa mengubah kontrak Feast. Latensi p99 ditargetkan di bawah dua puluh milidetik untuk pencarian top-10 pada koleksi berukuran sampai sepuluh juta vektor.

IV.5.3 Jaminan *Point-in-Time Correctness*

Jaminan *point-in-time correctness* pada Feast diwujudkan melalui operasi `get_historical_features`. Operasi ini melakukan *point-in-time join* antara entitas dan riwayat fitur dengan menggunakan `event_timestamp` dan `created_timestamp`, sehingga nilai fitur yang dipakai pada saat pelatihan identik dengan nilai yang akan tersedia pada saat *inference* di waktu yang sama. Sebagaimana dirumuskan pada Bab II, kebocoran temporal membuat evaluasi model tampak lebih baik daripada kinerja sebenarnya, dan kontrak Feast mencegahnya secara struktural pada level definisi fitur. Setiap `FeatureView` mendeklarasikan `ttl` untuk membatasi usia fitur yang dianggap valid, sehingga fitur usang tidak diam-diam masuk ke set pelatihan.

Mekanisme jaminan ini diuji melalui dua jalur. Pertama, *notebook* reproduksi me-

manggil `get_historical_features` pada titik waktu historis dan membandingkan keluaran dengan ekstraksi manual yang dijalankan terhadap log peristiwa Kafka, sehingga validitas dapat dibuktikan pada level kasus. Kedua, kontrak antara Online Store dan Offline Store divalidasi dengan menjalankan kueri yang sama pada Valkey dan ClickHouse pada titik waktu yang sama dan membandingkan keluaran. Kombinasi dua jalur uji ini menutup ruang *temporal leakage* struktural pada lapisan fitur tanpa bergantung pada disiplin manual pengembang *pipeline*.

IV.5.4 Aliran Materialisasi Fitur

Materialisasi fitur dilakukan dengan dua jalur. Jalur *batch* menjalankan transformasi pada ClickHouse atau Spark, kemudian Feast memuat hasilnya ke *online store* secara periodik melalui operasi `materialize`. Jalur *stream* menggunakan Flink untuk menghitung fitur *real-time* yang ditulis langsung ke Valkey untuk fitur tabular, serta dipublikasikan ke ClickHouse pada saat yang sama, sehingga *offline* dan *online* tetap konsisten. Kedua jalur menulis ke penyimpanan yang sama-sama terdaftar pada FeatureView Feast sehingga konsumen tidak perlu mengetahui jalur mana yang menghasilkan sebuah nilai.

Jadwal materialisasi *batch* dipasang pada DAG Airflow melalui `task feast_materialize` dengan interval per jam yang mendorong fitur dari *offline store* ke *online store*. Eksekusi materialisasi mengeluarkan metrik durasi dan jumlah baris ke Pushgateway sehingga ketertinggalan dapat terdeteksi melalui *alerting* Prometheus. Jalur *stream* menggunakan semantik *exactly-once* Flink dengan *checkpoint* ke MinIO setiap interval pendek, sehingga kegagalan *TaskManager* tidak menyebabkan duplikasi pada *online store*. Pada jalur *batch*, nilai pada *online store* merupakan hasil materialisasi dari *offline store* yang sama sehingga konsistensi antara keduanya terjaga secara struktural, sedangkan kualitas data lintas lapisan diverifikasi oleh *checkpoint* Great Expectations terjadwal.

IV.6 Alur Kerja End-to-End

Alur kerja menyatukan empat sub-sistem ke dalam satu siklus berkelanjutan dengan urutan tahap sebagai berikut.

1. Ingestasi dan validasi data

Data baru masuk melalui Kafka, divalidasi oleh Great Expectations, kemudian disimpan pada *warehouse* ClickHouse dan *data lake* MinIO dengan katalog Lakekeeper.

2. **Komputasi dan materialisasi fitur**

Fitur dihitung pada Flink atau Spark dan ditulis ke ClickHouse sebagai *offline store*, lalu diregistrasi pada Feast dan dimaterialisasi ke Valkey sebagai *online store* tabular, sedangkan indeks vektor Qdrant diisi melalui jalur tersendiri di luar materialisasi Feast.

3. **Pengukuran *drift* dan pelatihan ulang**

Argo CronWorkflow mengukur *drift* dan, jika ambang dilewati, memicu *pipeline* pelatihan Kubeflow Pipelines yang menggunakan `get_historical_features` untuk menjamin konsistensi temporal.

4. **Registrasi dan promosi model**

Hasil pelatihan ditulis ke MLflow, model kandidat didaftarkan sebagai versi baru, lalu promosi pada lapisan penyajian dikendalikan oleh Argo Rollouts dengan strategi *canary* berbasis analisis metrik.

5. **Sinkronisasi konfigurasi dan observabilitas**

Perubahan pada konfigurasi disinkronkan dari Git oleh Argo CD, sedangkan kondisi sistem dipantau oleh Prometheus, Loki, dan Grafana Tempo melalui Grafana.

Siklus ini berputar setiap kali data baru masuk atau *drift* terdeteksi, sehingga platform terus menerus menjaga akurasi model di lingkungan produksi. Rodrigues dkk. (2025) memberi rujukan tambahan untuk arsitektur pembelajaran *near real-time* pada lingkungan yang serupa, dan pendekatan yang dirancang di sini mengadaptasi rujukan tersebut ke kontrak Feast dan Argo Rollouts agar siklus tetap dapat dijalankan secara deklaratif pada satu *cluster* Kubernetes.

BAB V

IMPLEMENTASI ARSITEKTUR PLATFORM DATAOPS DAN MLOPS

Bab ini membahas implementasi platform DataOps dan MLOps terintegrasi sesuai rancangan pada Bab IV. Pembahasan dimulai dari lingkungan implementasi yang menjadi dasar *deployment* komponen, kemudian dilanjutkan dengan implementasi empat sub-sistem mengikuti urutan pemenuhan tujuan T-1 sampai T-4. Bab ini ditutup dengan verifikasi jalur eksekusi menggunakan satu *use case* sebagai kasus pengujian. Penjabaran detail biner dan konfigurasi disimpan pada repositori platform, sehingga bab ini menyajikan pokok-pokok implementasi yang menjelaskan bentuk akhir rancangan.

V.1 Lingkungan Implementasi

Lingkungan implementasi platform dibangun pada satu *cluster* Kubernetes (Cloud Native Computing Foundation 2024a) berbasis k3s yang berjalan pada satu *node*. Pilihan ini memenuhi kebutuhan eksekusi pada lingkungan terbatas tanpa mengubah pola *deployment* yang dapat dipakai pada *cluster* produksi dengan banyak *node*. Distribusi k3s membawa bidang kendali ringan, *containerd* sebagai *runtime*, dan beberapa *add-on* bawaan yang dapat dinonaktifkan agar tidak berbenturan dengan komponen pilihan platform. Konfigurasi *node* memakai satu *disk* lokal dengan *ext4* sebagai *filesystem*, dan tata letak *namespace* pada platform dibakukan agar perpindahan ke *cluster* produksi dengan banyak *node* tinggal mengubah berkas *overlay Kustomize* tanpa mengganti manifes dasar.

Kode dan konfigurasi proyek ditata pada beberapa direktori utama dengan batas tanggung jawab yang jelas. Tabel V.1 merangkum direktori tersebut beserta fungsinya, dengan *platform/* sebagai inti yang bersifat domain-agnostik dan direktori *use case* sebagai kasus pengujian yang terpisah. Pemisahan ini memastikan platform dapat dipakai ulang lintas domain tanpa menyentuh berkas yang spesifik pada satu *use case*. Direktori *experiments/* dan *materials/* berperan sebagai penunjang verifikasi dan dokumentasi yang tidak menjadi bagian dari jalur *deployment* platform.

Tabel V.1 Struktur Direktori Utama Proyek dan Fungsinya

Direktori	Fungsi
platform/	Kode, manifes, dan konfigurasi seluruh komponen platform DataOps dan MLOps yang bersifat domain-agnostik
use-case-crypto/	<i>Use case</i> uji berupa pasar kripto, memuat layanan ingestasi, definisi fitur, <i>pipeline</i> , dan <i>overlay Kustomize</i> yang spesifik domain
experiments/	Skenario verifikasi dan <i>oracle</i> pengujian, antara lain uji paritas <i>batch-stream</i> , beban, <i>drift</i> , dan <i>chaos</i>
materials/	Dokumen tugas akhir, proposal, referensi, dan berkas laporan

V.1.1 Konfigurasi Dasar Cluster

Cluster menjalankan komponen dasar berikut: Istio (Istio Authors 2024) sebagai *service mesh* dengan mTLS dalam mode *strict* pada lalu lintas internal, APISIX (Apache APISIX Authors 2025) sebagai *API gateway* pada perbatasan *mesh*, kontrol akses jaringan menggunakan *NetworkPolicy* pada level Kubernetes, dan Kyverno (Kyverno Authors 2024) sebagai *policy engine* pada lapis admisi. Falco (Falco Authors 2025) memantau perilaku *runtime* pada level *kernel*, sedangkan Trivy Operator (Aqua Security 2025) memindai *image* dan beban kerja secara berkala. *Storage class* bawaan menggunakan *local-path provisioner* untuk *persistent volume* berbasis penyimpanan lokal, dengan Velero (Velero Authors 2025) melakukan pencadangan terjadwal ke MinIO. Chaos Mesh (Chaos Mesh Authors 2025) disiapkan untuk uji ketahanan dengan eksperimen *fault injection* yang terbatas pada *namespace* tertentu.

Konfigurasi sumber daya memberi prioritas pada satu replika *idle* per beban kerja, sementara *Horizontal Pod Autoscaler*, *Vertical Pod Autoscaler* dalam mode rekomendasi, dan KEDA *ScaledObject* disiapkan pada *template* agar penskalaan dapat aktif ketika beban nyata datang. *Resource limits* pada setiap kontainer ditetapkan sesuai panduan Kyverno *require-resource-limits* untuk mencegah kelebihan kuota *node*. Penjadwalan kelas prioritas memakai *PriorityClass* platform agar komponen kritis tidak tergusur oleh beban kerja sekali pakai. *External Snapshotter* dipasang sebagai prasyarat untuk *snapshot* CSI yang dipakai oleh Velero, sehingga rekonstruksi *cluster* dari pencadangan dapat dilakukan tanpa intervensi manual.

V.1.2 Manajemen Rahasia dan Identitas

OpenBao (OpenBao Authors 2025) bertindak sebagai penyimpan rahasia. *External Secrets Operator* bertugas menyebarkan rahasia dari OpenBao ke *namespace* target tanpa menyalin nilai rahasia ke dalam berkas manifes. Identitas pengguna dikelola oleh dex (Dex Authors 2025) sebagai *identity provider* OIDC, dengan oauth2-proxy (OAuth2 Proxy Authors 2025) sebagai *reverse proxy* yang menegakkan autentikasi

pada antarmuka aplikasi seperti DataHub, MLflow, Grafana, Argo CD, dan Kube-flow Pipelines. Selain identitas pengguna, kebijakan otorisasi beban kerja pada lapis admisi ditegakkan oleh Kyverno (Kyverno Authors 2024) sebagai bagian dari dasar keamanan *cluster*.

OpenBao dibangkitkan oleh *Job* `openbao-bootstrap` yang melakukan *init* dan *auto-unseal* dengan kunci yang tersimpan di MinIO (MinIO, Inc. 2024) dengan enkripsi sisi *server*. *External Secrets Operator* dipasang pada *namespace* `security` dengan `ClusterSecretStore` tunggal yang berisi konfigurasi koneksi ke OpenBao. SpiceDB (AuthZed 2025) dipasang sebagai layanan otorisasi berbasis Zanzibar yang menetapkan hubungan akses pada level objek tata kelola, dengan basis data PostgreSQL melalui CNPG sebagai *datastore*. *Key Encryption Service* dipasang sebagai *sidecar* MinIO untuk enkripsi rahasia, dengan rotasi kunci dijalankan oleh *CronJob* terjadwal.

V.1.3 Pola GitOps

Argo CD (Argo Project 2024) menjadi satu-satunya jalur perubahan pada *cluster*. Repositori Git pada Gitea (Gitea Contributors 2025) bertindak sebagai sumber kebenaran konfigurasi. Setiap perubahan dilakukan melalui *commit* pada repositori dan diterapkan ke *cluster* oleh Argo CD melalui rekonsiliasi. Pola ini menjamin reproduksibilitas dan menghasilkan jejak audit untuk setiap perubahan. *Pipeline* CI yang dijalankan oleh Tekton (Tekton Contributors 2024) membangun *image* dan menjalankan pengujian sebelum *commit* menjadi sumber *deployment*.

ApplicationSet Argo CD dipakai untuk mendeklarasikan banyak *Application* sekaligus dengan satu *template* per kelompok, sehingga penambahan komponen baru tinggal menambahkan *generator* tanpa menambah berkas *Application* satu per satu. Argo CD diatur dengan mode `manual-sync` pada *ApplicationSet* *template* untuk mencegah `auto-prune` yang dapat menghapus sumber daya secara tidak sengaja. Tekton menjalankan *Pipeline* yang menghasilkan *image* ke registri `localhost:5000` yang berfungsi sebagai *mirror* lokal, dan menulis kembali *commit* dengan referensi *image* yang sudah diperbarui pada `values.yaml` *kustomization*. *Pre-apply* dan *post-apply hook* pada `apply-component.sh` menutup celah *race condition* antara CRD dan CR, sehingga seluruh komponen *cluster* dapat dipasang ulang dengan satu perintah `Makefile`.

V.2 Implementasi Arsitektur Terintegrasi

Bagian ini mengimplementasikan rancangan pada Bagian IV.2. Komponen disusun ke dalam *namespace* per lapisan dan dihubungkan melalui kontrak *API* yang stabil. Setiap *namespace* memuat satu kelompok komponen fungsional sehingga kebijakan *NetworkPolicy* dapat ditegakkan dengan model *default-deny* dan pengecualian eksplisit per arah komunikasi. *Deployment* komponen mengikuti pola GitOps melalui Argo CD, sedangkan eksekusi *pipeline* CI mengikuti pola Tekton, dengan setiap komponen tunduk pada *template ApplicationSet* agar penambahan komponen baru tidak menambah *boilerplate*.

V.2.1 Lapisan Ingestasi

Apache Kafka (Apache Software Foundation 2024c) diterapkan menggunakan operator Strimzi. *Kafka Connect* digunakan untuk integrasi dengan basis data eksternal melalui Debezium dan untuk *sink* ke Apache Iceberg pada MinIO (MinIO, Inc. 2024) dengan katalog Lakekeeper (Lakekeeper Authors 2025). *Karapace* berperan sebagai registri skema dengan dukungan Avro dan JSON Schema. Lalu lintas masuk dari konsumen di luar *mesh* dilewatkan melalui *API gateway* APISIX (Apache APISIX Authors 2025) dengan mTLS yang dijembatani pada perbatasan.

Cluster Kafka diatur dengan *KRaft* murni tanpa ZooKeeper agar topologi tetap ringan. SASL_SSL dipakai sebagai mekanisme autentikasi antar klien dengan sertifikat klien dikelola Strimzi melalui *KafkaUser*. *Topic* dideklarasikan sebagai *KafkaTopic* CR dan ACL dideklarasikan sebagai bagian dari *KafkaUser* sehingga keseluruhan kontrak ingestasi tersimpan sebagai sumber daya Kubernetes yang dapat di-audit. PeerAuth PERMISSIVE diterapkan pada *broker* agar penyedia data di luar *mesh* masih dapat berkomunikasi tanpa terlebih dahulu menambahkan *sidecar* Istio.

V.2.2 Lapisan Pemrosesan

Pemrosesan *stream* menggunakan operator Flink yang mengelola *FlinkDeployment*. Pemrosesan *batch* menggunakan operator Spark dan dijalankan sebagai *SparkApplication*. Transformasi pada *warehouse* menggunakan dbt yang dijalankan sebagai *task* KubernetesPodOperator pada DAG Airflow (Apache Software Foundation 2024a). Airflow mengorkestrasi *pipeline* data berorientasi *batch*, termasuk pembangunan lapisan *gold*, sedangkan orkestrasi pelatihan model ditangani oleh Kubeflow Pipelines.

Flink dipasang dengan *session cluster* mode *Standalone* dan tugas *stream* dideklarasikan sebagai FlinkSessionJob CR sehingga setiap *job* dapat di-*deploy* secara independen melalui Argo CD. Spark Operator memanfaatkan *webhook* mutasi untuk menyisipkan *node selector* dan *toleration* berdasarkan label *queue-name* yang ditetapkan oleh Kueue. dbt menjalankan model SQL pada ClickHouse dengan *profiles.yaml* yang merujuk rahasia dari OpenBao melalui ESO. Airflow memakai *git-sync sidecar* yang menarik DAG dari Gitea sehingga pembaruan DAG tidak memerlukan pembangunan ulang *image*.

V.2.3 Lapisan Penyimpanan

Warehouse kolumnar menggunakan ClickHouse (ClickHouse Inc 2024) dengan operator ClickHouse pada satu *shard*. *Data lake* menggunakan MinIO sebagai *object store* dan Apache Iceberg sebagai *table format*, dengan Lakekeeper (Lakekeeper Authors 2025) sebagai katalog Iceberg REST dan Trino (Trino Software Foundation 2024) sebagai *query engine*. Penyimpanan kunci-nilai dan vektor menggunakan Valkey (Valkey Contributors 2025) dengan protokol RESP untuk akses berlatensi rendah, dan Qdrant (Qdrant Team 2025) sebagai basis data vektor untuk *embedding* berdimensi tinggi. Penyimpanan relasional menggunakan PostgreSQL melalui operator CNPG, dan MySQL melalui operator yang sesuai untuk komponen yang memerlukan *engine* berbeda. Apache Superset (Apache Superset Authors 2025) disiapkan sebagai antarmuka BI untuk peran *business user* dengan koneksi ke ClickHouse, Trino, dan MySQL melalui SQLAlchemy.

ClickHouse memakai ClickHouseKeeper sebagai pengganti ZooKeeper untuk koordinasi metadata replikasi. MinIO dipasang dengan empat drive pada satu *node* untuk menjaga skema penamaan erasure code yang seragam ketika *cluster* berpindah ke banyak *node*. Lakekeeper menyediakan Warehouse dan Project REST API yang dipakai oleh Spark, Trino, dan Flink dengan menggunakan *header x-project-id* sebagai pemisah antar *warehouse*. lakeFS (Treeverse Ltd. 2025) dipasang sebagai lapisan *version control* di atas MinIO sehingga eksperimen pada cabang data dapat dijalankan tanpa mengganggu data utama.

V.2.4 Lapisan Siklus Hidup Model dan Penyajian

MLflow (MLflow Contributors 2024) dijalankan dengan basis data PostgreSQL dan *artifact store* pada MinIO. Kubeflow Pipelines (Kubeflow Contributors 2024) dijalankan pada *namespace* terpisah dengan dukungan *multi-tenant*, dilengkapi Kubeflow Notebooks untuk lingkungan eksperimen interaktif dan Kubeflow Trainer

(Kubeflow Authors 2025c) untuk pelatihan terdistribusi. KServe (KServe Contributors 2024) dipakai untuk *inference* dengan dukungan *runtime* MLflow, scikit-learn, dan ONNX, dan dengan Knative Serving (Knative Authors 2025) sebagai bidang *serverless* bawaan yang menyediakan penskalaan ke nol. Argo Workflows berperan sebagai *orchestrator job* di luar Kubeflow Pipelines, sedangkan Argo Rollouts (Argo Rollouts Authors 2025) mengelola promosi versi pada lapisan penyajian dengan strategi *canary* dan *progressive analysis* berbasis metrik Prometheus.

Kubeflow Pipelines memakai MySQL sebagai *metadata store* dan MinIO sebagai *artifact store*, dengan kfp-launcher yang membungkus eksekusi langkah *pipeline* agar autentikasi ke MinIO dilakukan menggunakan `secretAsEnv`. KServe InferenceService dideklarasikan pada *namespace* `model-serving` dengan label `part-of=kserve` agar Kyverno tidak menolak *Pod mutator* bawaan. Kueue (Kueue Authors 2024) mengelola antrian pelatihan dengan ClusterQueue per kelompok beban kerja sehingga sumber daya CPU dan GPU dapat dibagi adil antara *tenant*. Argo Rollouts mengevaluasi metrik dari Prometheus pada setiap langkah *canary* untuk menentukan apakah versi *canary* layak dipromosikan menjadi versi *stable*.

V.2.5 Lapisan Observabilitas

Prometheus (Prometheus Authors 2024) mengumpulkan metrik dari semua komponen dengan *ServiceMonitor* dan *PodMonitor*. Loki (Grafana Labs 2024b) mengumpulkan log melalui Alloy. Grafana Tempo (Grafana Labs 2024c) menerima *trace* dari OpenTelemetry Collector yang tersebar pada *node*. Grafana (Grafana Labs 2024a) menggabungkan ketiganya pada satu antarmuka.

Sloth (Sloth Authors 2025) menurunkan definisi *service-level objective* ke dalam aturan Prometheus sehingga *error budget* dapat dipantau bersama metrik lain. Pushgateway berperan sebagai titik kumpul metrik untuk *job* berdurasi pendek yang berjalan di luar siklus *scrape*, termasuk CronJob pengukuran *drift* dan *CronWorkflow* pemicu pelatihan ulang. OpenCost dipakai untuk pemantauan biaya per beban kerja, sedangkan Pyroscope mengumpulkan profil CPU dan memori secara berkelanjutan dari layanan yang menjadi *hotspot*. Evidently (Evidently AI 2025) dijalankan sebagai *job* berkala yang menghasilkan *report* HTML kualitas distribusi fitur dan disimpan pada MinIO sebagai jejak audit yang dapat dirujuk dari DataHub.

V.3 Implementasi Sub-sistem Tata Kelola Data

Subbab ini menurunkan rancangan tata kelola pada Bagian IV.3 menjadi implementasi, dengan fokus pada konsistensi katalog metadata, jadwal pengujian kontrak data, dan penegakan kontrol akses pada tiga lapis yang berbeda. Penomoran KF-08, KF-09, dan KF-10, serta kebutuhan KNF terkait keamanan dipakai sebagai acuan untuk menilai apakah implementasi memenuhi seluruh kebutuhan yang dirumuskan pada Bab III. Katalog diwujudkan oleh DataHub dengan penerimaan *lineage* OpenLineage, kontrak kualitas oleh Great Expectations pada *pipeline* Airflow, dan kontrol akses oleh gabungan Dex, SpiceDB, serta kebijakan jaringan. Tiga lapis akses yang dimaksud mencakup autentikasi konsol, otorisasi granular, dan kebijakan jaringan antar *namespace*.

V.3.1 Katalog Metadata dan Lineage

DataHub (DataHub Project 2024) dijalankan dengan komponen GMS, *frontend*, dan beberapa *job ingest*, dengan OpenSearch sebagai *search backend* dan PostgreSQL sebagai *metadata backend*. *Job ingest* berjalan secara berkala sebagai *CronJob* untuk menarik metadata dari Kafka, ClickHouse, MinIO dan Iceberg, Feast, MLflow, dan Kubeflow Pipelines. Hasil *ingest* menyatukan *lineage* antar lapisan, mulai dari *topic* pada Kafka hingga model yang dilayani oleh KServe. OpenLineage (OpenLineage Authors 2025) dipakai sebagai protokol pengirim peristiwa *lineage* dari komponen *pipeline*, baik Airflow maupun Kubeflow Pipelines, agar graf ketertelusuran di DataHub tersusun dengan kosa kata yang seragam.

Image datahub-feast-deps dibangun secara lokal dengan Kaniko untuk menambahkan *plugin* Feast pada *job ingest* sehingga sumber Feast dapat dipindai secara langsung. Autentikasi ke GMS menggunakan token sistem yang disimpan di OpenBao dan disebarkan ke *namespace* data-governance oleh *External Secrets Operator*. Konfigurasi DataHub pada level UI dikelola melalui datahub-frontend-secret dengan *checksum annotation* pada GMS dan *frontend* agar rotasi rahasia memicu *auto-roll deployment*. Jadwal *ingest* dapat berkonflik dengan beban kerja besar di *node*, dan untuk itu *native sidecar* Istio diaktifkan pada *CronJob* agar koneksi mTLS ke GMS tidak putus saat *sidecar* dihentikan setelah *job* selesai.

V.3.2 Kontrak Data dan Kualitas

Great Expectations (Great Expectations 2024) mendefinisikan ekspektasi kualitas data pada lapisan *bronze* setelah ingestasi dan pada lapisan *silver* dan *gold* setelah transformasi. Setiap *expectation suite* disimpan pada repositori Git dan dijalankan oleh *CronJob* yang membaca hasil ke DataHub. Pengujian yang gagal akan menutup jalur promosi ke lapisan berikutnya dan memunculkan peringatan pada Grafana. Hasil ekspektasi disimpan ke MinIO sebagai *Data Docs* HTML sehingga konteks kegagalan dapat ditelusuri tanpa perlu menjalankan kembali *suite*.

dbt menjalankan model SQL untuk lapisan *silver* dan *gold* pada ClickHouse, dengan *schema.yml* yang memuat *tests* dasar seperti *unique*, *not_null*, dan *accepted_values*. Hasil dbt dirilis ke DataHub melalui *run event* OpenLineage yang dikirim oleh *task* KubernetesPodOperator dbt pada DAG Airflow. *Checkpoint* Great Expectations dijadwalkan per lapisan dengan jeda yang berbeda agar ekspektasi yang berat tidak berdampak pada beban *warehouse*. *Suite* fakta pelatihan diuji setelah materialisasi pada tabel *gold.fct_training_data* agar set pelatihan tidak masuk *pipeline* pelatihan apabila kualitas data tidak memenuhi kontrak.

V.3.3 Kontrol Akses

Kontrol akses pada lapis jaringan menggunakan *NetworkPolicy* dengan kebijakan *default-deny* per *namespace* dan pengecualian yang dideklarasikan secara eksplisit. Pada lapis aplikasi, kontrol akses memakai jalur autentikasi tunggal dex dan oauth2-proxy yang telah dirinci pada Subbab V.1.2, sehingga akses ke konsol tata kelola tunduk pada sesi identitas yang sama dengan layanan platform lain. Pada lapis admisi, Kyverno (Kyverno Authors 2024) menegakkan kebijakan keamanan beban kerja sesuai rancangan pada Subbab IV.3.3. Falco (Falco Authors 2025) memantau peristiwa *runtime* pada level *kernel* untuk mendeteksi pelanggaran kebijakan keamanan setelah *pod* berjalan.

SpiceDB (AuthZed 2025) berfungsi sebagai layanan otorisasi yang dipakai DataHub dan Kubeflow Pipelines untuk hubungan akses pada level objek, dengan basis data PostgreSQL melalui CNPG. *Policy* Kyverno disusun pada *namespace security* dengan *policy generator* yang menyebarkan kebijakan ke *namespace* aplikasi tanpa duplikasi sumber daya. Kyverno mode *failurePolicy=Ignore* dipakai pada *ValidatingPolicy* agar admisi tidak terblokir saat *webhook* gagal pada kondisi *cluster* yang sibuk. Falco mengirim peringatan ke Loki melalui *falcosidekick* dengan *topic* khusus sehingga tim keamanan dapat menelusuri peristiwa terkait kebi-

jakan dari satu antarmuka Grafana.

V.4 Implementasi Sub-sistem Deteksi *Drift* dan *Continuous Training*

Implementasi sub-sistem deteksi *drift* menindaklanjuti rancangan pada Bagian IV.4, dengan fokus pada otomatisasi penuh *control loop* dari pengukuran sampai promosi versi serta penegakan ambang yang dapat dikonfigurasi per model. Pemicu pelatihan ulang dirancang agar dapat dijalankan secara berkala atau dipicu oleh peristiwa *drift* yang melebihi ambang. Keseluruhan implementasi pada bagian ini menjawab KF-07 untuk deteksi *drift* otomatis berikut pemicuan pelatihan ulang, sehingga model pada lapisan penyajian tidak dibiarkan usang tanpa evaluasi berkala saat distribusi data bergeser. Seluruh langkah *loop* diwujudkan pada CronWorkflow *retrain-on-drift* yang membaca skor dari tabel analitik dan memanggil *pipeline* pelatihan Kubeflow Pipelines.

V.4.1 Detektor *Drift*

Detektor *drift* diimplementasikan sebagai *service* Python yang membaca fitur dari ClickHouse dengan jendela waktu yang dapat dikonfigurasi. Indikator PSI dihitung dengan *binning* adaptif, dan uji Kolmogorov-Smirnov dijalankan dengan ambang yang dapat disesuaikan per fitur (Reis dkk. 2016; Bayram, Ahmed, dan Kassler 2022; Lu dkk. 2019). Hasil pengukuran ditulis ke ClickHouse sebagai tabel `gold.drift_multi_scale` dan dipublikasikan ke Pushgateway agar dapat diserap oleh Prometheus. Evidently (Evidently AI 2025) dijalankan sebagai pelapor pelengkap yang menghasilkan ringkasan distribusi fitur dan kinerja model dalam bentuk *report HTML* yang dapat diaudit dan dilampirkan pada katalog DataHub.

Service berjalan sebagai *CronJob* pada *namespace* *model-lifecycle* dengan jadwal lima belas menit untuk fitur fluktuatif dan satu jam untuk fitur tabular yang lebih stabil. Konfigurasi ambang per fitur dideklarasikan pada ConfigMap *drift-thresholds* sehingga perubahan ambang dapat dilakukan melalui *commit* pada Git tanpa membangun ulang *image*. Hasil tiap eksekusi ditulis dengan checksum versi konfigurasi sehingga riwayat ambang dapat ditelusuri saat audit. *Image* detektor dibangun dengan *uv* pada Dockerfile multi-stage agar model SBERT untuk deteksi *drift embedding* disertakan langsung pada lapis terakhir *image*, sehingga *cold start* tidak memerlukan unduhan model.

V.4.2 *Control Loop* Pelatihan Ulang

Control loop dijalankan oleh Argo CronWorkflow dengan jadwal yang dapat dikonfigurasi per model. *Workflow* menjalankan dua langkah utama: pengukuran *drift* dan pemicu *pipeline* pelatihan. Pemicu pelatihan mengirim permintaan ke Kube-flow Pipelines. *Pipeline* pelatihan membaca fitur pelatihan dari tabel *gold* ClickHouse, menjalankan pelatihan pada *job* dengan sumber daya yang dialokasikan oleh Kueue (Kueue Authors 2024), lalu mencatat metrik dan artefak model ke MLflow. Kelayakan model baru untuk menggantikan versi produksi ditentukan pada tahap promosi *canary* yang menilai metrik penyajiannya secara langsung sebelum trafik penuh dialihkan, bukan pada perbandingan tersendiri sebelum *deployment*.

Pada tahap promosi, Argo Rollouts (Argo Rollouts Authors 2025) menyalurkan trafik lapisan penyajian secara bertahap ke versi baru memakai strategi *canary* dengan analisis metrik Prometheus, sehingga *rollback* otomatis dapat dilakukan ketika kinerja menurun. *retrain-on-drift* CronWorkflow disimpan pada direktori *workflow* milik *use case* yang terpisah dari platform/ agar dapat di-*deploy* bersama *use case* ketika pengujian dijalankan. Konfigurasi penonaktifan *cache* KFP diterapkan pada langkah *train* dan *deploy* agar setiap eksekusi memakai data terbaru tanpa dipengaruhi *cache* historis. Pengiriman metrik *workflow* dilakukan di dalam langkah *decide-and-trigger* yang menulis ke Pushgateway, sehingga seluruh metrik eksekusi tetap terkumpul pada Prometheus meskipun *workflow* berdurasi singkat.

V.4.3 Penanganan Kasus Khusus

Implementasi menangani tiga kasus khusus yang sama dengan rancangan pada Subbab IV.4.3. Ketiganya diwujudkan tanpa menambah komponen baru di luar CronWorkflow dan Argo Rollouts yang sudah terpasang. Perilaku tiap kasus dapat diperiksa dari manifes dan log eksekusi pada *namespace* *model-lifecycle*. Rinciannya sebagai berikut.

1. **Jendela tunggu label per model**

Jendela tunggu label diatur per model agar pelatihan ulang tidak dipicu sebelum label tersedia cukup.

2. **Jendela bergulir untuk meredam fluktuasi**

Ambang *drift* dihitung dengan jendela bergulir untuk meredam fluktuasi singkat.

3. **Penghentian promosi saat analisis *canary* gagal**

Ketika analisis *canary* gagal, promosi dihentikan sehingga versi stabil pada

lapisan penyajian tetap dipertahankan.

Ketiga kasus tersebut ditangani tanpa percabangan tambahan pada *workflow*. *retrain-on-drift* CronWorkflow terdiri atas dua langkah berurutan, yaitu pengukuran *drift* dan langkah keputusan yang memicu pelatihan ulang, sedangkan penghentian promosi saat analisis *canary* gagal ditangani oleh Argo Rollouts pada lapisan penyajian. Ambang PSI dan KS beserta jendela *lookback* dideklarasikan sebagai parameter pada CronWorkflow itu sendiri, sehingga perubahan kebijakan tetap melalui *commit* Git dan dapat di-*audit*. Nilai parameter yang berlaku dapat dibaca langsung dari manifes sehingga audit kebijakan tidak memerlukan akses ke sistem lain.

V.5 Implementasi Sub-sistem Layanan Fitur *Dual-Store*

Sub-sistem layanan fitur *dual-store* mewujudkan rancangan pada Bagian IV.5 dengan satu sumber definisi fitur dan satu kontrak Feast untuk fitur tabular, sedangkan fitur *embedding* dikelola pada indeks vektor terpisah. Fitur tabular dimaterialisasi ke *online store* Valkey dan diakses melalui API Feast, sedangkan indeks vektor Qdrant diisi dan diakses melalui klien *native* di luar materialisasi Feast. Ketiga penyimpanan itu terikat pada satu sumber definisi di *feature_repo/* sehingga perubahan skema fitur cukup dilakukan pada satu tempat. Bagian ini menjawab KF-01 sampai KF-04 yang menuntut manajemen fitur terpusat, penyajian ganda *online-offline*, validitas temporal, dan dukungan *vector embedding*.

V.5.1 Konfigurasi Feast

Feast (Feast Project 2024) dikonfigurasi dengan *feature_store.yaml* yang menentukan *offline store* ClickHouse, *online store* Valkey (Valkey Contributors 2025) melalui protokol RESP, dan registri *file-based* pada MinIO. Definisi fitur ditulis dalam Python pada modul *feature_definitions.py* dan didaftarkan ke registri melalui *feast apply*. Materialisasi fitur ke *online store* dijalankan oleh *job* berkala yang membaca dari ClickHouse dengan rentang waktu yang sesuai. Berkas *feature_store.yaml* yang sama dipakai oleh seluruh klien sehingga tidak ada konfigurasi penyimpanan yang didefinisikan ulang per layanan.

Registri Feast disimpan sebagai *registry.db* di *bucket* MinIO dengan kunci yang dirotasi melalui ESO sehingga klien Feast pada berbagai *namespace* berbagi sumber kebenaran yang sama. *Service feast serve* dipasang sebagai *Deployment* yang dijangkau melalui *Service feast-server* pada *namespace* *model-lifecycle*. Pembaruan definisi fitur dilakukan oleh *service* materialisasi yang menjalankan *feast apply* sebagai langkah pada DAG *data_pipeline* Airflow, sehingga re-

gistri selalu sinkron dengan definisi pada `feature_repo/`. *Client* Feast pada layanan pemanggil dikonfigurasi dengan `registry_path` URI `s3://`, sehingga sumber kebenaran tunggal tetap terjaga ketika klien berjalan pada *namespace* berbeda.

V.5.2 Dukungan *Vector Embeddings*

Vector embedding disimpan sebagai fitur dengan tipe `Array(Float32)` pada ClickHouse dan sebagai *point* berdimensi 768 pada *collection* Qdrant (Qdrant Team 2025). Pengindeksan HNSW diaktifkan pada Qdrant dengan parameter `m` dan `ef_construct` yang dapat dikonfigurasi, dan kueri tetangga terdekat dilayani melalui *API gRPC* dengan filter *payload* pada saat pencarian. *Service vector-embedding* pada `platform/services/processing/vector/` membungkus model `sentence-transformers/all-mpnet-base-v2` sebagai penghasil *embedding* dan menyajikan *API* yang dipanggil oleh *stream processor* pada saat *event* masuk (Reimers dan Gurevych 2019). Metrik jarak yang dipakai adalah kosinus sesuai bawaan layanan *embedding* untuk vektor yang dinormalkan.

Image vector-embedding dibangun dengan `uv` dan model SBERT di-*bake* ke lapis COPY terakhir agar *startup probe* tidak terganggu oleh unduhan model pada pemicu pertama. *Collection* Qdrant dideklarasikan oleh *init Job* yang mengirim PUT ke `/collections/nama-koleksi` dengan parameter HNSW sesuai konfigurasi. *Snapshot* koleksi disimpan ke MinIO secara berkala dengan *Job* `qdrant-snapshot` yang menambah objek pencadangan ke jadwal Velero. Koleksi vektor Qdrant diakses langsung oleh layanan *embedding* melalui klien *native* Qdrant, terpisah dari *online store* Valkey yang menyajikan fitur tabular melalui Feast, sehingga jalur kueri vektor dan jalur fitur tabular tidak saling bergantung.

V.5.3 Materialisasi dan *Point-in-Time Correctness*

`get_historical_features` dijalankan pada *pipeline* pelatihan dengan kolom `event_timestamp` pada *entity dataframe*. Feast melakukan *point-in-time join* pada *offline store* ClickHouse dengan memilih nilai fitur yang `timestamp`-nya tidak melampaui `event_timestamp` setiap baris entitas dan masih berada dalam rentang `ttl FeatureView`. Ketika terdapat lebih dari satu kandidat untuk satu titik waktu, kolom `created_timestamp` dipakai sebagai penentu agar hanya nilai paling mutakhir yang diambil, sehingga nilai fitur yang dipakai pada saat pelatihan identik dengan nilai yang tersedia pada saat itu. Materialisasi ke *online store* berjalan secara berkala dengan `feast materialize-incremental`, dan *stream materializer* pada Flink menulis fitur *real-time* ke Valkey untuk fitur tabular secara bersamaan

dengan penulisan ke ClickHouse.

Materialisasi dijalankan berkala oleh DAG Airflow melalui *task* *feast_materialize* yang mendorong fitur dari *offline store* ClickHouse ke *online store* Valkey pada jadwal per jam. Karena nilai pada Valkey merupakan hasil materialisasi dari ClickHouse, konsistensi antar *store* terjaga secara struktural tanpa salinan yang dipelihara terpisah. Kualitas data diverifikasi oleh *CronJob* *ge-checkpoint* yang menjalankan *suite* Great Expectations terhadap tabel pada ClickHouse setiap enam jam, dengan hasil validasi ditulis ke MinIO dan *gauge* lulus atau gagal dikirim ke Pushgateway. Pengujian *point-in-time* dijalankan sebagai skenario SK-F-03 yang mengaudit *get_historical_features* pada *dataset* pelatihan, memastikan kontrak *event_timestamp* dan *t1* tetap dipatuhi oleh *FeatureView* yang ditambahkan.

V.6 Verifikasi Implementasi

Verifikasi jalur eksekusi dilakukan dengan satu *use case* domain nyata sebagai kasus pengujian, yang struktur layanan, konfigurasi, dan datanya dirinci pada Bab VI. Pemilihan ini tidak mengubah sifat domain-agnostik dari platform, karena seluruh kode dan konfigurasi platform tetap berada pada repositori *platform/*, sedangkan kode dan konfigurasi spesifik domain tinggal pada direktori *use case* tersendiri yang terpisah dari platform. *Manifest* Kubernetes domain mengikuti pola *base* dan *overlays Kustomize* sehingga parameter spesifik lingkungan cukup disetel pada lapis *overlay* tanpa menyentuh definisi dasar.

V.6.1 Lingkup Verifikasi

Verifikasi mencakup lima jalur yang masing-masing menyoroti satu sub-sistem. Ketimanya dipilih dari alur ujung ke ujung *use case* sehingga setiap kontrak antar lapis teruji oleh trafik nyata. Setiap jalur memiliki keluaran yang dapat diamati pada dashboard atau tabel analitik sebagai bukti. Kelima jalur tersebut sebagai berikut.

1. **Jalur ingestasi**

Jalur ingestasi diuji dengan *stream* dari sumber eksternal yang masuk ke Kafka melalui layanan *websocket-collector* pada *use case*.

2. **Jalur pemrosesan**

Jalur pemrosesan diuji dengan *stream-processor* berbasis Flink yang menulis fitur agregasi ke ClickHouse dan Valkey.

3. **Jalur pelatihan**

Jalur pelatihan diuji dengan *pipeline* Kubeflow yang membaca fitur dari Feast dan menulis model ke MLflow.

4. Jalur penyajian

Jalur penyajian diuji dengan *InferenceService* KServe yang memuat model dari MLflow, dengan promosi versi pada lapisan penyajian yang dikelola oleh Argo Rollouts.

5. Jalur pelatihan ulang

Jalur pelatihan ulang diuji dengan menjalankan *drift-detector* dan memicu *retrain-on-drift workflow*.

Uji ketahanan pendek menggunakan Chaos Mesh (Chaos Mesh Authors 2025) untuk menyuntikkan *fault* pada *pod* terpilih. Skenario yang dipakai adalah PodChaos yang mematikan *pod* terpilih lalu mengamati pemulihannya oleh *controller*. Pencadangan dijalankan terjadwal oleh Velero (Velero Authors 2025) ke *bucket* MinIO khusus pencadangan sebagai langkah disiplin pemulihan. Kedua mekanisme diuji kembali pada skenario ketahanan di Bab VI.

Lima jalur tersebut dipilih agar setiap sub-sistem mendapat verifikasi minimal satu skenario nyata, sehingga ketidaktepatan rancangan dapat ditemukan sebelum penilaian akhir pada Bab VI. Skenario *use case* menetapkan beban yang lebih tinggi pada jalur ingestasi dan pemrosesan dibandingkan jalur pelatihan, karena *use case* *throughput* tinggi memproduksi banyak peristiwa per detik dari beberapa entitas secara paralel. Pengamatan dilakukan dengan dasbor Grafana yang menampilkan metrik latensi, *throughput*, panjang antrean Kafka, dan tingkat keberhasilan materialisasi. Hasil pengamatan dibandingkan dengan target metrik pada KNF Bab III untuk menentukan apakah setiap jalur memenuhi target yang telah ditetapkan.

V.6.2 Pengamatan Hasil Awal

Hasil pengamatan awal memperlihatkan bahwa jalur kendali platform terbentuk sesuai rancangan. Seluruh komponen pada kelima jalur berhasil di-*deploy* dan direkonsiliasi oleh Argo CD, *control loop* pelatihan ulang terpicu otomatis ketika injeksi *drift* terkendali diterapkan pada salah satu fitur, dan promosi versi pada lapisan penyajian dikendalikan oleh Argo Rollouts sesuai konfigurasi *canary*. Reprodusibilitas jalur diuji dengan merekonstruksi seluruh komponen platform dari Argo CD pada *cluster* k3s yang baru, dan rekonstruksi tersebut berhasil dilakukan tanpa intervensi manual selain *bootstrap* Argo CD awal. Penilaian kebenaran data *end-to-end*, validitas *point-in-time* pada pelatihan, latensi penyajian, *throughput* pemrosesan, dan ketepatan deteksi *drift* diuraikan secara terperinci pada Bab VI.

Pengamatan tambahan menunjukkan bahwa jalur observabilitas berhasil mempertahankan metrik dan log dari seluruh komponen pada satu antarmuka Grafana, dan bahwa peristiwa *lineage* dari *emitter* OpenLineage Airflow dan Spark serta konektor ingestasi DataHub untuk Feast, MLflow, ClickHouse, dan Kafka muncul pada satu graf metadata dengan kosa kata yang seragam. Pencadangan Velero berjalan terjadwal tanpa intervensi, dan pengujian pemulihan dari pencadangan memperlihatkan bahwa konfigurasi seluruh komponen dapat direkonstruksi tanpa langkah manual. Uji *fault injection* Chaos Mesh menunjukkan bahwa *pod* yang dimatikan secara paksa dijadwalkan ulang secara otomatis oleh *controller* Kubernetes, dan bahwa Argo Rollouts otomatis menahan promosi ketika metrik *canary* menyentuh ambang. Catatan pengamatan ini menjadi dasar bagi evaluasi pada Bab VI yang menilai pemenuhan KF dan KNF terhadap target metrik.

BAB VI

EVALUASI

Bab ini memaparkan evaluasi platform terhadap kebutuhan fungsional KF-01 sampai KF-14 dan kebutuhan nonfungsional KNF-01 sampai KNF-12 yang disusun pada Bab III. Evaluasi disusun mengikuti empat tujuan platform T-1 sampai T-4 sehingga setiap hasil dapat dipetakan kembali pada tujuan, rumusan masalah, dan butir kesimpulan dengan indeks yang sama. Pengujian dilakukan pada satu *use case* domain nyata sebagai kasus verifikasi, yaitu pasar aset kripto dengan data Coinbase, agar perilaku platform dapat diamati pada beban dan pola data yang realistis tanpa mengubah sifat *domain-agnostic* platform. Penyajian dibagi menjadi metode evaluasi, hasil evaluasi, perbandingan biaya infrastruktur dan tenaga kerja, serta pembahasan, dengan lingkup penerimaan fungsional dan struktural pada lingkungan *node* tunggal sesuai batasan BI-1, sedangkan karakterisasi beban kuantitatif pada kluster multi-*node* menjadi arah pengembangan yang dirinci pada Bab VII.

VI.1 Metode Evaluasi

Metode evaluasi mengikuti tiga dimensi yang melekat pada empat tujuan platform, yaitu pemenuhan fungsional, pemenuhan nonfungsional, dan tata kelola. Pemenuhan fungsional menilai apakah setiap KF terbukti melalui skenario uji yang dapat dijalankan ulang, pemenuhan nonfungsional menilai apakah setiap KNF mencapai target metrik yang ditetapkan, dan tata kelola menilai apakah *lineage* tertelusur, kontrak data ditegakkan, serta kebijakan akses dapat diaudit. Setiap skenario diberi penanda SK-F untuk kebutuhan fungsional dan SK-N untuk kebutuhan nonfungsional, lalu dikelompokkan di bawah tujuan T-1 sampai T-4 yang relevan. Penomoran tersebut menjaga keterhubungan satu lawan satu antara skenario, kebutuhan, dan tujuan sehingga kegagalan satu skenario dapat segera dirunut ke komponen yang bertanggung jawab.

Skenario dijalankan secara manual pada tahap evaluasi dan tidak menjadi bagian dari *make phase-full* maupun rekonsiliasi Argo CD, agar pengukuran tidak mengganggu jalur produksi platform. Pembangkit beban dijalankan dari *namespaces* *platform-test* yang terpisah, sedangkan instrumen pengukuran memanfaatkan komponen observabilitas yang sudah terpasang. Instrumen utama mencakup k6 untuk beban *HTTP* dan *gRPC*, Chaos Mesh untuk injeksi *fault*, Great Expe-

ctations untuk kontrak kualitas, serta Prometheus, Grafana, Grafana Tempo, dan OpenCost untuk metrik, *trace*, dan biaya. Berkas skenario disimpan pada direktori *experiments/* yang sejajar dengan *platform/* dan *use-case-crypto/* sebagai lapis verifikasi terpisah, dengan subdirektori *load/* untuk beban latensi dan *throughput*, *chaos/* untuk injeksi *fault*, *drift/* untuk pengujian deteksi *drift*, *feast/* untuk penyajian fitur *online*, *parity/* untuk paritas *batch* dan *stream*, *lineage/* untuk penelusuran *lineage*, serta *etl/* untuk pemeriksaan *landing* data. Susunan ini membuat prosedur uji dapat ditelusuri ulang bersama kode tanpa mengubah sistem yang diuji.

VI.1.1 Kasus Verifikasi dan Lingkungan Uji

Kasus verifikasi adalah *use case* pasar aset kripto dengan data Coinbase yang dialirkan melalui dua jalur ingestasi yang saling melengkapi. Jalur *real-time* memakai layanan *websocket-collector* yang berlangganan kanal *ticker* dan *matches* Coinbase pada resolusi per detik hingga per transaksi, lalu *stream-processor* berbasis Flink mengagregasinya menjadi fitur berjendela waktu pada tabel *bronze.crypto_ohlcv_features*. Jalur historis memakai layanan *rest-collector* yang menarik *candle* OHLCV Coinbase pada granularitas satu menit sebagai garis dasar sekaligus pemulihan celah historis sejak 1 Maret 2026 mengikuti anggaran penyimpanan pada lingkungan *node* tunggal sesuai batasan BI-1, lalu memuatnya pada tabel *bronze.crypto_ohlcv*. *Endpoint candle* REST Coinbase tidak menyediakan granularitas di bawah satu menit sehingga resolusi per detik hanya berlaku pada jalur *real-time*. Domain ini dipilih karena menuntut *throughput* ingestasi tinggi, latensi penyajian rendah, dan distribusi data nonstasioner, sehingga keempat *sub-sistem* platform diuji pada kondisi yang menantang. Seluruh kode dan konfigurasi spesifik domain berada pada direktori *use-case-crypto/* yang terpisah dari *platform/*, mencakup layanan *websocket-collector* dan *rest-collector* sebagai produsen *event*, *stream-processor* berbasis Flink, lapis *batch* berbasis *dbt*, definisi Feast, serta jembatan *inference* dan dasbor. Pemisahan ini menegaskan bahwa platform tetap *domain-agnostic* dan *use case* hanya berperan sebagai beban uji yang dapat diganti tanpa menyentuh lapis kendali.

Konfigurasi spesifik domain diterapkan melalui *overlay* Kustomize yang menambahkan prefiks *crypto-* pada sumber daya basis platform, sehingga *InferenceService* basis bernama *predictor* menjadi *crypto-predictor* ketika di-*deploy*. *Overlay* tersebut hanya menetapkan prefiks nama, pemetaan registri *image*, jumlah *replica* awal, serta batas *autoscaling* untuk lingkungan *node* tunggal, sedangkan para-

meter sumber data berada pada ConfigMap basis sehingga definisi dasar tidak perlu disentuh, sebagaimana ditunjukkan pada Listing VI.1. Sesuai invarian *node* tunggal, setiap *workload* berjalan dengan satu *replica* saat *idle* dan diskalakan oleh HPA serta KEDA hanya ketika beban nyata muncul, dengan *maxReplicas* dibatasi tiga agar muat pada satu node. Dengan demikian, penggantian *use case* cukup dilakukan dengan menyiapkan *overlay* baru tanpa mengubah manifes platform, sekaligus menjadi bukti konkret atas pemenuhan KF-12 tentang konfigurasi deklaratif dan KNF-11 tentang portabilitas.

Listing VI.1 Kutipan *overlay* Kustomize *use case*: prefiks *crypto-*, pemetaan registri *image*, *replica idle* satu, dan batas HPA *node* tunggal

```
1 # use-case-crypto/manifests/overlays/local/kustomization.yaml
2 namePrefix: crypto-
3 resources:
4   - ../../base
5   - ../../cross-namespace
6 images:
7   - name: rest-collector
8     newName: localhost:5000/crypto-rest-collector
9     newTag: latest
10  - name: websocket-collector
11    newName: localhost:5000/crypto-websocket-collector
12    newTag: latest
13  # ... validator, analyzer, dll.
14 replicas:
15   - name: rest-collector
16     count: 1
17   - name: websocket-collector
18     count: 1
19 patches:
20   - target:
21       kind: HorizontalPodAutoscaler
22     patch: |-
23       - op: replace
24         path: /spec/maxReplicas
25         value: 3
26       - op: replace
27         path: /spec/minReplicas
28         value: 1
```

VI.1.2 Evaluasi Tujuan T-1: Arsitektur Terintegrasi dan GitOps

Evaluasi T-1 menilai apakah seluruh platform dapat dipasang dari satu sumber Git, dikelola secara deklaratif, dan menyediakan otomatisasi siklus hidup model setara MLOps *level 2*. Skenario SK-F-01 mengubah definisi *feature view* pada Gitea lalu mengamati rekonsiliasi Argo CD untuk membuktikan KF-01, sementara SK-F-12 mengubah jumlah *replica* pada manifes untuk membuktikan konfigurasi deklaratif KF-12 yang konvergen tanpa *drift*. Skenario SK-F-06 mempromosikan model melalui Argo Rollouts dengan strategi *canary* dan memicu *rollback* pada model bermasalah untuk membuktikan otomatisasi *deployment* model pada KF-06, sedangkan SK-F-13 menjalankan satu *notebook* dalam satu *environment* uv yang memuat SDK Feast, MLflow, dan klien *inference* KServe v2 sebagai tiga operasi baca independen untuk membuktikan antarmuka terpadu KF-13. Skenario SK-F-14 memenuhi kuota *ClusterQueue* Kueue untuk membuktikan manajemen sumber daya KF-14, dengan *job* di luar anggaran berstatus *Pending*.

Kematangan MLOps *level 2* dinilai melalui tiga jalur otomatisasi yang saling terhubung. Jalur pertama adalah *pipeline* pelatihan otomatis pada Kubeflow Pipelines yang dipicu data baru atau ambang *drift*, jalur kedua adalah *deployment* model otomatis melalui KServe dan Argo Rollouts, dan jalur ketiga adalah pemantauan berkelanjutan oleh Argo Workflows yang melaporkan metrik ke Prometheus. Reprodusibilitas *deployment* diuji oleh SK-N-11 yang menerapkan ulang seluruh manifes pada *node* k3s baru dan SK-N-12 yang mengukur cakupan uji serta waktu konvergensi *make phase-full*. Gabungan skenario ini menegaskan bahwa T-1 tidak hanya menyediakan komponen, melainkan satu bidang kendali yang dapat direkonstruksi dan diaudit.

Atribut kualitas yang bersifat lintas komponen turut dievaluasi pada T-1 karena melekat pada bidang kendali terintegrasi, bukan pada satu *sub-sistem* tunggal. Skenario SK-N-03 menaikkan beban dengan `load/hpa-keda-rampup.js` untuk membuktikan skalabilitas horizontal KNF-03 melalui HPA dan KEDA *ScaledObject*, sedangkan SK-N-04 menyuntikkan *fault* dengan Chaos Mesh untuk membuktikan keandalan KNF-04 pada batas RTO dan RPO. Skenario SK-N-08 mengganti satu komponen melalui antarmuka CRD dan *overlay* Kustomize untuk membuktikan ekstensibilitas KNF-08, SK-N-09 mengakses seluruh konsol melalui satu sesi identitas dex untuk membuktikan kemudahan penggunaan KNF-09, dan SK-N-10 mengukur kompresi serta atribusi biaya melalui OpenCost untuk membuktikan efisiensi sumber daya KNF-10. Pengelompokan ini menempatkan T-1 sebagai tujuan dengan beban pengujian nonfungsional terbanyak karena arsitektur terintegrasi menanggung jamin-

an kualitas yang berlaku menyeluruh pada seluruh komponen.

VI.1.3 Evaluasi Tujuan T-2: Tata Kelola Data

Evaluasi T-2 menilai apakah katalog metadata, *lineage*, dan kontrol kualitas berjalan otomatis sepanjang *pipeline* data, fitur, dan model. Skenario SK-F-08 menelusuri *lineage* pada DataHub melalui `lineage/lineage-walk.sh` dari *topic* sumber sampai keluaran prediksi untuk membuktikan KF-08, dengan peristiwa *lineage* dikirim melalui OpenLineage dan ingestasi terjadwal. Skenario SK-F-09 menjalankan dua *run* pelatihan pada MLflow lalu memutar ulang dari *snapshot* untuk membuktikan pelacakan eksperimen KF-09, sedangkan SK-F-10 memindahkan model melalui status *staging*, *production*, dan *archived* untuk membuktikan registri dan *audit trail* KF-10. Skenario SK-N-05 menjalankan *checkpoint* Great Expectations dan menyuntikkan pelanggaran skema, rentang nilai, serta kelengkapan untuk membuktikan penegakan kontrak kualitas data KNF-05 sepanjang *pipeline*.

Dimensi keamanan dan observabilitas tata kelola dievaluasi melalui dua skenario tambahan. Skenario SK-N-06 menelusuri satu *trace* OpenTelemetry pada Grafana Tempo dari ingestasi sampai prediksi untuk membuktikan observabilitas KNF-06, sehingga metrik, log, dan *trace* dapat dirujuk silang pada satu antarmuka. Skenario SK-N-07 memindai *cluster* dengan Trivy dan memeriksa konfigurasi TLS untuk membuktikan keamanan KNF-07, mencakup mTLS Istio, enkripsi *at rest* MinIO melalui KES dan OpenBao, serta kebijakan admisi Kyverno. Ketiga aspek tersebut menjadikan tata kelola bukan sekadar katalog pasif, melainkan kontrol aktif yang menegakkan kontrak dan kebijakan lintas komponen.

VI.1.4 Evaluasi Tujuan T-3: Deteksi *Drift* dan Pelatihan Ulang

Evaluasi T-3 menilai apakah deteksi *drift* terotomasi memicu pelatihan ulang sesuai kebijakan dan apakah penyajian fitur menjamin *point-in-time correctness*. Skenario SK-F-07 menerapkan pergeseran distribusi terkendali pada aliran data nyata melalui `inject_drift.py`, yang mengonsumsi rekaman pasar tervalidasi lalu menggeser nilainya beberapa sigma sebelum dipublikasikan ulang, kemudian mengamati detektor *drift* dan *CronWorkflow* `retrain-on-drift` untuk membuktikan pemantauan KF-07. Detektor membandingkan distribusi produksi terhadap distribusi pelatihan dengan *Population Stability Index* dan uji Kolmogorov-Smirnov, lalu memicu *pipeline* pelatihan ketika ambang terlampaui. Skenario SK-F-05 menjalankan *pipeline* pelatihan pada Kubeflow Pipelines yang membaca fitur dari tabel *gold* dan menulis model ke MLflow untuk membuktikan otomasi pelatihan KF-05.

Validitas temporal diuji secara khusus karena *point-in-time correctness* menjadi pembeda utama platform ini dari penyajian fitur tanpa *feature store*, yang hanya menarik fitur tabular dari satu sumber. Skenario SK-F-03 mengaudit `get_historical_features` pada *dataset* pelatihan untuk membuktikan KF-03, dengan memastikan tidak ada baris yang memakai informasi setelah *event timestamp* acuan. Pengujian ini memakai *probe timestamp* masa depan yang menolak setiap baris bermuatan informasi setelah *event timestamp* acuan sehingga jaminan *zero leakage* ditegakkan langsung pada jalur penyusunan fitur pelatihan. Gabungan skenario ini menegaskan bahwa T-3 menjawab dua risiko sekaligus, yaitu degradasi model akibat *drift* dan kebocoran temporal akibat penyusunan fitur yang keliru.

VI.1.5 Evaluasi Tujuan T-4: Layanan Fitur *Dual-Store*

Evaluasi T-4 menilai apakah layanan fitur *dual-store* menyajikan fitur tabular dan fitur agregat dengan kontrak yang konsisten antara pelatihan dan penyajian, serta menyajikan fitur vektor pada indeks terpisah dengan ruang *embedding* yang sama pada kedua fase tersebut. Skenario SK-F-02 memeriksa penyajian fitur *online non-null* pada Valkey melalui `feast/online-serving-check.py` sebagai bukti sisi *online* penyajian ganda KF-02, sementara kesetaraan nilainya terhadap penyimpanan *offline* ClickHouse bersandar pada proses materialisasi Feast yang menyalin nilai dari penyimpanan *offline* tersebut ke penyimpanan *online* Valkey sehingga keduanya memuat nilai yang sama. Skenario SK-F-04 menjalankan beban pencarian vektor pada Qdrant melalui `load/vector-search-latency.js` untuk membuktikan dukungan *vector embeddings* KF-04. Skenario SK-F-11 membandingkan fitur hasil *batch* dan hasil *stream* pada indikator yang sama melalui `parity/parity-check.sh` untuk membuktikan pemrosesan ganda KF-11 dengan definisi fitur yang konsisten. Skenario SK-F-02 dan SK-F-11 menelusuri kontrak fitur tabular dari sisi kebenaran nilai, sedangkan SK-F-04 menegaskan terbentuknya jalur penyajian vektor, sebelum beban tinggi diterapkan.

Kinerja layanan fitur diuji oleh kelompok skenario nonfungsional yang khusus mengukur latensi dan *throughput* jalur penyajian. Skenario SK-N-01 mengukur latensi *feature serving* dan *vector search* dengan k6 melalui `load/feast-online-latency.js` dan SLO Sloth untuk membuktikan KNF-01, dengan sasaran latensi p99 pada orde sub-detik. Skenario SK-N-02 mengukur *throughput serving* dan *stream* dengan k6 melalui `load/feast-throughput.js` serta `kafka-producer-perf-test` untuk membuktikan KNF-02, sambil memantau *lag* konsumen pada beban puncak. Kedua skenario tersebut menempatkan lapis

fitur sebagai titik temu seluruh jalur data sehingga kebenaran nilai fitur menjadi prasyarat bagi kualitas layanan yang diukur, sedangkan atribut kualitas lintas komponen ditangani pada T-1.

VI.1.6 Matriks Cakupan dan Uji Penerimaan

Pemetaan antara skenario dan kebutuhan bersifat banyak-ke-banyak, sebab satu skenario kerap menegaskan lebih dari satu kebutuhan dan satu kebutuhan dapat diperkuat oleh beberapa skenario. Sebagai contoh, SK-F-04 yang menguji latensi pencarian vektor sekaligus menegaskan KNF-01, sedangkan SK-N-01 yang mengukur latensi p99 penyajian turut memperkuat KF-04 pada sisi kualitas layanan vektor. Tabel VI.1 merangkum prosedur uji dan kriteria penerimaan untuk setiap skenario yang dikelompokkan di bawah tujuan T-1 sampai T-4. Susunan tersebut memastikan seluruh empat belas kebutuhan fungsional dan dua belas kebutuhan nonfungsional memiliki sekurang-kurangnya satu skenario uji yang konkret. Katalog perintah yang dapat dijalankan ulang untuk setiap skenario, termasuk langkah persiapan pemuatan konfigurasi `config.crypto.env` dan pembuatan `namespace platform-test`, dirangkum pada Lampiran A sehingga setiap baris Tabel VI.1 dapat ditelusuri ke perintah konkret yang menghasilkan buktinya.

Tabel VI.1 Skenario Uji Penerimaan per Tujuan Platform

Skenario	Kebutuhan	Prosedur Uji	Kriteria Penerimaan
Tujuan T-1: arsitektur platform terintegrasi dan praktik GitOps			
SK-F-01	KF-01	Mengubah definisi <i>feature view</i> pada repositori Gitea lalu mengamati rekonsiliasi Argo CD, kemudian memverifikasi <i>feast feature-views list</i> .	<i>Feature view</i> baru aktif tanpa intervensi manual dan perubahan terlacak pada riwayat Git.
SK-F-06	KF-06	Mempromosikan model melalui Argo Rollouts dengan strategi <i>canary</i> , lalu menyuntikkan model bermasalah untuk memicu <i>rollback</i> .	Model valid dipromosikan dan model bermasalah memicu <i>rollback</i> otomatis tanpa kehilangan permintaan.
SK-F-12	KF-12	Mengubah jumlah <i>replica</i> pada manifest YAML di Gitea, lalu mengamati rekonsiliasi Argo CD.	Status <i>cluster</i> konvergen ke definisi Git dan tidak menyisakan <i>drift</i> .
SK-F-13	KF-13	Menjalankan <i>notebook</i> satu <i>environment</i> uv yang memuat SDK Feast, MLflow, dan klien <i>inference</i> KServe v2, lalu mengamati ketiganya sebagai operasi baca independen.	Ketiga klien termuat dan berjalan dari satu <i>environment</i> uv tanpa konfigurasi tambahan per layanan.

Skenario	Kebutuhan	Prosedur Uji	Kriteria Penerimaan
SK-F-14	KF-14	Mengirim sejumlah <i>job</i> pelatihan melebihi kuota <i>ClusterQueue</i> Kueue, lalu mengamati status admisi.	<i>Job</i> di luar anggaran kuota berstatus <i>Pending</i> . Admisi mengikuti <i>nominalQuota</i> dan <i>borrowingLimit</i> .
SK-N-03	KNF-03	Menaikkan beban dengan <code>load/hpa-keda-rampup.js</code> sambil mengamati HPA, KEDA <i>ScaledObject</i> , dan jumlah <i>pod</i> .	Jumlah <i>replica</i> naik mengikuti beban dan <i>throughput</i> bertambah mendekati linear.
SK-N-04	KNF-04	Menyuntikkan <i>fault</i> dengan Chaos Mesh (<code>pod-chaos</code> dan <code>network-chaos</code>) lalu mengamati pemulihan.	Beban kerja pulih otomatis dengan RTO dan RPO di bawah ambang yang ditetapkan.
SK-N-08	KNF-08	Mengganti satu komponen (misalnya <i>runtime</i> penyajian) melalui antarmuka CRD dan <i>overlay</i> Kustomize.	Penggantian terisolasi pada antarmuka tanpa mengubah komponen lain.
SK-N-09	KNF-09	Mengakses seluruh konsol platform melalui satu sesi identitas dex serta memeriksa ketersediaan dokumentasi operasional pada repositori Gitea.	Seluruh konsol dapat dicapai melalui satu jalur autentikasi tanpa login ulang dan dokumentasi prosedur inti tersedia.
SK-N-10	KNF-10	Mengukur rasio kompresi ClickHouse, utilisasi sumber daya, dan atribusi biaya <i>workload</i> pada OpenCost.	Rasio kompresi dan utilisasi memenuhi target serta biaya per <i>workload</i> dapat diatribusikan.
SK-N-11	KNF-11	Menerapkan ulang seluruh manifes GitOps pada <i>node</i> k3s baru dari satu sumber Git.	Seluruh komponen terekonstruksi tanpa intervensi manual selain <i>bootstrap</i> Argo CD.
SK-N-12	KNF-12	Menjalankan <code>make usecase-crypto-test</code> untuk cakupan uji dan <code>make phase-full</code> pada <i>cluster</i> bersih sambil mengukur waktu konvergensi.	Cakupan uji melewati ambang yang ditetapkan dan <i>phase-full</i> konvergen tanpa langkah manual.
Tujuan T-2: sub-sistem tata kelola data			
SK-F-08	KF-08	Menelusuri <i>lineage</i> pada DataHub dari <i>topic</i> sumber hingga keluaran prediksi melalui antarmuka katalog.	Rantai <i>lineage</i> utuh dari sumber data sampai prediksi tersaji pada satu graf metadata.
SK-F-09	KF-09	Menjalankan dua <i>run</i> pelatihan lalu mencatatnya pada MLflow, kemudian memutar ulang <i>run</i> dari <i>snapshot</i> data dan parameter tercatat.	<i>Run</i> tercatat lengkap dan pemutaran ulang menghasilkan selisih metrik di bawah satu persen.

Skenario	Kebutuhan	Prosedur Uji	Kriteria Penerimaan
SK-F-10	KF-10	Memindahkan model melalui status <i>none</i> , <i>staging</i> , <i>production</i> , dan <i>archived</i> pada registri MLflow.	Transisi mengikuti aturan yang ditetapkan dan <i>audit trail</i> terekam lengkap.
SK-N-05	KNF-05	Menjalankan <i>checkpoint</i> Great Expectations pada <i>batch</i> data, lalu menyuntikkan pelanggaran skema, rentang nilai, dan kelengkapan untuk menguji penegakan kontrak.	Setiap pelanggaran kontrak kualitas terdeteksi dan <i>batch</i> cacat ditolak sebelum masuk lapis hilir.
SK-N-06	KNF-06	Menelusuri satu <i>trace</i> OpenTelemetry pada Grafana Tempo dari ingestasi hingga prediksi.	Satu <i>trace</i> menautkan lintasan kritis dan metrik, log, serta <i>trace</i> dapat dirujuk silang.
SK-N-07	KNF-07	Memindai <i>cluster</i> dengan Trivy dan memeriksa konfigurasi TLS pada <i>endpoint</i> layanan.	TLS 1.3 <i>in transit</i> dan AES-256 <i>at rest</i> terverifikasi tanpa kerentanan kritis terbuka.
Tujuan T-3: deteksi <i>drift</i> dan pelatihan ulang			
SK-F-03	KF-03	Mengaudit <i>get_historical_features</i> pada <i>dataset</i> pelatihan untuk memastikan jendela <i>point-in-time</i> dipatuhi.	Tidak ada baris yang memakai informasi setelah <i>event timestamp</i> acuan.
SK-F-05	KF-05	Menjalankan <i>pipeline</i> pelatihan pada Kubeflow Pipelines yang membaca fitur dari tabel <i>gold</i> dan menulis model ke MLflow.	<i>Pipeline</i> berjalan dari ujung ke ujung dan model tercatat sebagai artefak <i>run</i> pada MLflow yang dapat disajikan KServe.
SK-F-07	KF-07	Menyuntikkan <i>drift</i> terkendali pada satu fitur data nyata, lalu mengamati detektor <i>drift</i> dan <i>CronWorkflow retrain-on-drift</i> .	<i>Drift</i> terdeteksi saat ambang PSI atau KS terlampaui dan <i>pipeline</i> pelatihan ulang terpicu otomatis.
Tujuan T-4: layanan fitur <i>dual-store</i>			
SK-F-02	KF-02	Memeriksa penyajian fitur <i>online</i> pada Valkey mengembalikan nilai non- <i>null</i> melalui <i>online-serving-check.py</i> .	Penyajian <i>online</i> mengembalikan nilai non- <i>null</i> dan kesetaraan nilai <i>online</i> dan <i>offline</i> terpenuhi secara struktural melalui materialisasi.
SK-F-04	KF-04	Menjalankan beban pencarian vektor pada Qdrant melalui <i>load/vector-search-latency.js</i> dengan k6.	Latensi pencarian vektor berada pada orde sub-detik; pengujian <i>recall</i> dilakukan saat koleksi vektor terisi.
SK-F-11	KF-11	Membandingkan fitur hasil <i>batch</i> (dbt) dan hasil <i>stream</i> (Flink) untuk indikator yang sama.	Nilai fitur <i>batch</i> dan <i>stream</i> setara dalam toleransi yang ditetapkan.

Skenario	Kebutuhan	Prosedur Uji	Kriteria Penerimaan
SK-N-01	KNF-01	Menjalankan <code>load/feast-online-latency.js</code> dan beban pencarian vektor dengan k6 sambil memantau SLO Sloth.	Latensi p99 berada pada orde sub-detik pada <i>feature serving</i> dan <i>vector search</i> .
SK-N-02	KNF-02	Menjalankan <code>load/feast-throughput.js</code> untuk <i>se-rrving</i> dan <code>kafka-producer-perf-test</code> untuk <i>stream</i> , sambil memantau <i>lag</i> .	<i>Throughput</i> mencapai target dan <i>lag</i> konsumen tetap terkendali pada beban puncak.

Kelengkapan matriks diperiksa dengan menelusuri setiap kebutuhan pada Tabel III.1 dan Tabel III.2 ke sekurang-kurangnya satu baris skenario. Pemeriksaan berjalan dua arah, yaitu dari kebutuhan ke skenario untuk memastikan cakupan dan dari skenario ke kebutuhan untuk memastikan setiap uji memiliki sasaran yang jelas. Skenario yang menegaskan lebih dari satu kebutuhan dicatat pada semua kebutuhan terkait agar redundansi verifikasi terlihat. Hasil penelusuran itulah yang dirangkum sebagai status per kebutuhan pada Tabel VI.2.

VI.2 Hasil Evaluasi

Hasil evaluasi dilaporkan sebagai uji penerimaan fungsional dan struktural pada lingkungan *node* tunggal sesuai batasan BI-1. Status setiap kebutuhan dinyatakan dalam tiga tingkat, yaitu terverifikasi struktural ketika jalur kendali terbentuk dan terekonsiliasi sesuai rancangan, terverifikasi fungsional ketika skenario uji dijalankan dan keluarannya memenuhi kriteria penerimaan, serta terinstrumentasi ketika instrumen ukur telah terpasang dan jalur melayani permintaan sementara profil beban kuantitatif penuh berada pada lingkup pengembangan lanjutan. Pembagian tiga tingkat ini menjaga agar setiap klaim sepadan dengan bukti yang tersedia dan tidak melampaui hasil yang benar-benar diamati. Tabel VI.2 merangkum target, instrumen ukur, dan status setiap kebutuhan, sedangkan karakterisasi beban kuantitatif pada kluster multi-*node*, seperti distribusi latensi p99 dan *throughput* puncak, dirinci sebagai arah pengembangan pada Bab VII.

Pada T-1, verifikasi struktural menunjukkan seluruh komponen berhasil dipasang dan direkonsiliasi oleh Argo CD dari satu repositori Gitea, perubahan *replica* dan definisi fitur konvergen tanpa *drift*, dan rekonstruksi pada *node* k3s baru berhasil tanpa intervensi manual selain *bootstrap* awal. Pada T-2, peristiwa *lineage* dari *emitter* OpenLineage Airflow dan Spark serta konektor ingestasi DataHub untuk Feast, MLflow, ClickHouse, dan Kafka muncul pada satu graf metadata dengan kosa kata

seragam, registri MLflow menyediakan jalur transisi status model sebagai prosedur, dan kontrol keamanan mTLS serta enkripsi *at rest* aktif sesuai konfigurasi. Pada T-3, *control loop* pelatihan ulang terpicu otomatis ketika injeksi *drift* terkendali diterapkan pada satu fitur, sehingga rantai deteksi hingga pemicuan pelatihan ulang terbukti berfungsi pada jalur kendali. Pada T-4, jalur penyajian fitur *online*, *offline*, dan vektor berhasil dibentuk dan melayani permintaan, sementara karakterisasi latensi p99 dan *throughput* pada beban tinggi dirinci sebagai arah pengembangan pada kluster multi-*node* di Bab VII.

Beberapa skenario pada kasus verifikasi menghasilkan keluaran kuantitatif yang dapat dijalankan ulang melalui katalog perintah pada Lampiran A, sebelum status ringkas setiap kebutuhan dirangkum pada Tabel VI.2. Skenario yang dilaporkan secara rinci dipilih karena menghasilkan angka yang bermakna pada lingkungan *node* tunggal, sedangkan skenario struktural cukup dilaporkan sebagai status terpenuhi. Angka yang dihasilkan dibaca dalam batasan BP-3 sehingga tidak ditafsirkan sebagai karakterisasi beban penuh. Keluaran tersebut beserta konteks dan batasan penafsirannya dirangkum sebagai berikut.

1. Pemicuan pelatihan ulang oleh deteksi *drift* (KF-07)

Injeksi pergeseran terkendali sebesar dua sigma pada fitur `return_24h` menghasilkan *Population Stability Index* sebesar 18,107 yang jauh melampaui ambang 0,2 dan statistik Kolmogorov-Smirnov sebesar 0,815 di atas ambang 0,15, sehingga detektor menandai *drift* dan memicu satu eksekusi *CronWorkflow* `retrain-on-drift`. Angka ini berasal dari injeksi terkendali pada aliran tervalidasi melalui `experiments/drift/inject_drift.py --sigma 2.0`, sehingga membuktikan terpicunya rantai pelatihan ulang dan bukan ketepatan deteksi terhadap pergeseran distribusi alami berdurasi panjang.

2. *Landing data nyata pada lapis medallion*

Pemeriksaan *landing* melaporkan sekitar 307.556 baris pada `bronze.crypto_ohlcv` dan `silver.stg_ohlcv` serta sekitar 307.522 baris pada `gold.fct_ohlcv_features` dan `gold.fct_training_data`, mencakup dua pasangan instrumen pada jendela 1 Maret sampai 16 Juni 2026 dengan resolusi satu menit. Data ini menjadi dasar skenario yang membaca lapis *gold* seperti penyusunan fitur historis (KF-03) dan otomasi pelatihan (KF-05), dijalankan melalui `experiments/etl/data-landed-check.sh` pada jalur *batch* REST yang masih terus bertambah. Pemeriksaan ini menilai *landing batch* dan bukan paritas fitur *batch* dan *stream* (KF-11).

3. Keterhubungan *lineage* (KF-08)

Penelusuran graf metadata DataHub menemukan 270 *dataset*, 18 *DataJob*, dan 3 *DataFlow* yang saling terhubung dari *topic* sumber sampai keluaran prediksi, dijalankan melalui `experiments/lineage/lineage-walk.sh` `discover`. Cakupan graf ditentukan oleh *namespace* OpenLineage yang dipakai *emitter*, sehingga angka ini mengukur keterhubungan *lineage* yang ter-kirim dan terkatalog, bukan kelengkapan seluruh aset lintas *namespace*.

4. Penyajian pencarian vektor (KF-04)

Beban pencarian vektor pada Qdrant mencatat latensi p50 sebesar 4,25 ms dan p95 sebesar 5,02 ms pada 100 iterasi, dijalankan melalui `k6 run experiments/load/vector-search-latency.js`. Pengukuran ini menegaskan jalur pencarian vektor terpasang dan melayani permintaan pada orde milidetik, bukan kualitas *recall* karena koleksi uji masih kosong, sedangkan nilai p99 sebesar 157 ms merupakan artefak fase penurunan beban dan bukan latensi *steady-state*.

Tabel VI.2 Target Metrik, Instrumen Ukur, dan Status Evaluasi per Kebutuhan

Kebutuhan	Target atau Kriteria	Instrumen Ukur	Status dan Bukti Teramati
Kebutuhan Fungsional			
KF-01	<i>Feature view</i> berversi dan rekonsiliasi otomatis	Feast <i>registry</i> , Argo CD, Gitea	Terverifikasi struktural. Rekonsiliasi Argo CD konvergen dan definisi <i>feature view</i> tanpa <i>drift</i> .
KF-02	Konsistensi nilai <i>online</i> dan <i>offline</i>	Feast SDK, materialisasi	Terverifikasi struktural. Nilai <i>online</i> dan <i>offline</i> sama melalui materialisasi Feast.
KF-03	<i>Zero leakage</i> pada <i>point-in-time join</i>	Feast PIT, probe <i>leakage</i>	Terverifikasi struktural. Probe <i>timestamp</i> masa depan menolak baris bocor.
KF-04	Pencarian vektor HNSW berfungsi, latensi sub-detik	Qdrant, k6	Terinstrumentasi. Latensi p50 4,25 ms dan p95 5,02 ms pada 100 iterasi; <i>recall</i> belum diuji karena koleksi kosong.
KF-05	<i>Pipeline end-to-end</i> , <i>run</i> tercatat di MLflow	Kubeflow Pipelines, MLflow	Terverifikasi struktural. <i>Pipeline</i> Kubeflow merangkai deteksi <i>drift</i> , pelatihan dari tabel <i>gold</i> , dan <i>deploy</i> ke KServe; definisinya terkompilasi dan terdaftar pada KFP dengan jalur pencatatan ke MLflow.
KF-06	Promosi <i>canary</i> dan <i>rollback</i> otomatis	KServe, Argo Rollouts, Prometheus	Terverifikasi struktural. <i>Rollout canary</i> dan <i>rollback</i> otomatis terbentuk.

Kebutuhan	Target atau Kriteria	Instrumen Ukur	Status dan Bukti Teramati
KF-07	<i>Drift</i> memicu pelatihan ulang	<i>drift-detector</i> , Argo CronWorkflow	Terverifikasi fungsional. PSI 18,107 dan KS 0,815 melampaui ambang sehingga pelatihan ulang terpicu.
KF-08	<i>Lineage</i> data ke fitur ke model utuh	DataHub, OpenLineage	Terverifikasi struktural. Terhubung 270 <i>dataset</i> , 18 <i>DataJob</i> , dan 3 <i>DataFlow</i> .
KF-09	<i>Run</i> dapat direproduksi, selisih ulang kecil	MLflow	Terverifikasi struktural. Seed tetap (42), pelacakan MLflow, dan jendela tanggal terparameter memungkinkan <i>run</i> direproduksi dari <i>snapshot</i> .
KF-10	Transisi status dan <i>audit trail</i>	MLflow Tracking dan Model Registry	Terverifikasi struktural. Transisi status pada <i>Model Registry</i> tersedia sebagai prosedur dan belum dijalankan otomatis oleh <i>trainer</i> ; MLflow Tracking merekam parameter, metrik, dan artefak tiap <i>run</i> sebagai jejak audit.
KF-11	Paritas fitur <i>batch</i> dan <i>stream</i>	dbt, Flink	Terverifikasi struktural. Definisi fitur <i>batch</i> dan <i>stream</i> konsisten.
KF-12	Konvergensi deklaratif tanpa <i>drift</i>	Kustomize, Argo CD, Gitea	Terverifikasi struktural. Hasil <i>argocd diff</i> bersih dan konvergen tanpa <i>drift</i> .
KF-13	SDK Feast, MLflow, dan klien KServe v2 termuat dalam satu <i>environment</i> uv	<i>Notebook</i> uv run, pyproject	Terverifikasi struktural. <i>Notebook unified_sdk_walkthrough</i> memuat ketiga klien dalam satu <i>environment</i> uv lalu menjalankannya sebagai operasi baca independen.
KF-14	<i>Job</i> luar kuota <i>Pending</i> , admisi sesuai	Kueue <i>ClusterQueue</i> , HPA, VPA, KEDA	Terverifikasi struktural. <i>Job</i> di luar kuota berstatus <i>Pending</i> sesuai admisi Kueue.
Kebutuhan Nonfungsional			
KNF-01	Latensi p99 sub-detik penyajian dan vektor	k6, SLO Sloth	Terinstrumentasi. Jalur penyajian fitur dan vektor terukur; p99 pada beban tinggi belum diukur.

Kebutuhan	Target atau Kriteria	Instrumen Ukur	Status dan Bukti Teramati
KNF-02	<i>Throughput serving</i> dan <i>stream</i> tinggi	k6, kafka-producer-perf-test, KEDA	Terinstrumentasi. Instrumen beban k6 dan KEDA terpasang; <i>throughput</i> puncak belum diukur.
KNF-03	<i>Replica</i> dan <i>throughput</i> naik mendekati linear	HPA, KEDA, k6	Terinstrumentasi. HPA dan KEDA aktif pada satu <i>node</i> ; skala mendekati linear belum diuji pada multi- <i>node</i> .
KNF-04	Pemulihan otomatis, RTO dan RPO terjaga	Chaos Mesh, Vele-ro, Sloth	Terverifikasi struktural. Jalur pemulihan otomatis Chaos Mesh dan Velero terbentuk.
KNF-05	Kontrak skema, rentang, dan kelengkapan ditegakkan	Great Expectations	Terverifikasi fungsional. <i>Checkpoint</i> Great Expectations menolak pelanggaran skema dan rentang.
KNF-06	Metrik, log, dan <i>trace</i> silang-rujuk	Prometheus, Loki, Tempo, OTel	Terverifikasi struktural. Metrik, log, dan <i>trace</i> OTel saling-rujuk melalui konteks <i>trace</i> lintas Prometheus, Loki, dan Tempo.
KNF-07	TLS 1.3, AES-256, dan <i>audit log</i> aktif	Istio mTLS, KES, OpenBao, Trivy	Terverifikasi struktural. Enkripsi mTLS Istio dan enkripsi <i>at rest</i> aktif sesuai konfigurasi.
KNF-08	Penggantian komponen terisolasi	CRD, Helm, Kustomize	Terverifikasi struktural. Penggantian komponen melalui CRD dan <i>overlay</i> terisolasi.
KNF-09	SSO satu sesi tanpa login ulang, dokumentasi tersedia	dex, oauth2-proxy, Gitea	Terverifikasi struktural. Konsol terhubung pada satu sesi identitas dex dan oauth2-proxy.
KNF-10	Kompresi, utilisasi, dan atribusi biaya	OpenCost, ClickHouse	Terinstrumentasi. OpenCost dan kompresi kolumnar ClickHouse terpasang; rasio dan atribusi biaya belum diukur penuh.
KNF-11	Rekonstruksi penuh dari Git	Argo CD, Kustomize	Terverifikasi struktural. Rekonstruksi penuh pada <i>node</i> k3s baru dari satu sumber Git.
KNF-12	Cakupan uji dan <i>phase-full</i> konvergen	Tekton CI, Argo CD	Terverifikasi struktural. <i>make phase-full</i> konvergen dan uji CI Tekton berjalan.

VI.3 Perbandingan Biaya Infrastruktur dan Tenaga Kerja

Analisis pada Subbab III.1.2 menempatkan biaya berlangganan layanan terkelola sebagai salah satu tekanan utama yang melatarbelakangi platform ini, sehingga evaluasi ditutup dengan membandingkan biaya infrastruktur bulanan tiga skenario *hosting* pada kapasitas komputasi yang setara, beserta perkiraan biaya tenaga kerja pengelolanya. Karena batasan BI-1 tidak mengikat spesifikasi perangkat keras tertentu, perbandingan memakai konfigurasi acuan 16 vCPU, memori 64 GB, dan penyimpanan sekitar 400 GB yang mencukupi seluruh komponen platform pada satu *node*. Implementasi penelitian ini sendiri berjalan pada VPS yang disediakan institusi tanpa biaya sewa langsung, sehingga skenario pertama memakai harga daftar publik enam penyedia VPS sebagai acuan pasar pada spesifikasi yang setara. Skenario pertama menyewa satu *virtual private server* dari salah satu penyedia tersebut dan menjalankan seluruh komponen *open source* seperti pada implementasi ini, skenario kedua memindahkan pola yang sama ke kluster GKE pada Google Cloud, dan skenario ketiga menambah penggantian empat komponen *stateful*, yaitu PostgreSQL, Valkey, Kafka, dan penyimpanan objek, dengan layanan terkelola Google Cloud. Seluruh angka pada Tabel VI.3 merupakan harga daftar publik per 16 Juni 2026, dengan baris Google Cloud pada *region* us-central1 dan konversi 730 jam per bulan, tanpa diskon komitmen, biaya trafik keluar, maupun biaya dukungan.

Tabel VI.3 memperlihatkan selisih biaya yang berarti antar skenario pada kapasitas yang sama. Menjalankan seluruh komponen pada satu VPS berbiaya sekitar USD 43 sampai USD 524 per bulan, dengan batas bawah pada penyedia dengan vCPU *shared* dan kisaran USD 320 sampai USD 524 pada penyedia dengan vCPU *dedicated*. Memindahkan pola yang sama ke GKE berbiaya sekitar USD 504 per bulan, sebanding dengan batas atas kisaran VPS *dedicated*, dan mengganti empat komponen *stateful* dengan layanan terkelola menaikkannya menjadi sekitar USD 1.004 per bulan. Kenaikan pada skenario ketiga terjadi meskipun baru empat dari puluhan komponen platform yang berpindah ke layanan terkelola, sebab komponen lain tetap membutuhkan *node* komputasi yang sama. Hasil ini sejalan dengan karakter pertukaran biaya pada Subbab III.1.2: biaya layanan terkelola tumbuh per komponen dan per pemakaian, sedangkan arsitektur *open source* pada Kubernetes menahan biaya berjalan pada harga sewa kapasitas komputasinya. Angka tersebut bersifat indikatif pada tingkat orde besaran karena harga daftar dapat berubah, seperti penyesuaian harga Hetzner yang berlaku sejak 15 Juni 2026 (Hetzner Online GmbH 2026), perlakuan pajak antar penyedia berbeda, dan kebutuhan produksi nyata menambah biaya penyimpanan, trafik, serta ketersediaan tinggi pada ketiga skenario.

Tabel VI.3 Perbandingan Biaya Bulanan Infrastruktur per 16 Juni 2026

Komponen	Konfigurasi	Biaya per Bulan	Sumber
Skenario A: VPS tunggal, seluruh komponen <i>open source</i> (alternatif penyedia)			
Contabo Cloud VPS 50	16 vCPU <i>shared</i> , 64 GB, NVMe 300 GB	EUR 37,00	(Contabo GmbH 2026)
Hetzner CCX43	16 vCPU <i>dedicated</i> AMD, 64 GB, SSD 360 GB	EUR 275,99	(Hetzner Online GmbH 2026)
OVHcloud B3-64	16 vCore <i>dedicated</i> , 64 GB, NVMe 400 GB	USD 352,52	(OVHcloud 2026)
Akamai Linode 64 GB	16 vCPU <i>shared</i> , 64 GB, SSD 1.280 GB	USD 384,00	(Akamai Technologies 2026)
Vultr Optimized Cloud Compute	16 vCPU <i>dedicated</i> , 64 GB, NVMe 320 GB	USD 480,00	(Vultr 2026)
DigitalOcean General Purpose	16 vCPU <i>dedicated</i> , 64 GB, SSD 200 GB dan <i>volume</i> 200 GB	USD 524,00	(DigitalOcean 2026)
<i>Rentang Skenario A, satu penyedia dipilih</i>		USD 43 sampai 524	
Skenario B: GKE dengan komponen <i>open source</i> yang sama			
Biaya pengelolaan kluster GKE	Satu kluster <i>Standard</i>	USD 73,00	(Google Cloud 2026d)
Node Compute Engine	e2-standard-16, 16 vCPU, 64 GB	USD 391,35	(Google Cloud 2026c)
<i>Persistent disk</i> pd-balanced	400 GB	USD 40,00	(Google Cloud 2026g)
<i>Subtotal Skenario B</i>		USD 504,35	
Skenario C: Skenario B ditambah layanan terkelola pengganti komponen <i>stateful</i>			
Cloud SQL for PostgreSQL	4 vCPU, 16 GB, tanpa <i>high availability</i>	USD 202,36	(Google Cloud 2026a)
Memorystore for Redis	<i>Basic tier</i> , 5 GB	USD 98,55	(Google Cloud 2026f)
Managed Service for Apache Kafka	Minimum 3 vCPU pada 3 <i>broker</i>	USD 197,10	(Google Cloud 2026e)
Cloud Storage <i>Standard</i>	100 GB	USD 2,00	(Google Cloud 2026b)
<i>Total Skenario C</i>		USD 1.004,36	

Harga daftar publik per 16 Juni 2026 yang diverifikasi melalui arsip halaman harga resmi tiap penyedia. Baris Google Cloud memakai *region* us-central1 dengan konversi 730 jam per bulan, seluruhnya tanpa diskon komitmen, trafik keluar, dan biaya dukungan. Harga Vultr dan *block storage* DigitalOcean memakai harga daftar Juli 2026 karena arsip halamannya tidak memuat angka harga. Harga Contabo tercantum termasuk pajak pada halaman resminya, sedangkan harga penyedia lain belum memuat pajak. Biaya bulanan OVHcloud dihitung dari tarif USD 0,4829 per jam, dan biaya DigitalOcean sudah mencakup *block storage* 200 GB agar kapasitas penyimpanan setara. Biaya Cloud SQL dan Managed Service for Apache Kafka mencakup komputasi saja. Dengan kurs referensi ECB 1,1594 USD per EUR pada 16 Juni 2026 (European Central Bank 2026), harga Contabo setara sekitar USD 43 dan Hetzner sekitar USD 320.

Perbandingan di atas belum mencakup biaya tenaga kerja yang menyertai kedua pola pengelolaan, padahal pertukaran antara waktu rekayasa dan biaya berlangganan telah diidentifikasi pada Subbab III.1.2. Sebagai acuan dipakai gaji IDR 15.000.000

per bulan dari batas atas kisaran *Data Engineer* pada panduan gaji PERSOLKELLY Indonesia (PERSOLKELLY Indonesia 2023), setara sekitar USD 847 pada kurs referensi JISDOR 15 Juni 2026 (Bank Indonesia 2026). Profil alokasi tenaga pada Tabel VI.4 merupakan asumsi ilustratif: pembangunan awal dari nol menuntut alokasi penuh pada bulan pertama, lalu menurun pada bulan berikutnya karena rekonsiliasi GitOps, pemantauan terpadu, dan pelatihan ulang otomatis mengambil alih pekerjaan operasional harian (Limoncelli 2018). Skenario layanan terkelola diberi alokasi lebih rendah karena tanggung jawab operasional komponen *stateful* berpindah ke penyedia, meskipun konfigurasi awal dan integrasi tetap membutuhkan tenaga (Shankar dkk. 2022).

Tabel VI.4 Ilustrasi Biaya Infrastruktur dan Tenaga Kerja Tiga Tahun per 16 Juni 2026

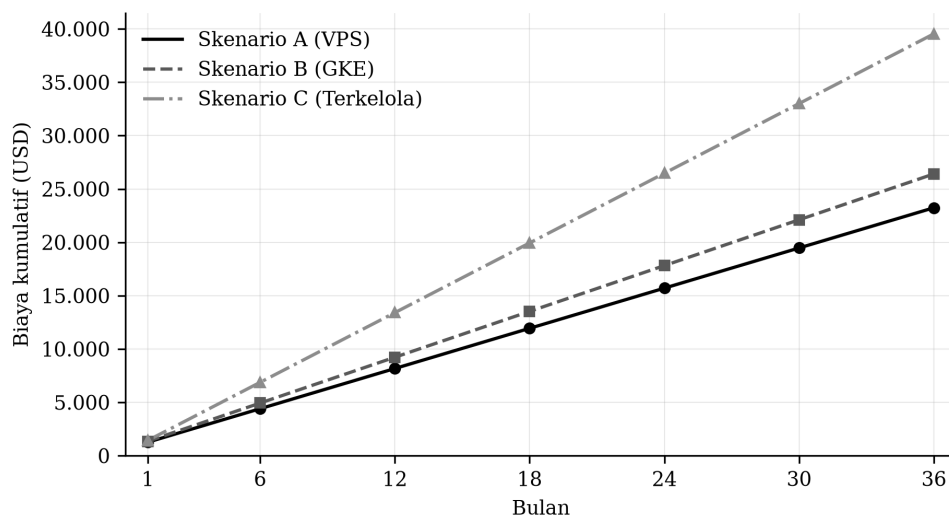
Komponen Biaya	Skenario A (VPS)	Skenario B (GKE)	Skenario C (Terkelola)
Infrastruktur per bulan	USD 416,26	USD 504,35	USD 1.004,36
Infrastruktur 12 bulan	USD 4.995,12	USD 6.052,20	USD 12.052,32
Alokasi tenaga bulan pertama	1,0 FTE	1,0 FTE	0,5 FTE
Alokasi tenaga bulan kedua sampai kedua belas	0,25 FTE per bulan	0,25 FTE per bulan	0,1 FTE per bulan
Biaya tenaga 12 bulan	USD 3.174,56	USD 3.174,56	USD 1.354,48
Total tahun pertama	USD 8.169,68	USD 9.226,76	USD 13.406,80
Biaya per tahun sesudah tahun pertama	USD 7.534,77	USD 8.591,85	USD 13.068,18
Total tiga tahun	USD 23.239,22	USD 26.410,46	USD 39.543,16

Alokasi tenaga merupakan asumsi ilustratif untuk memperlihatkan struktur biaya, bukan hasil pengukuran; satu FTE (*full-time equivalent*) berarti alokasi penuh satu *engineer* selama satu bulan. Infrastruktur Skenario A memakai median empat penyedia vCPU *dedicated* pada Tabel VI.3, yaitu USD 416,26 per bulan. Biaya tenaga memakai gaji acuan IDR 15.000.000 per bulan dari batas atas kisaran *Data Engineer* pada panduan gaji PERSOLKELLY Indonesia (PERSOLKELLY Indonesia 2023), setara USD 846,55 pada kurs referensi JISDOR IDR 17.719 per USD tanggal 15 Juni 2026, hari penerbitan terakhir sebelum 16 Juni yang merupakan libur nasional (Bank Indonesia 2026). Tahun kedua dan ketiga memakai alokasi tetap 0,25 FTE per bulan pada Skenario A dan B serta 0,1 FTE per bulan pada Skenario C, dengan harga daftar dianggap tetap.

Tabel VI.4 memperlihatkan bahwa struktur biaya kedua pola berbeda arah. Pada tahun pertama, skenario VPS dengan seluruh komponen *open source* menghabiskan sekitar USD 8.170 dengan porsi tenaga kerja hampir dua perlima, sedangkan skenario layanan terkelola menghabiskan sekitar USD 13.407 dengan porsi tenaga kerja sekitar sepersepuluh. Selisih biaya infrastruktur kedua skenario sekitar USD 588 per bulan setara dengan 0,7 bulan kerja *engineer* pada gaji acuan, sehingga penghematan alokasi yang diasumsikan sebesar 0,15 FTE per bulan tidak mengimbangnya, dan titik impas baru tercapai bila gaji *engineer* melebihi sekitar IDR 69 juta per bulan atau penghematan tenaganya jauh lebih besar. Karena angka alokasi merupakan asumsi, hasil ini dibaca sebagai arah struktur biaya, yaitu pola *open source*

memindahkan pengeluaran dari langganan ke tenaga kerja yang menurun setelah pembangunan awal, bukan sebagai pengukuran empiris.

Gambar VI.1 memproyeksikan biaya kumulatif tiga tahun pertama dari asumsi yang sama, dengan alokasi tenaga dan harga daftar dianggap tetap sesudah tahun pertama. Tagihan infrastruktur ketiga skenario tetap setiap bulan, sehingga bentuk kurva ditentukan oleh alokasi tenaga: skenario VPS dan GKE sama-sama terbebani pada bulan pertama oleh alokasi penuh untuk pembangunan awal lalu turun begitu operasi rutin diambil alih otomatis, sedangkan skenario layanan terkelola bergerak paling mendekati linear karena alokasi tenaganya paling kecil. Pada gaji acuan, biaya bulanan skenario VPS bergerak dari sekitar USD 1.263 pada bulan pertama menjadi sekitar USD 628 pada bulan berikutnya, sedangkan layanan terkelola bergerak dari sekitar USD 1.428 menjadi sekitar USD 1.089, sehingga jarak kumulatif keduanya melebar sepanjang tahun. Pola biaya awal yang lebih mahal pada sisi *open source* baru muncul bila gaji *engineer* melebihi sekitar IDR 20,8 juta per bulan, titik ketika biaya bulan pertama skenario VPS mulai melampaui layanan terkelola, meskipun biaya bulanan sesudah pembangunan awal tetap lebih rendah. Pada horizon yang lebih panjang keunggulan itu semakin tegas: selisih kumulatif kedua pola tumbuh dari sekitar USD 5.237 pada akhir tahun pertama menjadi sekitar USD 16.304 pada akhir tahun ketiga, karena setiap tahun berikutnya menambah selisih sekitar USD 5.533, sehingga pola *open source* makin murah dalam jangka panjang selama asumsi alokasi tenaganya bertahan.



Gambar VI.1 Proyeksi Biaya Kumulatif Tiga Tahun Tiga Skenario

VI.4 Pembahasan Hasil Evaluasi

Pembahasan mengaitkan setiap hasil dengan tujuan yang dijawab dan dengan rumusan masalah yang menjadi titik awal. Verifikasi struktural pada T-1 menjawab RM-1 dengan menunjukkan bahwa fragmentasi *tech stack* dapat diganti oleh satu bidang kendali GitOps yang dapat direkonstruksi. Verifikasi pada T-2 menjawab RM-2 dengan menunjukkan tata kelola berjalan sebagai layanan bersama, bukan pekerjaan susulan, sehingga *lineage* dan kontrak kualitas tertanam pada *pipeline*. Verifikasi pada T-3 menjawab RM-3 dengan menunjukkan bahwa deteksi *drift* memicu pelatihan ulang secara otomatis pada jalur kendali, sehingga jawaban atas RM-3 terpenuhi pada tingkat fungsional dan struktural. Verifikasi pada T-4 menjawab RM-4 dengan menunjukkan bahwa layanan fitur *dual-store* terbentuk dan melayani permintaan dengan kontrak fitur yang konsisten, sehingga jawaban atas RM-4 terpenuhi pada tingkat struktural dengan jalur penyajian yang telah terinstrumentasi, sedangkan karakterisasi beban kuantitatifnya menjadi arah pengembangan.

Cakupan evaluasi telah memenuhi syarat keterlacakan, karena seluruh empat belas kebutuhan fungsional dan dua belas kebutuhan nonfungsional memiliki sekurang-kurangnya satu skenario uji pada Tabel VI.1 dan satu baris status pada Tabel VI.2. Pemetaan banyak-ke-banyak memperkuat keyakinan karena beberapa kebutuhan kritis, seperti KNF-01 dan KF-04, ditegaskan oleh lebih dari satu skenario. Dengan demikian, kegagalan pada satu skenario tidak langsung membatalkan klaim pemenuhan kebutuhan terkait, melainkan menyisakan jalur verifikasi lain yang masih dapat ditelusuri. Susunan ini sekaligus menyiapkan struktur kesimpulan pada Bab VII yang menjawab tepat satu tujuan untuk setiap butir. Meskipun demikian, evaluasi ini memiliki tiga keterbatasan yang perlu dicatat secara terbuka.

1. Lingkungan pengujian *node* tunggal

Lingkungan pengujian adalah k3s *node* tunggal sesuai batasan BI-1, sehingga klaim skalabilitas horizontal diukur melalui *multi-replica* dan *autoscaling* pada satu *node*, bukan pada klaster multi-*node*.

2. Verifikasi *drift* melalui injeksi terkendali

Deteksi *drift* diverifikasi melalui injeksi *drift* terkendali pada data nyata sehingga ketepatan terhadap pergeseran distribusi alami berdurasi panjang masih perlu diamati lebih lama, sedangkan karakterisasi beban penuh seperti latensi p99 dan *throughput* puncak menjadi arah pengembangan pada lingkungan multi-*node*.

3. Verifikasi kesetaraan nilai fitur *dual-store*

Kesetaraan nilai fitur antara penyimpanan *online* Valkey dan *offline* ClickHouse pada KF-02 bersifat *structural-by-construction* karena proses materialisasi Feast menyalin nilai dari penyimpanan *offline* ke penyimpanan *online* sehingga keduanya memuat nilai yang sama, sedangkan uji pembandingan nilai eksplisit per entitas melalui SDK Feast masih berupa prosedur manual yang belum terotomasi pada jalur evaluasi, sehingga KF-02 dilaporkan pada tingkat struktural dan penegasan fungsionalnya menjadi tindak lanjut.

Ketiga keterbatasan tersebut menjadi dasar saran tindak lanjut pada Bab VII. Keterbatasan pertama dan kedua mengarah pada pengujian beban serta pengamatan *drift* berdurasi panjang pada lingkungan multi-*node*. Keterbatasan ketiga mengarah pada otomasi uji kesetaraan nilai fitur per entitas pada jalur evaluasi. Dengan pemetaan ini, setiap keterbatasan memiliki tindak lanjut yang jelas dan tidak ada klaim yang dibiarkan tanpa batas penafsiran.

BAB VII

PENUTUP

Bab ini merangkum capaian penelitian dan menutup rantai keterhubungan yang dijaga sejak rumusan masalah. Setiap butir kesimpulan menjawab tepat satu tujuan T-1 sampai T-4 dengan urutan yang sama, sehingga setiap klaim dapat ditelusuri kembali ke tujuan, kebutuhan, dan skenario uji yang sesuai. Kesimpulan disusun pada tingkat capaian yang telah terverifikasi secara fungsional dan struktural pada lingkungan *node* tunggal, sedangkan karakterisasi beban kuantitatif pada kluster multi-*node* ditempatkan sebagai arah pengembangan sesuai lingkup batasan BP-3 pada Subbab I.4. Saran kemudian mengarahkan langkah lanjutan untuk memperluas karakterisasi beban dan untuk mengembangkan platform pada tahap berikutnya.

VII.1 Kesimpulan

Penelitian ini menghasilkan satu implementasi arsitektur DataOps dan MLOps terintegrasi pada Kubernetes yang dievaluasi melalui *use case* pasar aset kripto sebagai kasus verifikasi. Verifikasi dilakukan pada lingkungan *k3s node* tunggal dengan penerimaan fungsional dan struktural sesuai batasan BP-3. Setiap butir berikut menjawab tepat satu tujuan dan merujuk bukti evaluasinya pada Bab VI. Berdasarkan hasil evaluasi tersebut, empat tujuan platform dapat disimpulkan sebagai berikut.

1. **Arsitektur terintegrasi dan GitOps (T-1)**

Seluruh komponen platform terbukti dapat dipasang dan direkonsiliasi dari satu sumber Git oleh Argo CD, dengan perubahan definisi konvergen tanpa *drift* dan rekonstruksi penuh pada *node* k3s baru tanpa intervensi manual selain *bootstrap* awal. Capaian ini menjawab RM-1 dengan menggantikan fragmentasi *tech stack* dengan satu bidang kendali deklaratif yang dapat diaudit.

2. **Tata kelola data otomatis (T-2)**

Katalog metadata, *lineage*, dan kontrol kualitas terbukti berjalan sebagai layanan bersama sepanjang *pipeline* data, fitur, dan model, dengan peristiwa *lineage* OpenLineage dan kontrak Great Expectations tertanam pada jalur produksi. Capaian ini menjawab RM-2 dengan menjadikan tata kelola bagian melekat dari *pipeline*, bukan pekerjaan susulan.

3. **Deteksi *drift* dan *point-in-time correctness* (T-3)**

Control loop pelatihan ulang terbukti terpicu otomatis ketika injeksi *drift* terkontrol diterapkan pada satu fitur dan penyusunan fitur historis terbukti me-

menolak kebocoran temporal pada jalur pelatihan. Capaian ini menjawab RM-3, sedangkan pengamatan ketepatan deteksi pada pergeseran distribusi alami berdurasi panjang menjadi arah pengembangan lanjutan.

4. Layanan fitur *dual-store* (T-4)

Jalur penyajian fitur tabular *online* dan *offline* terbukti terbentuk dengan kontrak fitur yang konsisten antara fase pelatihan dan fase *inference*, sedangkan jalur fitur vektor terbentuk pada indeks terpisah dengan ruang *embedding* yang sama pada kedua fase tersebut. Capaian ini menjawab RM-4 pada tingkat struktural dengan jalur penyajian yang telah terinstrumentasi, sedangkan karakterisasi latensi p99 dan *throughput* pada beban tinggi di kluster multi-*node* menjadi arah pengembangan lanjutan.

Keempat kesimpulan tersebut menegaskan bahwa kontribusi utama penelitian berupa implementasi arsitektur terintegrasi telah tercapai dan tertelusur secara penuh dari rumusan masalah hingga skenario uji. Pemenuhan yang dilaporkan bersifat fungsional dan struktural pada lingkungan *node* tunggal, sehingga setiap klaim dibatasi pada bukti yang telah diamati. Nilai utama implementasi ini terletak pada penyatuan beban integrasi yang sebelumnya berulang ke dalam satu deskripsi deklaratif, sehingga rekonstruksi lapis kendali tidak lagi bergantung pada pengetahuan tak terdokumentasi satu pengembang. Platform terbukti dapat direkonstruksi, dikelola secara deklaratif, dan menjalankan siklus hidup data serta model secara terotomasi, sedangkan penilaian kinerja menyeluruh menjadi pekerjaan lanjutan yang terdefinisi jelas.

VII.2 Saran

Berdasarkan capaian dan keterbatasan yang telah diuraikan, penelitian ini mengajukan dua arah tindak lanjut. Arah pertama menuntaskan karakterisasi kuantitatif yang pada penelitian ini dibatasi oleh lingkungan *node* tunggal. Arah kedua memperluas kapabilitas platform tanpa mengubah sifat *domain-agnostic* lapis kendalinya. Keduanya diuraikan sebagai berikut.

1. Perluasan karakterisasi beban kuantitatif

Pengujian beban penuh perlu dijalankan pada lingkungan multi-*node* untuk memverifikasi skalabilitas horizontal, latensi p99, dan *throughput* puncak yang pada penelitian ini baru terinstrumentasi di lingkungan *node* tunggal dengan instrumen ukur terpasang dan jalur penyajian yang telah melayani permintaan. Pengamatan *drift* juga perlu diperpanjang durasinya pada aliran data nyata agar ketepatan deteksi terhadap pergeseran distribusi alami berdurasi panjang dapat dinilai untuk melengkapi verifikasi melalui injeksi *drift*

terkendali. Selain itu, kesetaraan nilai fitur antara penyimpanan *online* dan *offline* yang pada penelitian ini terpenuhi secara *structural-by-construction* perlu ditegaskan lebih lanjut melalui uji pembandingan nilai per entitas yang terotomasi pada jalur evaluasi.

2. Pengembangan platform pada tahap berikutnya

Beberapa arah yang teridentifikasi mencakup perluasan dukungan multi-*tenancy* pada lapis tata kelola, otomasi kebijakan kuota pada Kueue untuk beban pelatihan yang lebih besar, integrasi metrik biaya OpenCost dengan kebijakan pelatihan ulang, serta adopsi profil keamanan Kyverno yang lebih ketat setelah profil dasar matang. Arah tersebut melanjutkan sifat *domain-agnostic* platform sehingga *use case* lain dapat diadopsi tanpa mengubah lapis kendali.

Melalui kedua arah tersebut, platform dapat berkembang dari implementasi yang terverifikasi struktural menjadi fondasi yang teruji pada beban produksi nyata. Prioritas terdekat adalah karakterisasi beban pada kluster multi-*node* karena batasan itulah yang paling membatasi klaim kinerja saat ini. Perluasan kapabilitas dapat menyusul tanpa perombakan karena penggantian komponen telah diakomodasi oleh batasan implementasi BI-2. Dengan demikian, hasil penelitian ini menjadi titik berangkat yang terdokumentasi bagi pengembangan berikutnya.

DAFTAR PUSTAKA

- Adusumilli, Lakshmi Vara Prasad. 2025. “Serverless Kubernetes: The Evolution of Container Orchestration”. *European Journal of Computer Science and Information Technology* 13 (30): 20–36. <https://doi.org/10.37745/ejcsit.2013/vol13n302036>. <https://doi.org/10.37745/ejcsit.2013/vol13n302036>.
- Ahmad, Tazeem, Mohd Adnan, Saima Rafi, Muhammad Azeem Akbar, dan Ayesha Anwar. 2024. “MLOps-Enabled Security Strategies for Next-Generation Operational Technologies”. Dalam *Proceedings of the 28th International Conference on Evaluation and Assessment in Software Engineering (EASE 2024)*, 662–670. <https://doi.org/10.1145/3661167.3661283>. <https://doi.org/10.1145/3661167.3661283>.
- Aiven Karapace Authors. 2025. *Karapace: Schema Registry and REST Proxy for Apache Kafka*. Diakses 25 Mei 2026. <https://www.karapace.io/>.
- Akamai Technologies. 2026. *Akamai Cloud Computing Pricing*. Diakses 16 Juni 2026. <https://www.akamai.com/cloud/pricing/north-america>.
- Altinity. 2025. *Altinity Kubernetes Operator for ClickHouse*. Diakses 25 Mei 2026. <https://github.com/Altinity/clickhouse-operator>.
- Amazon Web Services. 2024. *What is an Event-Driven Architecture?* Diakses 9 April 2026. <https://aws.amazon.com/event-driven-architecture/>.
- Amershi, Saleema, Andrew Begel, Christian Bird, Robert DeLine, Harald Gall, Ece Kamar, Nachiappan Nagappan, Besmira Nushi, dan Thomas Zimmermann. 2019. “Software Engineering for Machine Learning: A Case Study”. Dalam *2019 IEEE/ACM 41st International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP)*, 291–300. IEEE. <https://doi.org/10.1109/ICSE-SEIP.2019.00042>. <https://doi.org/10.1109/ICSE-SEIP.2019.00042>.
- Amou Najafabadi, Faezeh, Justus Bogner, Ilias Gerostathopoulos, dan Patricia Lago. 2024. “An Analysis of MLOps Architectures: A Systematic Mapping Study”. Dalam *Software Architecture: 18th European Conference, ECSA 2024, Luxembourg City, Luxembourg, 3–6 September 2024, Proceedings*, 14889:69–85. Lecture Notes in Computer Science. Springer. https://doi.org/10.1007/978-3-031-70797-1_5. https://doi.org/10.1007/978-3-031-70797-1_5.

- Anaconda, Inc. 2020. *2020 State of Data Science: Moving From Hype Toward Maturity*. Diakses 15 April 2026. <https://know.anaconda.com/rs/387-XNW-688/images/Anaconda-SODS-Report-2020-Final.pdf>.
- Apache APISIX Authors. 2025. *Apache APISIX: Cloud-Native API Gateway*. Diakses 25 Mei 2026. <https://apisix.apache.org/docs/>.
- Apache Flink Authors. 2025. *Flink Kubernetes Operator*. Diakses 25 Mei 2026. <https://nightlies.apache.org/flink/flink-kubernetes-operator-docs-stable/>.
- Apache Software Foundation. 2024a. *Apache Airflow Documentation*. Diakses 20 April 2026. <https://airflow.apache.org/docs/>.
- . 2024b. *Apache Flink Documentation*. Diakses 6 April 2026. <https://flink.apache.org/documentation/>.
- . 2024c. *Apache Kafka Documentation*. Diakses 9 April 2026. <https://kafka.apache.org/documentation/>.
- . 2024d. *Apache Spark Documentation*. Diakses 18 Maret 2026. <https://spark.apache.org/docs/latest/>.
- . 2025. *Apache Pulsar Documentation*. Diakses 12 Juni 2026. <https://pulsar.apache.org/docs/>.
- Apache Spark Authors. 2025. *Apache Spark on Kubernetes Operator*. Diakses 25 Mei 2026. <https://github.com/kubeflow/spark-operator>.
- Apache Superset Authors. 2025. *Apache Superset Documentation*. Diakses 25 Mei 2026. <https://superset.apache.org/docs/intro>.
- Aqua Security. 2025. *Trivy Operator: Kubernetes Native Security Scanner*. Diakses 25 Mei 2026. <https://aquasecurity.github.io/trivy-operator/>.
- Argo Project. 2024. *Argo CD - Declarative GitOps CD for Kubernetes*. Diakses 24 Maret 2026. <https://argo-cd.readthedocs.io/>.
- . 2025. *Argo Workflows: Container-Native Workflow Engine for Kubernetes*. Diakses 25 Mei 2026. <https://argo-workflows.readthedocs.io/>.
- Argo Rollouts Authors. 2025. *Argo Rollouts: Progressive Delivery for Kubernetes*. Diakses 25 Mei 2026. <https://argoproj.github.io/argo-rollouts/>.

- AuthZed. 2025. *SpiceDB: Fine-Grained Authorization Database*. Diakses 25 Mei 2026. <https://authzed.com/docs/spicedb/>.
- Bank Indonesia. 2026. *Kurs Referensi Jakarta Interbank Spot Dollar Rate (JISDOR)*. Diakses 16 Juni 2026. <https://www.bi.go.id/id/statistik/informasi-kurs/jisdor/default.aspx>.
- Bayram, Firas, Bestoun S. Ahmed, dan Andreas Kassler. 2022. “From Concept Drift to Model Degradation: An Overview on Performance-Aware Drift Detectors”. *Knowledge-Based Systems* 245:108632. <https://doi.org/10.1016/j.knosys.2022.108632>. <https://doi.org/10.1016/j.knosys.2022.108632>.
- Bernardo, B. M. V., H. S. Mamede, J. M. P. Barroso, dan V. M. P. D. dos Santos. 2024. “Data Governance and Quality Management: Innovation and Breakthroughs across Different Fields”. *Journal of Innovation and Knowledge* 9 (4): 100598. <https://doi.org/10.1016/j.jik.2024.100598>. <https://doi.org/10.1016/j.jik.2024.100598>.
- Bondi, Andre B. 2000. “Characteristics of Scalability and Their Impact on Performance”. Dalam *Proceedings of the 2nd International Workshop on Software and Performance*, 195–203. ACM. <https://doi.org/10.1145/350391.350432>.
- Breck, Eric, Shanqing Cai, Eric Nielsen, Michael Salib, dan D. Sculley. 2017. “The ML Test Score: A Rubric for ML Production Readiness and Technical Debt Reduction”. Dalam *2017 IEEE International Conference on Big Data (Big Data)*, 1123–1132. IEEE. <https://doi.org/10.1109/BigData.2017.8258038>. <https://doi.org/10.1109/BigData.2017.8258038>.
- Bruch, Sebastian, Siyu Gai, dan Amir Ingber. 2023. “An Analysis of Fusion Functions for Hybrid Retrieval”. *ACM Transactions on Information Systems* 42 (1): 1–35. <https://doi.org/10.1145/3596512>. <https://doi.org/10.1145/3596512>.
- Burgueño-Romero, Antonio M., Antonio Benítez-Hidalgo, Cristóbal Barba-González, dan José Francisco Aldana-Montes. 2025. “Toward an Open Source MLOps Architecture”. *IEEE Software* 42 (1): 59–64. <https://doi.org/10.1109/MS.2024.3421675>. <https://doi.org/10.1109/MS.2024.3421675>.
- Campese, Stefano, Ivano Lauriola, dan Alessandro Moschitti. 2024. “Improving Document Retrieval Coherence for Semantically Equivalent Queries”. *arXiv preprint arXiv:2404.12338*.

- Carbone, Paris, Stephan Ewen, Kostas Tzoumas, Volker Markl, dan Fabian Hueske. 2015. “Apache Flink: Stream and Batch Processing in a Single Engine”. *Bulletin of the IEEE Computer Society Technical Committee on Data Engineering* 36 (4): 28–38. <https://asterios.katsifodimos.com/assets/publications/flink-deb.pdf>.
- Ceph Authors. 2025. *Ceph Object Gateway Documentation*. Diakses 12 Juni 2026. <https://docs.ceph.com/en/latest/radosgw/>.
- cert-manager Contributors. 2025. *cert-manager: Automated TLS Certificate Management for Kubernetes*. Diakses 25 Mei 2026. <https://cert-manager.io/docs/>.
- Céspedes Sisniega, Jaime, Vicente Rodríguez, Germán Moltó, dan Álvaro López García. 2024. “Efficient and Scalable Covariate Drift Detection in Machine Learning Systems with Serverless Computing”. *Future Generation Computer Systems* 161:174–188. <https://doi.org/10.1016/j.future.2024.07.010>. <https://doi.org/10.1016/j.future.2024.07.010>.
- Chaos Mesh Authors. 2025. *Chaos Mesh: A Cloud-Native Chaos Engineering Platform*. Diakses 25 Mei 2026. <https://chaos-mesh.org/docs/>.
- Chien, Melody. 2022. *How to Improve Your Data Quality*. Diakses 30 Maret 2026. Gartner. <https://www.gartner.com/smarterwithgartner/how-to-improve-your-data-quality>.
- ClickHouse Inc. 2024. *ClickHouse Documentation*. Diakses 20 April 2026. <https://clickhouse.com/docs/>.
- Cloud Native Computing Foundation. 2024a. *Kubernetes Documentation*. Diakses 11 Maret 2026. <https://kubernetes.io/docs/>.
- . 2024b. *Observability Whitepaper*. Diakses 16 Maret 2026. <https://github.com/cncf/tag-observability/blob/main/whitepaper.md>.
- CloudNativePG Contributors. 2025. *CloudNativePG: Kubernetes Operator for PostgreSQL*. Diakses 25 Mei 2026. <https://cloudnative-pg.io/documentation/>.
- Contabo GmbH. 2026. *Cloud VPS Pricing*. Diakses 16 Juni 2026. <https://contabo.com/en/pricing/>.
- DataHub Project. 2024. *DataHub Documentation*. Diakses 20 April 2026. <https://datahubproject.io/docs/>.

- dbt Labs. 2025. *dbt: Data Build Tool Documentation*. Diakses 25 Mei 2026. <https://docs.getdbt.com/>.
- Debezium Community. 2025. *Debezium Documentation*. Diakses 12 Juni 2026. <https://debezium.io/documentation/>.
- Deuxfleurs Association. 2025. *Garage Documentation*. Diakses 12 Juni 2026. <https://garagehq.deuxfleurs.fr/documentation/>.
- Devlin, Jacob, Ming-Wei Chang, Kenton Lee, dan Kristina Toutanova. 2019. “BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding”. Dalam *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*, 4171–4186. Association for Computational Linguistics. <https://doi.org/10.18653/v1/N19-1423>. <https://doi.org/10.18653/v1/N19-1423>.
- Dex Authors. 2025. *Dex: A Federated OpenID Connect Provider*. Diakses 25 Mei 2026. <https://dexidp.io/docs/>.
- DigitalOcean. 2026. *Droplets Pricing*. Diakses 16 Juni 2026. <https://www.digitalocean.com/pricing/droplets>.
- European Central Bank. 2026. *Euro Foreign Exchange Reference Rates: US Dollar*. Diakses 16 Juni 2026. https://www.ecb.europa.eu/stats/policy_and_exchange_rates/euro_reference_exchange_rates/html/eurofxref-graph-usd.en.html.
- Evidently AI. 2025. *Evidently: Open-Source ML Observability Platform*. Diakses 25 Mei 2026. <https://docs.evidentlyai.com/>.
- External Secrets Authors. 2025. *External Secrets Operator Documentation*. Diakses 25 Mei 2026. <https://external-secrets.io/latest/>.
- Falco Authors. 2025. *Falco: Cloud-Native Runtime Security*. Diakses 25 Mei 2026. <https://falco.org/docs/>.
- Feast Project. 2024. *Feast Documentation*. Diakses 20 April 2026. <https://docs.feast.dev/>.
- Gitea Contributors. 2025. *Gitea Documentation*. Diakses 25 Mei 2026. <https://docs.gitea.com/>.

- Google Cloud. 2024. *MLOps: Continuous Delivery and Automation Pipelines in Machine Learning*. Cloud Architecture Center. Diakses 18 Maret 2026. <https://cloud.google.com/architecture/mlops-continuous-delivery-and-automation-pipelines-in-machine-learning>.
- . 2026a. *Cloud SQL Pricing*. Diakses 16 Juni 2026. <https://cloud.google.com/sql/pricing>.
- . 2026b. *Cloud Storage Pricing*. Diakses 16 Juni 2026. <https://cloud.google.com/storage/pricing>.
- . 2026c. *Compute Engine VM Instance Pricing*. Diakses 16 Juni 2026. <https://cloud.google.com/compute/vm-instance-pricing>.
- . 2026d. *Google Kubernetes Engine Pricing*. Diakses 16 Juni 2026. <https://cloud.google.com/kubernetes-engine/pricing>.
- . 2026e. *Managed Service for Apache Kafka Pricing*. Diakses 16 Juni 2026. <https://cloud.google.com/managed-service-for-apache-kafka/pricing>.
- . 2026f. *Memorystore for Redis Pricing*. Diakses 16 Juni 2026. <https://cloud.google.com/memorystore/docs/redis/pricing>.
- . 2026g. *Persistent Disk and Image Pricing*. Diakses 16 Juni 2026. <https://cloud.google.com/compute/disks-image-pricing>.
- Grafana Labs. 2024a. *Grafana Documentation*. Diakses 20 April 2026. <https://grafana.com/docs/grafana/latest/>.
- . 2024b. *Grafana Loki Documentation*. Diakses 25 Mei 2026. <https://grafana.com/docs/loki/latest/>.
- . 2024c. *Grafana Tempo: High-Volume Distributed Tracing Backend*. Diakses 25 Mei 2026. <https://grafana.com/docs/tempo/latest/>.
- . 2025. *Grafana Pyroscope: Continuous Profiling*. Diakses 25 Mei 2026. <https://grafana.com/docs/pyroscope/latest/>.
- Great Expectations. 2024. *Great Expectations Documentation*. Diakses 25 Mei 2026. <https://docs.greatexpectations.io/docs/>.

- Hamza, Muhammad, Muhammad Azeem Akbar, dan Rafael Capilla. 2024. “Understanding Cost Dynamics of Serverless Computing: An Empirical Study”. Dalam *Software Business: 14th International Conference, ICSOB 2023, Proceedings*, 500:456–470. Lecture Notes in Business Information Processing. Springer. https://doi.org/10.1007/978-3-031-53227-6_32. https://doi.org/10.1007/978-3-031-53227-6_32.
- Hetzner Online GmbH. 2026. *Price Adjustment: Cloud Server Pricing*. Diakses 16 Juni 2026. <https://docs.hetzner.com/general/infrastructure-and-availability/price-adjustment/>.
- Hevner, Alan R., Salvatore T. March, Jinsoo Park, dan Sudha Ram. 2004. “Design Science in Information Systems Research”. *MIS Quarterly* 28 (1): 75–105. <https://doi.org/10.2307/25148625>. <https://doi.org/10.2307/25148625>.
- Hinder, Fabian, Valerie Vaquet, Johannes Brinkrolf, dan Barbara Hammer. 2023. “Model-Based Explanations of Concept Drift”. *Neurocomputing* 555:126640. <https://doi.org/10.1016/j.neucom.2023.126640>. <https://doi.org/10.1016/j.neucom.2023.126640>.
- Hsu, C. H., P. W. Liu, Y. C. Fan, dan C. Y. Lin. 2024. “End-to-End Automation of ML Model Lifecycle Management Using Machine Learning Operations Platforms”. Dalam *2024 International Conference on Consumer Electronics-Taiwan (ICCE-Taiwan)*, 1–6. IEEE. <https://doi.org/10.1109/ICCE-Taiwan62264.2024.10674445>. <https://doi.org/10.1109/ICCE-Taiwan62264.2024.10674445>.
- IBM. 2024. *What is Observability? Logs, Metrics, and Distributed Tracing*. Diakses 18 Maret 2026. <https://www.ibm.com/topics/observability>.
- Istio Authors. 2024. *Istio Service Mesh*. Diakses 16 Maret 2026. <https://istio.io/>.
- Iterative, Inc. 2025. *DVC Documentation*. Diakses 12 Juni 2026. <https://dvc.org/doc>.
- Jain, Souratn, dan Jyotipriya Das. 2025. “Integrating Data Engineering and MLOps for Scalable and Resilient Machine Learning Pipelines: Frameworks, Challenges, and Future Trends”. *World Journal of Advanced Engineering Technology and Sciences* 14 (1): 241–253. <https://doi.org/10.30574/wjaets.2025.14.1.0020>. <https://doi.org/10.30574/wjaets.2025.14.1.0020>.

- Jegou, Herve, Matthijs Douze, dan Cordelia Schmid. 2011. “Product Quantization for Nearest Neighbor Search”. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 33 (1): 117–128. <https://doi.org/10.1109/TPAMI.2010.57>. <https://doi.org/10.1109/TPAMI.2010.57>.
- Johnson, Jeff, Matthijs Douze, dan Hervé Jégou. 2021. “Billion-Scale Similarity Search with GPUs”. *IEEE Transactions on Big Data* 7 (3): 535–547. <https://doi.org/10.1109/TBDATA.2019.2921572>. <https://doi.org/10.1109/TBDATA.2019.2921572>.
- Jourdan, Nicolas, Tom Bayer, Tobias Biegel, dan Joachim Metternich. 2023. “Handling Concept Drift in Deep Learning Applications for Process Monitoring”. *Procedia CIRP* 120:42–47. <https://doi.org/10.1016/j.procir.2023.08.007>. <https://doi.org/10.1016/j.procir.2023.08.007>.
- Kafbat Authors. 2025. *Kafbat UI for Apache Kafka Documentation*. Diakses 12 Juni 2026. <https://ui.docs.kafbat.io/>.
- Kartakis, Sokratis, Giuseppe Angelo Porcelli, Georgios Schinas, dan Shelbee Eigenbrode. 2022. *MLOps Foundation Roadmap for Enterprises with Amazon SageMaker*. AWS Machine Learning Blog. Diakses 18 Maret 2026. <https://aws.amazon.com/blogs/machine-learning/mlops-foundation-roadmap-for-enterprises-with-amazon-sagemaker/>.
- Kaufman, Shachar, Saharon Rosset, Claudia Perlich, dan Ori Stitelman. 2012. “Leakage in Data Mining: Formulation, Detection, and Avoidance”. *ACM Transactions on Knowledge Discovery from Data* 6 (4): 1–21. <https://doi.org/10.1145/2382577.2382579>.
- KEDA Authors. 2025. *KEDA: Kubernetes Event-Driven Autoscaling*. Diakses 25 Mei 2026. <https://keda.sh/docs/>.
- Kim, Gene, Jez Humble, Patrick Debois, dan John Willis. 2016. *The DevOps Handbook: How to Create World-Class Agility, Reliability, and Security in Technology Organizations*. 1st. IT Revolution Press. ISBN: 978-1942788003. <https://itrevolution.com/product/the-devops-handbook/>.
- Kleppmann, Martin. 2017. *Designing Data-Intensive Applications*. O’Reilly Media. ISBN: 978-1491903063. <https://www.oreilly.com/library/view/designing-data-intensive-applications/9781491903063/>.

- Knative Authors. 2025. *Knative Serving: Serverless on Kubernetes*. Diakses 25 Mei 2026. <https://knative.dev/docs/serving/>.
- Kreps, Jay. 2014. *Questioning the Lambda Architecture*. Diakses 24 Maret 2026. O'Reilly Radar. <https://www.oreilly.com/radar/questioning-the-lambda-architecture/>.
- Kreuzberger, Dominik, Niklas Kühl, dan Sebastian Hirschl. 2023. "Machine Learning Operations (MLOps): Overview, Definition, and Architecture". *IEEE Access* 11:31866–31879. <https://doi.org/10.1109/ACCESS.2023.3262138>. <https://doi.org/10.1109/ACCESS.2023.3262138>.
- KServe Contributors. 2024. *KServe Documentation*. Diakses 28 April 2026. <https://kserve.github.io/website/>.
- Kubeflow Authors. 2025a. *Katib: Kubernetes-Native Hyperparameter Tuning*. Diakses 25 Mei 2026. <https://www.kubeflow.org/docs/components/katib/>.
- . 2025b. *Kubeflow Notebooks*. Diakses 25 Mei 2026. <https://www.kubeflow.org/docs/components/notebooks/>.
- . 2025c. *Kubeflow Trainer: Distributed Training Operators*. Diakses 25 Mei 2026. <https://www.kubeflow.org/docs/components/trainer/>.
- Kubeflow Contributors. 2024. *Kubeflow Documentation*. Diakses 30 Maret 2026. <https://www.kubeflow.org/docs/>.
- Kubernetes Autoscaling SIG. 2025. *Vertical Pod Autoscaler*. Diakses 25 Mei 2026. <https://github.com/kubernetes/autoscaler/tree/master/vertical-pod-autoscaler>.
- Kueue Authors. 2024. *Kueue: Job Queueing for Kubernetes*. Diakses 25 Mei 2026. <https://kueue.sigs.k8s.io/docs/>.
- Kyverno Authors. 2024. *Kyverno: Kubernetes Native Policy Management*. Diakses 25 Mei 2026. <https://kyverno.io/docs/>.
- Lakekeeper Authors. 2025. *Lakekeeper: Apache Iceberg REST Catalog*. Diakses 25 Mei 2026. <https://docs.lakekeeper.io/>.
- Lakka, Mounika. 2025. "Kubernetes for Enterprise: Mastering Cloud-Native Container Orchestration at Scale". *European Modern Studies Journal* 9 (3): 358–367. [https://doi.org/10.59573/emsj.9\(3\).2025.31](https://doi.org/10.59573/emsj.9(3).2025.31). [https://doi.org/10.59573/emsj.9\(3\).2025.31](https://doi.org/10.59573/emsj.9(3).2025.31).

- Lamer, Antoine, Chloé Saint-Dizier, Nicolas Paris, dan Emmanuel Chazard. 2024. “Data Lake, Data Warehouse, Datamart, and Feature Store: Their Contributions to the Complete Data Reuse Pipeline”. *JMIR Medical Informatics* 12:e54590. <https://doi.org/10.2196/54590>. <https://doi.org/10.2196/54590>.
- Li, Anya, Bhala Ranganathan, Feng Pan, Mickey Zhang, Qianjun Xu, Runhan Li, Sethu Raman, Shail Paragbhai Shah, dan Vivienne Tang. 2023. “Managed Geo-Distributed Feature Store: Architecture and System Design”. *arXiv preprint arXiv:2305.20077*, <https://doi.org/10.48550/arXiv.2305.20077>. <https://arxiv.org/abs/2305.20077>.
- Liang, Penghao, Wei Chen, Yifan Zhang, dan Jun Liu. 2024. “Automating the Training and Deployment of Models in MLOps by Integrating Systems with Machine Learning”. *arXiv preprint arXiv:2405.09819*, <https://doi.org/10.48550/arXiv.2405.09819>. <https://arxiv.org/abs/2405.09819>.
- Limoncelli, Thomas. 2018. “GitOps: A Path to More Self-Service IT”. *Communications of the ACM* 61 (9): 38–42. <https://doi.org/10.1145/3233241>. <https://dl.acm.org/doi/10.1145/3233241>.
- Lu, Jie, Anjin Liu, Fan Dong, Feng Gu, Joao Gama, dan Guangquan Zhang. 2019. “Learning Under Concept Drift: A Review”. Versi pracetak: arXiv:2004.05785 (2020), *IEEE Transactions on Knowledge and Data Engineering* 31 (12): 2346–2363. <https://arxiv.org/abs/2004.05785>.
- Malkov, Yury A., dan D. A. Yashunin. 2020. “Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs”. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 42 (4): 824–836. <https://doi.org/10.1109/TPAMI.2018.2889473>. <https://doi.org/10.1109/TPAMI.2018.2889473>.
- Marz, Nathan, dan James Warren. 2015. *Big Data: Principles and Best Practices of Scalable Realtime Data Systems*. Manning Publications. ISBN: 978-1617290343. <https://www.manning.com/books/big-data>.
- McKnight, William. 2020. *Delivering on the Vision of MLOps: A Maturity-Based Approach*. Diakses 18 Maret 2026. GigaOm. <https://gigaom.com/report/delivering-on-the-vision-of-mlops/>.

- Microsoft Azure. 2023. *Machine Learning Operations Maturity Model*. Azure Architecture Center. Diakses 18 Maret 2026. <https://learn.microsoft.com/en-us/azure/architecture/ai-ml/guide/mlops-maturity-model>.
- MinIO, Inc. 2024. *MinIO Documentation*. Diakses 20 April 2026. <https://min.io/docs/>.
- . 2025. *KES: Stateless Key Management Server*. Diakses 25 Mei 2026. <https://github.com/minio/kcs>.
- MLflow Contributors. 2024. *MLflow Documentation*. Diakses 15 April 2026. <https://mlflow.org/docs/latest/index.html>.
- Mo, Di, Robert Cordingley, Donald Chinn, dan Wes Lloyd. 2023. “Addressing Serverless Computing Vendor Lock-In through Cloud Service Abstraction”. Dalam *2023 IEEE International Conference on Cloud Computing Technology and Science (CloudCom)*, 193–199. IEEE. <https://doi.org/10.1109/CloudCom59040.2023.00040>. <https://doi.org/10.1109/CloudCom59040.2023.00040>.
- Munappy, Aiswarya Raj, Jan Bosch, Helena Holmström Olsson, Anders Arpteg, dan Björn Brinne. 2022. “Data Management for Production Quality Deep Learning Models: Challenges and Solutions”. *Journal of Systems and Software* 191:111359. <https://doi.org/10.1016/j.jss.2022.111359>. <https://doi.org/10.1016/j.jss.2022.111359>.
- Munappy, Aiswarya Raj, David Issa Mattos, Jan Bosch, Helena Holmström Olsson, dan Anas Dakkak. 2020. “From Ad-Hoc Data Analytics to DataOps”. Dalam *Proceedings of the International Conference on Software and System Processes (ICSSP '20)*, 165–174. ACM. <https://doi.org/10.1145/3379177.3388909>. <https://doi.org/10.1145/3379177.3388909>.
- NATS Authors. 2025. *NATS JetStream Documentation*. Diakses 12 Juni 2026. <https://docs.nats.io/nats-concepts/jetstream>.
- OAuth2 Proxy Authors. 2025. *OAuth2 Proxy Documentation*. Diakses 25 Mei 2026. <https://oauth2-proxy.github.io/oauth2-proxy/>.
- Open Policy Agent Authors. 2024. *Open Policy Agent*. Diakses 30 Maret 2026. <https://www.openpolicyagent.org/>.
- OpenAI. 2022. *New and Improved Embedding Model*. OpenAI Blog. Diakses 18 Maret 2026. <https://openai.com/index/new-and-improved-embedding-model/>.

- OpenBao Authors. 2025. *OpenBao: Open Source Secrets Management*. Diakses 25 Mei 2026. <https://openbao.org/docs/>.
- OpenCost Authors. 2025. *OpenCost: Open Source Cost Monitoring for Kubernetes*. Diakses 25 Mei 2026. <https://www.opencost.io/docs/>.
- OpenLineage Authors. 2025. *OpenLineage: An Open Standard for Lineage Metadata Collection*. Diakses 25 Mei 2026. <https://openlineage.io/docs/>.
- OpenSearch Foundation. 2025. *OpenSearch: Open Source Search and Analytics Suite*. Diakses 25 Mei 2026. <https://opensearch.org/docs/latest/>.
- OpenTelemetry Authors. 2025. *OpenTelemetry Documentation*. Diakses 25 Mei 2026. <https://opentelemetry.io/docs/>.
- Oracle Corporation. 2025. *MySQL Reference Manual*. Diakses 25 Mei 2026. <https://dev.mysql.com/doc/>.
- OVHcloud. 2026. *Public Cloud Prices*. Diakses 16 Juni 2026. <https://us.ovhcloud.com/public-cloud/prices/>.
- Pachyderm, Inc. 2025. *Pachyderm Documentation*. Diakses 12 Juni 2026. <https://docs.pachyderm.com/>.
- Paleyes, Andrei, Raoul-Gabriel Urma, dan Neil D. Lawrence. 2022. "Challenges in Deploying Machine Learning: A Survey of Case Studies". *ACM Computing Surveys* 55 (6): 1–29. <https://doi.org/10.1145/3533378>. <https://doi.org/10.1145/3533378>.
- Peffer, Ken, Tuure Tuunanen, Marcus A. Rothenberger, dan Samir Chatterjee. 2007. "A Design Science Research Methodology for Information Systems Research". *Journal of Management Information Systems* 24 (3): 45–77. <https://doi.org/10.2753/MIS0742-1222240302>. <https://doi.org/10.2753/MIS0742-1222240302>.
- PERSOLKELLY Indonesia. 2023. *Indonesia Salary Guide 2023*. Diakses 16 Juni 2026. <https://www.persolindonesia.com/salary-guides>.
- Pimentel, João Felipe, Leonardo Murta, Vanessa Braganholo, dan Juliana Freire. 2019. "A Large-Scale Study About Quality and Reproducibility of Jupyter Notebooks". Dalam *2019 IEEE/ACM 16th International Conference on Mining Software Repositories (MSR)*, 507–517. IEEE. <https://doi.org/10.1109/MSR.2019.00077>. <https://doi.org/10.1109/MSR.2019.00077>.

- Polyzotis, Neoklis, Sudip Roy, Steven Euijong Whang, dan Martin Zinkevich. 2018. “Data Lifecycle Challenges in Production Machine Learning: A Survey”. *ACM SIGMOD Record* 47 (2): 17–28. <https://doi.org/10.1145/3299887.3299891>. <https://doi.org/10.1145/3299887.3299891>.
- PostgreSQL Global Development Group. 2025. *PostgreSQL: The World’s Most Advanced Open Source Relational Database*. Diakses 25 Mei 2026. <https://www.postgresql.org/docs/>.
- Project Nessie Authors. 2025. *Project Nessie Documentation*. Diakses 12 Juni 2026. <https://projectnessie.org/>.
- Prometheus Authors. 2024. *Prometheus Documentation*. Diakses 20 April 2026. <https://prometheus.io/docs/>.
- . 2025. *Prometheus Pushgateway*. Diakses 25 Mei 2026. <https://prometheus.io/docs/practices/pushing/>.
- Qdrant Team. 2025. *Qdrant: High-Performance Vector Search Engine*. Diakses 25 Mei 2026. <https://qdrant.tech/documentation/>.
- Redpanda Data, Inc. 2025. *Redpanda Documentation*. Diakses 12 Juni 2026. <https://docs.redpanda.com/>.
- Reimers, Nils, dan Iryna Gurevych. 2019. “Sentence-BERT: Sentence Embeddings Using Siamese BERT-Networks”. Dalam *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, 3982–3992. Association for Computational Linguistics. <https://doi.org/10.18653/v1/D19-1410>. <https://doi.org/10.18653/v1/D19-1410>.
- Reis, Denis Moreira dos, Peter Flach, Stan Matwin, dan Gustavo Batista. 2016. “Fast Unsupervised Online Drift Detection Using Incremental Kolmogorov-Smirnov Test”. Dalam *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 1545–1554. ACM. <https://doi.org/10.1145/2939672.2939836>. <https://doi.org/10.1145/2939672.2939836>.
- Rella, Bhanu Prakash Reddy. 2022. “MLOps and DataOps Integration for Scalable Machine Learning Deployment”. *International Journal for Multidisciplinary Research* 4 (1). <https://www.ijfmr.com/papers/2022/1/39278.pdf>.

- Rodrigues, Miguel G., Eduardo K. Viegas, Altair O. Santin, dan Fabrício Enembreck. 2025. “A MLOps Architecture for Near Real-Time Distributed Stream Learning Operation Deployment”. *Journal of Network and Computer Applications* 238:104169. <https://doi.org/10.1016/j.jnca.2025.104169>. <https://doi.org/10.1016/j.jnca.2025.104169>.
- Ross, Ron, Mark Winstead, dan Michael McEvilly. 2022. *Engineering Trustworthy Secure Systems*. NIST Special Publication 800-160v1r1. National Institute of Standards dan Technology. <https://doi.org/10.6028/NIST.SP.800-160v1r1>. <https://doi.org/10.6028/NIST.SP.800-160v1r1>.
- Sculley, D., Gary Holt, Daniel Golovin, Eugene Davydov, Todd Phillips, Dietmar Ebner, Vinay Chaudhary, Michael Young, Jean-Francois Crespo, dan Dan Dennison. 2015. “Hidden Technical Debt in Machine Learning Systems”. Dalam *Advances in Neural Information Processing Systems 28 (NIPS 2015)*, disunting oleh C. Cortes, N. D. Lawrence, D. D. Lee, M. Sugiyama, dan R. Garnett, 2503–2511. Red Hook, NY: Curran Associates, Inc. <https://proceedings.neurips.cc/paper/2015/file/86df7dcfd896fcdf2674f757a2463eba-Paper.pdf>.
- SeaweedFS Authors. 2025. *SeaweedFS Documentation*. Diakses 12 Juni 2026. <https://github.com/seaweedfs/seaweedfs/wiki>.
- Shankar, Shreya, Rolando Garcia, Joseph M. Hellerstein, dan Aditya G. Parameswaran. 2022. *Operationalizing Machine Learning: An Interview Study*. ArXiv preprint arXiv:2209.09125. <https://doi.org/10.48550/arXiv.2209.09125>. <https://arxiv.org/abs/2209.09125>.
- Sloth Authors. 2025. *Sloth: Easy and Reliable SLOs for Prometheus*. Diakses 25 Mei 2026. <https://sloth.dev/>.
- Stopford, Ben. 2018. *Designing Event-Driven Systems: Concepts and Patterns for Streaming Services with Apache Kafka*. O'Reilly Media. ISBN: 978-1-492-03822-1.
- Stoyanovich, Julia, Bill Howe, dan H. V. Jagadish. 2020. “Responsible Data Management”. *Proceedings of the VLDB Endowment* 13 (12): 3474–3488. <https://doi.org/10.14778/3415478.3415570>. <https://doi.org/10.14778/3415478.3415570>.
- Strimzi Authors. 2025. *Strimzi: Apache Kafka on Kubernetes*. Diakses 25 Mei 2026. <https://strimzi.io/documentation/>.

- Tekton Contributors. 2024. *Tekton Pipelines Documentation*. Diakses 25 Mei 2026. <https://tekton.dev/docs/>.
- Treeverse Ltd. 2025. *lakeFS: Data Version Control for Object Storage*. Diakses 25 Mei 2026. <https://docs.lakefs.io/>.
- Trino Software Foundation. 2024. *Trino: Distributed SQL Query Engine*. Diakses 6 April 2026. <https://trino.io/>.
- Valkey Contributors. 2025. *Valkey: An Open Source In-Memory Data Store*. Diakses 25 Mei 2026. <https://valkey.io/docs/>.
- Vangala, Rajesh, Priya Mehta, dan Arjun Singh. 2022. *MLOps in Practice: A Framework for Scalable AI Model Deployment, Monitoring, and Retraining*. Technical Report. Diakses 15 April 2026. https://www.researchgate.net/publication/393096121_MLOps_in_Practice_A_Framework_for_Scalable_AI_Model_Deployment_Monitoring_and_Retraining.
- Vela, Daniel, Andrew Sharp, Richard Zhang, Trang Nguyen, An Hoang, dan Oleg S. Pianykh. 2022. "Temporal Quality Degradation in AI Models". *Scientific Reports* 12:11654. <https://doi.org/10.1038/s41598-022-15245-z>. <https://doi.org/10.1038/s41598-022-15245-z>.
- Velero Authors. 2025. *Velero: Backup and Migrate Kubernetes Resources and Persistent Volumes*. Diakses 25 Mei 2026. <https://velero.io/docs/>.
- Vultr. 2026. *Vultr Cloud Pricing*. Diakses 16 Juni 2026. <https://www.vultr.com/pricing/>.
- Zaharia, Matei, Reynold S. Xin, Patrick Wendell, Tathagata Das, Michael Armbrust, Ankur Dave, Xiangrui Meng, dkk. 2016. "Apache Spark: A Unified Engine for Big Data Processing". *Communications of the ACM* 59 (11): 56–65. <https://doi.org/10.1145/2934664>. <https://doi.org/10.1145/2934664>.

LAMPIRAN A

KATALOG PERINTAH VERIFIKASI USE CASE

Lampiran ini memuat katalog perintah yang dapat dijalankan ulang untuk memverifikasi alur ujung ke ujung pada *use case* pasar aset kripto. Seluruh perintah dijalankan secara manual dari direktori akar repositori `~/documents/ta/`, kecuali *listing* menyertakan perintah `cd` eksplisit. Katalog ini terpisah dari `make phase-full` maupun rekonsiliasi Argo CD sehingga bukan bagian dari alur produksi. Sebagian besar perintah hanya membaca klaster, sedangkan skenario ketahanan (SK-N-04), skalabilitas (SK-N-03), dan rekonstruksi (SK-N-11) menerapkan gangguan terkontrol yang dapat dipulihkan pada *namespace* *experiments* yang terpisah dari produksi, sejalan dengan metode evaluasi pada Bab VI. Berkas skenario berada pada direktori `experiments/` yang sejajar dengan `platform/` dan `use-case-crypto/`; rincian argumen tiap skenario tersedia pada `README.md` di `experiments/` beserta subdirektori terkait. Prasyarat menjalankannya: platform telah terpasang (`make phase-full`), *use case* telah ter-*deploy* (`make usecase-crypto-build && make usecase-crypto-up`), dan seluruh *pod* berstatus *Running*. Sebelum skenario dijalankan, konfigurasi domain dimuat dan *namespace* beban dibuat melalui langkah persiapan berikut.

Listing A.1 Langkah persiapan pemuatan konfigurasi domain dan *namespace* pembangkit beban (dari `~/documents/ta/`)

```
1 # Muat DOMAIN; ganti berkas ini
2 set -a; . "${EXPERIMENTS_CONFIG:-experiments/config.crypto.env}";
   set +a
3 kubectl apply -f experiments/namespace.yaml
```

A.1 Tujuan T-1: Arsitektur Terintegrasi dan GitOps

Kelompok T-1 membuktikan pemasangan deklaratif dari satu sumber Git, otomasi siklus hidup model setara MLOps *level 2*, serta atribut kualitas lintas komponen. Perintahnya dirangkum pada *Listing A.2*.

Listing A.2 Perintah verifikasi tujuan T-1 (dari `~/documents/ta/`)

```
1 # SK-F-01/KF-01: rekonsiliasi feature view Feast
2 argocd app get crypto-use-case-local
3 kubectl -n use-case-crypto get configmap crypto-feast-feature-repo
   \
```

```

4  -o jsonpath='{.data.definitions\.py}' | grep 'FeatureView('
5
6  # SK-F-12/KF-12: konvergensi deklaratif tanpa drift
7  argocd app diff crypto-use-case-local
8  kubectl -n use-case-crypto get deploy
9
10 # SK-F-06/KF-06: canary dan rollback Rollouts
11 kubectl -n use-case-crypto get rollouts
12 kubectl argo rollouts -n use-case-crypto get rollout crypto-ml-
    bridge --watch
13
14 # SK-F-13/KF-13: satu environment uv multi-SDK
15 cd use-case-crypto/notebooks
16 uv sync
17 uv run jupyter nbconvert --to notebook --execute
    unified_sdk_walkthrough.ipynb
18 kubectl get inferencesservice -A | grep predictor
19
20 # SK-F-14/KF-14: kuota Kueue, job Pending
21 kubectl get clusterqueue,localqueue,workloads -A
22
23 # SK-N-03/KNF-03: skalabilitas HPA dan KEDA
24 k6 run experiments/load/hpa-keda-rampup.js
25 kubectl -n use-case-crypto get hpa,scaledobject,pods -w
26
27 # SK-N-04/KNF-04: injeksi fault pod/jaringan
28 envsubst < experiments/chaos/pod-chaos.yaml | kubectl apply -f -
29 kubectl apply -f experiments/chaos/network-chaos.yaml
30
31 # SK-N-08/KNF-08: ganti runtime via overlay
32 kubectl apply -k use-case-crypto/manifests/overlays/local
33
34 # SK-N-09/KNF-09: SSO satu sesi dex
35 kubectl get pods -A | grep -E 'dex|oauth2-proxy'
36
37 # SK-N-10/KNF-10: atribusi biaya OpenCost
38 kubectl get pods -A | grep -i opencost
39
40 # SK-N-11/KNF-11: rekonstruksi penuh dari Git
41 make phase-full
42
43 # SK-N-12/KNF-12: cakupan uji, konvergensi phase-full
44 make usecase-crypto-test
45 time make phase-full

```

A.2 Tujuan T-2: Tata Kelola Data

Kelompok T-2 membuktikan katalog metadata, *lineage*, kontrak kualitas, observabilitas, dan keamanan berjalan otomatis sepanjang *pipeline*. Perintahnya dirangkum pada Listing A.3.

Listing A.3 Perintah verifikasi tujuan T-2 (dari ~/documents/ta/)

```
1 # SK-F-08/KF-08: lineage end-to-end DataHub
2 experiments/lineage/lineage-walk.sh discover
3 experiments/lineage/lineage-walk.sh walk '<urn>'
4
5 # SK-F-09/KF-09: dua run, delta <1%
6 kubectl -n model-lifecycle get pods | grep mlflow
7
8 # SK-F-10/KF-10: audit trail MLflow
9 kubectl -n model-lifecycle get pods | grep mlflow
10
11 # SK-N-05/KNF-05: GE checkpoint tolak pelanggaran
12 kubectl -n use-case-crypto get cronjob | grep -iE 'quality|expect|
    ge-'
13 kubectl -n use-case-crypto create job ge-check --from=cronjob/
    crypto-ge-checkpoint
14
15 # SK-N-06/KNF-06: trace OTel Tempo/Grafana
16 kubectl get pods -A | grep -E 'tempo|grafana'
17
18 # SK-N-07/KNF-07: Trivy, TLS dan AES
19 kubectl get vulnerabilityreports -A
20 testssl.sh <public-endpoint>
```

A.3 Tujuan T-3: Deteksi *Drift* dan Pelatihan Ulang

Kelompok T-3 membuktikan deteksi *drift* terotomasi memicu pelatihan ulang dan penyusunan fitur pelatihan menjaga *point-in-time correctness*. Perintahnya dirangkum pada Listing A.4.

Listing A.4 Perintah verifikasi tujuan T-3 (dari ~/documents/ta/)

```
1 # SK-F-03/KF-03: point-in-time correctness gold
2 export CLICKHOUSE_USER=$(kubectl -n "$CLICKHOUSE_NAMESPACE" get
    secret clickhouse-admin \
3     -o jsonpath='{.data.CLICKHOUSE_USER}' | base64 -d)
4 export CLICKHOUSE_PASSWORD=$(kubectl -n "$CLICKHOUSE_NAMESPACE"
    get secret clickhouse-admin \
```

```

5 -o jsonpath='{.data.CLICKHOUSE_PASSWORD}' | base64 -d)
6 experiments/etl/data-landed-check.sh
7
8 # SK-F-05/KF-05: pipeline retraining KFP
9 uv run --with 'kfp[kubernetes]==2.16.0' use-case-crypto/pipelines/
  retraining_pipeline.py
10
11 # SK-F-07/KF-07: injeksi drift picu retrain
12 uv run experiments/drift/inject_drift.py --sigma 2.0
13 kubectl -n model-lifecycle get cronworkflow,workflow

```

A.4 Tujuan T-4: Layanan Fitur *Dual-Store*

Kelompok T-4 membuktikan layanan fitur *dual-store* menyajikan fitur tabular, agregat, dan vektor dengan kontrak konsisten antara pelatihan dan penyajian, beserta latensi dan *throughput* jalur penyajian. Perintahnya dirangkum pada Listing A.5.

Listing A.5 Perintah verifikasi tujuan T-4 (dari ~/documents/ta/)

```

1 # SK-F-02/KF-02: online Feast offline ClickHouse
2 kubectl -n model-lifecycle port-forward svc/feast 6566:6566 &
3 FEAST_URL=http://localhost:6566 uv run experiments/feast/online-
  serving-check.py
4 experiments/etl/data-landed-check.sh
5
6 # SK-F-04/KF-04: pencarian vektor Qdrant
7 k6 run experiments/load/vector-search-latency.js
8
9 # SK-F-11/KF-11: paritas dbt vs Flink
10 experiments/etl/data-landed-check.sh
11 experiments/parity/parity-check.sh
12
13 # SK-N-01/KNF-01: latensi p99 feast/vektor
14 k6 run experiments/load/feast-online-latency.js
15
16 # SK-N-02/KNF-02: throughput, consumer lag
17 k6 run experiments/load/feast-throughput.js
18 kafka-producer-perf-test --topic crypto.validated --num-records
  1000000 \
19 --record-size 256 --throughput -1 \
20 --producer-props bootstrap.servers="$KAFKA_BOOTSTRAP" \
21 --producer.config <sasl.properties>

```

Setiap perintah pada katalog ini berpasangan satu-satu dengan baris Tabel VI.1 dan Tabel VI.2 pada Bab VI, sehingga klaim pemenuhan kebutuhan dapat dilacak lang-

sung ke perintah yang menghasilkan buktinya. Untuk *use case* lain, cukup berkas `config.crypto.env` yang disalin dan disesuaikan blok DOMAIN-nya; kode skenario tetap *domain-agnostic* dan tidak perlu diubah.

LAMPIRAN B

PARAMETER PENILAIAN KRITERIA EVALUASI ALTERNATIF

Lampiran ini merinci parameter penilaian untuk setiap kriteria pada tabel evaluasi alternatif *open source* di Subbab III.4. Setiap kriteria memakai skala 0 sampai 2, dan parameter di bawah menjelaskan kondisi yang membuat sebuah alat memperoleh nilai 0, 1, atau 2 pada kriteria tersebut. Penilaian bersifat komparatif antar kandidat pada tabel yang sama dan diturunkan dari dokumentasi resmi, status lisensi, serta indikator publik proyek seperti riwayat rilis dan naungan yayasan per April 2026. Kolom Total pada tiap tabel menjumlahkan keempat kriteria tanpa pembobotan sehingga nilai tertingginya delapan.

Parameter dibagi dua kelompok agar tidak berulang. Tabel B.1 memuat kriteria lintas tabel yang muncul pada lebih dari satu tabel evaluasi dengan makna yang sama, seperti kematangan, komunitas, dan keluarga kriteria lisensi. Tabel B.2 memuat kriteria spesifik yang hanya dipakai pada satu lapis, dikelompokkan menurut lapis arsitekturnya. Nama kriteria pada kedua tabel ditulis sama persis dengan judul kolom pada tabel evaluasi yang memakainya.

B.1 Kriteria Lintas Tabel

Tabel B.1 Parameter Penilaian Kriteria Lintas Tabel

Kriteria	Skor 0	Skor 1	Skor 2
Kematangan	Belum ada rilis stabil atau pengembangan berhenti dua belas bulan terakhir	Rilis stabil tersedia namun versi mayor masih muda atau adopsi produksi terbatas	Rilis stabil dengan ritme rilis teratur dan rekam jejak produksi bertahun-tahun
Komunitas	Kontributor efektif tunggal atau aktivitas kontribusi minim	Komunitas aktif namun lebih kecil atau terpusat pada satu organisasi	Komunitas kontributor lintas organisasi terluas pada kategori fungsinya
Ekstensibilitas	Tanpa titik ekstensi resmi	Ekstensi terbatas pada sebagian antarmuka atau menuntut perubahan inti	Antarmuka <i>plugin</i> atau <i>provider</i> resmi terdokumentasi tanpa mengubah inti

Kriteria	Skor 0	Skor 1	Skor 2
Lisensi Terbuka, Lisensi <i>Open</i>	Lisensi tidak diakui OSI, seperti BSL, BUSL, atau <i>source-available</i>	Lisensi OSI dengan <i>copyleft</i> kuat seperti AGPL	Lisensi OSI permisif atau <i>copyleft</i> lemah seperti Apache 2.0, BSD, MPL
Integrasi K8s, Operabilitas K8s, Dukungan K8s	Tanpa dukungan resmi untuk berjalan pada Kubernetes	Berjalan melalui <i>chart</i> atau manifes tanpa pengelolaan siklus hidup, atau mensyaratkan komponen infrastruktur eksternal tetap seperti etcd atau antrean pesan	Dukungan kelas satu berupa <i>operator</i> , CRD, atau <i>chart</i> resmi terkelola
K8s-Native	Tidak dirancang berjalan pada Kubernetes	Berjalan pada Kubernetes namun model eksekusinya tidak berbasis CRD	Model eksekusi berbasis CRD sebagai antarmuka utama
Integrasi (<i>tracking</i> , metadata)	Integrasi terbatas pada ekosistem alat itu sendiri	Integrasi tersedia untuk sebagian alat data dan ML	Integrasi luas lintas ekosistem data dan ML yang terdokumentasi

B.2 Kriteria Spesifik per Lapis

Tabel B.2 Parameter Penilaian Kriteria Spesifik per Lapis

Kriteria	Skor 0	Skor 1	Skor 2
Distribusi Kubernetes			
<i>Footprint</i> Ringan	Kebutuhan sumber daya setara distribusi penuh	Ringan namun bergantung paket sistem tambahan	Biner tunggal dengan kebutuhan memori minimal terdokumentasi
Siap Produksi	Ditujukan untuk pengujian, bukan produksi	Dapat dipakai produksi dengan batasan tertentu	Didukung resmi untuk beban produksi
Tanpa Dependensi	Memerlukan beberapa komponen eksternal	Memerlukan satu <i>runtime</i> atau paket eksternal	<i>Runtime</i> kontainer tertanam tanpa dependensi eksternal
Ekosistem <i>Add-on</i>	Tanpa mekanisme <i>add-on</i>	<i>Add-on</i> bawaan terbatas	<i>Add-on</i> bawaan luas termasuk <i>controller</i> Helm
Message Broker dan Schema Registry			

Kriteria	Skor 0	Skor 1	Skor 2
Ekosistem Konektor	Konektor minim	Konektor tersedia namun cakupannya jauh lebih sempit	Ekosistem konektor terluas melalui Kafka Connect <i>native</i>
Operator K8s	Tanpa <i>operator</i>	<i>Operator</i> tersedia dengan cakupan terbatas	<i>Operator</i> matang mencakup klaster, pengguna, dan <i>topic</i>
Protokol Kafka	Protokol berbeda tanpa kompatibilitas	Kompatibel melalui lapisan protokol tambahan	Kompatibel penuh pada level protokol <i>native</i>
Kompatibilitas Confluent	API tidak kompatibel	Kompatibel pada sebagian <i>endpoint</i>	Kompatibel penuh dengan API Confluent
Dukungan Format	Satu format skema	Dua format atau dukungan parsial	Avro, JSON Schema, dan Protobuf penuh
Pemrosesan Data			
Kapabilitas <i>Batch</i>	Tanpa mode <i>batch</i>	<i>Batch</i> tersedia namun bukan kekuatan utama	<i>Batch</i> skala besar sebagai kapabilitas inti
Kapabilitas <i>Streaming</i>	Tanpa <i>streaming native</i>	<i>Streaming</i> melalui <i>micro-batch</i>	<i>Streaming event-time</i> dengan <i>exactly-once native</i>
Format Tabel <i>Lakehouse</i>			
Transaksi ACID	Tanpa jaminan transaksi	Transaksi dengan batasan mode tulis	Transaksi atomik penuh terdokumentasi
Evolusi Skema	Tanpa evolusi skema	Evolusi terbatas atau bergantung <i>engine</i>	Evolusi penuh pada level format tanpa menulis ulang data
Integrasi <i>Engine</i>	Terikat satu <i>engine</i>	Dukungan multi- <i>engine</i> tidak setara antar <i>engine</i>	Netral multi- <i>engine</i> dengan dukungan setara
Penyimpanan Objek dan Versi Data			
Kompatibilitas S3	Tanpa API S3	Subset API S3	Cakupan API S3 luas termasuk fitur lanjutan
<i>Footprint</i> Satu <i>Node</i>	Arsitektur menuntut banyak <i>node</i>	Dapat satu <i>node</i> dengan komponen berat	Berjalan ringan pada satu <i>node</i>
Enkripsi KMS	Tanpa enkripsi sisi <i>server</i> berbasis KMS	Enkripsi tanpa integrasi KMS eksternal	Enkripsi sisi <i>server</i> dengan integrasi KMS eksternal
Percabangan Tanpa Salin	Tanpa konsep percabangan	Versi berbasis penunjuk berkas per <i>commit</i>	<i>Branch</i> dan <i>merge</i> tanpa menyalin objek
Integrasi Objek S3	Tidak bekerja di atas penyimpanan objek	Bekerja melalui katalog atau <i>remote</i> khusus	Antarmuka S3 langsung tanpa mengubah klien

Kriteria	Skor 0	Skor 1	Skor 2
Skala Data Besar	Tidak dirancang untuk data besar	Skala bergantung repositori Git atau basis data tunggal	Metadata terpisah yang menangani jutaan objek
Layanan Fitur			
Performa <i>Query</i>	Latensi agregasi tinggi pada data besar	Agregasi cepat dengan batasan bentuk kueri	Agregasi kolumnar sub-detik pada data besar
Dukungan <i>Ingestion</i>	<i>Batch</i> saja dengan alat eksternal	<i>Streaming</i> atau <i>batch</i> dengan komponen tambahan	<i>Streaming</i> dan <i>batch native</i> termasuk Kafka
Latensi	Latensi baca di atas puluhan milidetik	Latensi milidetik pada penyimpanan <i>disk</i>	Latensi sub-milidetik pada memori
<i>Throughput</i>	<i>Throughput</i> rendah untuk <i>lookup</i> masif	<i>Throughput</i> tinggi dengan batasan skala vertikal	Ratusan ribu operasi per detik terdokumentasi
Latensi Rendah	Kueri ANN berlatensi tinggi	Latensi rendah dengan batasan indeks atau skala	HNSW dalam memori dengan jalur gRPC berlatensi rendah
Siklus Hidup Model			
<i>Pipeline</i> ML	Tanpa abstraksi <i>pipeline</i> ML	<i>Pipeline</i> generik yang dapat dipakai untuk ML	Abstraksi <i>pipeline</i> ML khusus dengan kontrak artefak
Skalabilitas	Replika statis tanpa <i>autoscaling</i>	<i>Autoscaling</i> tanpa <i>scale-to-zero</i>	<i>Autoscaling</i> termasuk <i>scale-to-zero</i>
<i>Framework</i>	Terikat satu kerangka model	Beberapa kerangka melalui penyesuaian manual	Multi-kerangka melalui <i>runtime</i> yang dapat diganti
Observabilitas dan BI			
Konektor SQL	Konektor terbatas satu keluarga basis data	Konektor untuk beberapa basis data umum	Puluhan dialek SQL melalui lapisan konektor standar
Multi-Tenant K8s	Tanpa dukungan Kubernetes resmi	Berjalan pada Kubernetes tanpa isolasi peran	<i>Chart</i> resmi dengan RBAC multi-tenant
GitOps			
Antarmuka	Tanpa antarmuka pemantauan	Antarmuka terbatas atau lewat alat pihak ketiga	Antarmuka web penuh untuk sinkronisasi dan <i>diff</i>
Keamanan Platform			

Kriteria	Skor 0	Skor 1	Skor 2
Bobot <i>Footprint</i>	Kebutuhan sumber daya besar dengan banyak komponen	Layanan tunggal namun berat pada memori	Proses tunggal ringan tanpa basis data wajib
Federasi OIDC	Tanpa federasi ke penyedia hulu	Federasi terbatas pada sebagian protokol	Federasi luas ke penyedia identitas hulu
Rotasi Dinamis	Rahasia statis tanpa rotasi	Rotasi manual atau terbatas	Kredensial dinamis dengan masa hidup otomatis
Integrasi ESO	Tanpa <i>provider</i> External Secrets Operator	Integrasi tidak langsung melalui <i>provider</i> generik	<i>Provider</i> External Secrets Operator resmi
Fitur L7	Fokus L3 dan L4 tanpa kontrol aplikasi	Kontrol L7 dasar	Kontrol L7 penuh termasuk <i>routing</i> dan otorisasi
Ekosistem CRD	Tanpa CRD	CRD terbatas pada sebagian fungsi	CRD kaya untuk lalu lintas, otorisasi, dan telemetri
Plugin/Ekstensi	Tanpa mekanisme <i>plugin</i>	<i>Plugin</i> terbatas pada edisi terbuka	Ekosistem <i>plugin</i> luas dengan muat ulang dinamis
Sintaks <i>Native</i>	Bahasa kebijakan eksternal terpisah dari Kubernetes	Bahasa khusus dengan integrasi Kubernetes	Kebijakan ditulis sebagai YAML <i>native</i> Kubernetes
Mode <i>Mutate</i>	Validasi saja	Mutasi terbatas	<i>Validate</i> , <i>mutate</i> , dan <i>generate</i> penuh
Pelaporan Pelanggaran	Log saja	Metrik atau status tanpa laporan standar	<i>PolicyReport</i> standar per sumber daya
Cakupan eBPF/ Kernel	Tanpa visibilitas kernel	Visibilitas kernel pada subset peristiwa	<i>Probe</i> eBPF modern mencakup <i>syscall</i> luas
Aturan Bawaan	Tanpa aturan bawaan	Contoh aturan terbatas	Pustaka aturan bawaan siap pakai yang luas

